玄铁 C910 用户手册（openc910）

2021 年 10 月 19 日

Copyright 2021 T-Head Semiconductor Co., Ltd.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

版本历史

版本	描述	日期
01	openc910 第一版发布。	2021.10.19

目录

第一章 概述 **1**

1.1 简介 1

1.2 特点 1

1.2.1 C910MP 处理器体系结构的主要特点 1

1.2.2 C910 核心的主要特点 2

1.3 配置 2

1.4 玄铁架构扩展技术 3

1.5 版本说明 3

1.6 命名规则 3

1.6.1 符号 3

1.6.2 术语 3

第二章 处理器简介 **6**

2.1 结构框图 6

2.2 核内子系统 7

2.2.1 指令提取单元 7

2.2.2 指令译码单元 7

2.2.3 执行单元 7

2.2.4 存储载入单元 7

2.2.5 指令退休单元 8

2.2.6 虚拟内存管理单元 8

2.2.7 物理内存保护单元 8

2.3 多核子系统 8

2.3.1 数据一致性接口单元 8

2.3.2 二级高速缓存 8

2.3.3 主设备接口 9

2.3.4 平台级中断控制器 9

2.3.5 计时器 9

2.4 接口概览 9

第三章 指令集 **10**

3.1 RV 基础指令集 10

3.1.1 整型指令集（RV64I） 10

3.1.2 乘除法指令集（RV64M） . . . 13
3.1.3 原子指令集（RV64A） . . . 14
3.1.4 单精度浮点指令集（RV64F） . . . 14
3.1.5 双精度浮点指令集（RV64D） . . . 16
3.1.6 压缩指令集（RV64C） . . . 18
3.2 玄铁扩展指令集 . . . 20
3.2.1 算术运算类指令 . . . 20
3.2.2 位操作类指令 . . . 21
3.2.3 内存访问类指令 . . . 21
3.2.4 Cache 指令 . . . 24
3.2.5 多核同步指令 . . . 25
3.2.6 半精度浮点类指令 . . . 25

第四章 处理器模式与寄存器 27
4.1 处理器模式 . . . 27
4.2 寄存器视图 . . . 27
4.3 通用寄存器 . . . 29
4.4 浮点寄存器 . . . 29
4.4.1 浮点寄存器与通用寄存器传输数据 . . . 30
4.4.2 维护寄存器精度的一致 . . . 30
4.5 系统控制寄存器 . . . 30
4.5.1 标准控制寄存器 . . . 30
4.5.2 扩展控制寄存器 . . . 32
4.6 数据格式 . . . 34
4.6.1 整型数据格式 . . . 34
4.6.2 浮点数据格式 . . . 34
4.7 大小端 . . . 36

第五章 异常与中断 37
5.1 概述 . . . 37
5.2 异常 . . . 39
5.2.1 异常响应 . . . 39
5.2.2 异常返回 . . . 39
5.2.3 非精确异常 . . . 40
5.3 中断 . . . 40
5.3.1 中断优先级 . . . 40
5.3.2 中断响应 . . . 40
5.3.3 中断返回 . . . 41

第六章 内存模型 42
6.1 内存模型概述 . . . 42
6.1.1 内存属性 . . . 42
6.1.2 内存一致性模型 . . . 43

6.2 虚拟内存管理 44
6.2.1 MMU 概述 44
6.2.2 TLB 组织形式 44
6.2.3 地址转换流程 44
6.2.4 系统控制寄存器 45
6.2.4.1 MMU 地址转换寄存器 (SATP) 46
6.2.4.2 MMU 控制寄存器 (SMCIR) 46
6.2.4.3 MMU Index 寄存器 (SMIR) 48
6.2.4.4 MMU EntryHi 寄存器 (SMEH) 49
6.2.4.5 MMU EntryLo 寄存器 (SMEL) 49
6.3 物理内存保护 51
6.3.1 PMP 概述 51
6.3.2 PMP 控制寄存器 52
6.3.2.1 物理内存保护设置寄存器（PMPCFG） 52
6.3.2.2 物理内存保护地址寄存器（PMPADDR） 54
6.4 内存访问顺序 54

第七章 内存子系统 56

7.1 内存子系统概述 56
7.2 L1 指令 Cache 56
7.2.1 概述 56
7.2.2 路预测 56
7.2.3 循环加速缓存器 57
7.2.4 分支历史表 57
7.2.5 分支跳转目标预测器 57
7.2.6 间接分支预测器 57
7.2.7 返回地址预测器 58
7.2.8 快速跳转目标预测器 58
7.3 L1 数据 Cache 59
7.3.1 概述 59
7.3.2 Cache 一致性 59
7.3.3 独占式访问 59
7.4 L2 Cache 60
7.4.1 L2 Cache 概要 60
7.4.2 Cache 一致性 60
7.4.3 组织形式 61
7.4.4 RAM 延时 61
7.5 内存加速访问 63
7.5.1 L1 I-Cache 指令预取 63
7.5.2 L1 D-Cache 多通道数据预取 63
7.5.3 L1 自适应的写分配机制 64
7.5.4 L2 预取机制 64

7.6 L1/L2 Cache 操作相关的指令和寄存器 64
7.6.1 L1 高速缓存扩展寄存器 65
7.6.2 L2 高速缓存扩展寄存器 65
7.6.3 L1/L2 Cache 操作指令 65

第八章 中断控制器 **67**
8.1 CLINT 中断控制器 67
8.1.1 CLINT 寄存器地址映射 67
8.1.2 软件中断 68
8.1.3 计时器中断 69
8.2 PLIC 中断控制器 70
8.2.1 中断的仲裁 71
8.2.2 中断的请求与响应 71
8.2.3 中断的完成 71
8.2.4 PLIC 寄存器地址映射 72
8.2.5 中断优先级配置寄存器（PLIC_PRIO） 74
8.2.6 中断等待寄存器（PLIC_IP） 74
8.2.7 中断使能寄存器（PLIC_IE） 75
8.2.8 PLIC 权限控制寄存器（PLIC_PER） 75
8.2.9 中断阈值寄存器（PLIC_TH） 76
8.2.10 中断响应/完成寄存器（PLIC_CLAIM） 76
8.3 多核中断 76
8.3.1 多个核心同时处理外部中断 77
8.3.2 核间发送软件中断 77

第九章 总线接口 **78**
9.1 AXI 主设备接口 78
9.1.1 AXI 主设备接口的特点 78
9.1.2 主设备接口的 Outstanding 能力 78
9.1.3 支持的传输类型 79
9.1.4 支持的响应类型 79
9.1.5 不同总线响应下的行为 80
9.1.6 AXI 主设备接口信号 80

第十章 调试 **84**
10.1 Debug 单元的功能 84
10.2 Debug 单元与 CPU Core 的连接 84
10.3 Debug 接口信号 86

第十一章 功耗管理 **88**
11.1 Power Domain 88
11.2 低功耗模式概要 88
11.3 核心 WFI 流程 88

11.4 单个核心下电流程 89
11.5 Cluster 下电流程（硬件清空 L2） 90
11.6 Cluster 下电流程（软件清空 L2） 91
11.7 简化场景：Cluster 整体下电流程（硬件清空 L2） 92
11.8 简化场景：Cluster 整体下电流程（软件清空 L2） 92
11.9 低功耗相关的编程模型和接口信号 94
11.9.1 编程模型 94
11.9.2 接口信号 94

第十二章 性能监测单元 95
12.1 PMU 简介 95
12.2 PMU 的编程模型 95
12.2.1 PMU 的基本用法 95
12.2.2 PMU 事件溢出中断 96
12.3 PMU 相关的控制寄存器 96
12.3.1 机器模式计数器访问授权寄存器（mcounteren） 96
12.3.2 超级用户模式计数器访问授权寄存器（scounteren） 97
12.3.3 机器模式计数禁止寄存器（mcountinhibit） 97
12.3.4 超级用户写使能寄存器（mcounterwen） 98
12.3.5 性能监测事件选择寄存器 98
12.3.6 事件计数器 100

第十三章 程序示例 101
13.1 处理器最优性能配置 101
13.2 MMU 设置示例 102
13.3 PMP 设置示例 105
13.4 高速缓存示例 106
13.4.1 高速缓存的开启示例 106
13.4.2 指令高速缓存与数据高速缓存的同步示例 107
13.4.3 TLB 与数据高速缓存的同步示例 108
13.5 同步原语示例 108
13.6 PLIC 设置示例 109
13.7 PMU 设置示例 110

第十四章 附录 A 标准指令术语 111
14.1 附录 A-1 I 指令术语 111
14.1.1 ADD——有符号加法指令 111
14.1.2 ADDI——有符号立即数加法指令 111
14.1.3 ADDIW——低 32 位有符号立即数加法指令 112
14.1.4 ADDW——低 32 位有符号加法指令 112
14.1.5 AND——按位与指令 113
14.1.6 ANDI——立即数按位与指令 113
14.1.7 AUIPC——PC 高位立即数加法指令 114

14.1.8 BEQ——相等分支指令 114
14.1.9 BGE——有符号大于等于分支指令 115
14.1.10 BGEU——无符号大于等于分支指令 115
14.1.11 BLT——有符号小于分支指令 116
14.1.12 BLTU——无符号小于分支指令 117
14.1.13 BNE——不等分支指令 117
14.1.14 CSRRC——控制寄存器清零传送指令 118
14.1.15 CSRRCI——控制寄存器立即数清零传送指令 119
14.1.16 CSRRS——控制寄存器置位传送指令 119
14.1.17 CSRRSI——控制寄存器立即数置位传送指令 120
14.1.18 CSRRW——控制寄存器读写传送指令 120
14.1.19 CSRRWI——控制寄存器立即数读写传送指令 121
14.1.20 EBREAK——断点指令 122
14.1.21 ECALL——环境异常指令 122
14.1.22 FENCE——存储同步指令 122
14.1.23 FENCE.I——指令流同步指令 123
14.1.24 JAL——直接跳转子程序指令 123
14.1.25 JALR——寄存器跳转子程序指令 124
14.1.26 LB——有符号扩展字节加载指令 125
14.1.27 LBU——无符号扩展字节加载指令 125
14.1.28 LD——双字加载指令 125
14.1.29 LH——有符号扩展半字加载指令 126
14.1.30 LHU——无符号扩展半字加载指令 126
14.1.31 LUI——高位立即数装载指令 127
14.1.32 LW——有符号扩展字加载指令 127
14.1.33 LWU——无符号扩展字加载指令 128
14.1.34 MRET——机器模式异常返回指令 128
14.1.35 OR——按位或指令 129
14.1.36 ORI——立即数按位或指令 129
14.1.37 SB——字节存储指令 130
14.1.38 SD——双字存储指令 130
14.1.39 SFENCE.VMA——虚拟内存同步指令 130
14.1.40 SH——半字存储指令 131
14.1.41 SLL——逻辑左移指令 132
14.1.42 SLLI——立即数逻辑左移指令 132
14.1.43 SLLIW——低 32 位立即数逻辑左移指令 133
14.1.44 SLLW——低 32 位逻辑左移指令 133
14.1.45 SLT——有符号比较小于置位指令 134
14.1.46 SLTI——有符号立即数比较小于置位指令 134
14.1.47 SLTIU——无符号立即数比较小于置位指令 135
14.1.48 SLTU——无符号比较小于置位指令 135
14.1.49 SRA——算数右移指令 136

14.1.50 SRAI——立即数算数右移指令 136
14.1.51 SRAIW——低 32 位立即数算数右移指令 137
14.1.52 SRAW——低 32 位算数右移指令 137
14.1.53 SRET——超级用户模式异常返回指令 137
14.1.54 SRL——逻辑右移指令 138
14.1.55 SRLI——立即数逻辑右移指令 138
14.1.56 SRLIW——低 32 位立即数逻辑右移指令 139
14.1.57 SRLW——低 32 位逻辑右移指令 139
14.1.58 SUB——有符号减法指令 140
14.1.59 SUBW——低 32 位有符号减法指令 140
14.1.60 SW——字存储指令 141
14.1.61 WFI——进入低功耗模式指令 141
14.1.62 XOR——按位异或指令 141
14.1.63 XORI——立即数按位异或指令 142
14.2 附录 A-2 M 指令术语 142
14.2.1 DIV——有符号除法指令 142
14.2.2 DIVU——无符号除法指令 143
14.2.3 DIVUW——低 32 位无符号除法指令 143
14.2.4 DIVW——低 32 位有符号除法指令 144
14.2.5 MUL——有符号乘法指令 145
14.2.6 MULH——有符号乘法取高位指令 145
14.2.7 MULHSU——有符号无符号乘法取高位指令 145
14.2.8 MULHU——无符号乘法取高位指令 146
14.2.9 MULW——低 32 位有符号乘法指令 146
14.2.10 REM——有符号取余指令 147
14.2.11 REMU——无符号取余指令 147
14.2.12 REMUW——低 32 位无符号取余指令 148
14.2.13 REMW——低 32 位有符号取余指令 148
14.3 附录 A-3 A 指令术语 149
14.3.1 AMOADD.D——原子加法指令 149
14.3.2 AMOADD.W——低 32 位原子加法指令 150
14.3.3 AMOAND.D——原子按位与指令 151
14.3.4 AMOAND.W——低 32 位原子按位与指令 151
14.3.5 AMOMAX.D——原子有符号取最大值指令 152
14.3.6 AMOMAX.W——低 32 位原子有符号取最大值指令 153
14.3.7 AMOMAXU.D——原子无符号取最大值指令 154
14.3.8 AMOMAXU.W——低 32 位原子无符号取最大值指令 155
14.3.9 AMOMIN.D——原子有符号取最小值指令 155
14.3.10 AMOMIN.W——低 32 位原子有符号取最小值指令 156
14.3.11 AMOMINU.D——原子无符号取最小值指令 157
14.3.12 AMOMINU.W——低 32 位原子无符号取最小值指令 158
14.3.13 AMOOR.D——原子按位或指令 159

14.3.14 AMOOR.W——低 32 位原子按位或指令 159
14.3.15 AMOSWAP.D——原子交换指令 160
14.3.16 AMOSWAP.W——低 32 位原子交换指令 161
14.3.17 AMOXOR.D——原子按位异或指令 162
14.3.18 AMOXOR.W——低 32 位原子按位异或指令 163
14.3.19 LR.D——双字加载保留指令 163
14.3.20 LR.W——字加载保留指令 164
14.3.21 SC.D——双字条件存储指令 165
14.3.22 SC.W——字条件存储指令 166
14.4 附录 A-4 F 指令术语 167
14.4.1 FADD.S——单精度浮点加法指令 167
14.4.2 FCLASS.S——单精度浮点分类指令 168
14.4.3 FCVT.L.S——单精度浮点转换成有符号长整型指令 169
14.4.4 FCVT.LU.S——单精度浮点转换成无符号长整型指令 170
14.4.5 FCVT.S.L——有符号长整型转换成单精度浮点数指令 171
14.4.6 FCVT.S.LU——无符号长整型转换成单精度浮点数指令 172
14.4.7 FCVT.S.W——有符号整型转换成单精度浮点数指令 172
14.4.8 FCVT.S.WU——无符号整型转换成单精度浮点数指令 173
14.4.9 FCVT.W.S——单精度浮点转换成有符号整型指令 174
14.4.10 FCVT.WU.S——单精度浮点转换成无符号整型指令 175
14.4.11 FDIV.S——单精度浮点除法指令 176
14.4.12 FEQ.S——单精度浮点比较相等指令 177
14.4.13 FLE.S——单精度浮点比较小于等于指令 177
14.4.14 FLT.S——单精度浮点比较小于指令 178
14.4.15 FLW——单精度浮点加载指令 178
14.4.16 FMADD.S——单精度浮点乘累加指令 179
14.4.17 FMAX.S——单精度浮点取最大值指令 180
14.4.18 FMIN.S——单精度浮点取最小值指令 180
14.4.19 FMSUB.S——单精度浮点乘累减指令 181
14.4.20 FMUL.S——单精度浮点乘法指令 182
14.4.21 FMV.W.X——单精度浮点写传送指令 183
14.4.22 FMV.X.W——单精度浮点寄存器读传送指令 183
14.4.23 FNMADD.S——单精度浮点乘累加取负指令 184
14.4.24 FNMSUB.S——单精度浮点乘累减取负指令 185
14.4.25 FSGNJ.S——单精度浮点符号注入指令 185
14.4.26 FSGNJN.S——单精度浮点符号取反注入指令 186
14.4.27 FSGNJX.S——单精度浮点符号异或注入指令 187
14.4.28 FSQRT.S——单精度浮点开方指令 187
14.4.29 FSUB.S——单精度浮点减法指令 188
14.4.30 FSW——单精度浮点存储指令 189
14.5 附录 A-5 D 指令术语 189
14.5.1 FADD.D——双精度浮点加法指令 189

14.5.2 FCLASS.D——双精度浮点分类指令 190
14.5.3 FCVT.D.L——有符号长整型转换成双精度浮点数指令 191
14.5.4 FCVT.D.LU——无符号长整型转换成双精度浮点数指令 192
14.5.5 FCVT.D.S——单精度浮点转换成双精度浮点指令 193
14.5.6 FCVT.D.W——有符号整型转换成双精度浮点数指令 194
14.5.7 FCVT.D.WU——无符号整型转换成双精度浮点数指令 194
14.5.8 FCVT.L.D——双精度浮点转换成有符号长整型指令 195
14.5.9 FCVT.LU.D——双精度浮点转换成无符号长整型指令 195
14.5.10 FCVT.S.D——双精度浮点转换成单精度浮点指令 196
14.5.11 FCVT.W.D——双精度浮点转换成有符号整型指令 197
14.5.12 FCVT.WU.D——双精度浮点转换成无符号整型指令 198
14.5.13 FDIV.D——双精度浮点除法指令 199
14.5.14 FEQ.D——双精度浮点比较相等指令 200
14.5.15 FLD——双精度浮点加载指令 200
14.5.16 FLE.D——双精度浮点比较小于等于指令 201
14.5.17 FLT.D——双精度浮点比较小于指令 201
14.5.18 FMADD.D——双精度浮点乘累加指令 202
14.5.19 FMAX.D——双精度浮点取最大值指令 203
14.5.20 FMIN.D——双精度浮点取最小值指令 203
14.5.21 FMSUB.D——双精度浮点乘累减指令 204
14.5.22 FMUL.D——双精度浮点乘法指令 205
14.5.23 FMV.D.X——双精度浮点写传送指令 206
14.5.24 FMV.X.D——双精度浮点读传送指令 206
14.5.25 FNMADD.D——双精度浮点乘累加取负指令 207
14.5.26 FNMSUB.D——双精度浮点乘累减取负指令 208
14.5.27 FSD——双精度浮点存储指令 208
14.5.28 FSGNJ.D——双精度浮点符号注入指令 209
14.5.29 FSGNJN.D——双精度浮点符号取反注入指令 209
14.5.30 FSGNJX.D——双精度浮点符号异或注入指令 210
14.5.31 FSQRT.D——双精度浮点开方指令 211
14.5.32 FSUB.D——双精度浮点减法指令 211
14.6 附录 A-6 C 指令术语 212
14.6.1 C.ADD——有符号加法指令 212
14.6.2 C.ADDI——有符号立即数加法指令 213
14.6.3 C.ADDIW——低 32 位有符号立即数加法指令 213
14.6.4 C.ADDI4SPN——4 倍立即数和堆栈指针相加指令 214
14.6.5 C.ADDI16SP——加 16 倍立即数到堆栈指针指令 215
14.6.6 C.ADDW——低 32 位有符号加法指令 215
14.6.7 C.AND——按位与指令 216
14.6.8 C.ANDI——立即数按位与指令 217
14.6.9 C.BEQZ——等于零分支指令 218
14.6.10 C.BNEZ——不等于零分支指令 219

14.6.11 C.EBREAK——断点指令 220
14.6.12 C.FLD——浮点双字加载指令 220
14.6.13 C.FLDSP——浮点双字堆栈加载指令 221
14.6.14 C.FSD——浮点双字存储指令 222
14.6.15 C.FSDSP——浮点双字堆栈存储指令 223
14.6.16 C.J——无条件跳转指令 223
14.6.17 C.JALR——寄存器跳转子程序指令 224
14.6.18 C.JR——寄存器跳转指令 225
14.6.19 C.LD——双字加载指令 225
14.6.20 C.LDSP——双字堆栈加载指令 226
14.6.21 C.LI——立即数传送指令 227
14.6.22 C.LUI——高位立即数传送指令 227
14.6.23 C.LW——字加载指令 228
14.6.24 C.LWSP——字堆栈加载指令 229
14.6.25 C.MV——数据传送指令 229
14.6.26 C.NOP——空指令 230
14.6.27 C.OR——按位或指令 230
14.6.28 C.SD——双字存储指令 231
14.6.29 C.SDSP——双字堆栈存储指令 232
14.6.30 C.SLLI——立即数逻辑左移指令 232
14.6.31 C.SRAI——立即数算数右移指令 233
14.6.32 C.SRLI——立即数逻辑右移指令 234
14.6.33 C.SW——字存储指令 235
14.6.34 C.SWSP——字堆栈存储指令 235
14.6.35 C.SUB——有符号减法指令 236
14.6.36 C.SUBW——低 32 位有符号减法指令 237
14.6.37 C.XOR——按位异或指令 238
14.7 附录 A-8 伪指令列表 238

第十五章 附录 B 平头哥扩展指令术语 241
15.1 附录 B-1 Cache 指令术语 241
15.1.1 DCACHE.CALL——DCACHE 清全部脏表项指令 241
15.1.2 DCACHE.CIALL——DCACHE 清全部脏表项后无效指令 242
15.1.3 DCACHE.CIPA ——DCACHE 按物理地址清脏表项并无效 242
15.1.4 DCACHE.CISW——DCACHE 按 way/set 清脏表项并无效指令 243
15.1.5 DCACHE.CIVA——DCACHE 按虚拟地址清脏表项并无效 243
15.1.6 DCACHE.CPA——DCACHE 按物理地址清脏表项 244
15.1.7 DCACHE.CPAL1 ——L1DCACHE 按物理地址清脏表项 244
15.1.8 DCACHE.CVA——DCACHE 按虚拟地址清脏表项 245
15.1.9 DCACHE.CVAL1——L1DCACHE 按虚拟地址清脏表项 245
15.1.10 DCACHE.IPA ——DCACHE 按物理地址无效指令 246
15.1.11 DCACHE.ISW ——DCACHE 按 set/way 无效指令 247

15.1.12 DCACHE.IVA ——DCACHE 按虚拟地址无效指令 247
15.1.13 DCACHE.IALL——DCACHE 无效所有表项指令 248
15.1.14 ICACHE.IALL——ICACHE 无效所有表项指令 248
15.1.15 ICACHE.IALLS——ICACHE 广播无效所有表项指令 249
15.1.16 ICACHE.IPA——ICACHE 按物理地址无效表项指令 249
15.1.17 ICACHE.IVA——ICACHE 按虚拟地址无效表项指令 250
15.1.18 L2CACHE.CALL——L2CACHE 清所有脏表项指令 250
15.1.19 L2CACHE.CIALL——L2CACHE 清所有脏表项并无效指令 251
15.1.20 L2CACHE.IALL——L2CACHE 无效指令 251
15.1.21 DCACHE.CSW ——DCACHE 按 set/way 清脏表项 252
15.2 附录 B-2 多核同步指令术语 252
15.2.1 SFENCE.VMAS——虚拟内存同步广播指令 253
15.2.2 SYNC——同步指令 254
15.2.3 SYNC.I——同步清空指令 254
15.2.4 SYNC.IS——同步清空广播指令 254
15.2.5 SYNC.S——同步广播指令 255
15.3 附录 B-3 算术运算指令术语 255
15.3.1 ADDSL——寄存器移位相加指令 255
15.3.2 MULA——乘累加指令 256
15.3.3 MULAH——低 16 位乘累加指令 256
15.3.4 MULAW——低 32 位乘累加指令 257
15.3.5 MULS——乘累减指令 257
15.3.6 MULSH——低 16 位乘累减指令 258
15.3.7 MULSW——低 32 位乘累减指令 258
15.3.8 MVEQZ——寄存器为 0 传送指令 259
15.3.9 MVNEZ——寄存器非 0 传送指令 259
15.3.10 SRRI——循环右移指令 260
15.3.11 SRRIW——低 32 位循环右移指令 260
15.4 附录 B-4 位操作指令术语 260
15.4.1 EXT——寄存器连续位提取符号位扩展指令 261
15.4.2 EXTU——寄存器连续位提取零扩展指令 261
15.4.3 FF0——快速找 0 指令 262
15.4.4 FF1——快速找 1 指令 262
15.4.5 REV——字节倒序指令 262
15.4.6 REVW——低 32 位字节倒序指令 263
15.4.7 TST——比特为 0 测试指令 264
15.4.8 TSTNBZ——字节为 0 测试指令 264
15.5 附录 B-5 存储指令术语 265
15.5.1 FLRD——浮点寄存器移位双字加载指令 265
15.5.2 FLRW——浮点寄存器移位字加载指令 265
15.5.3 FLURD——浮点寄存器低 32 位移位双字加载指令 266
15.5.4 FLURW——浮点寄存器低 32 位移位字加载指令 267

15.5.5 FSRD——浮点寄存器移位双字存储指令 267
15.5.6 FSRW——浮点寄存器移位字存储指令 268
15.5.7 FSURD——浮点寄存器低 32 位移位双字存储指令 268
15.5.8 FSURW——浮点寄存器低 32 位移位字存储指令 269
15.5.9 LBIA——符号位扩展字节加载基地址自增指令 269
15.5.10 LBIB——基地址自增符号位扩展字节加载指令 270
15.5.11 LBUIA——零扩展字节加载基地址自增指令 270
15.5.12 LBUIB——基地址自增零扩展字节加载指令 271
15.5.13 LDD——双寄存器加载指令 271
15.5.14 LDIA——符号位扩展双字加载基地址自增指令 272
15.5.15 LDIB——基地址自增符号位扩展双字加载指令 272
15.5.16 LHIA——符号位扩展半字加载基地址自增指令 273
15.5.17 LHIB——基地址自增符号位扩展半字加载指令 273
15.5.18 LHUIA——零扩展半字加载基地址自增指令 274
15.5.19 LHUIB——基地址自增零扩展半字加载指令 274
15.5.20 LRB——寄存器移位符号位扩展字节加载指令 275
15.5.21 LRBU——寄存器移位零扩展扩展字节加载指令 275
15.5.22 LRD——寄存器移位双字加载指令 276
15.5.23 LRH——寄存器移位符号位扩展半字加载指令 276
15.5.24 LRHU——寄存器移位零扩展扩展半字加载指令 277
15.5.25 LRW——寄存器移位符号位扩展字加载指令 277
15.5.26 LRWU——寄存器移位零扩展扩展字加载指令 277
15.5.27 LURB——寄存器低 32 位移位符号位扩展字节加载指令 278
15.5.28 LURBU——寄存器低 32 位移位零扩展字节加载指令 278
15.5.29 LURD——寄存器低 32 位移位双字加载指令 279
15.5.30 LURH——寄存器低 32 位移位符号位扩展半字加载指令 279
15.5.31 LURHU——寄存器低 32 位移位零扩展半字加载指令 280
15.5.32 LURW——寄存器低 32 位移位符号位扩展字加载指令 280
15.5.33 LURWU——寄存器低 32 位移位零扩展字加载指令 281
15.5.34 LWD——符号位扩展双寄存器字加载指令 281
15.5.35 LWIA——符号位扩展字加载基地址自增指令 282
15.5.36 LWIB——基地址自增符号位扩展字加载指令 282
15.5.37 LWUD——零扩展双寄存器字加载指令 283
15.5.38 LWUIA——零扩展字加载基地址自增指令 284
15.5.39 LWUIB——基地址自增零扩展字加载指令 284
15.5.40 SBIA——字节存储基地址自增指令 285
15.5.41 SBIB——基地址自增字节存储指令 285
15.5.42 SDD——双寄存器存储指令 286
15.5.43 SDIA——双字存储基地址自增指令 286
15.5.44 SDIB——基地址自增双字存储指令 287
15.5.45 SHIA——半字存储基地址自增指令 287
15.5.46 SHIB——基地址自增半字存储指令 287

15.5.47 SRB——寄存器移位字节存储指令 288
15.5.48 SRD——寄存器移位双字存储指令 288
15.5.49 SRH——寄存器移位半字存储指令 289
15.5.50 SRW——寄存器移位字存储指令 289
15.5.51 SURB——寄存器低 32 位移位字节存储指令 290
15.5.52 SURD——寄存器低 32 位移位双字存储指令 290
15.5.53 SURH——寄存器低 32 位移位半字存储指令 291
15.5.54 SURW——寄存器低 32 位移位字存储指令 291
15.5.55 SWIA——字存储基地址自增指令 292
15.5.56 SWIB——基地址自增字存储指令 292
15.5.57 SWD——双寄存器低 32 位存储指令 293
15.6 附录 B-6 浮点半精度指令术语 293
15.6.1 FADD.H——半精度浮点加法指令 293
15.6.2 FCLASS.H——半精度浮点分类指令 294
15.6.3 FCVT.D.H——半精度浮点转换成双精度浮点指令 295
15.6.4 FCVT.H.D——双精度浮点转换成半精度浮点指令 296
15.6.5 FCVT.H.L——有符号长整型转换成半精度浮点数指令 297
15.6.6 FCVT.H.LU——无符号长整型转换成半精度浮点数指令 297
15.6.7 FCVT.H.S——单精度浮点转换成半精度浮点指令 298
15.6.8 FCVT.H.W——有符号整型转换成半精度浮点数指令 299
15.6.9 FCVT.H.WU——无符号整型转换成半精度浮点数指令 300
15.6.10 FCVT.L.H——半精度浮点转换成有符号长整型指令 301
15.6.11 FCVT.LU.H——半精度浮点转换成无符号长整型指令 302
15.6.12 FCVT.S.H——半精度浮点转换成单精度浮点指令 303
15.6.13 FCVT.W.H——半精度浮点转换成有符号整型指令 303
15.6.14 FCVT.WU.H——半精度浮点转换成无符号整型指令 304
15.6.15 FDIV.H——半精度浮点除法指令 305
15.6.16 FEQ.H——半精度浮点比较相等指令 306
15.6.17 FLE.H——半精度浮点比较小于等于指令 306
15.6.18 FLH——半精度浮点加载指令 307
15.6.19 FLT.H——半精度浮点比较小于指令 307
15.6.20 FMADD.H——半精度浮点乘累加指令 308
15.6.21 FMAX.H——半精度浮点取最大值指令 309
15.6.22 FMIN.H——半精度浮点取最小值指令 309
15.6.23 FMSUB.H——半精度浮点乘累减指令 310
15.6.24 FMUL.H——半精度浮点乘法指令 311
15.6.25 FMV.H.X——半精度浮点写传送指令 312
15.6.26 FMV.X.H——半精度浮点寄存器读传送指令 312
15.6.27 FNMADD.H——半精度浮点乘累加取负指令 313
15.6.28 FNMSUB.H——半精度浮点乘累减取负指令 314
15.6.29 FSGNJ.H——半精度浮点符号注入指令 315
15.6.30 FSGNJN.H——半精度浮点符号取反注入指令 315

15.6.31 FSGNJX.H——半精度浮点符号异或注入指令 316
15.6.32 FSH——半精度浮点存储指令 316
15.6.33 FSQRT.H——半精度浮点开方指令 317
15.6.34 FSUB.H——半精度浮点减法指令 318

第十六章 附录 C 控制寄存器 319

16.1 附录 C-1 机器模式控制寄存器 319
16.1.1 机器模式信息寄存器组 319
16.1.1.1 机器模式供应商编号寄存器（MVENDORID） 319
16.1.1.2 机器模式架构编号寄存器（MARCHID） 319
16.1.1.3 机器模式硬件实现编号寄存器（MIMPID） 319
16.1.1.4 机器模式逻辑内核编号寄存器（MHARTID） 320
16.1.2 机器模式异常配置寄存器组 320
16.1.2.1 机器模式处理器状态寄存器（MSTATUS） 320
16.1.2.2 机器模式处理器指令集特性寄存器（MISA） 322
16.1.2.3 机器模式异常降级控制寄存器（MEDELEG） 323
16.1.2.4 机器模式中断降级控制寄存器（MIDELEG） 323
16.1.2.5 机器模式中断使能控制寄存器（MIE） 323
16.1.2.6 机器模式向量基址寄存器（MTVEC） 325
16.1.2.7 机器模式计数器访问授权寄存器（MCOUNTEREN） 325
16.1.3 机器模式异常处理寄存器组 325
16.1.3.1 机器模式异常临时数据备份寄存器（MSCRATCH） 325
16.1.3.2 机器模式异常保留程序计数器寄存器（MEPC） 326
16.1.3.3 机器模式异常事件向量寄存器（MCAUSE） 326
16.1.3.4 机器模式中断等待状态寄存器（MIP） 326
16.1.4 机器模式内存保护寄存器组 328
16.1.4.1 机器模式物理内存保护配置寄存器（PMPCFG） 328
16.1.4.2 机器模式物理内存地址寄存器（PMPADDR） 328
16.1.5 机器模式计数器寄存器组 328
16.1.5.1 机器模式周期计数器（MCYCLE） 328
16.1.5.2 机器模式退休指令计数器（MINSTRET） 328
16.1.5.3 机器模式事件计数器（MHPMCOUNTERn） 329
16.1.6 机器模式计数器配置寄存器组 329
16.1.6.1 机器模式事件选择器（MHPMEVENTn） 329
16.1.7 机器模式处理器控制和状态扩展寄存器组 329
16.1.7.1 机器模式扩展状态寄存器（MXSTATUS） 329
16.1.7.2 机器模式硬件配置寄存器（MHCR） 331
16.1.7.3 机器模式硬件操作寄存器（MCOR） 333
16.1.7.4 机器模式 L2Cache 控制寄存器（MCCR2） 334
16.1.7.5 机器模式隐式操作寄存器（MHINT） 335
16.1.7.6 机器模式复位向量基址寄存器（MRVBR） 338
16.1.7.7 超级户态计数器写使能寄存器（MCOUNTERWEN） 338

16.1.7.8 机器模式事件中断使能寄存器（MCOUNTERINTEN） 339
16.1.7.9 机器模式事件上溢出标注寄存器（MCOUNTEROF） 339
16.1.8 机器模式 Cache 访问扩展寄存器组 340
16.1.8.1 机器模式 Cache 指令寄存器（MCINS） 340
16.1.8.2 机器模式 Cache 访问索引寄存器（MCINDEX） 340
16.1.8.3 机器模式 Cache 数据寄存器（MCDATA0/1） 341
16.1.9 机器模式处理器型号寄存器组 342
16.1.9.1 机器模式处理器型号寄存器（MCPUID） 342
16.1.9.2 片上总线基地址寄存器（MAPBADDR） 342
16.1.10 多核扩展寄存器组 342
16.1.10.1 Snoop 监听使能寄存器（MSMPR） 342
16.2 附录 C-2 超级用户模式控制寄存器 343
16.2.1 超级用户模式异常配置寄存器组 343
16.2.1.1 超级用户模式处理器状态寄存器（SSTATUS） 343
16.2.1.2 超级用户模式中断使能控制寄存器（SIE） 343
16.2.1.3 超级用户模式向量基址寄存器（STVEC） 344
16.2.1.4 超级用户模式计数器访问授权寄存器（SCOUNTEREN） 344
16.2.2 超级用户模式异常处理寄存器组 344
16.2.2.1 超级用户模式异常临时数据备份寄存器（SSCRATCH） 344
16.2.2.2 超级用户模式异常保留程序计数器寄存器（SEPC） 345
16.2.2.3 超级用户模式异常事件向量寄存器（SCAUSE） 345
16.2.2.4 超级用户模式中断等待状态寄存器（SIP） 345
16.2.3 超级用户模式地址转换寄存器组 345
16.2.3.1 超级用户模式地址转换寄存器（SATP） 345
16.2.4 超级用户模式处理器控制和状态扩展寄存器组 346
16.2.4.1 超级用户模式扩展状态寄存器（SXSTATUS） 346
16.2.4.2 超级用户模式硬件控制寄存器（SHCR） 346
16.2.4.3 超级用户模式事件溢出中断使能寄存器（SCOUNTERINTEN） 346
16.2.4.4 超级用户模式事件上溢出标注寄存器（SCOUNTEROF） 346
16.2.4.5 超级用户模式周期计数器（SCYCLE） 346
16.2.4.6 超级用户模式退休指令计数器（SINSTRET） 347
16.2.4.7 超级用户模式事件计数器（SHPMCOUNTERn） 347
16.2.5 超级用户模式 MMU 扩展寄存器 347
16.2.5.1 超级用户模式 MMU 控制寄存器（SMCIR） 347
16.2.5.2 超级用户模式 MMU 控制寄存器（SMIR） 347
16.2.5.3 超级用户模式 MMU 控制寄存器（SMEH） 347
16.2.5.4 超级用户模式 MMU 控制寄存器（SMEL） 347
16.3 附录 C-3 用户模式控制寄存器 348
16.3.1 用户模式浮点控制寄存器组 348
16.3.1.1 浮点异常累积状态寄存器（FFLAGS） 348
16.3.1.2 浮点动态舍入模式寄存器（FRM） 348
16.3.1.3 浮点控制状态寄存器（FCSR） 348

16.3.2 用户模式计数/计时寄存器组 349
16.3.2.1 用户模式周期计数器（CYCLE） 349
16.3.2.2 用户模式时间计数器（TIME） 349
16.3.2.3 用户模式退休指令计数器（INSTRET） 350
16.3.2.4 用户模式事件计数器（HPMCOUNTERn） 350
16.3.3 用户模式扩展浮点控制寄存器组 350
16.3.3.1 用户模式浮点扩展控制寄存器（FXCR） 350

第一章　概述

本文档是开源版本 C910（openc910）的用户手册。文中出现的“C910”，“C910MP”，“核心”，“Core”等表述，均指代开源版本的 C910。

1.1　简介

C910MP 是基于 RISC-V 指令架构的 64 位高性能多核心处理器，主要面向对性能要求严格的边缘计算领域，如边缘服务器、边缘计算卡、高端机器视觉、高端视频监控、自动驾驶、移动智能终端、5G 基站等。C910MP 采用同构多核架构，支持双核。每个 C910 核心采用自主设计的微体系结构，并重点针对性能进行优化，引入 3 译码 8 执行的超标量架构和多通道的数据预取等高性能技术。此外，C910 核心支持实时检测并关断内部空闲功能模块，降低处理器动态功耗。

1.2　特点

1.2.1　C910MP 处理器体系结构的主要特点

- 同构多核架构，支持双核；
- 支持各个核心独立下电以及 cluster 下电；
- 支持 1 个 AXI4.0 Master 接口，128 比特的总线宽度；
- 两级高缓结构，哈佛结构一级高缓和共享的二级高缓；
- 一级指令/数据缓存分别为 64KB，缓存行 SIZE 为 64B；
- 一级缓存支持 MESI 的一致性协议，二级缓存支持 MOESI 的一致性协议；
- 二级高缓大小 1MB，缓存行 SIZE 为 64B，16 路组相联；
- 支持私有中断控制器 CLINT 和公有中断控制器 PLIC；
- 支持计时器功能；
- 支持自定义且接口兼容 RISC-V 的多核调试框架；

1.2.2　C910 核心的主要特点

- RISC-V 64GC 指令架构；
- 支持小端模式；
- 9~12 级深流水架构；
- 3 译码 8 执行的超标量架构，对软件完全透明；
- 按序取指，乱序发射，乱序完成和按序退休；
- 两级 TLB 内存管理单元，实现虚实地址转换与内存管理；
- 指令高缓和数据高缓大小为 64KB，缓存行为 64B；
- 指令预取功能，硬件自动检测并动态启动；
- 指令高缓路预测的低功耗访问技术；
- 短循环缓存的低功耗执行技术；
- 64Kb 的两级多路并行分支预测器；
- 1024 表项的分支目标缓存器；
- 支持 12 层的硬件返回地址堆栈；
- 256 表项的间接跳转分支预测器；
- 非阻塞发射，投机猜测执行；
- 基于物理寄存器的重命名技术；
- 支持 0 延时 move 指令；
- 双发射、全乱序执行的 load、store 指令；
- 支持读写各 8 路并发的总线访问；
- 支持写合并；
- 支持 8 个通道的数据缓存硬件预取，支持 stride 的预取方式；
- 包含浮点单元，支持半精度、单精度和双精度；

1.3　配置

C910MP 采用下列配置：

- 核心数量：2
- L1 I-Cache：64KB
- L1 D-Cache：64KB

- L2 Cache：1MB
- Master Interface：AXI4
- 外部中断个数：144

1.4　玄铁架构扩展技术

C910 兼容于玄铁 C 系列 1.0 扩展架构，具体包括：

- 运算指令扩展：在整型、浮点、load/store 等方面提高运算能力，是 RISC-V 基础指令集的有力补充
- Cache 操作扩展：方便地进行 Cache 维护操作，提高 Cache 效率
- 内存模型扩展：高效管理地址属性，提高内存访问效率
- 控制寄存器扩展：在标准 RISC-V 的基础上提供更加丰富的功能
- 多核同步指令扩展：提高多核一致性维护的效率
- PLIC 扩展：内嵌 PLIC 中断控制器

1.5　版本说明

C910 兼容 RISC-V 标准，具体版本为：

- *The RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2.*
- *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10.*
- 性能监测单元（PMU）增加了 *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20190125-Public-Review-draft* 中的 mcountinhibit 寄存器。

1.6　命名规则

1.6.1　符号

本文档用到的标准符号和操作符如 图 1.1 所示。

1.6.2　术语

- **逻辑 1** 是指对应于布尔逻辑真的电平值。
- **逻辑 0** 是指对应于布尔逻辑伪的电平值。

符号	功能
+	加
-	减
*	乘
/	除
>	大于
<	小于
=	等于
≥	大于或等于
≤	小于或等于
≠	不等于
&	与
|	或
⊕	异或
NOT	取反
:	连接
⇒	传输
⇔	交换
±	误差
0b0011	二进制数
0x0F	十六进制数
rd	整型目的寄存器
rs1	整型源寄存器1
rs2	整型源寄存器2
rs3	整型源寄存器3
fd	浮点目的寄存器
fs1	浮点源寄存器1
fs2	浮点源寄存器2
fs3	浮点源寄存器3
vd	矢量目的寄存器
vs1	矢量源寄存器1
vs2	矢量源寄存器2
vs3	矢量源寄存器3

图 1.1: 符号列表

- **置位**是指使得某个或某几个位达到逻辑 1 对应的电平值。
- **清除**是指使得某个或某几个位达到逻辑 0 对应的电平值。
- **保留位**是为功能的扩展而预留的，没有特殊说明时其值为 0。
- **信号**是指通过它的状态或状态间的转换来传递信息的电气值。
- **引脚**是表示一种外部电气物理连接，同一个引脚可以连接多个信号。
- **使能**是指使某个离散信号处在有效的状态：
  - 低电平有效信号从高电平切换到低电平；
  - 高电平有效信号从低电平切换到高电平。
- **禁止**是指使某个处在使能状态的信号状态改变：
- 低电平有效信号从低电平切换到高电平；
- 高电平有效信号从高电平切换到低电平。
- **LSB** 代表最低有效位，**MSB** 代表最高有效位。
- **信号**、**位域**、**控制位**的表示都使用一种通用的规则。
- **标识符后来跟着表示范围的数字**，从高位到低位表示一组信号。

  比如“addr[4:0]”就表示一组地址总线，最高位是 addr[4]，最低位是 addr[0]。
- **单个的标识符**就表示单个信号。

  例如“pad_cpu_rst_b”就表示单独的一个信号。

  有时候会在标识符后加上数字表示一定的意义，比如 addr15 就表示一组总线中的第 16 位。

第二章　处理器简介

2.1　结构框图

C910MP 结构框图如 图 2.1 所示。

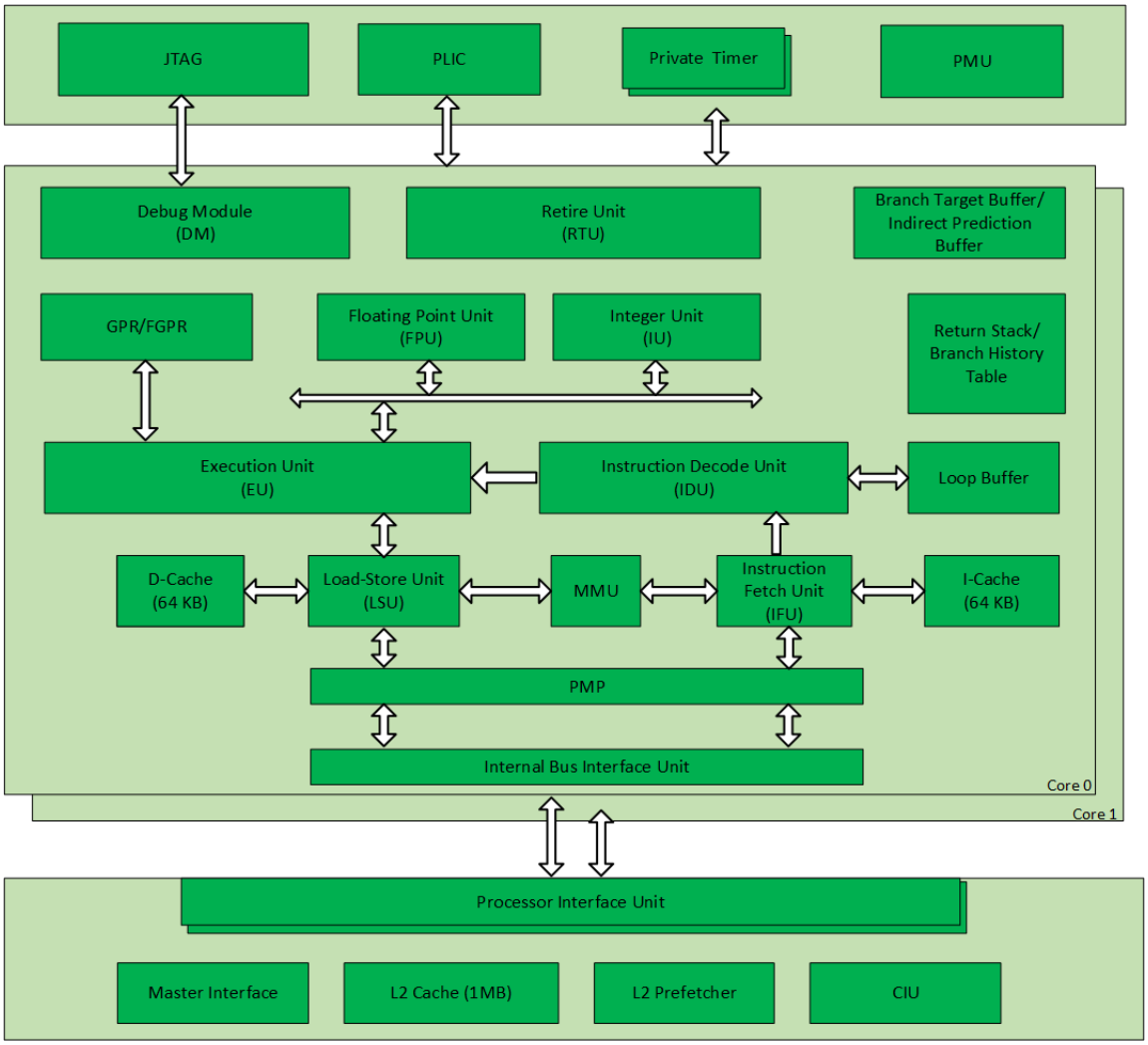


图 2.1: C910MP 微体系结构图

2.2　核内子系统

C910 核内子系统主要包含：指令提取单元（IFU）、指令译码单元（IDU）、整型执行单元（IU）、浮点单元（FPU）、存储载入单元（LSU）、指令退休单元（RTU）、虚拟内存管理单元（MMU）和物理内存保护单元（PMP）。

2.2.1　指令提取单元

指令提取单元（IFU），一次可最多提取八条指令并对其并行处理。实现了多项技术以提高访问效率，比如指令高缓路预测，指令暂存器，循环加速缓存器，直接/间接分支预测等。整个指令提取单元具有低功耗，高分支预测准确率，高指令预取效率的特点。

2.2.2　指令译码单元

指令译码单元（IDU）可以同时对三条指令进行译码并检测数据相关性。利用物理寄存器重命名技术解决指令间的数据相关性，并将指令乱序发送至下级流水线执行。指令译码单元支持指令的乱序调度分发，并通过投机性的发射缓解因数据相关性造成的性能损失。

2.2.3　执行单元

执行单元包含整型单元（IU）和浮点单元（FPU）。

整型单元包含算术逻辑单元（ALU）、乘法单元（MULT）、除法单元（DIV）和跳转单元（BJU）。ALU 执行 64 位整数操作。MULT 支持 16*16、32*32、64*64 整数乘法。除法器的设计采用了基 16 的 SRT 算法，执行周期视操作数而变化。BJU 可以在单周期内完成分支预测错误处理。

浮点单元包含浮点算术逻辑单元（FALU）、浮点融合乘累加单元（FMAU）和浮点除法开方单元（FDSU），支持半精度、单精度和双精度运算。浮点算术逻辑单元（FALU）负责加减、比较、转换、寄存器传输、符号注入、分类等操作。浮点融合乘累加单元 (FMAU) 负责普通乘法、融合乘累加等操作。浮点除法开方单元（FDSU）负责浮点除法、浮点开方等操作。

2.2.4　存储载入单元

存储载入单元（LSU）支持标量存储/加载指令的双发射、矢量存储/加载指令的单发射以及所有存储/加载指令的全乱序执行，支持高速缓存的非阻塞访问。支持字节、半字、字、双字和四字的存储/载入指令，并支持字节和半字的载入指令的符号位和零扩展。存储/加载指令可以流水执行，使得数据吞吐量达到一个周期存取一个数据。支持 8 路数据流硬件预取技术，将数据提前放入 L1 数据高缓中。当数据高缓缺失后，支持总线的并行访问。

2.2.5　指令退休单元

指令退休单元（RTU）包括一个重排序缓冲器与一个物理寄存器堆。其中，重排序缓冲器负责指令的乱序回收与按序退休，物理寄存器堆负责结果的乱序回收和传递。通过支持指令并行回收与快速退休提高指令退休效率。指令退休单元每个时钟周期并行退休三条指令，支持精确异常。

2.2.6　虚拟内存管理单元

虚拟内存管理单元（MMU）遵从 RISC-V SV39 标准，将 39 位虚拟地址转换为 40 位物理地址。C910 MMU 在 SV39 定义的硬件回填标准基础上，扩展了软件回填方式和地址属性。

具体信息参考内存模型。

2.2.7　物理内存保护单元

C910 物理内存保护单元（PMP）遵从 RISC-V 标准，支持 8 个表项，最小粒度为 4KB，不支持 NA4 模式。

具体信息参考内存模型。

2.3　多核子系统

C910 多核子系统包含：数据一致性接口单元（CIU）、二级高速缓存、主设备接口单元、平台级中断控制器（PLIC）、计时器和自定义多核单端口调试框架。

2.3.1　数据一致性接口单元

数据一致性接口单元（CIU），采用 MESI 协议维护各个 L1 数据高缓的一致性。设置两路监听缓冲器，可并行处理多个监听请求，最大化利用监听带宽。采用高效的数据旁路机制，当监听请求命中被监听的 L1 数据高缓时，直接将数据旁路给请求发起核心；另外，CIU 单元还支持 TLB 和 ICACHE 无效操作请求的广播，简化了 TLB/ICache 与 DCache 数据一致性的软件维护成本。

2.3.2　二级高速缓存

二级高速缓存，紧耦合于 CIU 单元，实现和 L1 数据高缓的同步访问；采用分块的流水线架构，单周期可并行处理两个访问请求，最大访问带宽可达到 1024 比特。二级高速缓存采用和 C910 相同的工作频率，TAG RAM 和 DATA RAM 的访问延时可以由软件配置。

2.3.3　主设备接口

主设备接口单元支持 AXI4.0 协议，支持关键字优先的地址访问，可以在不同的系统时钟与 CPU 时钟比例（1:1，1:2，1:3，1:4，1:5，1:6，1:7，1:8）下工作。

2.3.4　平台级中断控制器

平台级中断控制器（PLIC）支持 144 个外部中断源的采样和分发，支持电平和脉冲中断，可以设置 32 个级别的中断优先级。

具体信息参考中断控制器

2.3.5　计时器

多核系统中共用一个 64 位系统计时器，各个核心拥有私有的计时器比较值寄存器，通过采集系统计时器的数值与软件设置的私有计时器比较值寄存器进行比较，产生计时器信号。

具体信息参考中断控制器

2.4　接口概览

C910 的接口按照功能主要分为：时钟复位信号、总线系统、中断系统、调试系统、低功耗系统、DFT 系统和 CPU 运行观测信号。C910 的主要接口如 图 2.2 所示。

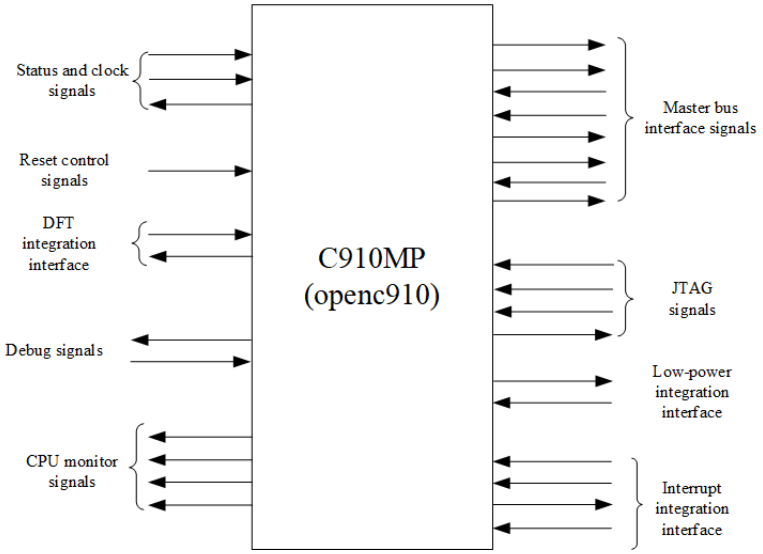


图 2.2: C910MP 接口概览

第三章　指令集

本章主要介绍 C910 中实现的指令集，分为两大部分：RV 基础指令集和玄铁扩展指令集。

3.1　RV 基础指令集

3.1.1　整型指令集（RV64I）

整型指令集按功能可以分为以下类型:

- 加减法指令
- 逻辑操作指令
- 移位指令
- 比较指令
- 数据传输指令
- 分支跳转指令
- 内存存取指令
- 控制寄存器操作指令:
- 低功耗指令
- 异常返回指令
- 特殊功能指令

表 3.1: 整型指令（RV64I）指令列表

指令名称	**指令描述**	**执行延时**
加减法指令		
ADD	有符号加法指令	1
ADDW	低 32 位有符号加法指令	1
ADDI	有符号立即数加法指令	1

下页继续

表 3.1 – 续上页

ADDIW	低 32 位有符号立即数加法指令	1
SUB	有符号减法指令	1
SUBW	低 32 位有符号减法指令	1
逻辑操作指令		
AND	按位与指令	1
ANDI	立即数按位与指令	1
OR	按位或指令	1
ORI	立即数按位或指令	1
XOR	按位异或指令	1
XORI	立即数按位异或指令	1
移位指令		
SLL	逻辑左移指令	1
SLLW	低 32 位逻辑字左移指令	1
SLLI	立即数逻辑左移指令	1
SLLIW	低 32 位立即数逻辑左移指令	1
SRL	逻辑右移指令	1
SRLW	低 32 位逻辑右移指令	1
SRLI	立即数逻辑右移指令	1
SRLIW	低 32 位立即数逻辑右移指令	1
SRA	算术右移指令	1
SRAW	低 32 位算数右移指令	1
SRAI	立即数算术右移指令	1
SRAIW	低 32 位立即数算数右移指令	1
比较指令		
SLT	有符号比较小于置位指令	1
SLTU	无符号比较小于置位指令	1
SLTI	有符号立即数比较小于置位指令	1
SLTIU	无符号立即数比较小于置位指令	1
数据传输指令		
LUI	高位立即数装载指令	1
AUIPC	PC 高位立即数加法指令	1
分支跳转指令		
BEQ	相等分支指令	1
BNE	不等分支指令	1
BLT	有符号小于分支指令	1
BGE	有符号大于等于分支指令	1
BLTU	无符号小于分支指令	1
BGEU	无符号大于等于分支指令	1

下页继续

表 3.1 – 续上页

JAL	直接跳转子程序指令	1
JALR	寄存器跳转子程序指令	1
内存存取指令		
LB	有符号扩展字节加载指令	WEAK ORDER LOAD: >=3 STORE: 1 STRONG ORDER 不定周期
LBU	无符号扩展字节加载指令	同上
LH	有符号扩展半字加载指令	同上
LHU	无符号扩展半字加载指令	同上
LW	有符号扩展字加载指令	同上
LWU	无符号扩展字加载指令	同上
LD	双字加载指令	同上
SB	字节存储指令	同上
SH	半字存储指令	同上
SW	字存储指令	同上
SD	双字存储指令	同上
控制寄存器操作指令		
CSRRW	控制寄存器读写传送指令	阻塞执行 不定周期
CSRRS	控制寄存器置位传送指令	同上
CSRRC	控制寄存器清零传送指令	同上
CSRRWI	控制寄存器立即数读写传送指令	同上
CSRRSI	控制寄存器立即数置位传送指令	同上
CSRRCI	控制寄存器立即数清零传送指令	同上
低功耗指令		
WFI	进入低功耗等待模式指令	不定周期
异常返回指令		
MRET	机器模式异常返回指令	阻塞执行 不定周期
SRET	超级用户模式异常返回指令	同上
特殊功能指令		
FENCE	存储同步指令	不定周期
FENCE.I	指令流同步指令	阻塞执行 不定周期
SFENCE.VMA	虚拟内存同步指令	同上
ECALL	环境异常指令	1
EBREAK	断点指令	1

具体指令说明和定义，请参考附录 *A-1 I* 指令术语

3.1.2　乘除法指令集（RV64M）

表 3.2: 整型乘除法 (RV64M) 指令列表

指令名称	指令描述	执行延时
MUL	有符号乘法指令	4
MULW	低 32 位有符号乘法指令	4
MULH	有符号乘法取高位指令	4
MULHS	有符号无符号乘法取高位指令	4
MULHU	无符号乘法取高位指令	4
DIV	有符号除法指令	3-20
DIVW	低 32 位有符号除法指令	3-12
DIVU	无符号除法指令	3-20
DIVUW	低 32 位无符号除法指令	3-12
REM	有符号取余指令	3-20
REMW	低 32 位有符号取余指令	3-12
REMU	无符号取余指令	3-20
REMUW	低 32 位无符号取余指令	3-12

具体指令说明和定义，请参考附录 *A-2 M* 指令术语

3.1.3　原子指令集（RV64A）

表 3.3: 原子指令 (RV64A) 指令列表

指令名称	指令描述	执行延时
LR.W	字加载保留指令	拆分为多条原子指令执行。 可能拆分出阻塞执行，指令延时不可预期。
LR.D	双字加载保留指令	
SC.W	字条件存储指令	
SC.D	双字条件存储指令	
AMOSWAP.W	低 32 位原子交换指令	
AMOSWAP.D	原子交换指令	
AMOADD.W	低 32 位原子加法指令	
AMOADD.D	原子加法指令	
AMOXOR.W	低 32 位原子按位异或指令	
AMOXOR.D	原子按位异或指令	
AMOAND.W	低 32 位原子按位与指令	
AMOAND.D	原子按位与指令	
AMOOR.W	低 32 位原子按位或指令	
AMOOR.D	原子按位或指令	
AMOMIN.W	低 32 位原子有符号取最小值指令	
AMOMIN.D	原子有符号取最小值指令	
AMOMAX.W	低 32 位原子有符号取最大值指令	
AMOMAX.D	原子有符号取最大值指令	
AMOMINU.W	低 32 位原子无符号取最小值指令	
AMOMINU.D	原子无符号取最小值指令	
AMOMAXU.W	低 32 位原子无符号取最大值指令	
AMOMAXU.D	原子无符号取最大值指令	

具体指令说明和定义，请参考*附录 A-3 A 指令术语* 。

3.1.4　单精度浮点指令集（RV64F）

单精度浮点指令集按功能可以分为以下类型：

- 运算指令
- 符号注入指令
- 数据传输指令
- 比较指令
- 数据类型转换指令
- 内存存储指令

- 浮点数分类指令

表 3.4: 单精度浮点（RV64F）指令列表

指令名称	指令描述	执行延时
运算指令		
FADD.S	单精度浮点加法指令	3
FSUB.S	单精度浮点减法指令	3
FMUL.S	单精度浮点乘法指令	4
FMADD.S	单精度浮点乘累加指令	5
FMSUB.S	单精度浮点乘累减指令	5
FNMADD.S	单精度浮点乘累加取负指令	5
FNMSUB.S	单精度浮点乘累减取负指令	5
FDIV.S	单精度浮点除法指令	4-10
FSQRT.S	单精度浮点开方指令	4-10
符号注入指令		
FSGNJ.S	单精度浮点符号注入指令	3
FSGNJN.S	单精度浮点符号取反注入指令	3
FSGNJX.S	单精度浮点符号异或注入指令	3
数据传输指令		
FMV.X.W	单精度浮点读传送指令	拆分执行 1+1
FMV.W.X	单精度浮点写传送指令	拆分执行 1+1
比较指令		
FMIN.S	单精度浮点取最小值指令	3
FMAX.S	单精度浮点取最大值指令	3
FEQ.S	单精度浮点比较相等指令	拆分执行 3+1
FLT.S	单精度浮点比较小于指令	拆分执行 3+1
FLE.S	单精度浮点比较小于等于指令	拆分执行 3+1
数据类型转换指令		
FCVT.W.S	单精度浮点转换成有符号整型指令	拆分执行 3+1
FCVT.WU.S	单精度浮点转换成无符号整型指令	拆分执行 3+1
FCVT.S.W	有符号整型转换成单精度浮点指令	拆分执行 3+1
FCVT.S.WU	无符号整型转换成单精度浮点指令	拆分执行 3+1
FCVT.L.S	单精度浮点转换成有符号长整型指令	拆分执行 3+1

下页继续

表 3.4 – 续上页

FCVT.LU.S	单精度浮点转换成无符号长整型指令	拆分执行 3+1
FCVT.S.L	有符号长整型转换成单精度浮点指令	拆分执行 1+3
FCVT.S.LU	无符号长整型转换成单精度浮点指令	拆分执行 1+3
内存存储指令		
FLW	单精度浮点加载指令	WEAK ORDER LOAD: >=3 STORE: 1 STRONG ORDER 不定周期
FSW	单精度浮点存储指令	同上
浮点数分类指令		
FCLASS.S	单精度浮点分类指令	1+1

具体指令说明和定义，请参考附录 *A-4 F 指令术语* 。

3.1.5　双精度浮点指令集（RV64D）

双精度浮点指令集按功能可以分为以下类型：

- 运算指令
- 符号注入指令
- 数据传输指令
- 比较指令
- 数据类型转换指令
- 内存存储指令

表 3.5: 双精度浮点（RV64D）指令列表

指令名称	**指令描述**	**执行延时**
运算指令		
FADD.D	双精度浮点加法指令	3
FSUB.D	双精度浮点减法指令	3
FMUL.D	双精度浮点乘法指令	4
FMADD.D	双精度浮点乘累加指令	5
FMSUB.D	双精度浮点乘累减指令	5

下页继续

表 3.5 – 续上页

FNMSUB.D	双精度浮点乘累加取反指令	5
FNMADD.D	双精度浮点乘累减取反指令	5
FDIV.D	双精度浮点除法指令	4-17
FSQRT.D	双精度浮点开方指令	4-17
符号注入指令		
FSGNJ.D	双精度浮点符号注入指令	3
FSGNJN.D	双精度浮点符号取反注入指令	3
FSGNJX.D	双精度浮点符号异或注入指令	3
数据传输指令		
FMV.X.D	双精度浮点读传送指令	1+1
FMV.D.X	双精度浮点写传送指令	1+1
比较指令		
FMIN.D	双精度浮点取最小值指令	3
FMAX.D	双精度浮点取最大值指令	3
FEQ.D	双精度浮点比较相等指令	拆分执行 3+1
FLT.D	双精度浮点比较小于指令	拆分执行 3+1
FLE.D	双精度浮点比较小于等于指令	拆分执行 3+1
数据类型转换指令		
FCVT.S.D	双精度浮点转换成单精度浮点指令	3
FCVT.D.S	单精度浮点转换成双精度浮点指令	3
FCVT.W.D	双精度浮点转换成有符号整型指令	拆分执行 3+1
FCVT.WU.D	双精度浮点转换成无符号整型指令	拆分执行 3+1
FCVT.D.W	有符号整型转换成双精度浮点指令	拆分执行 3+1
FCVT.D.WU	无符号整型转换成双精度浮点指令	拆分执行 3+1
FCVT.L.D	双精度浮点转换成有符号长整型指令	拆分执行 3+1
FCVT.LU.D	双精度浮点转换成无符号长整型指令	拆分执行 3+1
FCVT.D.L	有符号长整型转换成双精度浮点指令	拆分执行 3+1
FCVT.D.LU	无符号长整型转换成双精度浮点指令	拆分执行 3+1
内存存储指令		

下页继续

表 3.5 – 续上页

FLD	双精度浮点加载指令	WEAK ORDER LOAD: >=3 STORE: 1 STRONG ORDER 不定周期
FSD	双精度浮点存储指令	同上
浮点数分类指令		
FCLASS.D	双精度浮点分类指令	1+1

具体指令说明和定义，请参考附录 *A-5 D 指令术语*

3.1.6　压缩指令集（RV64C）

压缩指令集按功能可以分为以下类型：

- 加减法指令
- 逻辑操作指令
- 移位指令
- 数据传输指令
- 分支跳转指令
- 立即数偏移存取指令

表 3.6: 压缩指令（RV64C）指令列表

指令名称	**指令描述**	**执行延时**
加减法指令		
C.ADD	有符号加法指令	1
C.ADDW	低 32 位有符号加法指令	1
C.ADDI	有符号立即数加法指令	1
C.ADDIW	低 32 位有符号立即数加法指令	1
C.SUB	有符号减法压缩指令	1
C.SUBW	低 32 位有符号减法指令	1
C.ADDI16SP	加 16 倍立即数到堆栈指针	1
C.ADDI4SPN	4 倍立即数和堆栈指针相加	1
逻辑操作指令		
C.AND	按位与指令	1
C.ANDI	立即数按位与指令	1
C.OR	按位或指令	1

下页继续

表 3.6 – 续上页

C.XOR	按位异或指令	1
移位指令		
C.SLLI	立即数逻辑左移指令	1
C.SRLI	立即数逻辑右移指令	1
C.SRAI	立即数算术右移指令	1
数据传输指令		
C.MV	数据传送指令	1
C.LI	低位立即数传送指令	1
C.LUI	高位立即数传送指令	1
分支跳转指令		
C.BEQZ	等于零分支指令	1
C.BNEZ	不等于零分支指令	1
C.J	无条件跳转指令	1
C.JR	寄存器跳转指令	1
C.JALR	寄存器跳转子程序指令	1
立即数偏移存取指令		
C.LW	字加载指令	WEAK ORDER LOAD: >=3 STORE: 1 STRONG ORDER 不定周期
C.SW	字存储指令	同上
C.LWSP	字堆栈加载指令	同上
C.SWSP	字堆栈存储指令	同上
C.LD	双字加载指令	同上
C.SD	双字存储指令	同上
C.LDSP	双字堆栈加载指令	同上
C.SDSP	双字堆栈存储指令	同上
C.FLD	双精度加载指令	同上
C.FSD	双精度存储指令	同上
C.FLDSP	双精度堆栈存储指令	同上
C.FSDSP	双精度堆栈加载指令	同上
特殊指令		
C.NOP	空指令	1
C.EBREAK	断点指令	1

具体指令说明和定义，请参考*附录 A-6 C 指令术语*

3.2　玄铁扩展指令集

C910 除了支持 RV64GC 指令集之外，还在此基础上拓展了部分自定义指令。C910 的拓展指令集中，半精度浮点指令集可以直接使用，除此之外的所有 C910 拓展指令集需要在机器模式扩展状态寄存器（MXSTATUS）中打开扩展指令集使能位（THEADISAEE）才能正常使用，否则将产生非法指令异常。

3.2.1　算术运算类指令

表 3.7: 算术运算指令集

指令名称	指令描述	执行延时
加减法指令		
ADDSL	寄存器移位相加指令	1
MULA	乘累加指令	非累加数相关性:4
MULS	乘累减指令	非累加数相关性:4
MULAW	低 32 位乘累加指令	累加数相关性:1
MULSW	低 32 位乘累减指令	累加数相关性:1
MULAH	低 16 位乘累加指令	累加数相关性:1
MULSH	低 16 位成累减指令	累加数相关性:1
位移指令		
SRRI	循环右移指令	1
SRRIW	低 32 位循环右移指令	1
传送指令		
MVEQZ	寄存器为 0 传送指令	1
MVNEZ	寄存器非 0 传送指令	1

具体指令说明和定义，请参考*附录 B-3 算术运算指令术语* 。

3.2.2　位操作类指令

表 3.8: 位操指令集

指令名称	指令描述	执行延时
位操作指令		
TST	比特为 0 测试指令	1
TSTNBZ	字节为 0 测试指令	1
REV	字节倒序指令	1
REVW	低 32 位字节倒序指令	1
FF0	快速找 0 指令	1
FF1	快速找 1 指令	1
EXT	寄存器连续位提取符号位扩展指令	1
EXTU	寄存器连续位提取零扩展指令	1

具体指令说明和定义，请参考*附录 B-4 位操作指令术语* 。

3.2.3　内存访问类指令

表 3.9: 内存访问指令集

存储指令	执行延时	
FLRD	浮点寄存器移位双字加载指令	WEAK ORDER >=3 STRONG ORDER 不定周期
FLRW	浮点寄存器移位字加载指令	
FLURD	浮点寄存器低 32 位移位双字加载指令	
FLURW	浮点寄存器低 32 位移位字加载指令	
LRB	寄存器移位符号位扩展字节加载指令	
LRH	寄存器移位符号位扩展半字加载指令	
LRW	寄存器移位符号位扩展字加载指令	
LRD	寄存器移位双字加载指令	
LRBU	寄存器移位零扩展字节加载指令	

下页继续

表 3.9 – 续上页

LRHU	寄存器移位零扩展半字加载指令	
LRWU	寄存器移位零扩展字加载指令	
LURB	寄存器低 32 位移位符号位扩展字节加载指令	
LURH	寄存器低 32 位移位符号位扩展半字加载指令	
LURW	寄存器低 32 位移位符号位扩展字加载指令	
LURD	寄存器低 32 位移位双字加载指令	
LURBU	寄存器低 32 位移位零扩展字节加载指令	
LURHU	寄存器低 32 位移位零扩展半字加载指令	
LURWU	寄存器低 32 位移位零扩展字加载指令	
LBIA	符号位扩展字节加载基地址自增指令	拆分为 load 指令和 alu 指令执行
LBIB	基地址自增符号位扩展字节加载指令	
LHIA	符号位扩展半字加载基地址自增指令	
LHIB	基地址自增符号位扩展半字加载指令	
LWIA	符号位扩展字加载基地址自增指令	
LWIB	基地址自增符号位扩展字加载指令	
LDIA	符号位扩展双字加载基地址自增指令	
LDIB	基地址自增符号位扩展双字加载指令	
LBUIA	零扩展字节加载基地址自增指令	
LBUIB	基地址自增零扩展字节加载指令	
LHUIA	零扩展半字加载地址自增指令	
LHUIB	基地址自增零扩展半字加载指令	
LWUIA	零拓展字加载地址自增指令	
LWUIB	基地址自增零扩展字加载指令	
LDD	双寄存器加载指令	拆分为两条 load 执行

下页继续

表 3.9 – 续上页

LWD	符号位扩展双寄存器字加载指令	
LWUD	零扩展双寄存器字加载指令	
FSRD	浮点寄存器移位双字存储指令	WEAK ORDER 1 STRONG ORDER 不定周期
FSRW	浮点寄存器移位字存储指令	
FSURD	浮点寄存器低 32 位移位双字存储指令	
FSURW	浮点寄存器低 32 位移位字存储指令	
SRB	寄存器移位字节存储指令	
SRW	寄存器移位字存储指令	
SRD	寄存器移位双字存储指令	
SURB	寄存器低 32 位移位字节存储指令	
SURH	寄存器低 32 位移位半字存储指令	
SURW	寄存器低 32 位移位字存储指令	
SURD	寄存器低 32 位移位双字存储指令	
SBIA	字节存储基地址自增指令	拆分为 store 指令和 alu 指令执行
SBIB	基地址自增字节存储指令	
SHIA	半字存储基地址自增指令	
SHIB	基地址自增半字存储指令	
SWIA	字存储基地址自增指令	
SWIB	基地址自增字存储指令	
SDIA	双字存储基地址自增指令	
SDIB	基地址自增双字存储指令	
SDD	双寄存器加存储指令	拆分为两条 store 执行
SWD	双寄存器低 32 位存储指令	

具体指令说明和定义，请参考附录 *B-5 存储指令术语* 。

3.2.4 Cache 指令

表 3.10: Cache 指令列表

指令名称	指令描述	执行延时 (LMUL=1)
DCACHE.CALL	DCACHE 清全部脏表项指令	阻塞执行 不定周期
DCACHE.CIALL	DCACHE 清全部脏表项后无效指令	
DCACHE.CIPA	DCACHE 按物理地址清脏表项并无效指令（作用域包含 L2CACHE）	
DCACHE.CISW	DCACHE 按 set/way 清脏表项并无效指令	
DCACHE.CIVA	DCACHE 按虚拟地址清脏表项并无效指令（作用域包含 L2CACHE）	
DCACHE.CPA	DCACHE 按物理地址清脏表项指令（作用域包含 L2CACHE）	
DCACHE.CPAL1	L1DCACHE 按物理地址清脏表项指令	
DCACHE.CSW	DCACHE 按 set/way 清脏表项指令	
DCACHE.CVA	DCACHE 按虚拟地址清脏表项指令（作用域包含 L2CACHE）	
DCACHE.CVAL1	L1DCACHE 按虚拟地址清脏表项指令	
DCACHE.IPA	DCACHE 按物理地址无效指令（作用域包含 L2CACHE）	
DCACHE.ISW	DCACHE 按 set/way 无效指令	
DCACHE.IVA	DCACHE 按虚拟地址无效指令（作用域包含 L2CACHE）	
DCACHE.IALL	DCACHE 无效所有表项指令	
ICACHE.IALL	ICACHE 无效所有表项指令	不定周期
ICACHE.IALLS	ICACHE 广播无效所有表项指令	
ICACHE.IPA	ICACHE 按物理地址无效表项指令	
ICACHE.IVA	ICACHE 按虚拟地址无效表项指令	
L2CACHE.CALL	L2CACHE 清所有脏表项指令	
L2CACHE.CIALL	L2CACHE 清所有脏表项并无效指令	
L2CACHE.IALL	L2CACHE 无效指令	

具体指令说明和定义，请参考*附录 B-1 Cache 指令术语* 。

3.2.5　多核同步指令

表 3.11: 多核同步指令集

多核同步指令	描述
SFENCE.VMAS	虚拟内存同步广播指令
SYNC	同步指令
SYNC.S	同步广播指令
SYNC.I	同步清空指令
SYNC.IS	同步清空广播指令

具体指令说明和定义，请参考附录 *B-2* 多核同步指令术语 。

3.2.6　半精度浮点类指令

表 3.12: 半精度浮点指令集

指令名称	指令描述	执行延时
运算指令		
FADD.H	半精度浮点加法指令	3
FSUB.H	半精度浮点减法指令	3
FMUL.H	半精度浮点乘法指令	3
FMADD.H	半精度浮点乘累加指令	4
FMSUB.H	半精度浮点乘累减指令	4
FNMADD.H	半精度浮点乘累加取负指令	4
FNMSUB.H	半精度浮点乘累减取负指令	4
FDIV.H	半精度浮点除法指令	4-7
FSQRT.H	半精度浮点开方指令	4-7
符号注入指令		
FSGNJ.H	半精度浮点符号注入指令	3
FSGNJN.H	半精度浮点符号取反注入指令	3
FSGNJX.H	半精度浮点符号异或注入指令	3
数据传输指令		
FMV.X.H	半精度浮点读传送指令	1+1
FMV.H.X	半精度浮点写传送指令	1+1
比较指令		
FMIN.H	半精度浮点取最小值指令	3
FMAX.H	半精度浮点取最大值指令	3
FEQ.H	半精度浮点比较相等指令	拆分执行 3+1
FLT.H	半精度浮点比较小于指令	拆分执行 3+1

下页继续

表 3.12 – 续上页

FLE.H	半精度浮点比较小于等于指令	拆分执行 3+1
数据类型转换指令		
FCVT.S.H	半精度浮点转换成单精度浮点指令	3
FCVT.H.S	单精度浮点转换成半精度浮点指令	3
FCVT.D.H	半精度浮点转换成双精度浮点指令	3
FCVT.H.D	双精度浮点转换成半精度浮点指令	3
FCVT.W.H	半精度浮点转换成有符号整型指令	拆分执行 3+1
FCVT.WU.H	半精度浮点转换成无符号整型指令	拆分执行 3+1
FCVT.H.W	有符号整型转换成半精度浮点指令	拆分执行 3+1
FCVT.H.WU	无符号整型转换成半精度浮点指令	拆分执行 3+1
FCVT.L.H	半精度浮点转换成有符号长整型指令	拆分执行 3+1
FCVT.LU.H	半精度浮点转换成无符号长整型指令	拆分执行 3+1
FCVT.H.L	有符号长整型转换成半精度浮点指令	拆分执 3+1
FCVT.H.LU	无符号长整型转换成半精度浮点指令	
内存存储指令		
FLH	半精度浮点加载指令	WEAK ORDER LOAD: >=3 STORE: 1 STRONG ORDER 不定周期
FSH	半精度浮点存储指令	同上
浮点数分类指令		
FCLASS.H	单精度浮点分类指令	1+1

具体指令说明和定义，请参考*附录 B-6 浮点半精度指令术语* 。

第四章　处理器模式与寄存器

4.1　处理器模式

C910 支持 RISC-V 三种**特权模式**：机器模式、超级用户模式和用户模式。处理器复位后在机器模式下执行程序。三种运行模式对应不同的操作权限，区别主要体现在以下几个方面：

1. 对寄存器的访问；
2. 特权指令的使用；
3. 对内存空间的访问。

- **用户模式权限最低**。

  普通用户程序只允许访问指定给普通用户模式的寄存器。避免了普通用户程序接触特权信息，而操作系统通过协调普通用户程序的行为来为普通用户程序提供管理和服务。

- **超级用户模式权限比用户模式高，但比机器模式低**。

  超级用户模式下运行的程序不可以使用机器模式的控制寄存器，并且受到 PMP 的限制。使用基于页面的虚拟内存，这个功能构成了超级用户模式的核心。

- **机器模式拥有最高的权限**。

  在机器模式下运行的程序对内存、I/O 和一些对于启动和配置系统来说必要的底层功能有着完全的使用权。默认情况下（异常中断没有被降级处理），任何模式下发生的异常和中断都会切换到机器模式进行响应。

大多数指令在三种模式下都能执行，但是一些对系统产生重大影响的特权指令只能在超级用户模式或机器模式下执行，具体信息可参考附录 *A* 标准指令术语 和附录 *B* 平头哥扩展指令术语 ，查看指令的执行权限。

处理器的工作模式在异常响应时发生变化（响应异常的特权模式不同于异常发生时所处的特权模式），进入更高的特权模式响应异常，异常响应完之后再回到低特权模式。

4.2　寄存器视图

C910 的寄存器视图如 图 4.1 所示：

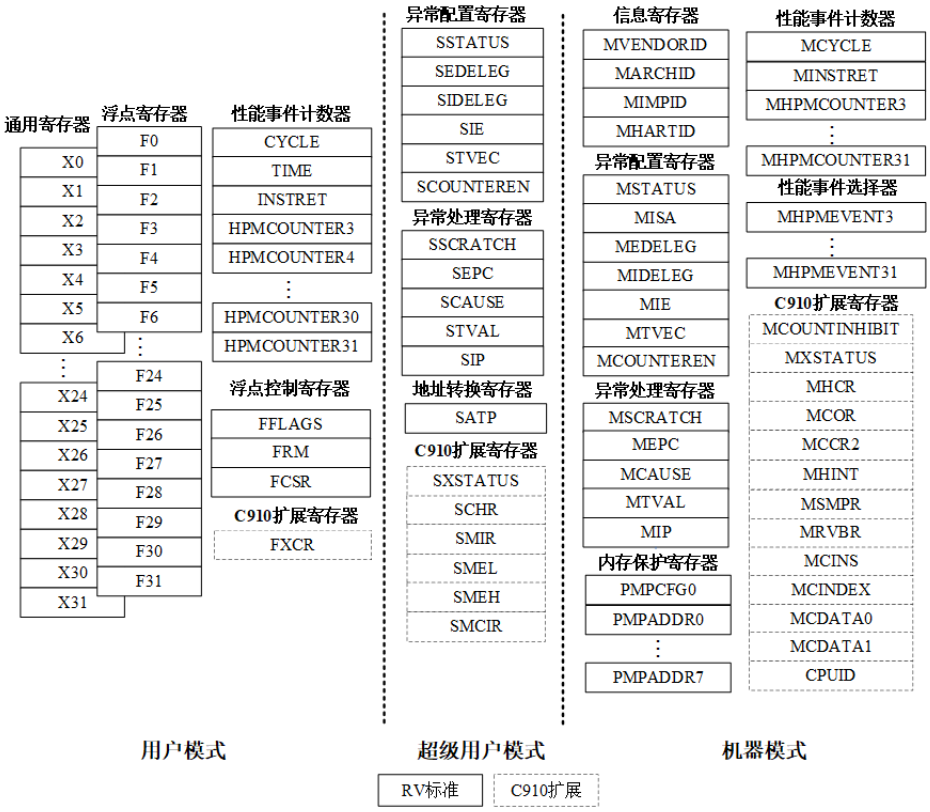

通用寄存器
X0
X1
X2
X3
X4
X5
X6
X24
X25
X26
X27
X28
X29
X30
X31
浮点寄存器
F0
F1
F2
F3
F4
F5
F6
F24
F25
F26
F27
F28
F29
F30
F31
性能事件计数器
CYCLE
TIME
INSTRET
HPMCOUNTER3
HPMCOUNTER4
HPMCOUNTER30
HPMCOUNTER31
浮点控制寄存器
FFLAGS
FRM
FCSR
C910扩展寄存器
FXCR
异常配置寄存器
SSTATUS
SEDELEG
SIDELEG
SIE
STVEC
SCOUNTEREN
异常处理寄存器
SSCRATCH
SEPC
SCAUSE
STVAL
SIP
地址转换寄存器
SATP
C910扩展寄存器
SXSTATUS
SCHR
SMIR
SMEL
SMEH
SMCIR
信息寄存器
MVENDORID
MARCHID
MIMPID
MHARTID
异常配置寄存器
MSTATUS
MISA
MEDELEG
MIDELEG
MIE
MTVEC
MCOUNTEREN
异常处理寄存器
MSCRATCH
MEPC
MCAUSE
MTVAL
MIP
内存保护寄存器
PMPCFG0
PMPADDR0
PMPADDR7
性能事件计数器
MCYCLE
MINSTRET
MHPMCOUNTER3
MHPMCOUNTER31
性能事件选择器
MHPMEVENT3
MHPMEVENT31
C910扩展寄存器
MCOUNTINHIBIT
MXSTATUS
MHCR
MCOR
MCCR2
MHINT
MSMPR
MRVBR
MCINS
MCINDEX
MCDATA0
MCDATA1
CPUID
用户模式
超级用户模式
机器模式
RV标准
C910扩展

图 4.1: 寄存器视图

4.3　通用寄存器

C910 拥有 32 个 64 位的通用寄存器，功能定义与 RISC-V 一致，如 表 4.1 所示。

表 4.1: 通用寄存器

寄存器	ABI 名称	描述
x0	zero	硬件绑 0
x1	ra	返回地址
x2	sp	堆栈指针
x3	gp	全局指针
x4	tp	线程指针
x5	t0	临时/备用链接寄存器
x6-7	t1-2	临时寄存器
x8	s0/fp	保留寄存器/帧指针
x9	s1	保留寄存器
x10-11	a0-1	函数参数/返回值
x12-17	a2-7	函数参数
x18-27	s2-11	保留寄存器
x28-31	t3-6	临时寄存器

通用寄存器用于保存指令操作数、指令执行结果以及地址信息。

4.4　浮点寄存器

C910 的浮点单元除了支持标准 RV64FD 指令集以外，还扩展支持了浮点半精度计算，拥有 32 个独立的 64 位浮点寄存器，可以在普通用户模式、超级用户模式和机器模式下被访问。

表 4.2: 浮点寄存器

寄存器	ABI 名称	描述
f0-7	ft0-7	浮点临时寄存器
f8-9	fs0-1	浮点保留寄存器
f10-11	fa0-1	浮点参数/返回值
f12-17	fa2-7	浮点参数
f18-27	fs2-11	浮点保留寄存器
f28-31	ft8-11	浮点临时寄存器

浮点寄存器 f0 和通用寄存器 x0 不同，并不是硬件绑死 0，而是和其他浮点寄存器一样，是可变的。单精度浮点数仅使用 64 位浮点寄存器的低 32 位，高 32 位必须全为 1，否则会被当做非数处理；半精度浮点数仅使用 64 位浮点寄存器的低 16 位，高 48 位必须全为 1，否则会被当作非数处理。

增加单独的浮点寄存器可以增大寄存器容量和带宽，进而提高处理器的性能。同时必须增加浮点加载和存储指令，还需要增加浮点和通用寄存器之间数据传递指令。

4.4.1　浮点寄存器与通用寄存器传输数据

通用寄存器与浮点寄存器之间的数据传输可以通过浮点寄存器传送指令实现。浮点寄存器传送指令包括：

- FMV.X.H/FMV.H.X 浮点寄存器半精度传送指令。
- FMV.X.W/FMV.W.X 浮点寄存器单精度传送指令。
- FMV.X.D/FMV.D.X 浮点寄存器双精度传送指令。

从通用寄存器传送一个半/单/双精度浮点数据到浮点寄存器中，数据格式不会因为传输而改变，所以程序可以直接使用这些寄存器而不必经过类型转换。

具体指令说明和定义可以参考*附录 A-4 F 指令术语*

4.4.2　维护寄存器精度的一致

浮点寄存器可以存储半精度浮点数、单精度浮点数、双精度浮点数和整形数据。举例说明，在浮点寄存器 f1 中所存的数据类型，取决于上一次的写操作，可能是四种数据类型中的任何一种。

浮点单元在硬件上不对数据类型做任何数据格式上的检测，硬件对浮点寄存器中数据格式的解析只取决于执行的浮点指令本身，而不关心这个寄存器上次的写操作用的数据格式。这完全是靠编译器或者程序本身来保证寄存器中的数据精度的一致性。

4.5　系统控制寄存器

4.5.1　标准控制寄存器

本章节描述 C910 实现的 RISC-V 标准控制寄存器，按照机器模式、超级用户模式、用户模式分别描述。

C910 中实现的 RISC-V 标准定义的机器模式控制寄存器如 表 4.3 所示。

表 4.3: RISC-V 标准机器模式控制寄存器

名称	读写权限	寄存器编号	描述
机器模式信息寄存器组			
mvendorid	机器模式只读	0xF11	供应商编号寄存器
marchid	机器模式只读	0xF12	架构编号寄存器
mimpid	机器模式只读	0xF13	机器模式硬件实现编号寄存器
mhartid	机器模式只读	0xF14	机器模式逻辑内核编号寄存器

下页继续

表 4.3 – 续上页

名称	读写权限	寄存器编号	描述
机器模式异常配置寄存器组			
mstatus	机器模式读写	0x300	机器模式处理器状态寄存器
misa	机器模式读写	0x301	机器模式处理器指令集特性寄存器
medeleg	机器模式读写	0x302	机器模式异常降级控制寄存器
mideleg	机器模式读写	0x303	机器模式中断降级控制寄存器
mie	机器模式读写	0x304	机器模式中断使能控制寄存器
mtvec	机器模式读写	0x305	机器模式向量基址寄存器
mcounteren	机器模式读写	0x306	机器模式计数器授权控制寄存器
mcountinhibit	机器模式读写	0x320	机器模式计数禁止寄存器
机器模式异常处理寄存器组			
mscratch	机器模式读写	0x340	机器模式异常临时数据备份寄存器
mepc	机器模式读写	0x341	机器模式异常保留程序计数器
mcause	机器模式读写	0x342	机器模式异常事件原因寄存器
mtval	机器模式读写	0x343	机器模式异常事件向量寄存器
mip	机器模式读写	0x344	机器模式中断等待状态寄存器
机器模式内存保护寄存器组			
pmpcfg0	机器模式读写	0x3A0	物理内存保护配置寄存器 0
pmpaddr0	机器模式读写	0x3B0	物理内存保护基址寄存器 0
……			
pmpaddr7	机器模式读写	0x3B7	物理内存保护基址寄存器 7
机器模式计数器/计时器			
mcycle	机器模式读写	0xB00	机器模式周期计数器
minstret	机器模式读写	0xB02	机器模式退休指令计数器
mhpmcounter3	机器模式读写	0xB03	机器模式计数器 3
……			
mhpmcounter31	机器模式读写	0xB1F	机器模式计数器 31
机器模式计数器配置寄存器组			
mhpmevent3	机器模式读写	0x323	机器模式事件选择寄存器 3
……			
mhpmevent31	机器模式读写	0x33F	机器模式事件选择寄存器 31

C910 中实现的 RISC-V 标准定义的超级用户模式控制寄存器如 表 4.4 所示。

表 4.4: RISC-V 标准超级用户模式控制寄存器

名称	读写权限	寄存器编号	描述
超级用户模式异常配置寄存器组			
sstatus	超级用户模式读写	0x100	超级用户模式处理器状态寄存器
sie	超级用户模式读写	0x104	超级用户模式中断使能控制寄存器
stvec	超级用户模式读写	0x105	超级用户模式向量基址寄存器
scounteren	超级用户模式读写	0x106	超级用户模式计数器使能控制寄存器
超级用户模式异常处理寄存器组			
sscratch	超级用户模式读写	0x140	超级用户模式异常临时数据备份寄存器
sepc	超级用户模式读写	0x141	超级用户模式异常保留程序计数器
scause	超级用户模式读写	0x142	超级用户模式异常事件原因寄存器
stval	超级用户模式读写	0x143	超级用户模式异常事件向量寄存器
sip	超级用户模式读写	0x144	超级用户模式中断等待状态寄存器
超级用户模式地址转换寄存器组			
satp	超级用户模式读写	0x180	超级用户虚拟地址转换和保护寄存器

C910 中实现的 RISC-V 标准定义的用户模式控制寄存器如*RISC-V 标准用户模式控制寄存器* 所示。

表 4.5: RISC-V 标准用户模式控制寄存器

名称	读写权限	寄存器编号	描述
用户模式浮点控制寄存器组			
fflags	用户模式读写	0x001	浮点异常累积状态寄存器
frm	用户模式读写	0x002	浮点动态舍入模式控制寄存器
fcsr	用户模式读写	0x003	浮点控制状态寄存器
用户模式计数/计时器			
cycle	用户模式只读	0xC00	用户模式周期计数器
time	用户模式只读	0xC01	用户模式时间计数器
instret	用户模式只读	0xC02	用户模式退休指令计数器
hpmcounter3	用户模式只读	0xC03	用户模式计数器 3
……			
hpmcounter31	用户模式只读	0xC1F	用户模式计数器 31

4.5.2　扩展控制寄存器

本章节描述 C910 实现的扩展控制寄存器，按照机器模式、超级用户模式、用户模式分别描述。

C910 中扩展的机器模式控制寄存器如 表 4.6 所示。

表 4.6: C910 扩展机器模式控制寄存器

名称	读写权限	寄存器编号	描述
机器模式处理器控制和状态扩展寄存器组			
mxstatus	机器模式读写	0x7C0	机器模式扩展状态寄存器
mhcr	机器模式读写	0x7C1	机器模式硬件配置寄存器
mcor	机器模式读写	0x7C2	机器模式硬件操作寄存器
mccr2	机器模式读写	0x7C3	机器模式 L2C ache 控制寄存器
mhint	机器模式读写	0x7C5	机器模式隐式操作寄存器
mrvbr	机器模式只读	0x7C7	机器模式复位向量基址寄存器
mcounterwen	机器模式读写	0x7C9	超级用户模式计数器写使能寄存器
mcounterinten	机器模式读写	0x7CA	机器模式事件中断使能寄存器
mcounterof	机器模式读写	0x7CB	机器模式事件上溢出标注寄存器
机器模式 Cache 访问扩展寄存器组			
mcins	机器模式读写	0x7D2	机器模式 C ache 指令寄存器
mcindex	机器模式读写	0x7D3	机器模式 Cache 访问索引寄存器
mcdata0	机器模式读写	0x7D4	机器模式 Ca che 数据寄存器 0
mcdata1	机器模式读写	0x7D5	机器模式 Ca che 数据寄存器 1
机器模式处理器型号扩展寄存器组			
mcpuid	机器模式只读	0xFC0	机器模式处理器型号寄存器
mapbaddr	机器模式只读	0xFC1	片上总线基地址
多核扩展寄存器组			
msmpr	机器模式读写	0x7F3	Snoop 监听使能寄存器

具体寄存器的定义和功能，请参考*附录 C-1 机器模式控制寄存器* 。

C910 中扩展的超级用户模式控制寄存器如 表 4.7 所示。

表 4.7: C910 扩展超级用户模式控制寄存器

名称	读写权限	寄存器编号	描述
超级用户模式处理器控制和状态扩展寄存器组			
sxtatus	超级用户模式读写	0x5C0	超级用户模式扩展状态寄存器
shcr	超级用户模式读写	0x5C1	超级用户模式硬件控制寄存器
scounterinten	超级用户模式读写	0x5C4	超级用户模式事件中断使能寄存器
scounterof	超级用户模式读写	0x5C5	超级用户模式事件上溢出标注寄存器
scycle	超级用户模式读写	0x5E0	超级用户模式周期计数器
……			
shpmcounter31	超级用户模式读写	0x5FF	超级用户模式计数器 31
超级用户模式 MMU 扩展寄存器组			
smir	超级用户模式读写	0x9C0	超级用户模式 MMU Index 寄存器
smel	超级用户模式读写	0x9C1	超级用户模式 MMU EntryLo 寄存器
smeh	超级用户模式读写	0x9C2	超级用户模式 MMU EntryHi 寄存器
smcir	超级用户模式读写	0x9C3	超级用户模式 MMU 控制寄存器

具体寄存器的定义和功能，请参考*附录 C-2 超级用户模式控制寄存器*

C910 中扩展的用户模式控制寄存器如 表 4.8 所示。

表 4.8: C910 扩展用户模式控制寄存器

名称	读写权限	寄存器编号	描述
用户模式扩展浮点控制寄存器组			
fxcr	用户模式读写	0x800	用户模式扩展浮点控制寄存器

具体寄存器的定义和功能，请参考*附录 C-3 用户模式控制寄存器* 。

4.6　数据格式

4.6.1　整型数据格式

寄存器内部的数值并没有大小端之分，只有有符号和无符号的区别。其格式均为从右至左表示逻辑低位到高位的排布，如 图 4.2 所示。

4.6.2　浮点数据格式

C910 浮点单元遵从 RISC-V 标准，兼容 ANSI/IEEE 754-2008 浮点标准，支持半精度、单精度和双精度浮点运算，数据格式如 图 4.3 所示。其中，单精度数据仅使用 64 位浮点寄存器的低 32 位，高 32 位需要全为 1，否则会被当被非数处理；半精度数据仅使用 64 位浮点寄存器的低 16 位，高 48 位需要全为 1，否则会被当作非数处理。

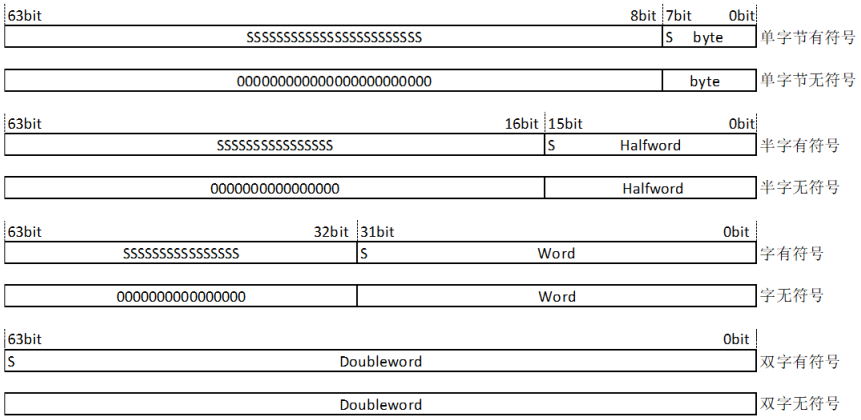


图 4.2: 寄存器中的整型数据组织结构

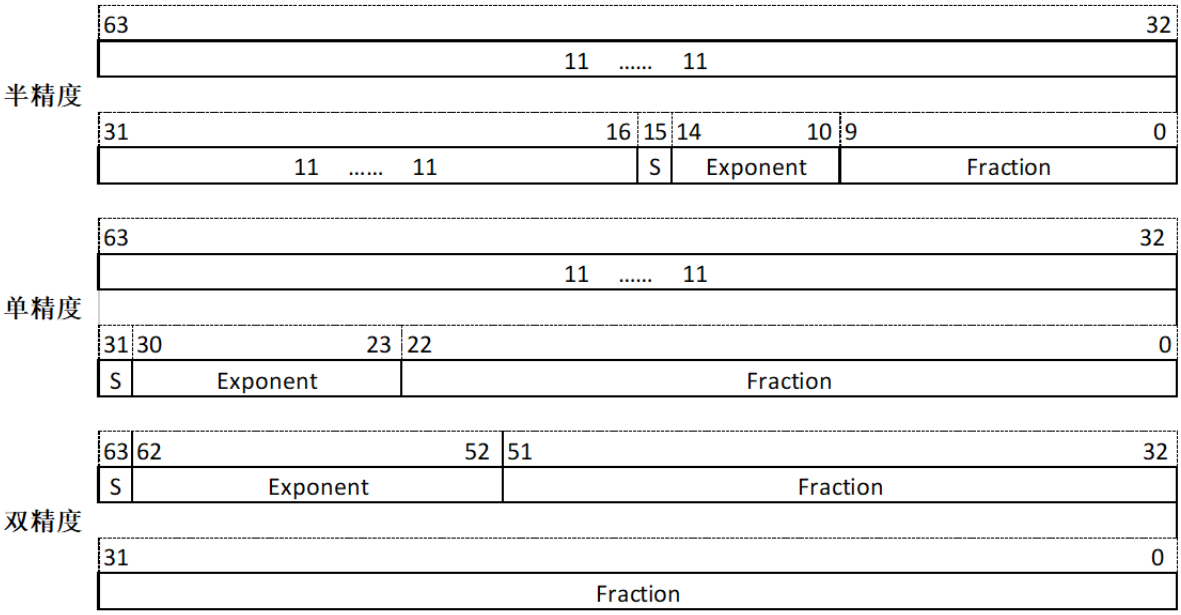


图 4.3: 寄存器中的浮点数据组织结构

4.7　大小端

大小端的概念是相对于存储器数据存储的格式而提出的。高地址字节存放至物理内存的低位被定义为大端；高地址字节存放至物理内存的高位被定义为小端，如 图 4.4 所示。

A+7	A+6	A+5	A+4	A+3	A+2	A+1	A	
Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0	Double word at A
Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0	Word at A
Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0	Half word at A
Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0	Byte at A

a) 小端模式

A+4	A+5	A+6	A+7	A	A+1	A+2	A+3	
Byte3	Byte2	Byte1	Byte0	Byte7	Byte6	Byte5	Byte4	Double word at A
Byte4	Byte5	Byte6	Byte7	Byte3	Byte2	Byte1	Byte0	Word at A
Byte6	Byte7	Byte4	Byte5	Byte1	Byte0	Byte3	Byte2	Half word at A
Byte7	Byte6	Byte5	Byte4	Byte0	Byte1	Byte2	Byte3	Byte at A

b)大端V1模式

A+7	A+6	A+5	A+4	A+3	A+2	A+1	A	
Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7	Double word at A
Byte4	Byte5	Byte6	Byte7	Byte0	Byte1	Byte2	Byte3	Word at A
Byte6	Byte7	Byte4	Byte5	Byte2	Byte3	Byte0	Byte1	Half word at A
Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0	Byte at A

b)大端V2模式

图 4.4: 内存中的数据组织形式

C910 仅支持小端模式，支持标准补码的二进制整数。每个指令操作数的长度既可以显式编码在程序中（load/store 指令），也可以隐含在指令操作中（index operation, byte extraction）。通常，指令接收 64 位操作数，产生 64 位结果。

第五章　异常与中断

5.1　概述

异常处理 (包括指令异常和外部中断) 是处理器的一项重要功能，在某些异常事件产生时，用来使处理器转入对这些事件的处理。这些事件包括硬件错误、指令执行错误、用户程序请求服务等。

异常处理的关键是在异常发生时，保存 CPU 当前运行的状态，在退出异常处理时恢复异常处理前的状态。异常能够在指令流水线的各个阶段被识别，CPU 硬件会保证后续指令不会改变 CPU 的状态。异常在指令的边界上被处理，即 CPU 在指令退休时响应异常，并保存退出异常处理时将被执行指令的地址。即使异常指令退休前被识别，异常也要在相应的指令退休时才会被处理。为了程序功能的正确性，CPU 在异常处理结束后要避免重复执行已执行完成的指令。

以在机器模式响应异常为例，具体步骤为:（这里的“异常”泛指指令异常和外部中断）

第一步：处理器保存 PC 到 mepc 中。

第二步：根据发生的异常类型更新 mcause 和 mtval。

第三步：将 mstatus 的中断使能位 MIE 保存到 MPIE 中，将 MIE 清零，禁止响应中断。

第四步：将发生异常之前的权限模式保存到 mstatus 的 MPP 中，切换到机器模式。

第五步：根据 mtvec 中的基址和模式，得到异常服务程序入口地址。处理器从异常服务程序的第一条指令处开始执行。

C910 遵从 RISC-V 标准的异常向量表，如 表 5.1 所示。

表 5.1: 异常和中断向量分配

中断标记	异常向量号	描述
1	0	未实现
1	1	超级用户模式软件中断
1	2	保留
1	3	机器模式软件中断
1	4	未实现
1	5	超级用户模式计时器中断
1	6	保留

下页继续

表 5.1 – 续上页

中断标记	异常向量号	描述
1	7	机器模式计时器中断
1	8	未实现
1	9	超级用户模式外部中断
1	10	保留
1	11	机器模式外部中断
1	17	性能检测溢出中断
1	其他	保留
0	0	未实现
0	1	取指令访问错误异常
0	2	非法指令异常
0	3	调试断点异常
0	4	加载指令非对齐访问异常
0	5	加载指令访问错误异常
0	6	存储/原子指令非对齐访问异常
0	7	存储/原子指令访问错误异常
0	8	用户模式环境调用异常
0	9	超级用户模式环境调用异常
0	10	保留
0	11	机器模式环境调用异常
0	12	取指页面错误异常
0	13	加载指令页面错误异常
0	14	保留
0	15	存储/原子指令页面错误异常
0	>= 16	保留

C910 支持异常和中断的降级响应 (delegation)。在超级用户模式发生异常或者中断时，处理器需要切换到机器模式响应，模式切换会造成处理器性能损失。Delegation 机制支持配置中断和异常在超级用户模式响应。其中，机器模式下发生的异常不受 delegation 控制，只在机器模式响应。机器模式外部中断、机器模式软件中断、机器模式计时器中断不支持降级到超级用户模式响应，其他中断均可以被降级到超级用户模式态。在机器模式下不响应被降级的中断。

在超级用户模式和用户模式下均可响应所有符合条件的中断和异常。对于未被降级的中断和异常，进入机器模式进行处理，更新机器模式异常处理寄存器。对于被降级的中断和异常均在超级用户模式响应，更新超级用户模式异常处理寄存器。

5.2　异常

5.2.1　异常响应

以在机器模式响应异常为例，具体步骤为：（这里的“异常”特指非法指令，访问错误等事件）

第一步：处理器保存发生异常的 PC 到 mepc 中。

第二步：设置 mcause 的中断标记为 0，将异常编号写入 mcause，并按照 表 5.2 的规则更新 mtval。

第三步：将 mstatus 的中断使能位 MIE 保存到 MPIE 中，将 MIE 清零，禁止响应中断。

第四步：将发生异常之前的权限模式保存到 mstatus 的 MPP 中，切换到机器模式。

第五步：PC 从 mtvec.Base 处取指令并执行。通常，取回的指令是一条跳转指令，跳转至顶层处理函数。该函数通过分析 mcause 获取异常编号，并调用该编号对应的处理函数。

表 5.2: 异常发生时 mtval 的更新

异常向量号	异常类型	mtval 更新值
1	取指令访问错误异常	取指访问的虚拟地址
2	非法指令异常	指令码
3	调试断点异常	0
4	加载指令非对齐访问异常	加载访问的虚拟地址
5	加载指令访问错误异常	0
6	存储/原子指令非对齐访问异常	存储/原子访问的虚拟地址
7	存储/原子指令访问错误异常	0
8	用户模式环境调用异常	0
9	超级用户模式环境调用异常	0
11	机器模式环境调用异常	0
12	取指页面错误异常	取指访问的虚拟地址
13	加载指令页面错误异常	加载访问的虚拟地址
15	存储/原子指令页面错误异常	存储/原子访问的虚拟地址

5.2.2　异常返回

执行 mret 指令可以实现异常返回。此时，处理器执行下列操作：

- 将 mepc 恢复到 PC。（mepc 保存的是发生异常的 PC。通过调整 mepc，可以跳过发生异常的指令；如果不调整，则重新执行发生异常的指令）
- 将 mstatus.MPIE 恢复到 mstatus.MIE。
- 从 mstatus.MPP 恢复发生异常之前的权限模式。

5.2.3　非精确异常

在极少数情况下，处理器可能表现出“非精确异常”的行为。非精确异常，是指发生异常时，mepc 没有指向触发该异常的指令。例如，CPU 执行了一条 load 指令，总线返回 Error。由于流水线具有指令快速退休的特性，当总线返回 Error 时，load 指令已经退休，所以 mepc 指向的是后续的某条指令，而不是 load 指令本身。

值得注意的是，非精确异常在实际系统中发生的概率极低，一旦发生，则意味着系统出现了 fatal 错误。

5.3　中断

5.3.1　中断优先级

当同时发生多个中断请求时，优先级按照下列顺序决定（从高到低）：

- M 态外部中断
- M 态软件中断
- M 态计时器中断
- S 态外部中断
- S 态软件中断
- S 态计时器中断
- PMU 溢出中断
- S 态外部中断（被降级）
- S 态软件中断（被降级）
- S 态计时器中断（被降级）
- PMU 溢出中断（被降级）

5.3.2　中断响应

以在机器模式响应中断为例，具体步骤为：

第一步：处理器执行完当前指令，保存下一条指令的 PC 到 mepc 中。

第二步：设置 mcause 的中断标记为 1，将中断编号写入 mcause，并更新 mtval 为 0。

第三步：将 mstatus 的中断使能位 MIE 保存到 MPIE 中，将 MIE 清零，禁止响应中断。

第四步：将发生中断之前的权限模式保存到 mstatus 的 MPP 中，切换到机器模式。

第五步（mtvec.Mode=0，直通中断）：PC 从 mtvec.Base 处取指令并执行。通常，取回的指令是一条跳转指令，跳转至顶层处理函数。该函数通过分析 mcause 获取中断编号，并调用该编号对应的处理函数。

第五步（mtvec.Mode=1，矢量中断）：PC 从 mtvec.Base + 4 * 中断编号处取指令并执行。通常，取回的指令是一条跳转指令，跳转至相应中断的处理函数。

5.3.3　中断返回

执行 mret 指令可以实现中断返回。此时，处理器执行下列操作：

- 将 mepc 恢复到 PC。（mepc 保存的是下一条指令的 PC，所以无需调整）
- 将 mstatus.MPIE 恢复到 mstatus.MIE。
- 从 mstatus.MPP 恢复发生中断之前的权限模式。

第六章　内存模型

6.1　内存模型概述

6.1.1　内存属性

C910 支持两种内存类型，分别是内存（Memory）和外设（Device），由 SO 位区分。其中，Memory 类型的特点是支持投机执行和乱序执行，根据是否可高缓（Cacheable，C）进一步分为可高缓内存（Cacheable memory）和不可高缓内存（Non-cacheable memory）。Device 类型的特点为不可投机执行且必须按序执行，因此 Device 一定带有不可高缓的属性。Device 根据是否可缓存（Bufferable，B）分为可缓存外设（Bufferable device）和不可缓存外设（Non-bufferable device）。Bufferable 表示写访问允许在某个中间节点快速返回写响应；反之，Non-bufferable 表示写访问只有在最终设备真正写完成后才返回写响应。

为了支持多核之间数据共享，C910 增加了可共享的页面属性（Shareable，SH）。对于可共享的页面，表示该页面在多核间共享，由硬件维护数据的一致性；对于不可共享的页面，表示被某个单核独占，不要求硬件维护数据的一致性。不可共享页面的多核数据一致性需要软件来维护。

可高缓内存可以配置 SH 的属性，而不可高缓内存类型和外设类型的 SH 属性不可配置，固定为可共享。

另外，C910 支持配置安全的页面属性（Security，SEC）。

表 6.1 给出了各个内存类型对应的页面属性。

表 6.1: 内存类型分类

内存类型	SO	C	B	SH	SEC
可高缓内存	0	1	1	可配	可配
不可高缓内存	0	0	1	1	可配
可缓存外设	1	0	1	1	可配
不可缓存外设	1	0	0	1	可配

CPU 可以通过两种方获取一个地址的页面属性：sysmap.h 或者页表项（pte）。具体说明如下：

1. 在所有不进行虚拟地址到物理地址转换的情况下（即：机器模式或者 MMU 关闭），地址的页面属性由 sysmap.h 决定。
2. 在所有进行虚拟地址到物理地址转换的情况下（即：非机器模式且 MMU 打开），地址的页面属性来源依赖于 mxstatus.maee。如果 maee 打开，则地址的页面属性由对应 pte 中扩展的页面属性决定。如果

maee 关闭，则地址的页面属性由 sysmap.h 决定。

sysmap.h 是 C910 扩展的配置文件，对用户开放，用户可以根据自身需求，定义不同地址段的页面属性。

sysmap.h 支持对 8 个地址空间的属性设定。第 i（i=0~7）个地址空间地址上限（不包含）由宏 SYSMAP_BASE_ADDRi（i=0~7）定义，地址下限（包含）由 SYSMAP_BASE_ADDR(i-1) 定义，即：

SYSMAP_BASE_ADDR(i-1) <= 第 i 个地址空间地址 < SYSMAP_BASE_ADDRi

第 0 个地址空间的下限是 0x0。内存地址不在 sysmap.h 文件设定的 8 个地址区间的地址属性默认为 cacheable/bufferable/shareable/security。每个地址空间上下边界是 4KB 对齐，因此宏 SYSMAP_BASE_ADDRi 定义的是地址的高 28 位。

落在第 i(i=0~7) 个地址空间内的地址的属性由宏 SYSMAP_FLAGi (i=0~7) 定义，属性的排布如图 图 6.1 所示：

4	3	2	1	0
Strong order	Cacheable	Bufferable	Shareable	Security

图 6.1: sysmap.h 地址属性格式

6.1.2　内存一致性模型

C910MP 采用宽松的内存一致性模型（Weak Memory Ordering），该模型具体定义如下：

- 各个 CORE 保证相同地址访问的顺序性，包括读后读、写后写、读后写和写后读;
- 各个 CORE 放松不同地址访问的顺序性，包括读后读、写后写、读后写和写后读;
- 保证 other-multi-copy 的原子性，即要求当一个核能获得另一个核的写数据时，保证其他核此时也能够获得该写数据；而一个核能够获得自己核的写数据时，不要求其他核此时也能够获得该写数据。

由于 Weak Memory Ordering 的模型，导致多核之间内存实际的读写顺序和程序给定的访问顺序会不一致。因此，C910 扩展了 SYNC 指令供软件强制规定内存访问的顺序性。

SYNC 指令限定了所有指令的执行顺序，保证了 SYNC 指令之前的所有指令一定在 SYNC 指令执行之前完成。另外，SYNC 指令还可以额外同步指令内存，即在 SYNC 指令前序指令完成时清空流水线，重新取指。具体指令如 表 6.2 所示。

表 6.2: SYNC 指令描述

助记符	指令描述	作用域
SYNC.IS	Synchronize data and instruction memory	Shareable
SYNC.I	Synchronize data and instruction memory	Non-shareable
SYNC.S	Synchronize data memory	Shareable
SYNC	Synchronize data memory	Non-shareable

6.2　虚拟内存管理

6.2.1　MMU 概述

C910 MMU（Memory Management Unit）兼容 RISC-V SV39 标准。其作用主要有:

- **地址转换**：将虚拟地址（39 位）转换成物理地址（40 位）。
- **页面保护**：通过对页面的访问者进行读/写/执行权限检查。
- **页面属性管理**：扩展地址属性位，根据访问地址，获取页面对应属性，供系统进一步使用。

6.2.2　TLB 组织形式

MMU 主要利用 TLB (Translation Look-aside Buffer) 来实现上述功能。TLB 将 CPU 访存所使用的虚拟地址作为输入，转换前检查 TLB 的页属性，再输出该虚拟地址所对应的物理地址。

C910 MMU 采用两级 TLB，第一级为 uTLB，分别为指令 I-uTLB 和数据 D-uTLB，第二级为 jTLB。在处理器复位后，硬件会将 uTLB 和 jTLB 的所有表项进行无效化操作，软件无需初始化操作。

I-uTLB 有 32 个全相联表项，可以混合存储 4K、2M 和 1G 三种大小的页面，取指请求命中 I-uTLB 时，当拍可以得到物理地址和相应权限属性。

D-uTLB 有 17 个全相联表项，可以混合存储 4K、2M 和 1G 三种大小的页面，加载和存储请求命中 D-uTLB 时，当拍可以得到物理地址和相应权限属性。

jTLB 为指令和数据共用，4 路组相联结构，表项大小 1024，可以混合存储 4K、2M 和 1G 三种大小页面。uTLB 缺失，jTLB 命中时，最快 3 个 cycle 返回物理地址和相应权限属性。

6.2.3　地址转换流程

MMU 的主要功能是将虚拟地址转换为物理地址并进行相应的权限检查。具体的地址映射关系和相应权限由操作系统进行配置，存放于页表中。C910 采用最多三级页表索引的方式实现地址转换。访问第一级页表得到第二级页表的基地址和相应的权限属性；访问第二级页表得到第三级页表的基地址和相应的权限属性；访问第三级页表得到最终物理地址和相应的权限属性。每一级访问都有可能得到最终的物理地址，即叶子表项。虚拟页面号 VPN 有 27-bit，等分为三个 9-bit 的 VPN[i]，每次访问使用一部分 VPN 进行索引。

叶子表项的内容（即由虚拟地址转换得到的物理地址和相应的权限属性）被缓存于 TLB 内以加速地址转换。若 uTLB 失配则访问 jTLB，若 jTLB 进一步失配，则 MMU 会启动 Hardware Page Table Walk，访问内存得到最终的地址转换结果。

页表用于存储下级页表的入口地址或者最终页表的物理信息，其结构如下:

Flags – 9:0 位页面属性

功能如*MMU EntryLo 寄存器 (SMEL)* 中所述。

Flags – 63:59 位页面属性

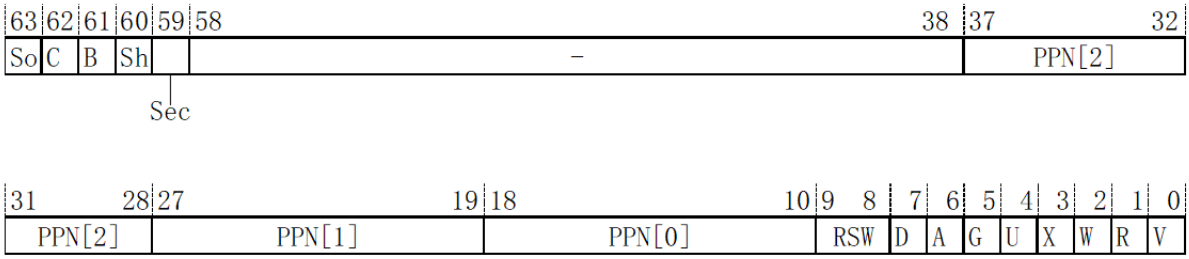


图 6.2: 页表结构说明

C910 自定义页面属性，当 mxstatus 寄存器打开了 MAEE 使能时存在，功能如*MMU EntryLo 寄存器 (SMEL)* 中所述。

PPN – 页表物理地址

PPN[i] 分别代表三级页表转换时所对应的 PPN 值。

地址转换的详细流程描述如下：

CPU 要访问某个虚拟地址，若 TLB 命中，则从 TLB 中直接获取物理地址及相关属性。若 TLB 缺失，则地址的转换具体步骤为：

1. 根据 SATP.PPN 和 VPN[2] 得到一级页表访存地址 {SATP.PPN, VPN[2], 3’ b0}，使用该地址访问 Dcache/内存，得到 64-bit 一级页表 PTE;
2. 检查 PTE 是否符合 PMP 权限，若不符合则产生相应 access error 异常；若符合则根据 表 6.4 所示的规则判断 X/W/R-bit 是否符合叶子页表条件，若符合叶子页表条件则说明已经找到最终物理地址，到第 3 步；若不符合则到第 1 步，使用 PTE.PPN 拼接下一级 VPN[]，再拼接 3’ b0 得到下一级页表访存地址继续访问 Dcache/内存；
3. 找到了叶子页表，结合 PMP 中的 X/W/R/L 位和 PTE 中的 X/W/R 位得到两者的最小权限进行权限检查，并将 PTE 的内容回填到 JTLB 中；
4. 在任何一步的 PMP 检查中，如果有权限违反，则根据访问类型产生对应的 access error 异常；
5. 若得到叶子页表，但：访问类型违反 A/D/X/W/R/U-bit 的设置，产生对应的 page fault 异常；若三次访问结束仍未得到叶子页表，则产生对应的 page fault 异常；若访问 Dcache/内存过程中得到 access error 响应，产生 page fault 异常。
6. 若得到叶子页表，但访问次数少于 3 次，则说明得到了大页表。检查大页表的 PPN 是否按照页表尺寸对齐，若未对齐，则产生 page fault 异常。

6.2.4　系统控制寄存器

C910 MMU 除了支持标准的 SATP 寄存器以外，还自定义扩展了 SMIR、SMCIR、SMEL 和 SMEH 控制寄存器。用户可以通过这几个扩展寄存器，直接对 TLB 进行读写、查询和无效操作。

6.2.4.1　MMU 地址转换寄存器 (SATP)

SATP 是 SV39 规范的 MMU 控制寄存器。

63 60	59 44	43 32
Mode	ASID	-

31 28	27 0
-	PPN

图 6.3: SATP 寄存器说明

Mode - MMU 地址翻译模式

表 6.3: MMU 地址翻译模式

RV64		
Value	**Name**	**Description**
0	Bare	No translation or protection
1-7	-	Reserved
8	Sv39	Page-based 39-bit virtual addressing
9	Sv48	Page-based 48-bit virtual addressing
10	Sv57	Reserved for page-based 57-bit virtual addressing
11	Sv64	Reserved for page-based 64-bit virtual addressing
12-15	-	Reserved

ASID – 当前 ASID

表示当前程序的 ASID 号。

PPN – 硬件回填根 PPN

第一级硬件回填使用的 PPN。

6.2.4.2　MMU 控制寄存器 (SMCIR)

SMCIR 寄存器实现对 MMU 进行多种操作，包括 TLB 查找、TLB 读写、TLB 无效等操作。

TLBP：TLB 查询

根据 EntryHi 寄存器去查询 TLB。

当查询命中时，用 TLB 的序号去更新 Index 寄存器。

TLBR：TLB 读

根据 Index 寄存器索引，读出对应的 TLB 表项的值，并用这些值来更新 SMEH 和 SMEL 寄存器。

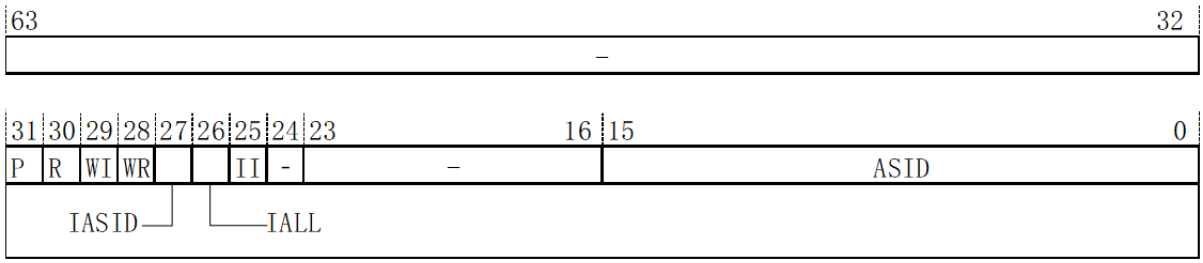


图 6.4: SMCIR 寄存器说明

TLBWI：TLB 索引写

根据 Index 寄存器索引值，将寄存器 SMEH 和 SMEL 写入 TLB 对应表项。

TLBWR：TLB 随机写

根据 Random 寄存器索引值，将寄存器 SMEH，SMEL 写入 TLB 对应表项。

TLBIASID：TLB 根据 ASID 无效

所有匹配 ASID 的 TLB 表项全部无效。

TLBIALL：TLB 初始化

将所有 TLB 表项无效，初始化。

TLBII：TLB 根据索引无效

根据 Index 寄存器索引值，将对应的 TLB 表项无效。

TLBIAW：TLB 根据世界无效

将可信世界以及非可信世界对应的所有 TLB 表项无效.

该位仅在配置有 TEE 扩展时有意义，C910 当前不支持。

ASID：ASID 号

TLBIASID 操作使用该 ASID 做匹配。SMCIR 寄存器实现对 MMU 进行多种操作，包括 TLB 查找、TLB 读写、TLB 无效等操作。

TLBP – TLB 查询

根据 EntryHi 寄存器去查询 TLB，当查询命中时，用 TLB 的序号去更新 Index 寄存器。

TLBR – TLB 读

根据 Index 寄存器索引，读出对应的 TLB 表项的值，并用这些值更新 SMEH 和 SMEL 寄存器。

TLBWI – TLB 索引写

根据 Index 寄存器索引值，将寄存器 SMEH，SMEL 写入 TLB 对应表项。

TLBWR – TLB 随机写

根据 Random 寄存器索引值，将寄存器 SMEH，SMEL 写入 TLB 对应表项。

TLBIASID – TLB 根据 ASID 无效

所有匹配 ASID 的 TLB 表项全部无效。

TLBIALL – TLB 初始化

将所有 TLB 表项无效，初始化。

TLBII – TLB 根据索引无效

根据 Index 寄存器索引值，将对应的 TLB 表项无效。

ASID – ASID 号

TLBIASID 操作使用该 ASID 做匹配。

6.2.4.3　MMU Index 寄存器 (SMIR)

MMU 索引寄存器，用于索引 TLB。在进行 TLB 查询时，会更新命中表项的 index。在 TLB 写索引时，通过写 SMIR 的 index 域，可以将映射关系写入 jTLB 中对应 index 处。

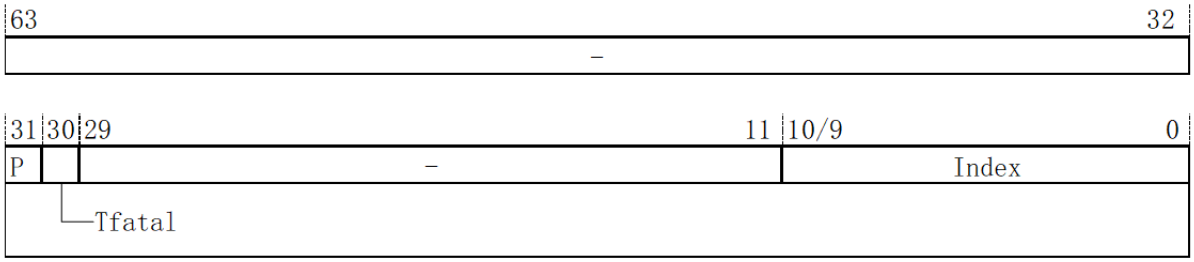


图 6.5: SMIR 寄存器说明

P – Probe Failure

0: TLBP 查询匹配命中。

1: TLBP 查询没命中。

Tfatal – Probe multiple

执行 TLBP 指令是，是否发生多重匹配。

0: 没有多重匹配。

1: 发生多重匹配。

Index – TLB Index

1024-entry 配置：Index[9:8] 为 way 索引，Index[7:0] 为 set/entry 索引；（4 way，256 entries）

2048-entry 配置：Index[10:9] 为 way 索引，Index[8:0] 为 set/entry 索引；（4 way，512 entries）

6.2.4.4　MMU EntryHi 寄存器 (SMEH)

SMEH 寄存器有两个功能：包含 TLB 访问的虚拟地址信息和 TLB 异常时当前 VPN 的值，ASID 表示当前页面对应的进程号。

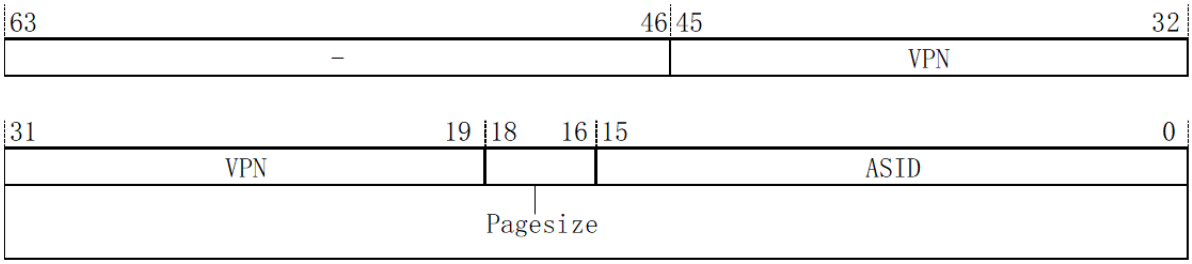


图 6.6: SMEH 寄存器说明

VPN – 虚拟页帧号

该域在 TLB 读和发生页面错误异常时硬件更新，在软件写 TLB 表项之前由软件预先写入。

Pagesize – 页面尺寸

Onehot 从低到高表示 4K，2M，1G 页面大小。

该域在 TLB 读时硬件更新，在软件写 TLB 表项之前由软件预先写入。

ASID – 地址空间标识

该域通常用来保存操作系统所看到的当前地址空间的标识，用于区分不同进程。

在 TLB 读时硬件更新，在软件写 TLB 表项之前由软件预先写入。

6.2.4.5　MMU EntryLo 寄存器 (SMEL)

SMEL 包含 TLB 访问的物理地址和页面属性信息。

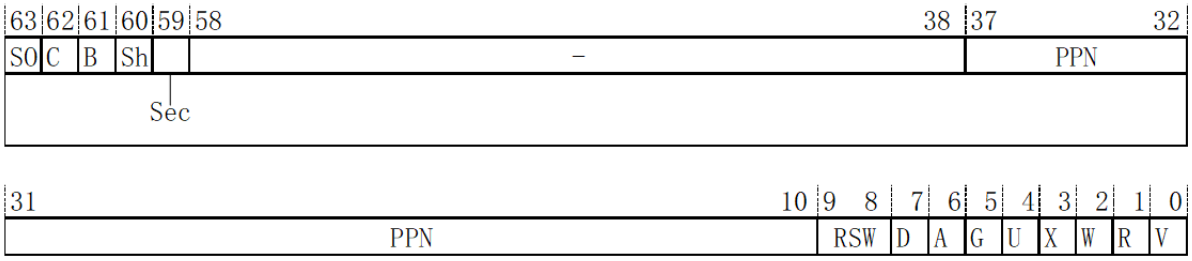


图 6.7: SMEL 寄存器说明

PPN – 物理页帧号, 28-bit

SO – Strong Order

用于表示内存对访问顺序的要求

1’b0: No strong order（Normal memory）

1’b1: Strong order（Device）

C – Cacheable

1’b0: Uncacheable

1’b1: Cacheable

B – Buffer

1’b0: Unbufferable

1’b1: Bufferable

SH – Shareable

用于表征页面的共享属性

1’b0: Unshareable

1’b1: Shareable

Sec (T – Trustable)

用于表征页面属于可信世界或者非可信世界，该位仅在配有 TEE pro 扩展时有意义

1’b0: non-trustalbe

1’b1: trustable

RSW – Reserved for Software

用于预留给软件做自定义页表功能的位。default 为 2’b0

D – Dirty

D 位为 1 时，表明该页是否被改写。

1’b0: 当前页未被写/不可写

1’b1: 当前页已经被写/可写

D 位为 0 时，对此页面进行写操作会触发 Page Fault (Store) 异常，通过软件在异常服务程序中操控 D 位来维护 D 位满足是否被改写/可写的定义。

A – Accessed

A 位为 1 时，表明该页可访问。为 0 时不可访问，否则会触发 Page Fault (对应访问类型) 异常。

1’b0: 当前页不可访问

1’b1: 当前页可访问

G – Global

全局页面标识，当前页可供多个进程共享

1’b0: 非共享页面，进程号 ASID 私有

1’b1: 共享页面

U – User

用户模式可访问

1’b0: 用户模式不可访问，当用户模式访问，出 page fault 异常。**default is 1’b0**

1’b1: 用户模式可访问

X W R – 可执行，可写，可读

表 6.4: XWR 权限说明

X	W	R	Meaning
0	0	0	Pointer to next level of page table
0	0	1	Read-only page
0	1	0	Reserved for future use
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Reserved for future page
1	1	1	Read-write-execute page

V – Valid

表明物理页在内存中是否分配好，访问一个 V=0 的页面，将触发 Page Fault 异常。

1’b0: 当前页没有分配好

1’b1: 当前页已分配好

6.3　物理内存保护

6.3.1　PMP 概述

C910 PMP（Physical Memory Protection）遵从 RISC-V 标准。PMP 单元负责对物理地址的访问权限进行检查，判定当前工作模式下 CPU 是否具备对该地址的读/写/执行权限。

C910 PMP 单元的主要特征有：

- 支持 8 个 PMP 表项，每个表项通过 0-7 的号码来标识和索引
- 地址划分最小粒度为 4KB
- 支持 OFF、TOR、NAPOT 三种地址匹配模式，不支持 NA4 匹配模式

- 支持可读、可写、可执行三种权限的配置
- PMP 表项支持软件 Lock

6.3.2　PMP 控制寄存器

PMP 表项主要由一个 8 比特的设置寄存器和一个 64 比特的地址寄存器构成，所有 PMP 控制寄存器都只能在机器模式下访问，其他模式访问将产生非法指令异常。

6.3.2.1　物理内存保护设置寄存器（PMPCFG）

物理内存保护设置寄存器提供 8 个表项的权限设置。

63 56	55 48	47 40	39 32	31 24	23 16	15 8	7 0	
entry7_cfg	entry6_cfg	entry5_cfg	entry4_cfg	entry3_cfg	entry2_cfg	entry1_cfg	entry0_cfg	pmpcfg0
8	8	8	8	8	8	8	8	

图 6.8: 物理内存保护设置寄存器整体分布

7	6 5	4 3	2	1	0
L(WARL)	0(WARL)	A(WARL)	X(WARL)	W(WARL)	R(WARL)
1	2	2	1	1	1

图 6.9: 物理内存保护设置寄存器

PMP 控制寄存器描述其具体的描述如 表 6.5 所示。

表 6.5: PMP 控制寄存器描述

位	名称	描述
0	R	表项的可读属性： **0**：表项匹配地址不可读 **1**：表项匹配地址可读
1	W	表项的可写属性： **0**：表项匹配地址不可写 **1**：表项匹配地址可写
2	X	表项的可执行属性： **0**：表项匹配地址不可执行 **1**：表项匹配地址可执行
4:3	A	表项的地址匹配模式： **00**：OFF，无效表项 **01**：TOR（Top of range），使用相邻表项的地址作为匹配区间的模式 **10**：NA4（Naturally aligned four-byte region），区间大小为 4 字节的匹配模式，该模式不支持 **11**：NAPOT（Naturally aligned power-of-2 regions），区间大小为 2 的幂次方的匹配模式，至少为 4KB
7	L	表项的 Lock 使能位： **0**：机器模式的访问都将成功 系统模式/用户模式的访问根据 R/W/X 判定是否成功 **1**：表项被锁住，无法对相关表项进行修改 当配置 TOR 模式，其前一个表项的地址寄存器也无法被修改 所有模式都需要根据 R/W/X 判定是否访问成功

对于 TOR 模式，假设访问地址为 A，则其命中表项 i 的条件为：pmpaddr(i-1)　A < pmpaddr(i)。对于 0 号表项，其使用 0 作为下边界。

对于 NAPOT 模式，其地址与区间大小的关系如 表 6.6 所示。

表 6.6: 保护区间编码

pmpaddr	pmpcfg.A	保护区大小	备注
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa	NA4	4B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0	NAPOT	8B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aa01	NAPOT	16B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a011	NAPOT	32B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_0111	NAPOT	64B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0_1111	NAPOT	128B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aa01_1111	NAPOT	256B	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a011_1111	NAPOT	512B	不支持

下页继续

表 6.6 – 续上页

pmpaddr	pmpcfg.A	保护区大小	备注
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_0111_1111	NAPOT	1KB	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aaa0_1111_1111	NAPOT	2KB	不支持
aaaa_aaaa_aaaa_aaaa_aaaa_aa01_1111_1111	NAPOT	4KB	支持
aaaa_aaaa_aaaa_aaaa_aaaa_a011_1111_1111	NAPOT	8KB	支持
aaaa_aaaa_aaaa_aaaa_aaaa_0111_1111_1111	NAPOT	16KB	支持
aaaa_aaaa_aaaa_aaaa_aaa0_1111_1111_1111	NAPOT	32KB	支持
aaaa_aaaa_aaaa_aaaa_aa01_1111_1111_1111	NAPOT	64KB	支持
aaaa_aaaa_aaaa_aaaa_a011_1111_1111_1111	NAPOT	128KB	支持
aaaa_aaaa_aaaa_aaaa_0111_1111_1111_1111	NAPOT	256KB	支持
aaaa_aaaa_aaaa_aaa0_1111_1111_1111_1111	NAPOT	512KB	支持
aaaa_aaaa_aaaa_aa01_1111_1111_1111_1111	NAPOT	1M	支持
aaaa_aaaa_aaaa_a011_1111_1111_1111_1111	NAPOT	2M	支持
aaaa_aaaa_aaaa_0111_1111_1111_1111_1111	NAPOT	4M	支持
aaaa_aaaa_aaa0_1111_1111_1111_1111_1111	NAPOT	8M	支持
aaaa_aaaa_aa01_1111_1111_1111_1111_1111	NAPOT	16M	支持
aaaa_aaaa_a011_1111_1111_1111_1111_1111	NAPOT	32M	支持
aaaa_aaaa_0111_1111_1111_1111_1111_1111	NAPOT	64M	支持
aaaa_aaa0_1111_1111_1111_1111_1111_1111	NAPOT	128M	支持
aaaa_aa01_1111_1111_1111_1111_1111_1111	NAPOT	256M	支持
aaaa_a011_1111_1111_1111_1111_1111_1111	NAPOT	512M	支持
aaaa_0111_1111_1111_1111_1111_1111_1111	NAPOT	1G	支持
aaa0_1111_1111_1111_1111_1111_1111_1111	NAPOT	2G	支持
aa01_1111_1111_1111_1111_1111_1111_1111	NAPOT	4G	支持
a011_1111_1111_1111_1111_1111_1111_1111	NAPOT	8G	支持
0111_1111_1111_1111_1111_1111_1111_1111	NAPOT	16G	支持
1111_1111_1111_1111_1111_1111_1111_1111	Reserved	-	支持

需要说明的是，C910 PMP 支持的最小粒度为 4KB。

6.3.2.2 物理内存保护地址寄存器（PMPADDR）

PMP 共实现了 8 个地址寄存器 pmpaddr0-pmpaddr7，存放表项的物理地址。

RISC-V 规定 PMP 地址寄存器存放的是物理地址的 [39:2] 比特，因为 C910 PMP 表项粒度最低支持 4KB，因此 bit[8:0] 不会用于地址鉴权逻辑。

6.4 内存访问顺序

在不同的场景下，C910 对地址空间的访问过程简要归纳如下：

	63 38	37 9	8 0
	0	address [37:9] (WARL)	0(WARL)
Reset	0	0	0

图 6.10: PMP 地址寄存器

场景 1：不进行 VA-PA 转换

- CPU 要访问 PA；
- 通过 sysmap.h 得到该地址的属性；
- PMP 检查，确认读/写/执行权限符合 PMP 的设定；
- 执行对该地址的访问。

场景 2：进行 VA-PA 转换

- CPU 要访问 VA；
- 通过 MMU 进行地址翻译，得到页表项（pte）；
- 从 pte 可以得到以下信息：PA、地址属性（注 1）和读/写/执行权限；
- PMP 检查，确认读/写/执行权限符合 PMP 的设定；（最终的读/写/执行权限取 PMP 与 pte 的“最小值”）
- 执行对该地址的访问。

（注 1）当 maee=1 时，地址属性来自 pte；当 maee=0 时，地址属性来自 sysmap.h。

第七章　内存子系统

7.1　内存子系统概述

C910 的每个核心有单独的指令 Cache 和数据 Cache，双核共享一个 L2 Cache。核心之间的数据一致性由硬件维护。

7.2　L1 指令 Cache

7.2.1　概述

L1 指令高速缓存的主要特征如下：

- 大小为 64KB，2 路组相联，缓存行大小为 64B；
- 虚拟地址索引，物理地址标记（VIPT）；
- 访问数据位宽为 128 比特；
- 采用先进先出的替换策略；
- 支持对整个指令高速缓存的无效操作，支持对单条缓存行的无效操作；
- 支持指令预取功能；
- 支持路预测；
- 指令高速缓存缺失的请求会 snoop 数据高速缓存（存在开关位控制）。

7.2.2　路预测

C910 指令高速缓存采用两路组相联结构，为了减少并行访问两路缓存的功耗，C910MP 实现了指令高速缓存的路预测功能。在路预测信息有效时，关闭无效数据路的访问，仅访问预测路的数据。用户可以通过配置隐式操作寄存器 MHINT.IWPE，使能指令高速缓存的路预测功能。

根据取指行为的不同，路预测可分为以下两类：

- **顺序访问**：当进行连续行内取指时，根据上次访问的路命中信息预测此次访问的路信息。

- **跳转访问**：分支指令在获取跳转目标地址的同时获取了目标缓存行的路预测信息，并根据该信息访问其中一路缓存。

7.2.3　循环加速缓存器

针对程序中存在的大量短循环，C910 设置了一个 32B 的循环加速缓存器。当检测到短循环指令序列时，该循环体被加载到循环加速缓存器中；当后续取指命中该缓存器时，直接从缓存器中获取指令以及跳转的目标地址，关闭对指令高速缓存和分支历史表、分支跳转目标预测器的访问，从而降低了取指的动态功耗。用户可以通过配置隐式操作寄存器 MHINT.LPE 使能短循环加速功能。

7.2.4　分支历史表

C910 采用分支历史表对条件分支的跳转方向进行预测。分支历史表容量为 64Kb，使用 BI-MODE 预测器作为预测机制，每周期支持一条分支结果预测。分支历史表由预测器和选择器两部分组成。其中预测器又分为跳转预测器和非跳转预测器，并根据分支历史信息对各预测器进行实时维护。分支历史表通过分支历史信息以及当前分支指令地址对各路进行索引，获得分支指令跳转方向的预测结果。

分支历史表进行预测的条件分支指令包括：

BEQ、BNE、BLT、BLTU、BGE、BGEU、C.BEQZ、C.BNEZ

7.2.5　分支跳转目标预测器

C910 使用分支跳转目标预测器对分支指令的跳转目标地址进行预测。分支跳转目标预测器对分支指令历史目标地址进行记录。如果当前分支指令命中分支跳转目标预测器，则将记录目标地址作为当前分支指令预测目标地址。

分支跳转目标预测器主要特征包括：

- 支持 1024 表项；
- 两路组相联结构，根据分支指令低位 PC 选择替换；
- 维护指令高速缓存路预测信息；
- 使用当前分支指令部分 PC 进行索引；

分支跳转目标预测器进行预测的分支指令包括：

- BEQ、BNE、BLT、BLTU、BGE、BGEU、C.BEQZ、C.BNEZ
- JAL、C.J

7.2.6　间接分支预测器

C910 使用间接分支预测器负责对间接分支的目标地址进行预测。间接分支指令通过寄存器获取目标地址，一条间接分支指令可包含多个分支目标地址，无法通过传统分支跳转目标预测器进行预测。因此，C910

采用基于分支历史的间接分支预测机制，将间接分支指令的历史目标地址与该分支之前的分支历史信息进行关联，用不同的分支历史信息将同一条间接分支的不同目标地址进行离散，从而实现多个不同目标地址的预测。

间接分支指令包括：

- JALR：源寄存器为 X1、X5 除外
- C.JALR：源寄存器为 X5 除外
- C.JR：源寄存器为 X1、X5 除外

7.2.7　返回地址预测器

返回地址预测器用于函数调用结束时，返回地址的快速准确预测。当取指单元译码得到有效的函数调用指令时，将函数返回地址压栈存入返回地址预测器；当取指单元译码得到有效的函数返回指令时，则从返回地址预测器弹栈，获取函数返回目标地址。返回地址预测器最多支持 12 层函数调用嵌套，超出嵌套次数会导致目标地址预测错误。

- 函数调用指令包括：JAL、JALR、C.JALR
- 函数返回指令包括：JALR、C.JR、C.JALR

指令功能的具体划分可以如 表 7.1 。

表 7.1: 指令功能具体划分

rd	rs1	rs1=rd	RAS action
!link	!link	-	none
!link	link	-	pop
link	!link	-	push
link	link	0	push and pop
link	link	1	push

7.2.8　快速跳转目标预测器

为了加快连续跳转时取指单元的取指效率，C910 在取指单元的第一级增加了快速跳转目标预测器。当取指单元发生连续跳转时，快速跳转目标预测器将会记录连续跳转的第二条跳转指令的地址和跳转的目标地址。若取指时命中了快速跳转目标预测器，则在第一级发起跳转，减少至少一个周期的性能损失。

快速跳转目标预测器进行预测的分支指令包括：

- BEQ、BNE、BLT、BLTU、BGE、BGEU、C.BEQZ、C.BNEZ
- JAL、C.J
- 函数返回指令

7.3　L1 数据 Cache

7.3.1　概述

L1 数据高速缓存的主要特征如下：

- 大小为 64KB，2 路组相联，缓存行大小为 64B；
- 物理地址索引，物理地址标记 (PIPT)；
- 每次读访问的最大宽度为 128 比特，支持字节/半字/字/双字/四字访问；
- 每次写访问的最大宽度为 256 比特，支持任意字节组合的访问；
- 写策略支持写回-写分配模式和写回-写不分配模式；
- 采用先进先出的替换策略；
- 支持对整个数据高速缓存的无效和清除操作，支持对单条缓存行的无效和清除操作；
- 指令多通道的数据预取功能。

7.3.2　Cache 一致性

对于页面属性配置为 shareable 且 cacheable 的请求，硬件维护数据在不同核心的 L1 数据高速缓存的一致性。

对于页面属性配置为 non-shareable 且 cacheable 的请求，处理器不维护数据在多个 L1 数据高速缓存上的一致性。如果需要该属性的页面在多个核心上共享，则需要软件维护数据一致性。

C910MP 一级高速缓存采用 MESI 协议维护多个处理器核心数据高速缓存的一致性。MESI 代表了每个缓存行在数据高速缓存上的 4 个状态，分别是：

- M：表示缓存行仅位于此数据高速缓存中，且被写脏；(UniqueDirty)
- E：表示缓存行仅位于此数据高速缓存中，且是干净的；(UniqueClean)
- S：表示缓存行可能位于多个数据高速缓存中，且是干净的；(ShareClean)
- I：表示缓存行不在该数据高速缓存中。(Invalid)

7.3.3　独占式访问

C910 支持独占式的内存访问指令 LR 和 SC。用户可以使用这两条指令构成原子锁等同步原语实现同一个核不同进程之间或者不同核之间的同步。通过 LR 指令标记需要独占访问的地址，SC 指令判断被标记的地址是否被其他进程抢占。C910 为每个核心上设置了一个位于 L1 数据高缓的局部监测器和一个位于二级高缓的全局监测器。每个监测器由一个状态机和一个地址缓存器组成，其中，状态机包含两个状态：IDLE 和 EXCLUSIVE。

对于属性设置为可高缓的页面，通过局部监测器就能够实现独占式访问。LR 指令在执行过程中设置局部监测器的状态机为 EXCLUSIVE 态并将访问的地址和 Size 保存到缓存器中；SC 指令在执行过程中读取局部监测器的状态、地址和 Size，如果状态为 EXCLUSIVE 并且地址和 Size 完全匹配，那么执行该写操作，返回写成功，并清除状态机回到 IDLE 态；否则如果状态或者地址/Size 有一项不满足条件，或者数据高速缓存未使能时，不执行该写操作，返回写失败，并清除状态机回到 IDLE 态。其他核的写操作在相同 cacheline 地址匹配局部监测器时，也会将状态机清回到 IDLE 态；本核的写操作或不同地址的独占访问不影响局部监测器。此外，在进程切换时需要清除局部监测器。

对于属性设置为不可高缓的页面，需要局部监测器和全局监测器共同作用实现独占式访问。LR 在执行过程中不仅要设置局部监测器还需要设置全局监测器；SC 在局部监测器检查通过后需要进一步检查全局监测器，只有当全局监测器也通过检查，才执行写操作、返回写成功，清除状态机；否则不执行写操作，返回写失败，清除状态机。其他核的写操作在地址匹配某个全局监测器时，会将该全局监测器的状态清回 IDLE 态。

在基于 C910 的系统中，推荐使用 LR 和 SC 指令实现原子锁操作。如果原子锁的地址属性是 cacheable（含 shared 和 non-shared），则不需要 SoC 系统做特别的设计。这是典型情况。如果原子锁的地址属性是 Non-cacheable/Device/Strongly Ordered，则需要用户在系统里（e.g. Slave 端）集成 exclusive monitor 功能。使用其他方式，操作结果为 UNPREDICTABLE。

7.4　L2 Cache

7.4.1　L2 Cache 概要

L2 高速缓存的主要特征如下：

- 大小为 1MB，16 路组相联，缓存行大小为 64B；
- L2 Cache 与 L1 D-Cache 是严格的 Inclusive 关系。L2 cache 与 L1 I-Cache 是非严格的 Inclusive 关系；
- 物理地址索引，物理地址标记（PIPT）；
- 每次访问的最大宽度为 64B；
- 支持写回-写分配和写回-写不分配的写策略；
- 采用先进先出的替换策略；
- 可编程的 RAM 访问延时；
- 支持指令预取和 TLB 预取机制；
- 采用分块的流水线技术。

7.4.2　Cache 一致性

C910MP 二级高速缓存采用 MOESI 协议维护多个处理器核心数据高速缓存的一致性。MOESI 代表了每个缓存行在数据高速缓存上的 5 个状态，分别是：

- M：表示缓存行仅位于此数据高速缓存中，且被写脏；(UniqueDirty)
- O：表示缓存行可能位于多个数据高速缓存中，且被写脏；(ShareDirty)
- E：表示缓存行仅位于此数据高速缓存中，且是干净的；(UniqueClean)
- S：表示缓存行可能位于多个数据高速缓存中，且是干净的；(ShareClean)
- I：表示缓存行不在该数据高速缓存中。(Invalid)

7.4.3　组织形式

C910MP L2 高速缓存在组织形式上采用了分块的流水线架构，将访问地址离散在两个不同的块中，允许多个访问的并行处理，从而提高访问效率。

分块机制如 图 7.1 所示。

- TAG RAM 按照地址 PA[6] 分为两个标记子块，分别为 Tag bank0 和 Tag bank1，以支持同个时钟周期并行处理 2 个访问请求。
- 同样，DATA RAM 也按照地址 PA[6] 分为 2 个数据子块，分别为 Data bank0 和 Data bank1；对应每一个数据子块，又被进一步分为四个微块，每个微块的数据宽度为 128bit，从而实现并行获取一条缓存行的目的。

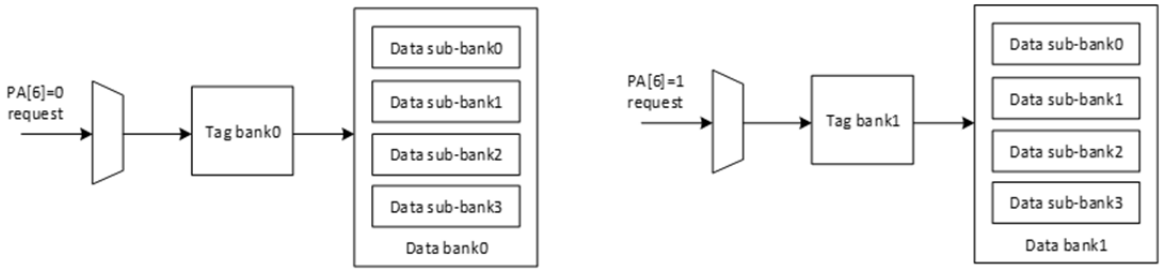


图 7.1: L2 Cache 组织形式

7.4.4　RAM 延时

由于 L2 Cache 较大，所以访问延迟较长，通常需要多个时钟周期才能完成访问。C910MP 提供了可配的访问延迟，在不同工艺下能够根据所用 RAM 的 setup time 和 latency 进行手动设置。配置详情如 表 7.2 所示。

表 7.2: RAM 访问延迟配置

配置选项	功能	说明
L2 TAG setup	L2 Cache Tag RAM setup： **1b0** 0 cycle. 默认值 **1b1** 1 cycle.	L2 Cache Tag RAM 的配置只影响 TAG RAM 的访问
L2 TAG latency	L2 Cache Tag RAM latency： **3b000** 1 cycle. 默认值 **3b001** 2 cycle. **3b010** 3 cycle. **3b011** 4 cycle. **3b1xx** 5 cycle.	
L2 DATA setup	L2 Cache Data RAM setup: **1b0** 0 cycle. 默认值 **1b1** 1 cycle.	L2 Cache Data RAM 的配置只影响 DATA RAM 的访问
L2 DATA latency	L2 Data RAM latency: **3b000** 1 cycle. 默认值 **3b001** 2 cycle. **3b010** 3 cycle. **3b011** 4 cycle. **3b100** 5 cycle. **3b101** 6 cycle. **3b110** 7 cycle. **3b111** 8 cycle.	

用户根据所用 RAM 的访问时长，配置 latency；setup 默认值为 0，当所用 RAM 的 setup time 较长，或者绕线较长时，可以选择将 setup 配置成 1。

在配置以上选项后，所得的访问周期数如 表 7.3 所示。

表 7.3: TAG RAM 有效访问延迟

TAG latency	**TAG RAM 有效访问延迟**	
/	**TAG setup = 0**	**TAG setup = 1**
000	1	2
001	2	3
010	3	4
011	4	5
1xx	5	5

表 7.4: DATA RAM 有效访问延迟

TAG latency	DATA RAM 有效访问延迟	
/	TAG setup = 0	TAG setup = 1
000	1	2
001	2	3
010	3	4
011	4	5
100	5	6
101	6	7
110	7	8
111	8	8

- L2 Tag latency 最大有效延迟为 5 周期；
- TAG setup 设置为 1 时增加 1 周期访问，访问 SRAM 之前会对 SRAM 输入端信号进行 flop；
- L2 Data latency 最大有效延迟为 8 周期；
- DATA setup 设置为 1 时增加 1 周期访问，访问 SRAM 之前会对 SRAM 输入端信号进行 flop。

7.5　内存加速访问

本小节集中描述 C910 L1/L2 cache 在内存加速访问方面的特性。

7.5.1　L1 I-Cache 指令预取

L1 指令高速缓存支持指令预取功能，通过配置隐式操作寄存器 MHINT.IPLD 实现。在当前缓存行访问缺失时，开启下一条连续缓存行的预取，并将预取结果缓存到预取缓冲器中。当指令访问命中预取缓冲器时，直接从缓冲器获取指令并回填指令高速缓存，从而降低了取指延迟。

指令预取要求预取的缓存行与当前访问的缓存行位于同一个页面，从而保证取指地址的安全。此外，读敏感的外设地址空间也禁止被分配到指令区空间。

7.5.2　L1 D-Cache 多通道数据预取

为了减少 DDR 等大内存的存储访问延时，C910 支持数据预取功能。通过检测数据高速缓存的缺失，匹配出固定的访问模式，然后硬件自动预取缓存行并回填 L1 数据高速缓存。

C910 最多支持 8 个通道的数据预取，并实现了连续预取和间隔预取（stride<=32 个缓存行）这 2 种不同的预取方式。

此外，还实现了正向预取和反向预取（即 stride 为负数），从而支持各种可能的访问模式。

在处理器执行数据高速缓存无效和清除操作时，停止数据预取功能。

用户可通过设置隐式寄存器 MHINT.DPLD，使能数据预取功能；并通过设置 MHINT.DPLD_DIS，决定一次预取的缓存行数量。

支持数据预取的指令如下：

- LB、LBU、LH、LHU、LW、LWU、LD
- FLW、FLD
- LRB、LRH、LRW、LRD、LRBU、LRHU、LRWU、LURB、LURH、LURW、LURD、LURBU、LURHU、LURWU、LBI、LHI、LWI、LDI、LBUI、LHUI、LWUI、LDD、LWD、LWUD

7.5.3　L1 自适应的写分配机制

C910 的 L1 实现了自适应的写分配机制。当处理器检测到连续的内存写入操作时，页面的写分配属性会自动关闭。

用户可通过设置隐式寄存器 MHINT.AMR 来开启 L1 自适应写分配。

当执行数据高缓的无效或者清除操作时，处理器的自适应写分配机制会被关闭；在高速缓存操作完成后，重新检测内存连续写入行为。

支持自适应写分配的指令如下：

- SB、SH、SW、SD
- FSW、FSD
- SRB、SRH、SRW、SRD、SURB、SURH、SURW、SURD、SBI、SHI、SWI、SDI、SDD、SWD

7.5.4　L2 预取机制

L2 高速缓存具有预取功能，支持取指令与 TLB 访问的预取工作。支持的特性如下：

- 软件可配的指令预取数量（0，1，2，3），所有预取都会回填 L2 高速缓存；
- TLB 预取数固定为 1；
- 预取机制以 4KB 页表为边界，对于预取时发生跨 4KB 边界地址会主动停止预取；
- 可以通过 MCCR2 对预取机制进行配置。

7.6　L1/L2 Cache 操作相关的指令和寄存器

在处理器复位后，指令和数据高速缓存会自动进行无效化操作，且默认关闭指令和数据高速缓存。

类似地，在处理器复位后，L2 高速缓存会自动进行无效化操作，完成操作后 L2 将自动开启，且不可关闭。值得注意的是，当 L1 高速缓存关闭时，L2 在缺失时不会发起回填操作。

7.6.1　L1 高速缓存扩展寄存器

C910 L1 高速缓存相关扩展寄存器，按功能主要分为：

- 高速缓存使能和模式配置：机器模式硬件配置寄存器 (mhcr) 可以实现对指令和数据高速缓存的开关以及写分配和写回模式的配置。超级用户模式硬件配置寄存器（shcr）是 mhcr 的映射，为只读寄存器。
- 脏表项清除和无效化操作：机器模式高速缓存操作寄存器（mcor）可以对指令和数据高速缓存进行脏表项和无效化操作。
- 高速缓存读操作：机器模式高速缓存访问指令寄存器（mcins）、高速缓存访问索引寄存器（mcindex）和高速缓存访问数据寄存器 0/1（mcdata0/1），通过这三个寄存器可以实现对指令和数据高速缓存的读操作。

具体控制寄存器说明可以参考*机器模式处理器控制和状态扩展寄存器组* 和*机器模式 Cache 访问扩展寄存器组*。

7.6.2　L2 高速缓存扩展寄存器

C910 L2 高速缓存相关扩展寄存器，按功能主要分为：

- L2 高速缓存使能和延时配置：机器模式 L2 高速缓存使能寄存器 (mccr2) 可以实现对 L2 高速缓存访问延时设置。
- L2 高速缓存读操作：机器模式高速缓存访问指令寄存器（mcins）、高速缓存访问索引寄存器（mcindex）和高速缓存访问数据寄存器 0/1（mcdata0/1），通过这三个寄存器可以实现对 L2 高速缓存的读操作。

具体相关控制寄存器的定义和说明可以参考*机器模式处理器控制和状态扩展寄存器组* 和*机器模式 Cache 访问扩展寄存器组*。

7.6.3　L1/L2 Cache 操作指令

C910 扩展了 L1/L2 高速缓存操作指令，包括按地址进行无效化、无效化全部、按地址清脏表项、清全部脏表项、按地址清脏表项并无效化和清全部脏表项并无效化，具体如 表 7.5 所示。

表 7.5: L1/L2 Cache 操作指令

ICACHE.IALL	ICACHE 无效所有表项
ICACHE.IALLS	ICACHE 广播无效所有表项
ICACHE.IPA	ICACHE 按物理地址无效表项
ICACHE.IVA	ICACHE 按虚拟地址无效表项
DCACHE.CALL	DCACHE 清全部脏表项
DCACHE.CIALL	DCACHE 清全部脏表项并无效
DCACHE.CIPA	DCACHE 按物理地址清脏表项并无效
DCACHE.CISW	DCACHE 按 set/way 清脏表项并无效
DCACHE.CIVA	DCACHE 按虚拟地址清脏表项并无效
DCACHE.CPA	DCACHE 按物理地址清脏表项
DCACHE.CPAL1	L1 DCACHE 按物理地址清脏表项
DCACHE.CVA	DCACHE 按虚拟地址清脏表项
DCACHE.CSW	DCACHE 按 set/way 清脏表项
DCACHE.CVAL1	L1 DCACHE 按虚拟地址清脏表项
DCACHE.IPA	DCACHE 按物理地址无效
DCACHE.ISW	DCACHE 按 set/way 无效
DCACHE.IVA	DCACHE 按虚拟地址无效
DCACHE.IALL	DCACHE 无效所有表项
L2CACHE.CALL	L2CACHE 清所有脏表项
L2CACHE.CIALL	L2CACHE 清所有脏表项并无效
L2CACHE.IALL	L2CACHE 全部表项无效

指令的具体说明请参考*附录 B-1 Cache 指令术语*

第八章　中断控制器

8.1　CLINT 中断控制器

C910 实现了处理器核局部中断控制器（以下简称 CLINT），是一个内存地址映射的模块，用于处理软件中断和计时器中断。

8.1.1　CLINT 寄存器地址映射

CLINT 中断控制器占据 64KB 内存空间。其高 13 位地址由 SoC 硬件集成决定，低 27 位地址映射如表 8.1 所示。所有寄存器仅支持字对齐的访问。

表 8.1: CLINT 寄存器存储器映射地址

地址	名称	类型	初始值	描述
0x4000000	MSIP0	读/写	0x00000000	核 0 机器模式软件中断 高位绑 0，bit[0] 有效
0x4000004	MSIP1	读/写	0x00000000	核 1 机器模式软件中断 配置寄存器高位绑 0，bit[0] 有效
Reserved	-	-	-	-
0x4004000	MTIMECMPL0	读/写	0xFFFFFFFF	核 0 机器模式时钟计时器 比较值寄存器 (低 32 位)
0x4004004	MTIMECMPH0	读/写	0xFFFFFFFF	核 0 机器模式时钟计时器 比较值寄存器 (高 32 位)
0x4004008	MTIMECMPL1	读/写	0xFFFFFFFF	核 1 机器模式时钟计时器 比较值寄存器 (低 32 位)
0x400400C	MTIMECMPH1	读/写	0xFFFFFFFF	核 1 机器模式时钟计时器 比较值寄存器 (高 32 位)
Reserved	-	-	-	-
0x400C000	SSIP0	读/写	0x00000000	核 0 超级用户模式软件中断 配置寄存器高位绑 0，bit[0] 有效
0x400C004	SSIP1	读/写	0x00000000	核 1 超级用户模式软件中断配置寄存器 高位绑 0，bit[0] 有效
Reserved	-	-	-	-
0x400D000	STIMECMPL0	读/写	0xFFFFFFFF	核 0 超级用户模式时钟计时器比较值寄存器 (低 32 位)
0x400D004	STIMECMPH0	读/写	0xFFFFFFFF	核 0 超级用户模式时钟计时器比较值寄存器 (高 32 位)
0x400D008	STIMECMPL1	读/写	0xFFFFFFFF	核 1 超级用户模式时钟计时器比较值寄存器 (低 32 位)
0x400D00C	STIMECMPH1	读/写	0xFFFFFFFF	核 1 超级用户模式时钟计时器比较值寄存器 (高 32 位)
Reserved	-	-	-	-

8.1.2　软件中断

CLINT 可用于生成软件中断。

软件中断通过配置地址映射的软件中断配置寄存器进行控制。其中机器模式软件中断由机器模式软件中断配置寄存器（MSIP）控制，超级用户模式软件中断由超级用户模式软件中断配置寄存器（SSIP）控制。

用户可通过将 xSIP 位置 1 的方式，产生软件中断；可通过将 xSIP 位清 0 的方式，清除软件中断。其中 CLINT 超级用户模式软件中断请求，仅在对应核使能 CLINTEE 位时有效。

机器模式下拥有修改访问所有软件中断相关寄存器的权限；超级用户模式下仅具有访问修改超级用户模

式软件中断配置寄存器（SSIP）的权限；普通用户模式没有权限。

两组寄存器的结构相同，其寄存器位分布和位定义如 图 8.1 所示。

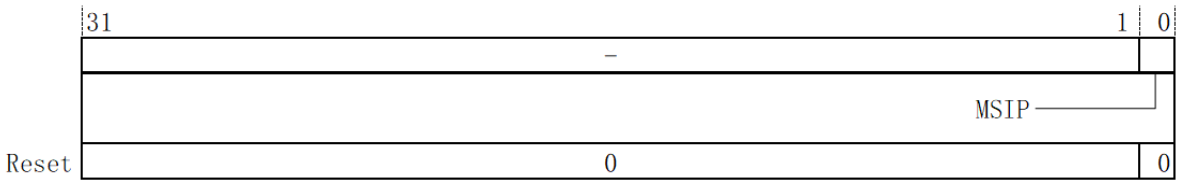


图 8.1: 机器模式软件中断配置寄存器（MSIP）

- **MSIP：机器模式软件中断等待位**

  该位表示机器模式软件中断的中断状态。

  – 当 MSIP 位置 1，当前有有效的机器模式软件中断请求。

  – 当 MSIP 位置 0，当前没有有效的机器模式软件中断请求。

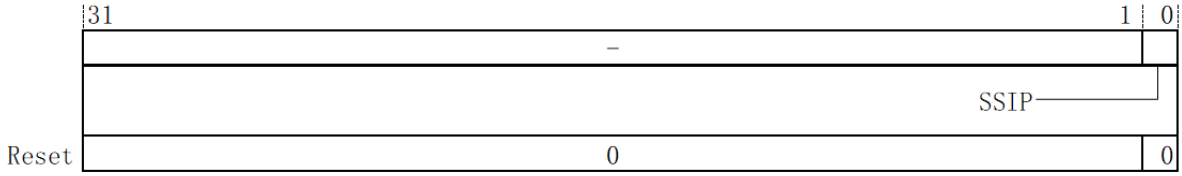


图 8.2: 超级用户模式软件中断配置寄存器（SSIP）

- **SSIP：超级用户模式软件中断等待位**

  该位表示超级用户模式软件中断的中断状态。

  – 当 SSIP 位置 1，当前有有效的超级用户软件中断请求。

  – 当 SSIP 位置 0，当前没有有效的超级用户软件中断请求。

8.1.3　计时器中断

CLINT 可用于生成计时器中断。

在多核系统中仅存在一个 64 位系统计时器 MTIME，要求在 always-on 电压域进行工作。系统计时器不可写，仅能通过 reset 清 0。系统计时器的当前值可通过读取 PMU 的 TIME 寄存器获取。系统计时器的主要作用是为多个核心提供统一的时间基准。

每一个核均有一组 64 位的机器模式时钟计时器比较值寄存器（*MTIMECMPL, MTIMECMPH*）和一组 64 位的超级用户模式时钟计时器比较值寄存器（*STIMECMPL, STIMECMPH*）。这些寄存器均可以通过地址字对齐访问的方式，分别修改其高 32 位或低 32 位。

CLINT 通过比较 {CMPH[31:0],CMPL[31:0]} 的值与系统计时器的当前值，确定是否产生计时器中断。当 {CMPH[31:0],CMPL[31:0]} 大于系统计时器的值时不产生中断；当 {CMPH[31:0],CMPL[31:0]} 小于或等于系统计时器的值时 CLINT 产生对应的计时器中断。软件可通过改写 MTIMECMP/STIMECMP 的值来清除对应的计时器中断。其中超级用户模式计时器中断请求，仅在对应核使能 CLINTEE 位时有效。

机器模式下拥有修改访问所有计时器中断相关寄存器的权限；超级用户模式下仅具有访问修改超级用户模式时钟计时器比较值寄存器（STIMECMPL, STIMECMPH）的权限；普通用户模式没有权限。

每组寄存器结构相同，其寄存器位分布和位定义如 图 8.3 所示。

	63 … 32	31 … 0
	MTIMECMPH	MTIMECMPL
Reset	0xffffffff	0xffffffff

图 8.3: 机器模式计时器中断比较值寄存器（高/低）

- **MTIMECMPH/MTIMECMPL：机器模式计时器中断比较值寄存器高位/低位**

  该寄存器存储了计时器比较值

  – MTIMECMPH：计时器比较值高 32 位；

  – MTIMECMPL：计时器比较值低 32 位；

	63 … 32	31 … 0
	STIMECMPH	STIMECMPL
Reset	0xffffffff	0xffffffff

图 8.4: 超级用户模式计时器中断比较值寄存器（高/低）

- **STIMECMPH/STIMECMPL：超级用户模式计时器中断比较值寄存器高位/低位**

  该寄存器存储了计时器比较值

  – STIMECMPH：计时器比较值高 32 位；

  – STIMECMPL：计时器比较值低 32 位；

8.2　PLIC 中断控制器

平台级别中断控制器（以下简称 PLIC），用于对外部中断源进行采样，优先级仲裁和分发。

在 PLIC 模型中每个核的机器模式和超级用户模式均可作为有效中断目标。

C910 实现的 PLIC 单元基本功能如下：

- 支持 2 个核/4 个中断目标的中断分发；
- 144 个中断源采样，支持电平中断，脉冲中断；

- 32 个级别的中断优先级；
- 每个中断目标的中断使能独立维护；
- 每个中断目标的中断阈值独立维护；
- PLIC 寄存器访问权限可配置。

8.2.1　中断的仲裁

在 PLIC 中，只有符合条件的中断源才会参与对某个中断目标的仲裁。满足的条件如下：

- 中断源处于等待状态（IP = 1）
- 中断优先级大于 0。
- 对于该中断目标的使能位打开。

当 PLIC 中对某个中断目标有多个中断处于 Pending 状态时，PLIC 仲裁出优先级最高的中断。C910 的 PLIC 实现中，机器模式中断优先级始终高于超级用户模式中断。当模式相同的情况下，优先级配置寄存器的值越大，优先级越高，优先级为 0 的中断无效；如多个中断拥有相同的优先级，则 ID 较小的优先处理。

PLIC 会将仲裁结果以中断 ID 的形式更新入对应中断目标的中断响应/完成寄存器。

8.2.2　中断的请求与响应

当 PLIC 对特定中断目标存在有效中断请求，且优先级大于该中断目标的中断阈值时，会向该中断目标发起中断请求。当该中断目标收到中断请求，且可响应该中断请求时，需要向 PLIC 发送中断响应消息。

中断响应机制如下：

- 中断目标向其对应的中断响应/完成寄存器发起一个读操作。该读操作将返回一个 ID，表示当前 PLIC 仲裁出的中断 ID。中断目标根据所获得的 ID 进行下一步处理。如果获得的中断 ID 为 0，表示没有有效中断请求，中断目标结束中断处理。
- 当 PLIC 收到中断目标发起的读操作，且返回相应 ID 后，会将该 ID 对应的中断源 IP 位清 0，且在中断完成之前屏蔽该中断源的后续采样。

8.2.3　中断的完成

当中断目标完成中断处理后，需要向 PLIC 发送中断完成消息。中断完成机制如下：

- 中断目标向中断响应/完成寄存器发起写操作，写操作的值为本次完成的中断 ID。如果中断类型为电平中断，则发起上述写操作之前还需清除外部中断源。
- PLIC 收到该中断完成请求后，不更新中断响应/完成寄存器，解除 ID 对应的中断源采样屏蔽，结束整个中断处理过程。

8.2.4 PLIC 寄存器地址映射

PLIC 中断控制器占据 64MB 内存空间。其高 13 位地址由 SoC 硬件集成决定，低 27 位地址映射如 表 8.2 所示。所有寄存器仅支持地址字对齐的访问。（*PLIC 的寄存器要通过字访问指令（Load word 指令）访问，访问结果放在 64 位 GPR 的低 32 位。*）

注：不在开源版 C910 支持范围内的寄存器，属性为“保留”。

表 8.2: PLIC 地址映射

地址	名称	类型	初始值	描述
0x0000000	-	-	-	-
0x0000004	PLIC_PRIO1	R/W	0x0	中断源 1 ~ 1023 优先级配置寄存器
0x0000008	PLIC_PRIO2	R/W	0X0	
0x000000C	PLIC_PRIO3	R/W	0x0	
...	...	...	...	
0x0000FFC	PLIC_PRIO1023	R/W	0x0	
0x0001000	PLIC_IP0	R/W	0x0	1 ~ 31 号中断 中断等待寄存器
0x0001004	PLIC_IP1	R/W	0x0	32 ~ 63 号中断 中断等待寄存器
...	...	...	...	...
0x000107C	PLIC_IP31	R/W	0x0	992 ~ 1023 号中断 中断等待寄存器
Reserved	-	-	-	-
0x0002000	PLIC_H0_MIE0	R/W	0x0	核 0 1 ~ 31 号 机器模式中断使能寄存器
0x0002004	PLIC_H0_MIE1	R/W	0x0	核 0 32 ~ 63 机器模式中断使能寄存器
...	...	...	...	...
0x000207C	PLIC_H0_MIE31	R/W	0x0	核 0 992 ~ 1023 号 机器模式中断使能寄存器
0x0002080	PLIC_H0_SIE0	R/W	0x0	核 0 1 ~ 31 号 超级用户模式 中断使能寄存器
0x0002084	PLIC_H0_SIE1	R/W	0x0	核 0 32 ~ 63 号 超级用户模式 中断使能寄存器
...	...	...	...	...
0x00020FC	PLIC_H0_SIE31	R/W	0x0	核 0 992 ~ 1023 号 超级用户模式中断使能寄存器

下页继续

表 8.2 – 续上页

地址	名称	类型	初始值	描述
0x0002100	PLIC_H1_MIE0	R/W	0x0	核 1 1 ~ 31 号 机器模式中断使能寄存器
0x0002104	PLIC_H1_MIE1	R/W	0x0	核 1 32 ~ 63 号 机器模式中断使能寄存器
...	...	...	...	...
0x000217C	PLIC_H1_MIE31	R/W	0x0	核 1 992 ~ 1023 号 机器模式中断使能寄存器
0x0002180	PLIC_H1_SIE0	R/W	0x0	核 1 1 ~ 31 号 超级用户模式 中断使能寄存器
0x0002184	PLIC_H1_SIE1	R/W	0x0	核 1 32 ~ 63 号 超级用户模式 中断使能寄存器
...	...	...	...	...
0x00021FC	PLIC_H1_SIE31	R/W	0x0	核 1 992 ~ 1023 号 超级用户模式中断使能寄存器
Reserved	-	-	-	-
0x01FFFFC	PLIC_PER	R/W	0x0	PLIC 权限控制寄存器
0x0200000	PLIC_H0_MTH	R/W	0x0	核 0 机器模式中断 阈值寄存器
0x0200004	PLIC_H0_MCLAIM	R/W	0x0	核 0 机器模式中断 响应/完成寄存器
Reserved	-	-	-	-
0x0201000	PLIC_H0_STH	R/W	0x0	核 0 超级用户模式中断 阈值寄存器
0x0201004	PLIC_H0_SCLAIM	R/W	0x0	核 0 超级用户模式中断 响应/完成寄存器
Reserved	-	-	-	-
0x0202000	PLIC_H1_MTH	R/W	0x0	核 1 机器模式中断 阈值寄存器
0x0202004	PLIC_H1_MCLAIM	R/W	0x0	核 1 机器模式中断 响应/完成寄存器
Reserved	-	-	-	-
0x0203000	PLIC_H1_STH	R/W	0x0	核 1 超级用户模式中断 阈值寄存器
0x0203004	PLIC_H1_SCLAIM	R/W	0x0	核 1 超级用户模式中断 响应/完成寄存器
Reserved	-	-	-	-

8.2.5　中断优先级配置寄存器（PLIC_PRIO）

该寄存器设定中断源的优先级。寄存器读写权限参考权限控制寄存器（PLIC_PER）描述。寄存器位分布和位定义如 图 8.5 所示。

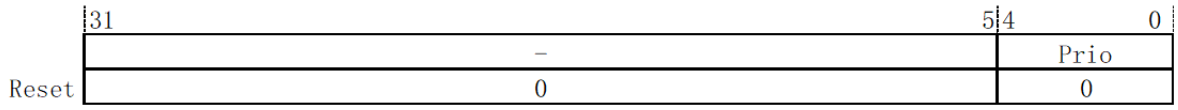


图 8.5: 中断优先级配置寄存器 (PLIC_PRIO)

- **PRIO：中断优先级**

  优先级配置寄存器低 5 位可写，支持 32 个不同级别的优先级。其中优先级设置为 0 表示该中断无效。

  机器模式中断优先级无条件高于超级用户模式中断。当模式相同时，优先级 1 为最低优先级，优先级 31 为最高。当优先级相同时，进一步比较中断源 ID，ID 较小的有高优先级。

8.2.6　中断等待寄存器（PLIC_IP）

每一个中断源的等待状态都可以通过读取中断等待寄存器中的信息获取。对于中断 ID 为 N 的中断，其中断信息存储于 PLIC_IP x（x=N/32）寄存器中的 IP y 上（y = N mod 32）。其中 PLIC_IP0 寄存器的第 0 位固定绑 0。寄存器读写权限参考权限控制寄存器（PLIC_PER）描述。寄存器位分布和位定义如 图 8.6 所示。

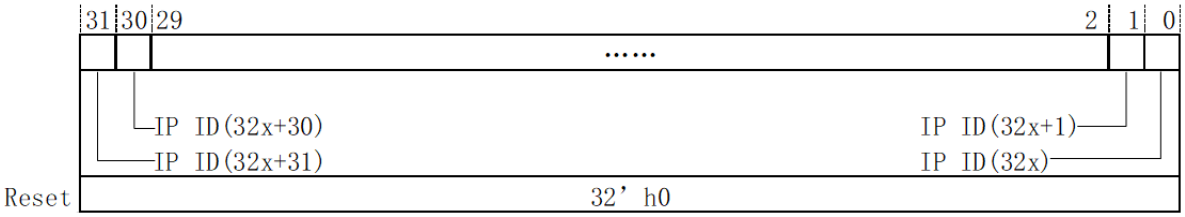


图 8.6: PLIC_IP x 中断等待寄存器 (PLIC_IP)

- **IP：中断等待状态**

  该位表示对应中断源的中断等待状态。

  – 当 IP 位为 1 时，表示当前该外部中断源存在等待响应的中断。该位可通过内存存储指令置 1。在对应中断源采样逻辑采样到有效电平或脉冲中断后也会将该位置 1。

  – 当 IP 位为 0 时，表示当前该外部中断源没有等待响应的中断。该位可通过内存存储指令清 0。当中断被响应后，PLIC 会将对应 IP 位清除。

8.2.7　中断使能寄存器（PLIC_IE）

每个中断目标对每个中断源均有一个中断使能位，可用于使能对应的中断。其中机器模式中断使能寄存器用于使能机器模式外部中断，超级用户模式中断使能寄存器用于使能超级用户模式外部中断。

对于中断 ID 为 N 的中断，其中断使能信息存储于 PLIC_IE x（x=N/32）寄存器中的 IE y 上（y = N mod 32）。其中 ID0 对应的 IE 位固定绑 0。寄存器读写权限参考权限控制寄存器（PLIC_PER）描述。

寄存器位分布和位定义如 图 8.7 所示。

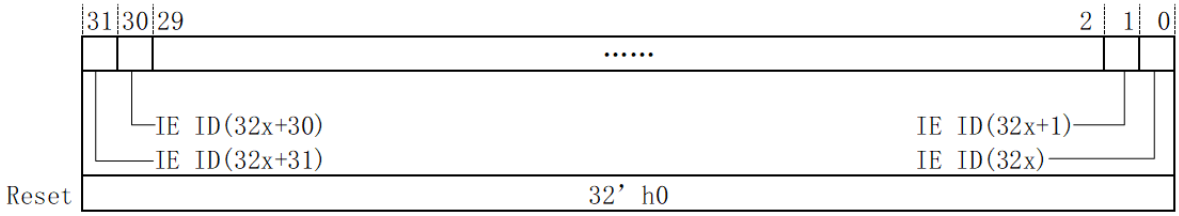


图 8.7: PLIC_IE x 中断使能寄存器 (PLIC_IE)

- **IE 中断使能：**

  该位表示对应中断源的中断使能状态。

  – 当 IE 位为 1 时，表示中断对该目标使能。

  – 当 IE 位为 0 时，表示中断对该目标屏蔽。

8.2.8　PLIC 权限控制寄存器（PLIC_PER）

PLIC 权限控制寄存器用于控制超级用户模式对 PLIC 部分寄存器的访问权限。

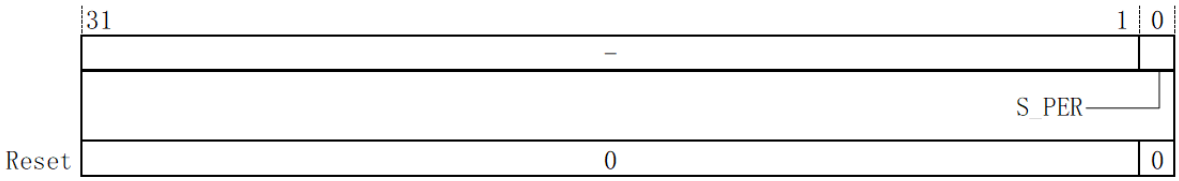


图 8.8: PLIC 权限控制寄存器 (PLIC_PER)

- **S_PER 访问权限控制位：**

  在 S_PER 为 0 时，仅机器模式拥有访问 PLIC 所有寄存器的权限。超级用户模式没有 PLIC 权限控制寄存器，中断优先级配置寄存器，中断等待寄存器和中断使能寄存器的访问权限，仅能访问超级用户模式中断阈值寄存器和超级用户模式中断响应/完成寄存器。普通用户模式没有任何 PLIC 寄存器的访问权限。

  在 S_PER 为 1 时，机器模式拥有所有权限。超级用户模式拥有除 PLIC 权限控制寄存器以外的所有 PLIC 寄存器权限。普通用户模式没有任何 PLIC 寄存器的访问权限。

8.2.9　中断阈值寄存器（PLIC_TH）

每一个中断目标均有一个对应的中断阈值寄存器。仅有优先级大于中断阈值的有效中断才会向中断目标发起中断请求。寄存器读写权限参考权限控制寄存器（PLIC_PER）描述。

寄存器位分布和位定义如 图 8.9 所示。

	31 … 5	4 … 0
	–	PRIOTHRESHOLD
Reset	0	0

图 8.9: 中断阈值寄存器 (PLIC_TH)

- **PRIOTHRESHOLD 优先级阈值：**

  指示当前中断目标的中断阈值。阈值配置为 0，表示允许所有中断。

8.2.10　中断响应/完成寄存器（PLIC_CLAIM）

每一个中断目标均有一个对应的中断响应/完成寄存器。该寄存器在 PLIC 完成仲裁时更新，更新值为 PLIC 本次仲裁结果的中断 ID。寄存器读写权限参考权限控制寄存器（PLIC_PER）描述。

寄存器位分布和位定义如 图 8.10 所示。

	31 … 10	9 … 0
	–	CLAIM_ID
Reset	0	0

图 8.10: 中断响应/完成寄存器 (PLIC_CLAIM)

- **CLAIM_ID 中断请求 ID：**

  对该寄存器的读操作：返回寄存器当前存储的 ID 值。该读操作表示对应 ID 的中断已开始处理。PLIC 开始中断响应处理。

  对该寄存器的写操作：表示写入值对应 ID 的中断已完成处理，该写操作不会更新中断响应/完成寄存器。PLIC 开始中断完成处理。

8.3　多核中断

下面简要说明两个常见的多核中断应用场景。

8.3.1　多个核心同时处理外部中断

在 PLIC 的模型下，允许把一个中断源同时映射到多个核心。当该中断源产生中断请求时，它相对于多个核心同时处于 Pending 状态。由于核心运行状态的不同，各个核心会先后响应这个中断并读取 CLAIM 寄存器以获得中断 ID。PLIC 的设计能够保证：只有第一个读取 CLAIM 寄存器的核心能够获得真正的 ID，而其他核心得到的是无效的 ID（即 ID=0），从而不作处理。因此，这个中断只会被处理一次。

将一个中断同时映射到多个核心，可以缩短总体中断响应时间（多个核心中的任何一个都可能处理这个中断），同时多占用一部分处理器资源（得到无效 ID 的核心白白消耗了带宽）。

8.3.2　核间发送软件中断

在 CLINT 的编程模型中，软件中断有专门的寄存器，分别是：

- M 态软件中断：MSIP0 MSIP1
- S 态软件中断：SSIP0 SSIP1

上述寄存器的地址对于所有的核心都是统一而且可见的，因此每个核心只要针对上述寄存器执行写操作，即可实现向任意核心（包括自己）发送软件中断的功能。

第九章　总线接口

9.1　AXI 主设备接口

C910MP 的主设备接口支持 AMBA 4.0 AXI 协议。请参考 *AMBA 规格说明—AMBA® AXI™ and ACE™ Protocol Specification*。

9.1.1　AXI 主设备接口的特点

AXI 主设备接口负责 C910 和 AXI 系统总线之间的地址控制和数据传输，其基本特点包括：

- 支持 AMBA 4.0 AXI 总线协议；
- 支持 128 位总线宽度；
- 支持系统时钟与 CPU 主时钟的不同频率比；
- 所有的输出信号 flopped out、输入信号 flopped 以获得较好的时序。

9.1.2　主设备接口的 Outstanding 能力

本小节列出了 C910 AXI 主设备接口的 Outstanding 能力，具体如下：

表 9.1: AXI 主设备接口 Outstanding 能力

参数	数值	说明
Read Issuing Capability	8n+28 n= 核心数量	每个核心最多发出 8 个 Non-cacheable 和 Device 读请求。全局最多 28 个 Cacheable 读请求。
Write Issuing Capability	8n+32 n= 核心数量	每个核心最多发出 8 个 Non-cacheable 和 Device 写请求。全局最多 32 个 Cacheable 写请求。

表 9.2: AXI 主设备接口 ARID 编码

ARID[7:0]	适用场景	每个 ID 的 Outstanding
{2’ b10, 6’ b??????}	Cacheable 读请求	每个 ID 无 outstanding，所有 cacheable 读请求 outstanding 总共 28 个。
{1’ b0, 2’ b(coreid), 5’ h18}	Non-cacheable weak-ordered 读请求	所有 Non-cacheable 读请求 outstanding 共 31 个。
{1’ b0, 2’ b(coreid), 5’ h1d}	Non-cacheable strong-ordered 读请求	

表 9.3: AXI 主设备接口 AWID 编码

AWID[7:0]	适用场景	每个 ID 的 Outstanding
{3’ b111, 5’ b?????}	Cacheable 写请求	每个 ID 无 outstanding，所有 cacheable 写请求 outstanding 总共 32 个。
{4’ b0000, 4’ b????}	Non-cacheable weak-ordered 写请求	每个 ID 无 outstanding，所有 Non-cacheable weak-ordered 写请求 outstanding 总共 16 个。
{1’ b0, 2’ b(coreid), 5’ h1d}	Non-cacheable strong-ordered 写请求	Non-cacheable strong-ordered 写请求 outstanding 为 31 个。

注意：上述 ARID/AWID 的编码可能随着处理器版本的演进而发生变化，因此，SoC 的集成不应该依赖于特定的 ID 值，而应该遵从 AXI 协议的通用规则。

9.1.3　支持的传输类型

主设备接口支持的传输特性如下：

- BURST 支持 INCR 和 WRAP 传输，其它突发类型均不支持；
- LEN 支持传输长度为 1 或者 4，其他传输长度均不支持；
- 支持独占式访问；
- SIZE 支持四字、双字、字、半字和字节传输，其它传输大小不支持；
- 支持读和写操作。

注意：C910 的主设备接口只实现了全部 AXI 传输的一个子集。但是，SoC 的集成不应该依赖于特定的传输类型，而应该遵从 AXI 协议的通用规则。

9.1.4　支持的响应类型

主设备接口接收从设备的响应类型为：

- OKAY

- EXOKAY
- SLVERR
- DECERR

9.1.5　不同总线响应下的行为

总线上出现不同总线响应时 CPU 的行为如 表 9.4 。

表 9.4: 总线异常处理

RRESP/BRESP	结果
OKAY	普通传输访问成功，或 exclusive 传输访问失败；读传输 exclusive 访问失败代表总线不支持 exclusive 传输，产生访问错误异常，写传输 exclusive 访问失败仅代表抢锁失败，不会返回异常
EXOKAY	exclusive 访问成功；
SLVERR/DECERR	访问出错，读传输产生访问错误异常，写传输忽略此错误；

9.1.6　AXI 主设备接口信号

AXI4.0 主设备的接口信号为 表 9.5 中所有信号。

表 9.5: AXI 协议通道接口信号

信号名	I/O	Reset	定义
读地址通道相关接口			
biu_pad_arid[7:0]	O	0	读请求地址 ID
biu_pad_araddr[39:0]	O	0	读请求地址
biu_pad_arlen[1:0]	O	0	读请求 burst 长度 00：1 个 transfer 11：4 个 transfer
biu_pad_arsize[2:0]	O	0	读请求每拍数据位宽 000：1 byte 001：2 bytes 010：4 bytes 011：8 bytes 100：16 bytes
biu_pad_arburst[1:0]	O	0	读请求对应的传输类型： 01：INCR 10：WRAP

下页继续

表 9.5 – 续上页

信号名	I/O	Reset	定义
biu_pad_arlock	O	0	读请求对应的访问方式： 0：normal access 1：exclusive access
biu_pad_arcache[3:0]	O	0	读请求对应的 memory 访问类型： 0000：Device No n-bufferable (strong order) 0001：Device Bufferable (strong order) 0011：Normal Non-cacheable Bufferable（weak order） 1111：Cacheable
biu_pad_arprot[2:0]	O	0	读请求的保护类型： 0 | 1 [2]: Data | Instruction [1]: Secure | N on-Secure，固定为 1； [0]: User | Privileged
biu_pad_aruser[2:0]	O	0	读请求用户定义信号： [2]: L2 Cache 预取请求 [1]:machine mode 请求 [0]:mmu 回填请求
biu_pad_arvalid	O	0	读地址通道有效信号
pad_biu_arready	I	-	读地址通道 slave ready 信号
读数据通道相关接口			
pad_biu_rid[7:0]	I	-	读请求数据 ID
pad_biu_rdata[127:0]	I	-	读请求数据
pad_biu_rresp[3:0]	I	-	读请求的 response 信息 [1:0]：00：OKAY 01：EXOKAY 10：SLVERR 11：DECERR [2]：PASSDIRTY [3]：ISSHARED
pad_biu_rlast	I	-	读请求的最后一拍数据
pad_biu_rvalid	I	-	读请求数据的有效信号
biu_pad_rready	O	1	读数据通道的 ready 信息
写地址通道相关接口			
biu_pad_awid[7:0]	O	0	写请求地址 ID

下页继续

表 9.5 – 续上页

信号名	I/O	Reset	定义
biu_pad_awaddr[39:0]	O	0	写请求地址
biu_pad_awlen[1:0]	O	0	写请求 burst 长度 00：1 拍 11：4 拍
biu_pad_awsize[2:0]	O	0	写请求每拍数据位宽 000：1 byte 001：2 bytes 010：4 bytes 011：8 bytes 100：16 bytes
biu_pad_awburst[1:0]	O	0	写请求对应的传输类型： 01：INCR 10：WRAP
biu_pad_awlock	O	0	写请求对应的访问方式： 0：normal access 1：exclusive access
biu_pad_awcache[3:0]	O	0	写请求对应的 memory 访问类型： 0000：Device No n-bufferable (strong order) 0001：Device Bufferable (strong order) 0011：Normal Non-cacheable Bufferable（weak order） 0111：Write-back No-allocate 1111：Write-back Cacheable
biu_pad_awprot[2:0]	O	0	写请求的保护类型： 0 | 1 [2]: Data | Instruction [1]: Secure | N on-Secure，固定为 1； [0]: User | Privileged
biu_pad_awvalid	O	0	写地址通道有效信号
pad_biu_arready	I	-	写地址通道 slave ready 信号
写数据通道相关接口			
biu_pad_wdata[127:0]	O	0	写请求数据
biu_pad_wstrb[15:0]	O	0	数据中有效数据域
biu_pad_wlast	O	0	最后一笔数据
biu_pad_wvalid	O	0	写数据通道有效信号

下页继续

表 9.5 – 续上页

信号名	I/O	Reset	定义
biu_pad_wready	I	-	写数据通道 slave ready 信号
写应答通道相关接口			
pad_biu_bid[7:0]	I	-	写应答的 ID
pad_biu_bresp[1:0]	I	-	写应答信息 写请求的 response 信息 [1:0]：00：OKAY 01：EXOKAY 10：SLVERR 11：DECERR
pad_biu_bvalid	I	-	写应答通道的 valid 信号
biu_pad_bready	O	1	写应答通道的 ready 信号

第十章　调试

10.1　Debug 单元的功能

调试接口是软件与处理器交互的通道。用户可以通过调试接口获取 CPU 的寄存器以及存储器内容等信息，包括其他的片上设备信息。此外，程序下载等操作也可以通过调试接口完成。

C910MP 支持兼容 IEEE-1149.1 标准的 JTAG 通信协议（通称 JTAG5），可以同已有的 JTAG 部件或独立的 JTAG 控制器集成在一起。

调试接口的主要特性如下：

- 使用标准的 JTAG 接口进行调试；
- 支持同步调试和异步调试，保证在极端恶劣情况下使处理器进入调试模式；
- 支持软断点；
- 可以设置多个内存断点；
- 检查和设置 CPU 寄存器的值；
- 检查和改变内存值；
- 可进行指令单步执行或多步执行；
- 快速下载程序；
- 可在 CPU 复位之后进入调试模式。

C910 的调试工作是由调试软件、调试代理服务程序、调试器和调试接口一起配合完成的。调试接口在整个 CPU 调试环境中的位置如 图 10.1 所示。其中，调试软件和调试代理服务程序通过网络互联，调试代理服务程序与调试器通过 USB 连接，调试器与 CPU 的调试接口以 JTAG 模式通信。

10.2　Debug 单元与 CPU Core 的连接

C910MP 采用多核单端口的调试框架，通过一个共享的 JTAG 接口访问各个 CORE 的辅助调试单元（HAD），触发 CORE 进出调试模式和访问处理器资源。通过在 JTAG 接口中设置 HACR 的 CORESEL 域的方式指定目标 CORE，然后再配置目标 CORE 的 HAD 寄存器。

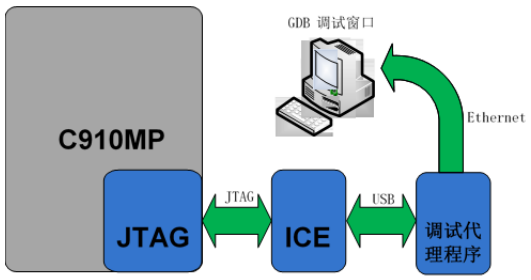


图 10.1: 调试接口在整个 CPU 调试环境中的位置

在多核调试场景下，当某一个 CORE 进入调试模式时，需要将其他某个或者多个核都拉入调试模式；而当某个 CORE 退出调试模式时，也需要将其他某个或者多个核都拉出调试模式。因此，C910MP 设计了一个集中的事件传输模块（ETM），用于多核之间调试事件（进入调试和退出调试）的传输。当 C910 核心在接收到 ICE 发送的调试命令产生调试进入或者调试退出事件时，同时将该事件发送到 ETM，由 ETM 将该事件转发到其他的 CORE，实现多个 CORE 同步进入和退出调试的目的。C910 多核调试框架如 图 10.2 所示（双核的情况）。

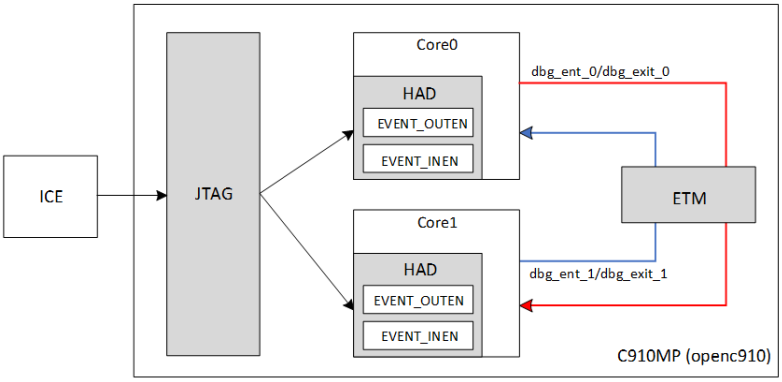


图 10.2: 多核调试整体框架

- **多核系统进入调试的场景：**

  某个 CORE 进入调试模式时，会产生一个向外输出的 DBG_ENT 信号，该 CORE 的 EVENT_OUTEN 寄存器控制该 DBG 信号是否能够传输到 ETM 模块。当 EVENT_OUTEN 寄存器使能时，该 DBG 信号经过 ETM 模块传输给其他 CORE。此时其他 CORE 是否能够被拉入调试状态，取决于各自的 EVENT_INEN 寄存器。

- **多核系统退出调试的场景：**

  某个 CORE 退出调试模式时，会产生一个向外输出的 DBG_EXIT 信号，该 CORE 的 EVENT_OUTEN 寄存器控制该 DBG_EXIT 信号是否能够传输到 ETM 模块。当 EVENT_OUTEN 寄存器使能时，该 DBG_EXIT 信号经过 ETM 模块传输给其他 CORE。此时其他 CORE 是否能够被拉出调试状态，取决于各自的 EVENT_INEN 寄存器。

10.3 Debug 接口信号

调试模块与外部的接口主要是与 JTAG 相关的接口信号和调试相关的接口信号。表 10.1 列出了与调试相关的接口信号。

表 10.1: 调试模块与外部的接口信号

信号名	方向
corex_pad_halted	输出
pad_corex_dbgrq_b	输入
corex_pad_jdb_pm[1:0]	输出
pad_corex_dbg_mask	输入
pad_had_jtg_tclk	输入
pad_had_jtg_trst_b	输入
pad_had_jtg_tdi	输入
had_pad_jtg_tdo	输出
had_pad_jtg_tdo_en	输出
pad_had_jtg_tms	输入

corex_pad_halted

高电平表示对应的 CORE 处于调试模式中。

pad_corex_dbgrq_b

同步进入调试模式请求信号，低电平有效。该信号是 CORE 的外部输入信号，在 CORE 中被系统时钟同步之后输入到 HAD 中，HAD 使用该信号欲让 CORE 同步进入调试模式。拉低该信号与设置 HAD 寄存器 HCR 的 DR 位有相同的作用。

corex_pad_jdb_pm[1:0]

corex_pad_jdb_pm[1:0] 信号指示对应 CORE 当前工作模式，可以通过该信号确定 CPU 是否已进入调试模式。具体如 表 10.2 所示。

表 10.2: PM 指示的当前 CPU 状态

had_pad_jdb_pm[1:0]	说明
00	普通模式
01	低功耗模式
10	调试模式
11	保留

pad_corex_dbg_mask

调试请求屏蔽信号，由 SoC 驱动，用来屏蔽针对 COREx 的调试请求。该信号需要在下电流程中置高。

pad_had_jtg_tclk

JTAG 时钟信号。该信号为外部输入信号，一般为调试器产生的时钟信号，要保证该时钟信号的频率低于 CPU 时钟信号的频率 1/2 才能保证调试模块与 CORE 之间的正常工作。

pad_had_jtg_trst_b

pad_had_jtg_trst_b 信号为 JTAG 复位信号，可以复位 TAP 状态机及其他相关控制信号。

JTAG5 相关信号

- pad_had_jtg_tdi 信号为 HAD 端 JTAG 串行输入信号，HAD 端在 JTAG 时钟信号 tclk 的上升沿对其采样，而外部调试器在 JTAG 时钟的下降沿设置该信号；
- pad_had_jtg_tms 信号为 JTAG 模式选择信号，该信号由调试器发出，用于控制 HAD 中 TAP 状态机的运作。
- had_pad_jtg_tdo 信号为 HAD 端 JTAG 串行输出信号，HAD 端在 JTAG 时钟信号 tclk 的下降沿对其设置，而外部调试器在 JTAG 时钟的上升沿对其采样；
- had_pad_jtg_tdo_en 信号表示 had_pad_jtg_tdo 信号有效，通常外部调试器监视该信号以确定 had_pad_jtg_tdo 信号是否有效，调试器也可以不观察该信号而通过调试器内部的 TAP 状态机的状态以决定对 had_pad_jtg_tdo 信号的采样。该信号主要用作在实现多个 TAP 状态机时，确定是哪个 JTAG 的输出。

第十一章　功耗管理

C910 支持各种功耗管理功能，包括支持多个 power domain，支持单个 core 下电，支持 cluster 下电，支持通过外部硬件接口清空 L2 cache 等。本章将详细介绍 C910 的功耗管理特性。

11.1　Power Domain

C910 最多可以划分为 3 个 Power Domain，分别是：

- 每个核心是一个 Power Domain，包括核心的计算单元、控制逻辑和 Cache RAM；
- L2 子系统（也称为“顶层”）是一个 Power Domain，包括 CIU、L2C、Debug、PLIC、CLINT 和 SYSIO 等子模块。

11.2　低功耗模式概要

C910 支持下列几种低功耗模式：

- 正常模式：各个核心与 L2 都处于正常运行的状态。
- 核心 WFI 模式：个别核心处于 WFI 模式。
- 单个核心下电：个别核心处于下电状态。
- Cluster 下电：整个 cluster，包括所有核心和 L2，都处于下电状态。

11.3　核心 WFI 流程

核心执行 WFI 低功耗指令，即可进入 WFI 模式，同时输出信号 core(x)_pad_lpmd_b[1:0]=2’b00，表示该核心已经进入 WFI 模式。此时，L2 子系统将在 cluster 内部关闭该核心的全局 ICG。

当发生下列事件时，核心会被唤醒并退出 WFI 模式：

- 复位；
- 中断请求：PLIC 或者 CLINT 发送的外部中断、软件中断或者 timer 中断请求。
- 调试请求。

当发生下列事件时，核心会被临时唤醒以处理该事件，处理完毕重新进入低功耗模式，但整个过程不会退出 WFI 模式：

- Snoop 请求：其他核心发送过来的 Snoop 请求。

11.4　单个核心下电流程

系统可通过关断核心电源的方式完全关断核心的静态功耗。核心电源关断流程如下：

C910 被关断核心执行的操作：

- 通知 SoC 即将执行单个核心下电流程，具体方式取决于 SoC 的设计。
- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器的中断使能位。如果下电流程执行在 M 态，则关闭 MSTATUS 和 MIE 的中断使能；如果下电流程执行在 S 态，则关闭 SSTATUS 和 SIE 的中断使能。
- 关闭数据预取。
- 核心执行 D-Cache INV&CLR ALL，将 dirty line 写回 L2 Cache。
- 核心关闭 D-Cache（注意：在清 cache 和关 cache 之间不能有 store 指令）。
- 关闭核心的 SMPEN 位，屏蔽对该核心的 snoop 请求。
- 核心执行 fence iorw, iorw 指令。
- 核心执行低功耗指令 WFI，核心进入低功耗模式。

系统执行的操作：

- 系统检测到核心的低功耗输出信号 core(x)_pad_lpmd_b 有效。
- 系统将 pad_core(x)_dbg_mask 置高，以屏蔽针对待下电核心的调试请求。
- 激活待下电核心的输出信号钳位。
- 系统拉低待下电核心的复位信号 pad_core(x)_rst_b。
- 关闭核心的电源。

核心在断电的状态下，只有通过复位才能重新启动。核心的重新上电流程如下：

- 系统检测到特定的事件，决定对核心重新上电（也可以称为“唤醒”）。
- 系统设置被唤醒核心的复位地址。
- 拉低核心的复位信号。
- 打开电源，保持复位信号不释放。
- 释放核心的输出信号钳位。
- 释放核心复位的信号。

- 被唤醒核心执行初始化程序，开启 SMPEN 位，执行 MMU、DCACHE 使能等初始化操作。

11.5　Cluster 下电流程（硬件清空 L2）

首先，确保 Cluster 内除了主核以外，其余核心的电源均已关闭。这里的“主核”是指最后一个下电的核心，它可以是 2 个核心中的任意一个。

主核执行的操作：

- 通知 SoC 即将执行 Cluster 下电流程。具体方式取决于 SoC 的设计。
- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器的中断使能位。
- 关闭数据预取。
- 执行 D-Cache INV&CLR ALL 操作。
- 关闭 D-Cache（注意：在清 cache 和关 cache 之间不能有 store 指令）。
- 关闭核心的 SMPEN 位。
- 执行 fence iorw，iorw 指令。
- 执行低功耗指令 WFI，进入低功耗模式。

系统执行的操作：

- 系统检测到主核的低功耗输出信号 core(x)_pad_lpmd_b 有效。
- 系统将 pad_core(x)_dbg_mask 置高，以屏蔽针对主核的调试请求。
- 激活主核的输出信号钳位。
- 系统拉低主核的复位信号 pad_core(x)_rst_b。
- 关闭主核的电源。
- 系统拉高 pad_cpu_l2cache_flush_req，开始清空 L2 cache 的过程。
- 等待 C910 返回 cpu_pad_l2cache_flush_done=1。
- 系统拉低 pad_cpu_l2cache_flush_req。（随后 C910 将拉低 cpu_pad_l2cache_flush_done）
- 等待 C910 返回 cpu_pad_no_op=1。
- 激活顶层的输出信号钳位。
- 拉低 L2 复位信号 pad_cpu_rst_b。
- 关闭顶层电源。

Cluster 通过复位重新上电，流程如下：

- 拉低 Cluster 内所有核心和顶层的复位信号。

- 打开电源，保持复位信号不释放，pll 稳定。
- 释放各核心和顶层的输出信号钳位。
- 释放各核心和顶层的复位信号。
- 执行复位异常服务程序，恢复 CPU 状态。

11.6　Cluster 下电流程（软件清空 L2）

首先，确保 Cluster 内除了主核以外，其余核心的电源均已关闭。在这个场景下，推荐 SoC 对“主核”与“副核”的角色加以区分，“主核”可以由 Core 0 担当。

主核执行的操作：

- 通知 SoC 即将执行 Cluster 下电流程。具体方式取决于 SoC 的设计。
- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器的中断使能位。
- 关闭数据预取。
- 执行 D-Cache INV&CLR ALL 操作。
- 关闭 D-Cache（注意：在清 cache 和关 cache 之间不能有 store 指令）。
- 关闭核心的 SMPEN 位。
- 执行 fence iorw，iorw 指令。
- 执行 CLR & INV L2 Cache 操作。
- 执行 fence iorw，iorw 指令。
- 执行低功耗指令 WFI，进入低功耗模式。

系统执行的操作：

- 系统检测到主核的低功耗输出信号 core(x)_pad_lpmd_b 有效。
- 系统将 pad_core(x)_dbg_mask 置高，以屏蔽针对主核的调试请求。
- 激活主核的输出信号钳位。
- 系统拉低主核的复位信号 pad_core(x)_rst_b。
- 关闭主核的电源。
- 等待 C910 返回 cpu_pad_no_op=1。
- 激活顶层的输出信号钳位。
- 拉低 L2 复位信号 pad_cpu_rst_b。
- 关闭顶层电源。

11.7　简化场景：Cluster 整体下电流程（硬件清空 L2）

在某些系统中，SoC 设计人员可能采取比较简单的方式来划分 power domain，即：把 C910 Cluster（2 核心 +L2）整体作为一个 power domain，整体下电，不区分各个单独的核心。在这样的场景下，Cluster 下电流程（硬件清空 L2）可以采取如下的步骤：

系统执行的操作：

- 通知 SoC 即将进入 Cluster 整体下电流程，具体方式取决于 SoC 的设计。

核心执行的操作：（此时，不必区分“主核”与“副核”，他们的流程相同。）

- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器中断使能位。
- 关闭数据预取。
- 核心执行 INV&CLR D-Cache ALL，将 dirty line 写回 L2 Cache。
- 核心关闭 D-Cache（在清 cache 和关 cache 之间不能有 store 指令）。
- 核心关闭 SMPEN 位，屏蔽对该核心的 snoop 请求。
- 核心执行 fence iorw, iorw 指令。
- 核心执行 WFI。

系统执行的操作：

- 等待全部 core(x)_pad_lpmd_b[1:0]==2’b00，即：所有 CPU 都已进入低功耗状态。
- 将全部 pad_core(x)_dbg_mask 置高，屏蔽调试请求。
- 拉高 pad_cpu_l2cache_flush_req，开始硬件清空 L2 cache 的过程。
- 等待 C910 返回 cpu_pad_l2cache_flush_done=1，代表 L2 已经清空。
- 拉低 pad_cpu_l2cache_flush_req。（随后 C910 将拉低 cpu_pad_l2cache_flush_done。）
- 等待 cpu_pad_no_op==1’b1，即 L2 进入空闲状态。（此时所有的 CPU 也仍然处于低功耗状态）
- 激活 cluster 输出信号的 clamp。
- Assert 全部复位信号。
- 将整个 cluster 掉电。

11.8　简化场景：Cluster 整体下电流程（软件清空 L2）

类似地，本小节的内容也适用于简化的 power domain 划分，即：把 C910 Cluster（2 核心 +L2）整体作为一个 power domain，整体下电，不区分各个单独的核心。在这样的场景下，Cluster 下电流程（软件清空 L2）可以采取如下的步骤：

系统执行的操作：

- 通知 SoC 即将进入 Cluster 整体下电流程，具体方式取决于 SoC 的设计。

“副核”（e.g. CPU1）执行的操作：

- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器中断使能位。
- 关闭数据预取。
- 核心执行 INV&CLR D-Cache ALL，将 dirty line 写回 L2 Cache。
- 核心关闭 D-Cache（在清 cache 和关 cache 之间不能有 store 指令）。
- 核心关闭 SMPEN 位，屏蔽对该核心的 snoop 请求。
- 核心执行 fence iorw, iorw 指令。
- 核心执行 WFI。

“主核”（e.g. CPU0）执行的操作：

- 屏蔽核心的所有中断请求，包括外部中断、软中断和 timer 中断，关闭 MSTATUS/SSTATUS 寄存器中断使能位（mie、sie）和 MIE/SIE 寄存器中断使能位。
- 关闭数据预取。
- 核心执行 INV&CLR D-Cache ALL，将 dirty line 写回 L2 Cache。
- 核心关闭 D-Cache（在清 cache 和关 cache 之间不能有 store 指令）。
- 核心关闭 SMPEN 位，屏蔽对该核心的 snoop 请求。
- 核心执行 fence iorw, iorw 指令。
- 主核等待所有副核都进入 WFI。（具体方式取决于 SoC 的设计）
- 核心执行 INV&CLR L2 Cache ALL ，清空 L2 cache。
- 核心执行 fence iorw, iorw 指令。
- 核心执行 WFI。

系统执行的操作：

- 等待全部 core(x)_pad_lpmd_b[1:0]==2’b00，且 cpu_pad_no_op==1’b1，即：所有 CPU 都已进入低功耗状态，且 L2 进入空闲状态。
- 将全部 pad_core(x)_dbg_mask 置高，屏蔽调试请求。
- 激活 cluster 输出信号的 clamp。
- Assert 全部复位信号。
- 将整个 cluster 掉电。

11.9　低功耗相关的编程模型和接口信号

11.9.1　编程模型

机器模式侦听使能寄存器（MSMPR）

该寄存器宽度为 64 位，只有 bit[0] 有定义（=SMPEN），默认值为 0。该寄存器的功能是：控制核心能否接受侦听（snoop）请求。

- MSMPR.SMPEN = 0 时，核心不能够处理侦听请求，顶层屏蔽发送侦听请求给核心。
- MSMPR.SMPEN = 1 时，核心能够处理侦听请求，顶层发送侦听请求给核心。

核心下电前，要求设置核心对应的 SMPEN=0；核心上电后，软件在打开 D-Cache 和 MMU 之前需要设置 SMPEN=1。核心在正常工作模式下，必须保持 SMPEN=1。

机器模式复位向量基址寄存器（MRVBR）

每个核心有各自的 MRVBR，用于决定该核心的复位启动地址。MRVBR 的访问权限为“MRO”。各个核心的 MRVBR 初始值由硬件信号 pad_core(x)_rvba[39:0] 决定。注意：pad_core(x)_rvba[0] 要绑为 0。

11.9.2　接口信号

复位信号的控制

C910 top level 有 3 个复位信号：pad_core0_rst_b，pad_core1_rst_b 和 pad_cpu_rst_b。SoC 可以通过上述信号控制 Core 0，Core 1 和 L2 的复位。

C910 与 SoC 功耗管理单元的沟通主要通过以下信号实现：

- core(x)_pad_lpmd_b：可以用来判断一个核心是否处于 WFI 模式。2’b11 代表正常模式；2’b00 代表 WFI 模式。
- cpu_pad_no_op：L2 Cache 空闲指示信号。当所有核心都进入低功耗模式，且 L2 Cache 完成了全部传输时，该信号有效（高电平有效）。
- pad_cpu_l2cache_flush_req 与 cpu_pad_l2cache_flush_done：这一组信号用于在 SoC 的控制下清空 L2 cache，应用场景是 Cluster 下电。“req” 信号由 SoC 驱动，“done” 信号由 C910 驱动。操作顺序是：SoC 首先拉高并保持“req”以开始清空 L2 的过程；C910 完成清空 L2 的动作以后，返回“done”=1；SoC 拉低“req”信号；随后 C910 拉低“done”信号。

第十二章　性能监测单元

12.1　PMU 简介

C910 性能监测单元（PMU）遵从 RISC-V 标准，用于统计程序运行中的软件信息和部分硬件信息，供软件开发人员进行程序优化。

性能监测单元统计的软硬件信息分为以下几类：

- 运行时钟数和时间
- 指令信息统计
- 处理器关键部件信息统计

12.2　PMU 的编程模型

12.2.1　PMU 的基本用法

PMU 的基本用法如下：

- 通过 mcountinhibit 寄存器禁止所有事件的计数。
- 把各个 PMU counter 的当前值清零，包括：mcycle，minstret，mhpmcounter3 ~ mhpmcounter31。
- 为各个 PMU counter 配置对应的事件。C910 的“事件-计数器”对应关系是固定的，因此必须按照固定的 pattern 进行配置。例如，mhpmevent3 必须写入 0x1，表示 mhpmcounter3 固定针对 0x1 号事件（L1 ICache 访问次数）计数；mhpmevent4 必须写入 0x2，表示 mhpmcounter4 固定针对 0x2 号事件（L1 ICache Miss 次数）计数；以此类推。
- 访问权限授权：通过 mcounteren 决定 S 态是否可以访问 PMU counter；通过 scounteren 决定 U 态是否可以访问 PMU counter。
- 通过 mcountinhibit 寄存器解除禁止，开始计数。

具体例子可参考*PMU 设置示例*

12.2.2　PMU 事件溢出中断

C910 自定义实现了机器模式事件上溢出标注寄存器（MCOUNTEROF）和机器模式事件中断使能寄存器 (MCOUNTERINTEN)，其寄存器功能和读写权限等详见附录 c-1 机器模式控制寄存器。机器模式事件上溢寄存器（MCOUNTEROF）中各个 bit 与事件计数器一一对应，分别用于表示各事件计数是否发生了上溢出。机器模式事件中断使能寄存器 (MCOUNTERINTEN) 中各个 bit 与事件计数器一一对应，分别用于控制对应事件计数器发生上溢出情况下是否会发起中断请求。

由性能检测单元发起的溢出中断统一中断向量号为 17。中断的使能及处理过程同普通私有中断，详见异常与中断 。

12.3　PMU 相关的控制寄存器

12.3.1　机器模式计数器访问授权寄存器（mcounteren）

机器模式计数器访问授权寄存器（mcounteren），用于授权超级用户模式是否可以访问用户模式计数器。

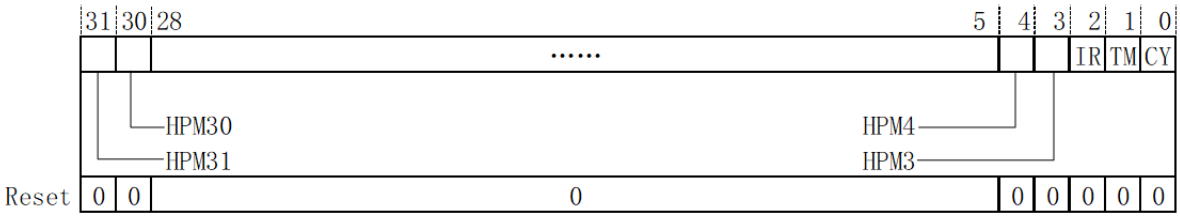


图 12.1: 机器模式计数器访问授权寄存器（MCOUNTEREN）

表 12.1: 机器模式计数器访问授权寄存器说明

位	读写	名称	介绍
31：3	读写	HPMn	hpmcounter$n$ 寄存器 S-mode 访问位： 0：S-mode 访问 h pmcounter$n$ 将发生非法指令异常 1：S-mode 能正常访问 hpmcounter$n$
2	读写	IR	minstret 寄存器 S-mode 访问位： 0：S-mode 访问 minstret 寄存器将发生非法指令异常 1：S-mode 能正常访问 minstret 寄存器
1	读写	TM	time 寄存器 S-mode 访问位： 0：S-mo de 访问 time 寄存器将发生非法指令异常 1：当 mco unteren 对应位为 1，S-mode 能正常访问 time 寄存器，否则将发生非法指令异常
0	读写	CY	mcycle 寄存器 S-mode 访问位： 0：S-mod e 访问 cycle 寄存器将发生非法指令异常 1：S-mode 能正常访问 cycle 寄存器

12.3.2　超级用户模式计数器访问授权寄存器（scounteren）

超级用户模式计数器访问授权寄存器（scounteren），用于授权用户模式是否可以访问用户模式计数器。

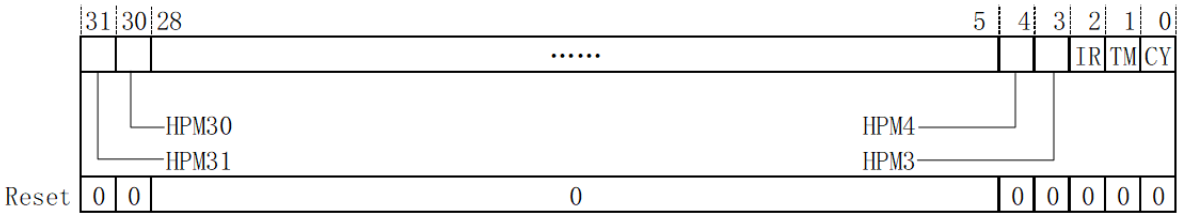


图 12.2: 超级用户模式计数器访问授权寄存器（SCOUNTEREN）

表 12.2: 超级用户模式计数器访问授权寄存器说明

位	读写	名称	介绍
31：3	读写	HPMn	hpmcounter$n$ 寄存器 U-mode 访问位： 0：U-mode 访问 h pmcounter$n$ 将发生非法指令异常 1：当 mcounteren 对应位为 1，则 U-mode 能正常访问 hpmcounter$n$，否则将发生非法指令异常
2	读写	IR	instret 寄存器 U-mode 访问位： 0：U-mode 访问 instret 寄存器将发生非法指令异常 1：当 mcount eren 对应位为 1，U-mode 能正常访问 ins tret 寄存器，否则将发生非法指令异常
1	读写	TM	time 寄存器 U-mode 访问位： 0：U-mo de 访问 time 寄存器将发生非法指令异常 1：当 mco unteren 对应位为 1，U-mode 能正常访问 time 寄存器，否则将发生非法指令异常
0	读写	CY	cycle 寄存器 U-mode 访问位： 0：U-mod e 访问 cycle 寄存器将发生非法指令异常 1：当 mcou nteren 对应位为 1，U-mode 能正常访问 c ycle 寄存器，否则将发生非法指令异常

12.3.3　机器模式计数禁止寄存器（mcountinhibit）

机器模式禁止计数寄存器（mcountinhibit），可以禁止机器模式计数器计数。在不需要性能分析的场景下，关闭计数器，可以降低处理器的功耗。

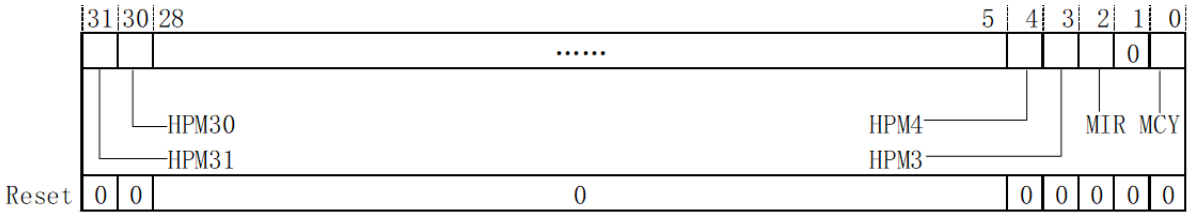


图 12.3: 机器模式计数禁止授权寄存器（MCOUNTINHIBIT）

表 12.3: 机器模式计数禁止授权寄存器寄存器说明

位	读写	名称	介绍
31：3	读写	MHPMn	mhpmcounter$n$ 寄存器禁止计数位： 0：正常计数 1：禁止计数
2	读写	MIR	minstret 寄存器禁止计数位： 0：正常计数 1：禁止计数
1	-	-	-
0	读写	MCY	mcycle 寄存器禁止计数位： 0：正常计数 1：禁止计数

12.3.4　超级用户写使能寄存器（mcounterwen）

超级用户态计数器写使能寄存器（MCOUNTERWEN），用于授权超级用户模式是否可以写超级用户模式事件计数器。该寄存器为机器模式拓展寄存器，寄存器具体描述参见*附录 C-1 机器模式控制寄存器* 。

12.3.5　性能监测事件选择寄存器

性能监测事件选择器（mhpmevent3-31），用于选择每个计数器对应的计数事件。C910 中，每个计数器对应一个事件，不能修改，因此每个事件选择器只能写入对应的事件号。只有当事件选择器写入对应事件的索引值，并通过 csrw 指令初始化对应事件计数器后，才能正常计数。

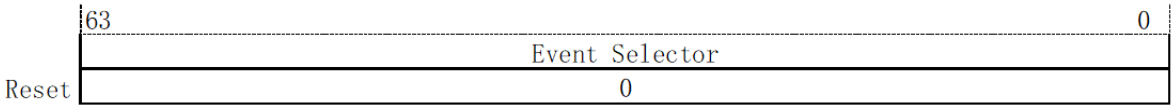


图 12.4: 机器模式性能监测事件选择寄存器（MHPMEVENT）

表 12.4 为机器模式性能监测事件选择寄存器说明。

表 12.4: 机器模式性能监测事件选择寄存器说明

位	读写	名称	介绍
63:0	读写	事件索引	性能监测事件索引： 0：没有事件； 0x1~0x1A：硬件实现的性能监测事件，具体见 表 12.5 ； >0x1A：硬件未定义的性能监测事件，为软件自定义事件使用。

表 12.5 为事件选择器和事件以及计数器之间的对应关系。

表 12.5: 计数器事件对应列表

事件选择器	索引	事件	计数器
mhpmevent3	0x1	L1 ICache Access Counter	mhpmcounter3
mhpmevent4	0x2	L1 ICache Miss Counter	mhpmcounter4
mhpmevent5	0x3	I-UTLB Miss Counter	mhpmcounter5
mhpmevent6	0x4	D-UTLB Miss Counter	mhpmcounter6
mhpmevent7	0x5	JTLB Miss Counter	mhpmcounter7
mhpmevent8	0x6	Conditional Branch Mispredict Counter	mhpmcounter8
mhpmevent9	0x7	reserved	mhpmcounter9
mhpmevent10	0x8	Indirect Branch Mispredict Counter	mhpmcounter10
mhpmevent11	0x9	Indirect Branch Instruction Counter	mhpmcounter11
mhpmevent12	0xA	LSU Spec Fail Counter	mhpmcounter12
mhpmevent13	0xB	Store Instruction Counter	mhpmcounter13
mhpmevent14	0xC	L1 DCache read access Counter	mhpmcounter14
mhpmevent15	0xD	L1 DCache read miss Counter	mhpmcounter15
mhpmevent16	0xE	L1 DCache write access Counter	mhpmcounter16
mhpmevent17	0xF	L1 DCache write miss Counter	mhpmcounter17
mhpmevent18	0x10	L2 Cache read access Counter	mhpmcounter18
mhpmevent19	0x11	L2 Cache read miss Counter	mhpmcounter19
mhpmevent20	0x12	L2 Cache write access Counter	mhpmcounter20
mhpmevent21	0x13	L2 Cache write miss Counter	mhpmcounter21
mhpmevent22	0x14	RF Launch Fail Counter	mhpmcounter22
mhpmevent23	0x15	RF Reg Launch Fail Counter	mhpmcounter23
mhpmevent24	0x16	RF Instruction Counter	mhpmcounter24
mhpmevent25	0x17	LSU Cross 4K Stall Counter	mhpmcounter25
mhpmevent26	0x18	LSU Other Stall Counter	mhpmcounter26
mhpmevent27	0x19	LSU SQ Discard Counter	mhpmcounter27
mhpmevent28	0x1A	LSU SQ Data Discard Counter	mhpmcounter28
mhpmevent29~31	>= 0x1B	暂未定义	mhpmcounter29~31

12.3.6　事件计数器

事件计数器有三组，分别为：机器模式事件计数器、用户模式事件计数器和 C910 扩展的超级用户模式事件计数器。具体如 表 12.6 所示。

表 12.6: 机器模式事件计数器列表

名称	索引	读写	初始值	介绍
MCYCLE	0xB00	MRW	0x0	cycle counter
MINSTRET	0xB02	MRW	0x0	instructions-retired counter
MHPMCOUNTER3	0xB03	MRW	0x0	performance-monitoring counter
MHPMCOUNTER4	0xB04	MRW	0x0	performance-monitoring counter
...	...	...	...	...
MHPMCOUNTER31	0xB1F	MRW	0x0	performance-monitoring counter

用户模式事件计数器列表如 表 12.7 所示。

表 12.7: 用户模式事件计数器列表

名称	索引	读写	初始值	介绍
CYCLE	0xC00	URO	0x0	cycle counter
TIME	0xC01	URO	0x0	timer
INSTRET	0xC02	URO	0x0	instructions-retired counter
HPMCOUNTER3	0xC03	URO	0x0	performance-monitoring counter
HPMCOUNTER4	0xC04	URO	0x0	performance-monitoring counter
...	...	...	...	...
HPMCOUNTER31	0xC1F	URO	0x0	performance-monitoring counter

表 12.8: 超级用户模式事件计数器列表

名称	索引	读写	初始值	介绍
SCYCLE	0x5E0	SRO	0x0	cycle counter
SINSTRET	0x5E2	SRO	0x0	instructions-retired counter
SHPMCOUNTER3	0x5E3	SRO	0x0	performance-monitoring counter
SHPMCOUNTER4	0x5E4	SRO	0x0	performance-monitoring counter
...	...	...	...	...
SHPMCOUNTER31	0x5FF	SRO	0x0	performance-monitoring counter

其中，用户模式的 CYCLE、INSTRET 和 HPMCOUNTERn 是对应机器模式事件计数器的只读映射，TIMER 计数器是 MTIME 寄存器的只读映射；超级用户模式的 SCYCLE、SINSTRET 和 SHPMCOUNTERn 是对应机器模式事件计数器的映射。

第十三章　程序示例

本章主要介绍多种程序示例，包含：MMU 设置示例、PMP 设置示例、高速缓存设置示例、多核启动示例、同步原语示例、PLIC 设置示例和 PMU 设置示例。

13.1　处理器最优性能配置

采用下列配置，可以发挥 C910 的最优性能：

- MHCR = 0x11ff
- MHINT = 0x6e30c
- MCCR2 = 0xe0000009（注：mccr2 包含 RAM 延时的设定，本例子中所有延时 =0。客户需要根据实际情况设定合适的 RAM 延时）
- MXSTATUS = 0x638000
- MSMPR = 0x1

```
# mhcr
  li x3, 0x11ff
  csrs mhcr,x3

#mhint
li x3, 0x6e30c
csrs mhint,x3

# mxstatus
li   x3, 0x638000
csrs mxstatus,x3

# msmpr
csrsi msmpr,0x1

# mccr2
```

（下页继续）

(续上页)

```
li x3, 0xe0000009
csrs mccr2,x3
```

13.2　MMU 设置示例

```
/*****************************************************************************

* Function: An example of setting C910MP MMU.
* Memory space: Virtual address <-> physical address.
*
* Pagesize 4K: vpn: {vpn2,vpn1,vpn0} <-> ppn: {ppn2,ppn1,ppn0}
* Pagesize 2M: vpn: {vpn2,vpn1} <-> ppn:{ppn2,ppn1}
* Pagesize 1G: vpn: {vnp2} <-> ppn: {ppn2}
*
*****************************************************************************/

  /*C910 will invalidate all MMU TLB entries automatically when reset*/
  /*You can use sfence.vma to invalid all MMU TLB entries if necessary*/
  sfence.vma x0, x0

  /* Pagesize 4K: vpn: {vpn2, vpn1, vpn0} <-> ppn: {ppn2, ppn1, ppn0}*/
  /* First-level page addr base: PPN (defined in satp)*/
  /* Second-level page addr base: BASE2 (self define)*/
  /* Third-level page addr base: BASE3 (self define)*/
  /* 1. Get first-level page addr base: PPN and vpn*/
  /* Get PPN*/
  csrr x3, satp
  li x4, 0xfffffffffff
  and x3, x3, x4

  /*2. Config first-level page*/
  /*First-level page addr: {PPN, vpn2, 3' b0}, first-level page pte:{ 44' b BASE2, 10' b1}
↪ */
  /*Get first-level page addr*/
  slli x3, x3, 12
  /*Get vpn2*/
  li x4, VPN
  li x5, 0x7fc0000
```

(下页继续)

(续上页)

```
  and x4, x4, x5
  srli x4, x4, 15
  and x5, x3, x4
  /*Store pte at first-level page addr*/
  li x6, {44' b BASE2, 10' b1}
  sd x6, 0(x5)

  /*3. Config second-level page*/
  /*Second-level page addr: {BASE2, vpn1, 3' b0}, second-level page pte:{ 44' b BASE3, 10'
b1} */
  /*Get second-level page addr*/
  /* VPN1*/
  li x4, VPN
  li x5, 0x3fe00
  and x4, x4, x5
  srli x4, x4, 9
  /*BASE2*/
  li x5, BASE2
  srli x5, x5, 12
  and x5, x5, x4
  /*Store pte at second-level page addr*
  li x6, {44' b BASE3, 10' b1}
  sd x6, 0(x5)
  /*4. Config third-level page*/
  /*Third-level page addr: {BASE3, vpn0, 3' b0}, third-level page pte:{
  theadflag, ppn2, ppn1, ppn0, 9' b flags,1' b1} */
  /*Get second-level page addr*/
  /* VPN0*/
  li x4, VPN
  li x5, 0x1ff
  and x4, x4, x5
  srli x4, x4, 3
  /*BASE3*/
  li x5, BASE3
  srli x5, x5, 12
  and x5, x5, x4
  /*Store pte at second-level page addr*/
  li x6, { theadflag, ppn2, ppn1, ppn0, 9' b flags, 1' b1}
  sd x6, 0(x5)
```

(下页继续)

(续上页)

```
/* Pagesize 2M: vpn: {vpn2, vpn1} <-> ppn: {ppn2, ppn1}*/
/*First-level page addr base: PPN (defined in satp)*/
/*Second-level page addr base: BASE2 (self define)*/

/*1. Get first-level page addr base: PPN and vpn*/
/* Get PPN*/
csrr x3, satp
li x4, 0xfffffffffff
and x3, x3, x4

/*2. Config first-level page*/
/*First-level page addr: {PPN, vpn2, 3' b0}, first-level page pte:{ 44' b
BASE2, 10' b1}*/
/*Get first-level page addr*/
slli x3, x3, 12
/*Get vpn2*/
li x4, VPN
li x5, 0x7fc0000
and x4, x4, x5
srli x4, x4, 15
and x5, x3, x4
/*Store pte at first-level page addr*/
li x6, {44' b BASE2, 10' b1}
sd x6, 0(x5)

/*3. Config second-level page*/
/*Second-level page addr: {BASE2, vpn1, 3' b0}, second-level page pte:{
theadflag, ppn2, ppn1, 9' b0, 9' b flags,1' b1} */
/*Get second-level page addr*/
/*VPN1*/
li x4, VPN
li x5, 0x3fe00
and x4, x4, x5
srli x4, x4, 9
/*BASE2*/
li x5, BASE2
srli x5, x5, 12
and x5, x5, x4
/*Store pte at second-level page addr*/
```

(下页继续)

(续上页)

```
li x6, { theadflag, ppn2, ppn1, 9' b0, 9' b flags,1' b1}
sd x6, 0(x5)

/* Pagesize 1G: vpn: {vpn2} <-> ppn: {ppn2}*/
/*First-level page addr base: PPN (defined in satp)*/
/*1. Get first-level page addr base: PPN and vpn*/
/* Get PPN*/
csrr x3, satp
li x4, 0xfffffffffff
and x3, x3, x4

/*2. Config first-level page*/
/*First-level page addr: {PPN, vpn2, 3' b0}, first-level page pte:{
theadflag, ppn2, 9' b0, 9' b0, 9' b flags,1' b1}*/
/*Get first-level page addr*/
slli x3, x3, 12
/*Get vpn2*/
li x4, VPN
li x5, 0x7fc0000
and x4, x4, x5
srli x4, x4, 15
and x5, x3, x4
/*Store pte at first-level page addr*/
li x6, { theadflag, ppn2, 9' b0, 9' b0, 9' b flags,1' b1}
sd x6, 0(x5)
```

13.3　PMP 设置示例

```
/*******************************************************************************
* Function: An example of setting C910MP PMP.
* 0x0 ~ 0xf0000000, TOR 模式，RWX
* 0xf0000000 ~ 0xf8000000, NAPOT 模式，RW
*0xfff73000 ~ 0xfff74000, NAPOT 模式，RW
*0xfffc0000 ~ 0xfffc2000, NAPOT 模式，RW
* 以上 4 片区域配置了不同的执行权限。另外，为了防止 CPU 在不同模式下投机执行到不支持的地址区
域，尤其是在默认拥有全部执行权限的机器模式下，需要对 PMP 进行相关配置。具体来说，就
是配置完需要执行权限的地址区域后，将剩余地址区域配置成无任何权限，如下例所示。
*******************************************************************************/
```

(下页继续)

(续上页)

```
# pmpaddr0,0x0 ~ 0xf0000000, TOR 模式，读写可执行权限
li x3, (0xf0000000 >> 2)
csrw pmpaddr0, x3
# pmpaddr1,0xf0000000 ~ 0xf8000000, NAPOT 模式，读写权限
li x3, ( 0xf0000000 >> 2 | (0x8000000-1) >> 3))
csrw pmpaddr1, x3
# pmpaddr2,0xfff73000 ~ 0xfff74000, NAPOT 模式，读写权限
li x3, ( 0xfff73000 >> 2 | (0x1000-1) >> 3))
csrw pmpaddr2, x3
# pmpaddr3,0xfffc0000 ~ 0xfffc2000, NAPOT 模式，读写权限
li x3, ( 0xfffc0000 >> 2 | (0x2000-1) >> 3))
csrw pmpaddr3, x3
# pmpaddr4,0xf0000000 ~ 0x100000000, NAPOT 模式，无任何权限
li x3, ( 0xf0000000 >> 2 | (0x10000000-1) >> 3))
csrw pmpaddr4, x3
# pmpaddr5,0x100000000 ~ 0xffffffffff, TOR 模式，无任何权限
li x3, (0xffffffffff >> 2)
csrw pmpaddr5, x3
# PMPCFG0，配置各表项执行权限/模式/lock 位，
lock 为 1 时，该表项在机器模式下才有效
li x3,0x88989b9b9b8f
csrw pmpcfg0, x3
# pmpaddr5,0x100000000 ~ 0xffffffffff, TOR 模式，0x100000000 <= addr <
0xffffffffff 时都会命中 pmpaddr5，但是 0xfffffff000 ~
0xffffffffff 地址区间无法命中 pmpaddr5（C910 中 PMP 的最小粒度为 4K），如果需要屏蔽 1T 空间
的最后一个 4K 空间，需要再配置一个 NAPOT 模式的表项。
```

13.4　高速缓存示例

13.4.1　高速缓存的开启示例

```
/*C910 will invalidate all I-cache automatically when reset*/
/*You can invalidate I-cache by yourself if necessary*/
/*Invalidate I-cache*/
li x3, 0x33
csrc mcor, x3
li x3, 0x11
```

(下页继续)

(续上页)

```
csrs mcor, x3
// You can also use icache instrucitons to replace the invalidate sequence
// if theadisaee is enabled.
//icache.iall
//sync.is

/*Enable I-cache*/
li x3, 0x1
csrs mhcr, x3

/*C910 will invalidate all D-cache automatically when reset*/
/*You can invalidate D-cache by yourself if necessary*/
/*Invalidate D-cache*/
li x3, 0x33
csrc mcor, x3
li x3, 0x12
csrs mcor, x3

// You can also use dcache instrucitons to replace the invalidate sequence
// if theadisaee is enabled.
// dcache.iall
// sync.is

/*Enable D-cache*/
li x3, 0x2
csrs mhcr, x3

/*C910 will invalidate all L2 cache automatically when reset*/
/*You can invalidate L2 by yourself if necessary*/
/*Invalidate L2-cache if theadisaee is enabled*/
l2cache.iall
sync.is

/*Enable L2-cache*/
li x3, 8
csrs mccr2, x3
```

13.4.2　指令高速缓存与数据高速缓存的同步示例

CPU0

```
sd x3,0(x4) // a new instruction defined in x3
            // is stored to program memory address defined in x4.
dcache.cval1 r0 // clean the new instrcution to the shared L2 cache.
sync.s          // ensure completion of clean operation.
                // the dcache clean is not necessarily if INSDE is not enabled.
icache.iva r0  // invalid icache according to shareable configuraiton.
sync.s/fence.i  // ensure completion in all CPUs.
sd x5,0(x6)    // set flag to signal operation completion.
sync.is
jr x4 // jmp to new code
```

CPU1~CPU3

```
WAIT_FINISH:
  ld x7,0(x6)
  bne x7,x5, WAIT_FINISH // wait CPU0 modification finish.
  sync.is
  jr x4                 // jmp to new code
```

13.4.3　TLB 与数据高速缓存的同步示例

CPU0

```
sd x4,0(x3) // update a new translation table entry
sync.is/fence.i // ensure completion of update operation.
sfence.vma x5,x0 // invalid the TLB by va
sync.is/fence.i // ensure completion of TLB invalidation and
                // synchronises context
```

13.5　同步原语示例

CPU0

```
li x1, 0x1
li x6, 0x0

ACQUIRE_LOCK:              // (x3) is the lock address. 0: Free; 1: Busy.
lr x4, 0(x3)               // Read lock
bnez x4, ACQUIRE_LOCK      // Try again if the lock is in use
```

(下页继续)

(续上页)

```
sc x5, x1, 0(x3)           // Attempt to store new value
bne x6, x5, ACQUIRE_LOCK   // Try again if fail
sync.s

...                        // Critical section code
```

CPU1

```
sync.s/fence.i      // Ensure all operations are observed before clearing the lock.
sd x0, 0(x3)        // Clear the lock.
```

13.6　PLIC 设置示例

```
//Init id 1 machine mode int for hart 0
/*1.set hart threshold if needed*/
li x3, (plic_base_addr + 0x200000) // h0 mthreshold addr
li x4, 0xa //threshold value
sw x4,0x0(x3) // set hart0 threshold as 0xa

/*2.set priority for int id 1*/
li x3, (plic_base_addr + 0x0) // int id 1 prio addr
li x4, 0x1f // prio value
sw x4,0x4(x3) // init id1 priority as 0x1f

/*3.enable m-mode int id1 to hart*/
li x3, (plic_base_addr + 0x2000) // h0 mie0 addr
li x4, 0x2
sw x4,0x0(x3) // enable int id1 to hart0

/*4.set ip or wait external int*/
/*following code set ip*/
li x3, (plic_base_addr + 0x1000) // h0 mthreshold addr
li x4, 0x2 // id 1 pending
sw x4, 0x0(x3) // set int id1 pending

/*5.core enters interrupt handler, read PLIC_CLAIM and get ID*/

/*6.core takes interrupt*/
```

(下页继续)

(续上页)

```
/*7.core needs to clear external interrupt source if LEVEL(not PULSE)
configured, then core writes ID to PLIC_CLAIM and exits interrupt*/
```

13.7　PMU 设置示例

```
/*1.inhibit counters counting*/
li x3, 0xffffffff
csrw mcountinhibit, x3

/*2.C910 will initial all pmu counters when reset*/
/*you can initial pmu counters manually if necessarily*/
csrw mcycle, x0
csrw minstret, x0
csrw mhpmcounter3, x0
……
csrw mhpmcounter31, x0

/*3.configure mhpmevent*/
li x3, 0x1
csrw mhpmevent3, x3 // mhpmcounter3 count event: L1 ICache Access Counter
li x3, 0x2
csrw mhpmevent4, x3 // mhpmcounter4 count event: L1 ICache Miss Counter
……
li x3, 0x13
csrw mhpmevent21, x3 // mhpmcounter21 count event: L2 Cache write miss Counter

/*4. configure mcounteren and scounteren*/
li x3, 0xffffffff
csrw mcounteren, x3 // enable super mode to read hpmcounter
li x3, 0xffffffff
csrw scounteren, x3 // enable user mode to read hpmcounter

/*5. enable counters to count when you want*/
csrw mcountinhibit, x0
```

第十四章　附录 A 标准指令术语

C910MP 实现了 RV64IMAFDC 指令集包，以下各章节按照不同指令集对每条指令做具体描述。

14.1　附录 A-1 I 指令术语

以下是对 C910 实现的 RISC-V I 指令集的具体描述，指令按英文字母顺序排列。

本节指令位宽默认为 32 位，但是系统在特定情况下会将某些指令汇编成 16 位的压缩指令，关于压缩指令具体描述请见*附录 A-6 C 指令术语* 。

14.1.1　ADD——有符号加法指令

语法：

add rd, rs1, rs2

操作：

rd ← rs1 + rs2

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	000	rd	0110011

14.1.2　ADDI——有符号立即数加法指令

语法：

addi rd, rs1, imm12

操作：

rd ← rs1 + sign_extend(imm12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	000	rd	0010011

14.1.3　ADDIW——低 32 位有符号立即数加法指令

语法：

addiw rd, rs1, imm12

操作：

tmp[31:0] ← rs1[31:0] + sign_extend(imm12)[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	000	rd	0011011

14.1.4　ADDW——低 32 位有符号加法指令

语法：

addw rd, rs1, rs2

操作：

tmp[31:0] ← rs1[31:0] + rs2[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	000	rd	0111011

14.1.5　AND——按位与指令

语法：

and rd, rs1, rs2

操作：

rd ← rs1 & rs2

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	111	rd	0110011

14.1.6　ANDI——立即数按位与指令

语法：

andi rd, rs1, imm12

操作：

rd ← rs1 & sign_extend(imm12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	111	rd	0010011

14.1.7　AUIPC——PC 高位立即数加法指令

语法：

auipc rd, imm20

操作：

rd ← current pc + sign_extend(imm20<<12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 12	11 7	6 0
imm20[19:0]	rd	0010111

14.1.8　BEQ——相等分支指令

语法：

beq rs1, rs2, label

操作：

if (rs1 == rs2)

　　next pc = current pc +sign_extend(imm12<<1)

else

　　next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

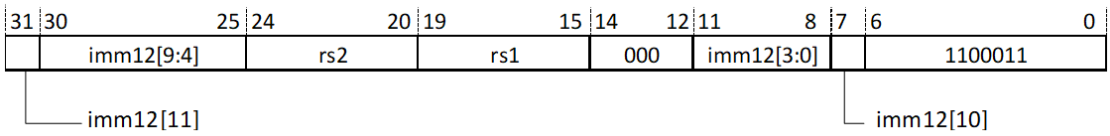
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.9　BGE——有符号大于等于分支指令

语法：

bge rs1, rs2, label

操作：

if (rs1 >= rs2)

　　next pc = current pc + sign_extend(imm12 <<1)

else

　　next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

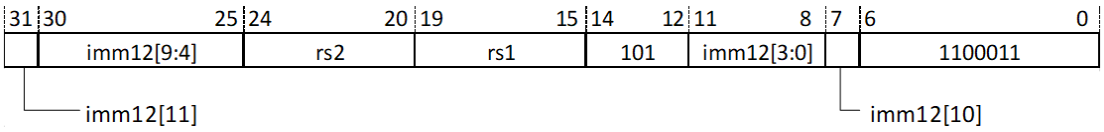
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.10　BGEU——无符号大于等于分支指令

语法：

bgeu rs1, rs2, label

操作：

if (rs1 >= rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

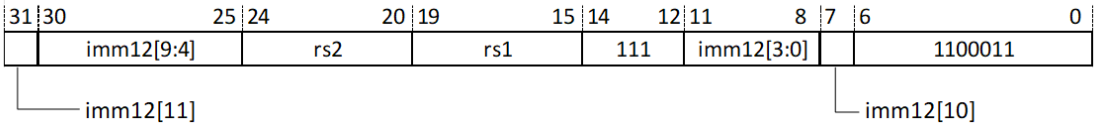
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.11　BLT——有符号小于分支指令

语法：

blt rs1, rs2, label

操作：

if (rs1 < rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

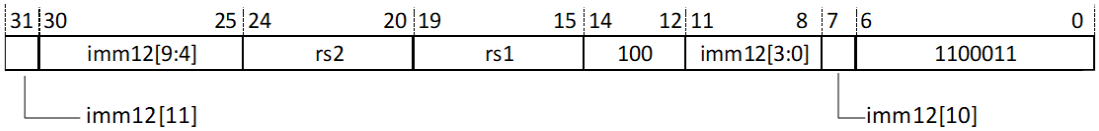
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.12　BLTU——无符号小于分支指令

语法：

bltu rs1, rs2, label

操作：

if (rs1 < rs2)

　　next pc = current pc + sign_extend(imm12<<1)

else

　　next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

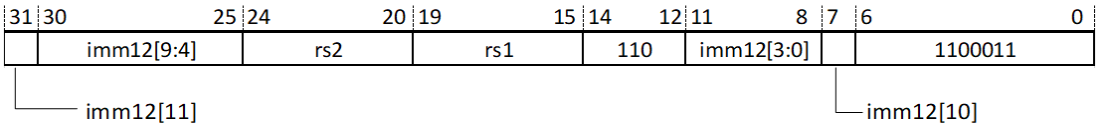
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.13　BNE——不等分支指令

语法：

bne rs1, rs2, label

操作：

if (rs1 != rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

执行权限：

M mode/S mode/U mode

异常：

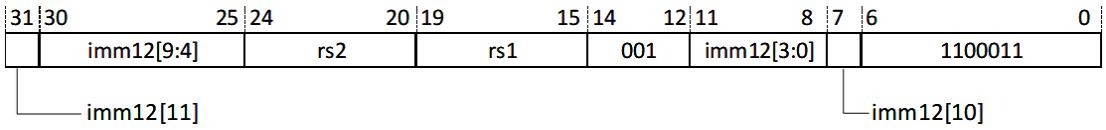
无

说明：

汇编器根据 label 算出 imm12

指令跳转范围为±4KB 地址空间

指令格式：



14.1.14　CSRRC——控制寄存器清零传送指令

语法：

csrrc rd, csr, rs1

操作：

rd ← csr

csr ← csr & (~rs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	rs1	011	rd	1110011

14.1.15　CSRRCI——控制寄存器立即数清零传送指令

语法：

csrrci rd, csr, imm5

操作：

rd ← csr

csr ← csr & ~zero_extend(imm5)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	imm5	111	rd	1110011

14.1.16　CSRRS——控制寄存器置位传送指令

语法：

csrrs rd, csr, rs1

操作：

rd ← csr

csr ← csr | rs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	rs1	010	rd	1110011

14.1.17　CSRRSI——控制寄存器立即数置位传送指令

语法：

csrrsi rd, csr, imm5

操作：

rd ← csr

csr ← csr | zero_extend(imm5)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	imm5	110	rd	1110011

14.1.18　CSRRW——控制寄存器读写传送指令

语法：

csrrw rd, csr, rs1

操作：

rd ← csr

csr ← rs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	rs1	001	rd	1110011

14.1.19　CSRRWI——控制寄存器立即数读写传送指令

语法：

csrrwi rd, csr, imm5

操作：

rd ← csr

csr[4:0] ← imm5

csr[63:5] ← csr[63:5]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

各权限下允许访问的控制寄存器不同，具体请参考控制寄存器章节。

当 rs1=x0 时，该指令不产生写操作，不会产生写行为引发的异常。

指令格式：

31 20	19 15	14 12	11 7	6 0
csr	imm5	101	rd	1110011

14.1.20　EBREAK——断点指令

语法：

ebreak

操作：

产生断点异常或者进入调试模式

执行权限：

M mode/S mode/U mode

异常：

断点异常

指令格式：

31 20	19 15	14 12	11 7	6 0
000000000001	00000	000	00000	1110011

14.1.21　ECALL——环境异常指令

语法：

ecall

操作：

产生环境异常

执行权限：

M mode/S mode/U mode

异常：

用户模式环境调用异常、超级用户模式环境调用异常、机器模式环境调用异常

指令格式：

31 20	19 15	14 12	11 7	6 0
000000000000	00000	000	00000	1110011

14.1.22　FENCE——存储同步指令

语法：

fence iorw, iorw

操作：

保证该指令前序所有读写存储器或外设指令比该指令后序所有读写存储器或外设指令更早被观察到。

执行权限：

M mode/S mode/U mode

异常：

无

说明：

pi=1, so=1，指令语法为 fence i,o，以此类推

指令格式：

31 28	27	26	25	24	23	22	21	20	19 15	14 12	11 7	6 0
0000	pi	po	pr	pw	si	so	sr	sw	00000	000	00000	0001111

14.1.23　FENCE.I——指令流同步指令

语法：

fence.i

操作：

清空 icache, 保证该指令前序所有数据访存结果能够被指令后的取指操作访问到。

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 28	27 24	23 20	19 15	14 12	11 7	6 0
0000	0000	0000	00000	001	00000	0001111

14.1.24　JAL——直接跳转子程序指令

语法：

jal rd, label

操作：

next pc ← current pc + sign_extend(imm20<<1)

rd ← currect pc + 4

执行权限：

M mode/S mode/U mode

异常：

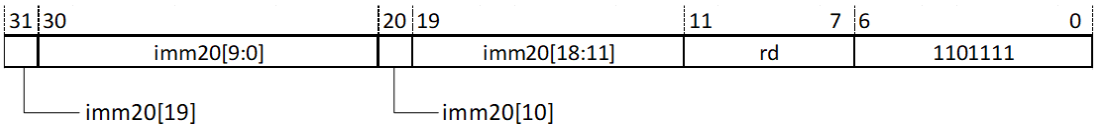
无

说明：

汇编器根据 label 算出 imm20

指令跳转范围为±1MB 地址空间

指令格式：



14.1.25　JALR——寄存器跳转子程序指令

语法：

jalr rd, rs1, imm12

操作：

next pc ← (rs1 + sign_extend(imm12)) & 64’ hfffffffffffffe

rd ← currect pc + 4

执行权限：

M mode/S mode/U mode

异常：

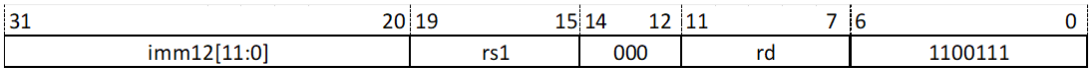
无

说明：

机器模式或者 MMU 关闭时，指令跳转范围为全部 1TB 地址空间

非机器模式且 MMU 打开时，指令跳转范围为全部 512GB 地址空间

指令格式：



14.1.26　LB——有符号扩展字节加载指令

语法：

lb rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← sign_extend(mem[address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	000	rd	0000011

14.1.27　LBU——无符号扩展字节加载指令

语法：

lbu rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← zero_extend(mem[address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	100	rd	0000011

14.1.28　LD——双字加载指令

语法：

ld rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← mem[(address+7):address]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	011	rd	0000011

14.1.29　LH——有符号扩展半字加载指令

语法：

lh rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← sign_extend(mem[(address+1):address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	001	rd	0000011

14.1.30　LHU——无符号扩展半字加载指令

语法：

lhu rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← zero_extend(mem[(address+1):address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	101	rd	0000011

14.1.31　LUI——高位立即数装载指令

语法：

lui rd, imm20

操作：

rd←sign_extend(imm20<<12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 12	11 7	6 0
imm20[19:0]	rd	0110111

14.1.32　LW——有符号扩展字加载指令

语法：

lw rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← sign_extend(mem[(address+3):address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	010	rd	0000011

14.1.33　LWU——无符号扩展字加载指令

语法：

lwu rd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

rd ← zero_extend(mem[(address+3):address])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	110	rd	0000011

14.1.34　MRET——机器模式异常返回指令

语法：

mret

操作：

next pc← mepc

mstatus.mie ←mstatus.mpie

mstatus.mpie ←1

执行权限：

M mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0011000	00010	00000	000	00000	1110011

14.1.35　OR——按位或指令

语法：

or rd, rs1, rs2

操作：

rd ← rs1 | rs2

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	110	rd	0110011

14.1.36　ORI——立即数按位或指令

语法：

ori rd, rs1, imm12

操作：

rd ← rs1 | sign_extend(imm12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	110	rd	0010011

14.1.37　SB——字节存储指令

语法：

sb rs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[:address] ← rs2[7:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	rs2	rs1	000	imm12[4:0]	0100011

14.1.38　SD——双字存储指令

语法：

sd rs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[(address+7):address] ← rs2

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	rs2	rs1	011	imm12[4:0]	0100011

14.1.39　SFENCE.VMA——虚拟内存同步指令

语法：

sfence.vma rs1,rs2

操作：

用于虚拟内存的无效和同步操作

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mstatus.tvm=1，在超级用户模式下执行该指令引起非法指令异常。

rs1：虚拟地址，rs2：asid

- rs1=x0，rs2=x0 时，无效 TLB 中所有的表项。
- rs1!=x0，rs2=x0 时，无效 TLB 中所有命中 rs1 虚拟地址的表项。
- rs1=x0，rs2!=x0 时，无效 TB 中所有命中 rs2 进程号的表项。
- rs1!=x0，rs2!=x0 时，无效 TLB 中所有命中 rs1 虚拟地址和 rs2 进程号的表项。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001001	rs2	rs1	000	00000	1110011

14.1.40　SH——半字存储指令

语法：

sh rs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[(address+1):address] ← rs2[15:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	rs2	rs1	001	imm12[4:0]	0100011

14.1.41　SLL——逻辑左移指令

语法：

sll rd, rs1, rs2

操作：

rd← rs1 << rs2[5:0]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	001	rd	0110011

14.1.42　SLLI——立即数逻辑左移指令

语法：

slli rd, rs1, shamt6

操作：

rd← rs1 << shamt6

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
000000	shamt6	rs1	001	rd	0010011

14.1.43　SLLIW——低 32 位立即数逻辑左移指令

语法：

slliw rd, rs1, shamt5

操作：

tmp[31:0]←(rs1[31:0] << shamt5)[31:0]

rd← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	shamt5	rs1	001	rd	0011011

14.1.44　SLLW——低 32 位逻辑左移指令

语法：

sllw rd, rs1, rs2

操作：

tmp[31:0]← (rs1[31:0] << rs2[4:0])[31:0]

rd←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	001	rd	0111011

14.1.45　SLT——有符号比较小于置位指令

语法：

slt rd, rs1, rs2

操作：

if (rs1 < rs2)

　　rd←1

else

　　rd←0

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	010	rd	0110011

14.1.46　SLTI——有符号立即数比较小于置位指令

语法：

slti rd, rs1, imm12

操作：

if (rs1 <sign_extend(imm12))

　　rd←1

else

　　rd←0

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	010	rd	0010011

14.1.47　SLTIU——无符号立即数比较小于置位指令

语法：

sltiu rd, rs1, imm12

操作：

if (rs1 <zero_extend(imm12))

　　rd←1

else

　　rd←0

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	011	rd	0010011

14.1.48　SLTU——无符号比较小于置位指令

语法：

sltu rd, rs1, rs2

操作：

if (rs1 < rs2)

　　rd←1

else

　　rd←0

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	011	rd	0110011

14.1.49　SRA——算数右移指令

语法：

sra rd, rs1, rs2

操作：

rd←rs1 >>> rs2[5:0]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	rs2	rs1	101	rd	0110011

14.1.50　SRAI——立即数算数右移指令

语法：

srai rd, rs1, shamt6

操作：

rd← rs1 >>>shamt6

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
010000	shamt6	rs1	101	rd	0010011

14.1.51　SRAIW——低 32 位立即数算数右移指令

语法：

sraiw rd, rs1, shamt5

操作：

tmp[31:0]←(rs1[31:0] >>> shamt5)[31:0]

rd← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	shamt5	rs1	101	rd	0011011

14.1.52　SRAW——低 32 位算数右移指令

语法：

sraw rd, rs1, rs2

操作：

tmp←(rs1[31:0] >>> rs2[4:0])[31:0]

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	rs2	rs1	101	rd	0111011

14.1.53　SRET——超级用户模式异常返回指令

语法：

sret

操作：

next pc← sepc

sstatus.sie ←sstatus.spie

sstatus.spie ←1

执行权限：

S mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001000	00010	00000	000	00000	1110011

14.1.54　SRL——逻辑右移指令

语法：

srl rd, rs1, rs2

操作：

rd←rs1 >> rs2[5:0]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	101	rd	0110011

14.1.55　SRLI——立即数逻辑右移指令

语法：

srli rd, rs1, shamt6

操作：

rd← rs1 >> shamt6

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
000000	shamt6	rs1	101	rd	0010011

14.1.56　SRLIW——低 32 位立即数逻辑右移指令

语法：

srliw rd, rs1, shamt5

操作：

tmp[31:0]←(rs1[31:0] >> shamt5)[31:0]

rd← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	shamt5	rs1	101	rd	0011011

14.1.57　SRLW——低 32 位逻辑右移指令

语法：

srlw rd, rs1, rs2

操作：

tmp←(rs1[31:0] >> rs2[4:0])[31:0]

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	101	rd	0111011

14.1.58　SUB——有符号减法指令

语法：

sub rd, rs1, rs2

操作：

rd ← rs1 - rs2

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	rs2	rs1	000	rd	0110011

14.1.59　SUBW——低 32 位有符号减法指令

语法：

subw rd, rs1, rs2

操作：

tmp[31:0] ← rs1[31:0] - rs2[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	rs2	rs1	000	rd	0111011

14.1.60　SW——字存储指令

语法：

sw rs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[(address+3):address] ← rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	rs2	rs1	010	imm12[4:0]	0100011

14.1.61　WFI——进入低功耗模式指令

语法：

wfi

操作：

处理器进入低功耗模式，此时 CPU 时钟关闭，大部分外设时钟也关闭

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001000	00101	00000	000	00000	1110011

14.1.62　XOR——按位异或指令

语法：

xor rd, rs1, rs2

操作：

rd ← rs1 ^ rs2

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	rs2	rs1	100	rd	0110011

14.1.63　XORI——立即数按位异或指令

语法：

xori rd, rs1, imm12

操作：

rd ← rs1 & sign_extend(imm12)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	100	rd	0010011

14.2　附录 A-2 M 指令术语

以下是对 C910 实现的 RISC-V M 指令集的具体描述，本节指令位宽为 32 位，指令按英文字母顺序排列，

14.2.1　DIV——有符号除法指令

语法：

div rd, rs1, rs2

操作：

rd ← rs1 / rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，除法结果为 0xffffffffffffff

产生 overflow 时，除法结果为 0x8000000000000000

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	100	rd	0110011

14.2.2　DIVU——无符号除法指令

语法：

divu rd, rs1, rs2

操作：

rd ← rs1 / rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，除法结果为 0xffffffffffffff

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	101	rd	0110011

14.2.3　DIVUW——低 32 位无符号除法指令

语法：

divuw rd, rs1, rs2

操作：

tmp[31:0] ← (rs1[31:0] / rs2[31:0])[31:0]

rd←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，除法结果为 0xffffffffffffff

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	101	rd	0111011

14.2.4　DIVW——低 32 位有符号除法指令

语法：

divw rd, rs1, rs2

操作：

tmp[31:0] ← (rs1[31:0] / rs2[31:0])[31:0]

rd←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，除法结果为 0xffffffffffffff

产生 overflow 时，除法结果为 0xffffffff80000000

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	100	rd	0111011

14.2.5　MUL——有符号乘法指令

语法：

mul rd, rs1, rs2

操作：

rd ← (rs1 * rs2)[63:0]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	000	rd	0110011

14.2.6　MULH——有符号乘法取高位指令

语法：

mulh rd, rs1, rs2

操作：

rd ← (rs1 * rs2)[127:64]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	001	rd	0110011

14.2.7　MULHSU——有符号无符号乘法取高位指令

语法：

mulusu rd, rs1, rs2

操作：

rd ← (rs1 * rs2)[127:64]

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 有符号数，rs2 无符号数

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	010	rd	0110011

14.2.8　MULHU——无符号乘法取高位指令

语法：

mulhu rd, rs1, rs2

操作：

rd ← (rs1 * rs2)[127:64]

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	011	rd	0110011

14.2.9　MULW——低 32 位有符号乘法指令

语法：

mulw rd, rs1, rs2

操作：

tmp ← (rs1[31:0] * rs2[31:0])[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	000	rd	0111011

14.2.10　REM——有符号取余指令

语法：

rem rd, rs1, rs2

操作：

rd ← rs1 % rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，求余结果为被除数

产生 overflow 时，余数结果为 0x0

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	110	rd	0110011

14.2.11　REMU——无符号取余指令

语法：

remu rd, rs1, rs2

操作：

rd ← rs1 % rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，求余结果为被除数

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	111	rd	0110011

14.2.12　REMUW——低 32 位无符号取余指令

语法：

remw rd, rs1, rs2

操作：

tmp ← (rs1[31:0] % rs2[31:0])[31:0]

rd ← sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，求余结果是对被除数 [31] 位符号位扩展后的结果

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	111	rd	0111011

14.2.13　REMW——低 32 位有符号取余指令

语法：

remw rd, rs1, rs2

操作：

tmp[31:0] ← (rs1[31:0] % rs2[31:0])[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

说明：

除数是 0 时，求余结果是对被除数 [31] 位符号位扩展后的结果

产生 overflow 时，余数结果为 0x0

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	rs2	rs1	110	rd	0111011

14.3　附录 A-3 A 指令术语

以下是对 C910 实现的 RISC-V A 指令的具体描述，本节指令位宽为 32 位，指令按英文字母顺序排列。

14.3.1　AMOADD.D——原子加法指令

语法：

amoadd.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← mem[rs1+7:rs1] + rs2

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoadd.d rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amoadd.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0: 对应的汇编指令 amoadd.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

- aq=1,rl=1: 对应的汇编指令 amoadd.d.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00000	aq	rl	rs2	rs1	011	rd	0101111

14.3.2　AMOADD.W——低 32 位原子加法指令

语法：

amoadd.w.aqrl rd, rs2, (rs1)

操作：

rd ←sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1]← mem[rs1+3:rs1] + rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoadd.w rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amoadd.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0: 对应的汇编指令 amoadd.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amoadd.w.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00000	aq	rl	rs2	rs1	010	rd	0101111

14.3.3　AMOAND.D——原子按位与指令

语法：

amoand.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← mem[rs1+7:rs1] & rs2

执行权限： M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoand.d rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amoand.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0: 对应的汇编指令 amoand.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amoand.d.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
01100	aq	rl	rs2	rs1	011	rd	0101111

14.3.4　AMOAND.W——低 32 位原子按位与指令

语法：

amoand.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← mem[rs1+3:rs1] & rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoand.w rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amoand.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0: 对应的汇编指令 amoand.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amoand.w.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
01100	aq	rl	rs2	rs1	010	rd	0101111

14.3.5　AMOMAX.D——原子有符号取最大值指令

语法：

amomax.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← max(mem[rs1+7:rs1], rs2)

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomax.d rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amomax.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0: 对应的汇编指令 amomax.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amomax.d.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
10100	aq	rl	rs2	rs1	011	rd	0101111

14.3.6　AMOMAX.W——低 32 位原子有符号取最大值指令

语法：

amomax.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1]← max(mem[rs1+3:rs1], rs2[31:0])

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomax.w rd, rs2, (rs1)。
- aq=0,rl=1: 对应的汇编指令 amomax.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。

- aq=1,rl=0: 对应的汇编指令 amomax.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amomax.w.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
10100	aq	rl	rs2	rs1	010	rd	0101111

14.3.7 AMOMAXU.D——原子无符号取最大值指令

语法：

amomaxu.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← max(mem[rs1+7:rs1], rs2)

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomaxu.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amomaxu.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amomaxu.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1: 对应的汇编指令 amomaxu.d.aqrl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
11100	aq	rl	rs2	rs1	011	rd	0101111

14.3.8　AMOMAXU.W——低 32 位原子无符号取最大值指令

语法：

amomaxu.w.aqrl rd, rs2, (rs1)

操作：

rd ← zero_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← max(mem[rs1+3:rs1], rs2[31:0])

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomaxu.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amomaxu.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amomaxu.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amomaxu.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
11100	aq	rl	rs2	rs1	010	rd	0101111

14.3.9　AMOMIN.D——原子有符号取最小值指令

语法：

amomin.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← min(mem[rs1+7:rs1],rs2)

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomin.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amomin.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amomin.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amomin.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
10000	aq	rl	rs2	rs1	011	rd	0101111

14.3.10　AMOMIN.W——低 32 位原子有符号取最小值指令

语法：

amomin.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← min(mem[rs1+3:rs1], rs2[31:0])

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amomin.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amomin.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amomin.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amomin.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
10000	aq	rl	rs2	rs1	010	rd	0101111

14.3.11　AMOMINU.D——原子无符号取最小值指令

语法：

amominu.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← min(mem[rs1+7:rs1], rs2)

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amominu.d rd, rs2, (rs1)。

- aq=0,rl=1：对应的汇编指令 amominu.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amominu.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amominu.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
11000	aq	rl	rs2	rs1	011	rd	0101111

14.3.12　AMOMINU.W——低 32 位原子无符号取最小值指令

语法：

amominu.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← min(mem[rs1+3:rs1], rs2[31:0])

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amominu.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amominu.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amominu.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amominu.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
11000	aq	rl	rs2	rs1	010	rd	0101111

14.3.13　AMOOR.D——原子按位或指令

语法：

amoor.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← mem[rs1+7:rs1] | rs2

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoor.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amoor.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoor.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoor.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
01000	aq	rl	rs2	rs1	011	rd	0101111

14.3.14　AMOOR.W——低 32 位原子按位或指令

语法：

amoor.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← mem[rs1+3:rs1] | rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoor.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amoor.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoor.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoor.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
01000	aq	rl	rs2	rs1	010	rd	0101111

14.3.15　AMOSWAP.D——原子交换指令

语法：

amoswap.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ←rs2

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位： 无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoswap.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amoswap.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoswap.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoswap.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00001	aq	rl	rs2	rs1	011	rd	0101111

14.3.16　AMOSWAP.W——低 32 位原子交换指令

语法：

amoswap.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ←rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位： 无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoswap.w rd, rs2, (rs1)。

- aq=0,rl=1：对应的汇编指令 amoswap.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoswap.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoswap.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00001	aq	rl	rs2	rs1	010	rd	0101111

14.3.17　AMOXOR.D——原子按位异或指令

语法：

amoxor.d.aqrl rd, rs2, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] ← mem[rs1+7:rs1] ^ rs2

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoxor.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amoxor.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoxor.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoxor.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00100	aq	rl	rs2	rs1	011	rd	0101111

14.3.18　AMOXOR.W——低 32 位原子按位异或指令

语法：

amoxor.w.aqrl rd, rs2, (rs1)

操作：

rd ← sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] ← mem[rs1+3:rs1] ^ rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 amoxor.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 amoxor.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 amoxor.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 amoxor.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00100	aq	rl	rs2	rs1	010	rd	0101111

14.3.19　LR.D——双字加载保留指令

语法：

lr.d.aqrl rd, (rs1)

操作：

rd ← mem[rs1+7: rs1]

mem[rs1+7:rs1] is reserved

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 lr.d rd, (rs1)。
- aq=0,rl=1：对应的汇编指令 lr.d.rl rd, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 lr.d.aq rd, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 lr.d.aqrl rd, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00010	aq	rl	00000	rs1	011	rd	0101111

14.3.20　LR.W——字加载保留指令

语法：

lr.w.aqrl rd, (rs1)

操作：

rd ←sign_extend(mem[rs1+3: rs1])

mem[rs1+3:rs1] is reserved

执行权限：

M mode/S mode/U mode

异常： 原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位： 无

说明： aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 lr.w rd, (rs1)。
- aq=0,rl=1：对应的汇编指令 lr.w.rl rd, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 lr.w.aq rd, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 lr.w.aqrl rd, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00010	aq	rl	00000	rs1	010	rd	0101111

14.3.21 SC.D——双字条件存储指令

语法：

sc.d.aqrl rd, rs2, (rs1)

操作：

If(mem[rs1+7:rs1] is reserved)

　　mem[rs1+7: rs1] ← rs2

　　rd ← 0

else

　　rd ← 1

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 sc.d rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 sc.d.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。
- aq=1,rl=0：对应的汇编指令 sc.d.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 sc.d.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00011	aq	rl	rs2	rs1	011	rd	0101111

14.3.22　SC.W——字条件存储指令

语法：

sc.w.aqrl rd, rs2, (rs1)

操作：

if(mem[rs1+3:rs1] is reserved)

　　mem[rs1+3:rs1] ← rs2[31:0]

　　rd ← 0

else

　　rd ← 1

执行权限：

M mode/S mode/U mode

异常：

原子指令非对齐访问异常、原子指令访问错误异常、原子指令页面错误异常

影响标志位：

无

说明：

aq 位和 rl 位决定该指令前序和后序访问存储器指令的执行顺序：

- aq=0,rl=0: 对应的汇编指令 sc.w rd, rs2, (rs1)。
- aq=0,rl=1：对应的汇编指令 sc.w.rl rd, rs2, (rs1)，该指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到。

- aq=1,rl=0：对应的汇编指令 sc.w.aq rd, rs2, (rs1)，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。
- aq=1,rl=1：对应的汇编指令 sc.w.aqrl rd, rs2, (rs1)，指令前序所有访问存储的指令的结果必须在该指令执行之前被观察到，该指令后序所有访问存储的指令必须等该指令执行完成后才开始执行。

指令格式：

31 27	26	25	24 20	19 15	14 12	11 7	6 0
00011	aq	rl	rs2	rs1	010	rd	0101111

14.4　附录 A-4 F 指令术语

以下是对 C910 实现的 RISC-V F 指令集的具体描述，本节指令位宽为 32 位，指令按英文字母顺序排列，

对于单精度浮点指令，如果源寄存器的高 32 位不全为 1，则该单精度数据当作 qNaN 处理。

当 mstatus.fs==2’b00 时，执行本节所有指令会产生非法指令异常，当 mstatus.fs != 2’b00 时，执行本节任意指令后 mstatus.fs 置位为 2’b11。

14.4.1　FADD.S——单精度浮点加法指令

语法：

fadd.s fd, fs1, fs2, rm

操作：

frd ← fs1 + fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fadd.s fd, fs1,fs2,rne。
- 3’b001: 向零舍入，对应的汇编指令 fadd.s fd, fs1,fs2,rtz。

- 3’b010: 向负无穷舍入，对应的汇编指令 fadd.s fd, fs1,fs2,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fadd.s fd, fs1,fs2,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fadd.s fd, fs1,fs2,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fadd.s fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	fs2	fs1	rm	fd	1010011

14.4.2 FCLASS.S——单精度浮点分类指令

语法：

fclass.s rd, fs1

操作：

if (fs1 = -inf)

　　rd ← 64’h1

if (fs1 = -norm)

　　rd ← 64’h2

if (fs1 = -subnorm)

　　rd ← 64’h4

if (fs1 = -zero)

　　rd ← 64’h8

if (fs1 = +zero)

　　rd ← 64’h10

if (fs1 = +subnorm)

　　rd ← 64’h20

if (fs1 = +norm)

　　rd ← 64’h40

if (fs1 = +Inf)

　　rd ← 64’h80

if (fs1 = sNaN)

rd ← 64’ h100

if (fs1 = qNaN)

rd ← 64’ h200

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110000	00000	fs1	001	rd	1010011

14.4.3 FCVT.L.S——单精度浮点转换成有符号长整型指令

语法：

fcvt.l.s rd, fs1, rm

操作：

rd ← single_convert_to_signed_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’ b000: 就近向偶数舍入，对应的汇编指令 fcvt.l.s rd,fs1,rne。
- 3’ b001: 向零舍入，对应的汇编指令 fcvt.l.s rd,fs1,rtz。
- 3’ b010: 向负无穷舍入，对应的汇编指令 fcvt.l.s rd,fs1,rdn。

- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.l.s rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.l.s rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.l.s rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100000	00010	fs1	rm	rd	1010011

14.4.4 FCVT.LU.S——单精度浮点转换成无符号长整型指令

语法：

fcvt.lu.s rd, fs1, rm

操作：

rd ← single_convert_to_unsigned_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.lu.s rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.lu.s rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.lu.s rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.lu.s rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.lu.s rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。

- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.lu.s rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100000	00011	fs1	rm	rd	1010011

14.4.5 FCVT.S.L——有符号长整型转换成单精度浮点数指令

语法：

fcvt.s.l fd, rs1, rm

操作：

fd ← signed_long_convert_to_single(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.s.l fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.s.l fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.s.l fd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.s.l fd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.s.l fd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.s.l fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101000	00010	rs1	rm	fd	1010011

14.4.6　FCVT.S.LU——无符号长整型转换成单精度浮点数指令

语法：

fcvt.s.l fd, fs1, rm

操作：

fd ← unsigned_long_convert_to_single_fp(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.s.lu fd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.s.lu fd, fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.s.lu fd, fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.s.lu fd, fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.s.lu fd, fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.s.lu fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101000	00011	rs1	rm	fd	1010011

14.4.7　FCVT.S.W——有符号整型转换成单精度浮点数指令

语法：

fcvt.s.w fd, rs1, rm

操作：

fd ← signed_int_convert_to_single(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.s.w fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.s.w fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.s.w fd,rs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.s.w fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.s.w fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.s.w fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101000	00000	rs1	rm	fd	1010011

14.4.8 FCVT.S.WU——无符号整型转换成单精度浮点数指令

语法：

fcvt.s.wu fd, rs1, rm

操作：

fd ← unsigned_int_convert_to_single_fp(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.s.wu fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.s.wu fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.s.wu fd,rs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.s.wu fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.s.wu fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.s.wu fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101000	00001	rs1	rm	fd	1010011

14.4.9　FCVT.W.S——单精度浮点转换成有符号整型指令

语法：

fcvt.w.s rd, fs1, rm

操作：

tmp ← single_convert_to_signed_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.w.s rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.w.s rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.w.s rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.w.s rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.w.s rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.w.s rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100000	00000	fs1	rm	rd	1010011

14.4.10　FCVT.WU.S——单精度浮点转换成无符号整型指令

语法：

fcvt.wu.s rd, fs1, rm

操作：

tmp ← single_convert_to_unsigned_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.wu.s rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.wu.s rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.wu.s rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.wu.s rd,fs1,rup。

- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.wu.s rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.wu.s rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100000	00001	fs1	rm	rd	1010011

14.4.11 FDIV.S——单精度浮点除法指令

语法：

fdiv.s fd, fs1, fs2, rm

操作：

fd ← fs1 / fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/DZ/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fdiv.s fs1,fs2,rne。
- 3’b001: 向零舍入，对应的汇编指令 fdiv.s fd fs1,fs2,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fdiv.s fd, fs1,fs2,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fdiv.s fd, fs1,fs2,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fdiv.s fd, fs1,fs2,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fdiv.s fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001100	fs1	fs2	rm	fd	1010011

14.4.12　FEQ.S——单精度浮点比较相等指令

语法：

feq.s rd, fs1, fs2

操作：

if(fs1 == fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010000	fs2	fs1	010	rd	1010011

14.4.13　FLE.S——单精度浮点比较小于等于指令

语法：

fle.s rd, fs1, fs2

操作：

if(fs1 <= fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010000	fs2	fs1	000	rd	1010011

14.4.14　FLT.S——单精度浮点比较小于指令

语法：

flt.s rd, fs1, fs2

操作：

if(fs1 < fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010000	fs2	fs1	001	rd	1010011

14.4.15　FLW——单精度浮点加载指令

语法：

flw fd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

fd[31:0] ← mem[(address+3):address]

fd[63:32] ← 32’hffffffff

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

影响标志位：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	010	fd	0000111

14.4.16　FMADD.S——单精度浮点乘累加指令

语法：

fmadd.s fd, fs1, fs2, fs3, rm

操作：

rd ← fs1*fs2 + fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3, rup。

- 3’b100: 就近向大值舍入，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmadd.s fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	00	fs2	fs1	rm	fd	1000011

14.4.17　FMAX.S——单精度浮点取最大值指令

语法：

fmax.s fd, fs1, fs2

操作：

if(fs1 >= fs2)

　　fd ← fs1

else

　　fd ← fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010100	fs2	fs1	001	fd	1010011

14.4.18　FMIN.S——单精度浮点取最小值指令

语法：

fmin.s fd, fs1, fs2

操作：

if(fs1 >= fs2)

　　fd ← fs2

else

　　fd ← fs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010100	fs2	fs1	000	fd	1010011

14.4.19　FMSUB.S——单精度浮点乘累减指令

语法：

fmsub.s fd, fs1, fs2, fs3, rm

操作：

fd ← fs1*fs2 - fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmsub.s fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmsub.s fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmsub.s fd,fs1, fs2, fs3, rdn。

- 3’b011: 向正无穷舍入，对应的汇编指令 fmsub.s fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmsub.s fd, fs1, fs2, fs3,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmsub.s fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	00	fs2	fs1	rm	fd	1000111

14.4.20　FMUL.S——单精度浮点乘法指令

语法：

fmul.s fd, fs1, fs2, rm

操作：

fd ← fs1 * fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmul.s fd, fs1, fs2, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmul.s fd, fs1, fs2, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmul.s fd, fs1, fs2, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmul.s fd, fs1, fs2, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmul.s fd, fs1,fs2, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。

- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmul.s fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001000	fs2	fs1	rm	fd	1010011

14.4.21　FMV.W.X——单精度浮点写传送指令

语法：

fmv.w.x fd, rs1

操作：

fd[31:0] ← rs[31:0]

fd[63:32] ← 32’hffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1111000	00000	rs1	000	fd	1010011

14.4.22　FMV.X.W——单精度浮点寄存器读传送指令

语法：

fmv.x.w rd, fs1

操作：

tmp[31:0] ← fs1[31:0]

rd ← sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110000	00000	fs1	000	rd	1010011

14.4.23　FNMADD.S——单精度浮点乘累加取负指令

语法：

fnmadd.s fd, fs1, fs2, fs3, rm

操作：

fd ←-(fs1*fs2 + fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmadd.s fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	00	fs2	fs1	rm	fd	1001111

14.4.24　FNMSUB.S——单精度浮点乘累减取负指令

语法：

fnmsub.s fd, fs1, fs2, fs3, rm

操作：

fd ← -(fs1*fs2 - fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmsub.s fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	00	fs2	fs1	rm	fd	1001011

14.4.25　FSGNJ.S——单精度浮点符号注入指令

语法：

fsgnj.s fd, fs1, fs2

操作：

fd[30:0] ← fs1[30:0]

fd[31] ← fs2[31]

fd[63:32] ← 32’hffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010000	fs2	fs1	000	fd	1010011

14.4.26　FSGNJN.S——单精度浮点符号取反注入指令

语法：

fsgnjn.s fd, fs1, fs2

操作：

fd[30:0] ← fs1[30:0]

fd[31] ← ! fs2[31]

fd[63:32] ← 32’hffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010000	fs2	fs1	001	fd	1010011

14.4.27　FSGNJX.S——单精度浮点符号异或注入指令

语法：

fsgnjx.s fd, fs1, fs2

操作：

fd[30:0] ← fs1[30:0]

fd[31] ← fs1[31] ^ fs2[31]

fd[63:32] ← 32’hffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010000	fs2	fs1	010	fd	1010011

14.4.28　FSQRT.S——单精度浮点开方指令

语法：

fsqrt.s fd, fs1, rm

操作：

fd ← sqrt(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsqrt.s fd, fs1,rne
- 3’b001: 向零舍入，对应的汇编指令 fsqrt.s fd, fs1,rtz
- 3’b010: 向负无穷舍入，对应的汇编指令 fsqrt.s fd, fs1,rdn
- 3’b011: 向正无穷舍入，对应的汇编指令 fsqrt.s fd, fs1,rup
- 3’b100: 就近向大值舍入，对应的汇编指令 fsqrt.s fd, fs1,rmm
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111：动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsqrt.s fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0101100	00000	fs1	rm	fd	1010011

14.4.29　FSUB.S——单精度浮点减法指令

语法：

fsub.s fd, fs1, fs2, rm

操作：

fd ← fs1 - fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsub.fd, fs1,fs2,rne
- 3’b001: 向零舍入，对应的汇编指令 fsub.s fd, fs1,fs2,rtz
- 3’b010: 向负无穷舍入，对应的汇编指令 fsub.s fd, fs1,fs2,rdn
- 3’b011: 向正无穷舍入，对应的汇编指令 fsub.s fd, fs1,fs2,rup
- 3’b100: 就近向大值舍入，对应的汇编指令 fsub.s fd, fs1,fs2,rmm

- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111：动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsub.s fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000100	fs2	fs1	rm	fd	1010011

14.4.30　FSW——单精度浮点存储指令

语法：

fsw fs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[(address+31):address] ← fs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	fs2	rs1	010	imm12[4:0]	0100111

14.5　附录 A-5 D 指令术语

以下是对 C910 实现的 RISC-V D 指令集的具体描述，本节指令位宽为 32 位，指令按英文字母顺序排列。

当 mstatus.fs==2’b00 时，执行本节所有指令会产生非法指令异常，当 mstatus.fs != 2’b00 时，执行本节任意指令后 mstatus.fs 置位为 2’b11。

14.5.1　FADD.D——双精度浮点加法指令

语法：

fadd.d fd, fs1, fs2, rm

操作：

fd ← fs1 + fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fadd.d fd, fs1,fs2,rne
- 3’b001: 向零舍入，对应的汇编指令 fadd.d fd, fs1,fs2,rtz
- 3’b010: 向负无穷舍入，对应的汇编指令 fadd.d fd, fs1,fs2,rdn
- 3’b011: 向正无穷舍入，对应的汇编指令 fadd.d fd, fs1,fs2,rup
- 3’b100: 就近向大值舍入，对应的汇编指令 fadd.d fd, fs1,fs2,rmm
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fadd.d fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	fs2	fs1	rm	fd	1010011

14.5.2 FCLASS.D——双精度浮点分类指令

语法：

fclass.d rd, fs1

操作： if (fs1 = -Inf)

rd ← 64’h1

if (fs1 = -norm)

rd ← 64’h2

if (fs1 = -subnorm)

rd ← 64’ h4

if (fs1 = -zero)

rd ← 64’ h8

if (fs1 = +Zero)

rd ← 64’ h10

if (fs1 = +subnorm)

rd ← 64’ h20

if (fs1 = +norm)

rd ← 64’ h40

if (fs1 = +Inf)

rd ← 64’ h80

if (fs1 = sNaN)

rd ← 64’ h100

if (fs1 = qNaN)

rd ← 64’ h200

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110001	00000	fs1	001	rd	1010011

14.5.3 FCVT.D.L——有符号长整型转换成双精度浮点数指令

语法：

fcvt.d.l fd, rs1, rm

操作：

fd ← signed_long_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.d.l fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.d.l fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.d.l fd,rs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.d.l fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.d.l fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.d.l fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101001	00010	rs1	rm	fd	1010011

14.5.4 FCVT.D.LU——无符号长整型转换成双精度浮点数指令

语法：

fcvt.d.lu fd, rs1, rm

操作：

fd ← unsigned_long_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.d.lu fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.d.lu fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.d.lu fd,rs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.d.lu fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.d.lu fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.d.lu fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101001	00011	rs1	rm	fd	1010011

14.5.5 FCVT.D.S——单精度浮点转换成双精度浮点指令

语法：

fcvt.d.s fd, fs1

操作：

fd ← single_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100001	00000	fs1	000	fd	1010011

14.5.6　FCVT.D.W——有符号整型转换成双精度浮点数指令

语法：

fcvt.d.w fd, rs1

操作：

fd ← signed_int_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101001	00000	rs1	000	fd	1010011

14.5.7　FCVT.D.WU——无符号整型转换成双精度浮点数指令

语法：

fcvt.d.wu fd, rs1

操作：

fd ← unsigned_int_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101001	00001	rs1	000	fd	1010011

14.5.8　FCVT.L.D——双精度浮点转换成有符号长整型指令

语法：

fcvt.l.d rd, fs1, rm

操作：

rd ← double_convert_to_signed_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.l.d rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.l.d rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.l.d rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.l.d rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.l.d rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.l.d rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100001	00010	fs1	rm	rd	1010011

14.5.9　FCVT.LU.D——双精度浮点转换成无符号长整型指令

语法：

fcvt.lu.d rd, fs1, rm

操作：

rd ← double_convert_to_unsigned_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.lu.d rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.lu.d rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.lu.d rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.lu.d rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.lu.d rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.lu.d rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100001	00011	fs1	rm	rd	1010011

14.5.10　FCVT.S.D——双精度浮点转换成单精度浮点指令

语法：

fcvt.s.d fd, fs1, rm

操作：

fd ← double_convert_to_single(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.s.d fd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.s.d fd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.s.d fd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.s.d fd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.s.d fd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.s.d fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	00001	fs1	rm	fd	1010011

14.5.11　FCVT.W.D——双精度浮点转换成有符号整型指令

语法：

fcvt.w.d rd, fs1, rm

操作：

tmp ← double_convert_to_signed_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.w.d rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.w.d rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.w.d rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.w.d rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.w.d rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.w.d rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100001	00000	fs1	rm	rd	1010011

14.5.12　FCVT.WU.D——双精度浮点转换成无符号整型指令

语法：

fcvt.wu.d rd, fs1, rm

操作：

tmp ← double_convert_to_unsigned_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.wu.d rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.wu.d rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.wu.d rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.wu.d rd,fs1,rup。

- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.wu.d rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.wu.d rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100001	00001	fs1	rm	rd	1010011

14.5.13 FDIV.D——双精度浮点除法指令

语法：

fdiv.d fd, fs1, fs2, rm

操作：

fd ← fs1 / fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/DZ/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fdiv.d fd, fs1,fs2,rne。
- 3’b001: 向零舍入，对应的汇编指令 fdiv.d fd, fs1,fs2,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fdiv.d fd, fs1,fs2,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fdiv.d fd, fs1,fs2,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fdiv.d fd, fs1,fs2,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fdiv.d fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001101	fs2	fs1	rm	fd	1010011

14.5.14　FEQ.D——双精度浮点比较相等指令

语法：

feq.d rd, fs1, fs2

操作：

if(fs1 == fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010001	fs2	fs1	010	rd	1010011

14.5.15　FLD——双精度浮点加载指令

语法：

fld fd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

fd[63:0] ← mem[(address+7):address]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

影响标志位：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	011	fd	0000111

14.5.16　FLE.D——双精度浮点比较小于等于指令

语法：

fle.d rd, fs1, fs2

操作：

if(fs1 <= fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010001	fs2	fs1	000	rd	1010011

14.5.17　FLT.D——双精度浮点比较小于指令

语法：

flt.d rd, fs1, fs2

操作：

if(fs1 < fs2)

　　rd ← 1

else

rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010001	fs2	fs1	001	rd	1010011

14.5.18　FMADD.D——双精度浮点乘累加指令

语法：

fmadd.d fd, fs1, fs2, fs3, rm

操作：

fd ← fs1*fs2 + fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3, rmm。

- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmadd.d fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	01	fs2	fs1	rm	fd	1000011

14.5.19 FMAX.D——双精度浮点取最大值指令

语法：

fmax.d fd, fs1, fs2

操作：

if(fs1 >= fs2)

　　fd ← fs1

else

　　fd ← fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010101	fs2	fs1	001	fd	1010011

14.5.20 FMIN.D——双精度浮点取最小值指令

语法：

fmin.d fd, fs1, fs2

操作：

if(fs1 >= fs2)

fd ← fs2

else

fd ← fs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010101	fs2	fs1	000	fd	1010011

14.5.21 FMSUB.D——双精度浮点乘累减指令

语法：

fmsub.d fd, fs1, fs2, fs3, rm

操作：

fd ← fs1*fs2 - fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3, rup。

- 3’b100: 就近向大值舍入，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmsub.d fd, fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	01	fs2	fs1	rm	fd	1000111

14.5.22 FMUL.D——双精度浮点乘法指令

语法：

fmul.d fd, fs1, fs2, rm

操作：

fd ← fs1 * fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmul.d fd, fs1, fs2, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmul.d fd, fs1, fs2, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmul.d fd, fs1, fs2, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmul.d fd, fs1, fs2, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmul.d fd, fs1, fs2, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmul. fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001001	fs2	fs1	rm	fd	1010011

14.5.23　FMV.D.X——双精度浮点写传送指令

语法：

fmv.d.x fd, rs1

操作：

fd← rs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

说明：

整型寄存器搬运到浮点寄存器

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1111001	00000	rs1	000	fd	1010011

14.5.24　FMV.X.D——双精度浮点读传送指令

语法：

fmv.x.d rd, fs1

操作：

rd← fs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

说明：

浮点寄存器搬运到整型寄存器

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110001	00000	fs1	000	rd	1010011

14.5.25　FNMADD.D——双精度浮点乘累加取负指令

语法：

fnmadd.d fd, fs1, fs2, fs3, rm

操作：

fd ←-(fs1*fs2 + fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmadd.d fd, fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmadd.d fd, fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmadd.d fd, fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmadd.d fd, fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmadd.d fd, fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmadd.d fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	01	fs2	fs1	rm	fd	1001111

14.5.26　FNMSUB.D——双精度浮点乘累减取负指令

语法：

fnmsub.d fd, fs1, fs2, fs3, rm

操作：

fd ← -(fs1*fs2 - fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmsub.d fd, fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmsub.d fd, fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmsub.d fd, fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmsub.d fd, fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmsub.d fd, fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmsub.d fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	01	fs2	fs1	rm	fd	1001011

14.5.27　FSD——双精度浮点存储指令

语法：

fsd fs2, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

mem[(address+63):address] ← fs2[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	fs2	rs1	011	imm12[4:0]	0100111

14.5.28　FSGNJ.D——双精度浮点符号注入指令

语法：

fsgnj.d fd, fs1, fs2

操作：

fd[62:0] ← fs1[62:0]

fd[63] ← fs2[63]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010001	fs2	fs1	000	fd	1010011

14.5.29　FSGNJN.D——双精度浮点符号取反注入指令

语法：

fsgnjn.d fd, fs1, fs2

操作：

fd[62:0] ← fs1[62:0]

fd[63] ← !fs2[63]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010001	fs2	fs1	001	fd	1010011

14.5.30　FSGNJX.D——双精度浮点符号异或注入指令

语法：

fsgnjx.d fd, fs1, fs2

操作：

fd[62:0] ← fs1[62:0]

fd[63] ← fs1[63] ^ fs2[63]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010001	fs2	fs1	010	fd	1010011

14.5.31　FSQRT.D——双精度浮点开方指令

语法：

fsqrt.d fd, fs1, rm

操作：

fd ← sqrt(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsqrt.d fd, fs1, rne。
- 3’b001: 向零舍入，对应的汇编指令 fsqrt.d fd, fs1, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fsqrt.d fd, fs1, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fsqrt.d fd, fs1, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fsqrt.d fd, fs1, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsqrt.d fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0101101	00000	fs1	rm	fd	1010011

14.5.32　FSUB.D——双精度浮点减法指令

语法：

fsub.d fd, fs1, fs2, rm

操作：

fd ← fs1 - fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsub.fd, fs1, fs2, rne。
- 3’b001: 向零舍入，对应的汇编指令 fsub.d fd, fs1, fs2, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fsub.d fd, fs1, fs2, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fsub.d fd, fs1, fs2, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fsub.d fd, fs1, fs2, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsub.dfd, fs1, fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000101	fs2	fs1	rm	fd	1010011

14.6　附录 A-6 C 指令术语

以下是对 C910 实现的 RISC-V C 指令的具体描述，本节指令位宽为 16 位，指令按英文字母顺序排列。

14.6.1　C.ADD——有符号加法指令

语法：

c.add rd, rs2

操作：

rd ← rs1 + rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 = rd != 0

rs2 ! = 0

指令格式：

15 13	12	11 7	6 2	1 0
100	1	rs1/rd	rs2	10

14.6.2　C.ADDI——有符号立即数加法指令

语法：

c.addi rd, nzimm6

操作：

rd ← rs1 + sign_extend(nzimm6)

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 = rd != 0

nzimm6!=0

指令格式：

15 13	12	11 7	6 2	1 0
000		rs1/rd	nzimm6[4:0]	01

nzimm6[5]

14.6.3　C.ADDIW——低 32 位有符号立即数加法指令

语法：

c.addiw rd, imm6

操作：

tmp[31:0] ← rs1[31:0] + sign_extend(imm6)

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

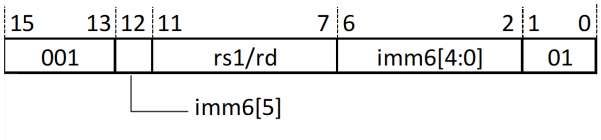
异常：

无

说明：

rs1 = rd != 0

指令格式：



14.6.4　C.ADDI4SPN——4 倍立即数和堆栈指针相加指令

语法：

c.addi4spn rd, sp, nzuimm8<<2

操作：

rd ← sp + zero_extend(nzuimm8<<2)

执行权限：

M mode/S mode/U mode

异常：

无

说明：

nzuimm8 != 0

rd 编码代表寄存器如下:

- 000 x8
- 001 x9
- 010 x10

- 011 x11
- 100 x12
- 101 x13
- 110 x14
- 111 x15

指令格式：

15 13	12 5	4 2	1 0
000	nzuimm8[3:2|7:4|0|1]	rd	00

14.6.5　C.ADDI16SP——加 16 倍立即数到堆栈指针指令

语法：

c.addi16sp sp, nzuimm6<<4

操作：

sp ← sp + sign_extend(nzuimm6<<4)

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

15 13	12	11 7	6 2	1 0
011		00010		01
	nzimm6[5]		nzimm6[0|2|4:3|1]	

14.6.6　C.ADDW——低 32 位有符号加法指令

语法：

c.addw rd, rs2

操作：

tmp[31:0] ← rs1[31:0] + rs2[31:0]

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 = rd

rd/rs1，rs2 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：

15 13	12	11 10	9 7	6 5	4 2	1 0
100	1	11	rs1/rd	01	rs2	01

14.6.7　C.AND——按位与指令

语法：

c.and rd, rs2

操作：

rd ← rs1 & rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 = rd

rd/rs1，rs2 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式:

15 13	12	11 10	9 7	6 5	4 2	1 0
100	0	11	rs1/rd	11	rs2	01

14.6.8 C.ANDI——立即数按位与指令

语法:

c.andi rd, imm6

操作:

rd ← rs1 & sign_extend(imm6)

执行权限:

M mode/S mode/U mode

异常:

无

说明:

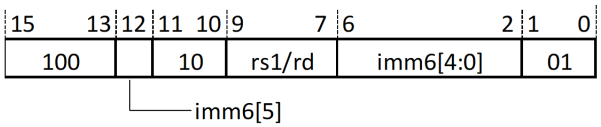
rs1 = rd

rd/rs1 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13

- 110: x14
- 111: x15

指令格式：



14.6.9　C.BEQZ——等于零分支指令

语法：

c.beqz rs1, label

操作：

if (rs1 == 0)

　　next pc = current pc + imm8<<1;

else

　　next pc = current pc + 2;

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

汇编器根据 label 算出 imm8

指令跳转范围为±256B 地址空间

指令格式：

15 13	12 10	9 7	6 2	1 0
110		rs1	Imm8[6:5|1:0|4]	01

imm8[7|3:2]

14.6.10　C.BNEZ——不等于零分支指令

语法：

c.bnez rs1, label

操作：

```
if (rs1 != 0)
    next pc = current pc + imm8<<1;
else
    next pc = current pc + 2;
```

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

汇编器根据 label 算出 imm12

指令跳转范围为±256B 地址空间

指令格式：

15　13	12　10	9　7	6　2	1　0
111		rs1	imm[6:5|1:0|4]	01

imm8[7|3:2]

14.6.11　C.EBREAK——断点指令

语法：

c.ebreak

操作：

产生断点异常或者进入调试模式

执行权限：

M mode/S mode/U mode

异常：

断点异常

指令格式：

15　13	12	11　7	6　2	1　0
100	1	00000	00000	10

14.6.12　C.FLD——浮点双字加载指令

语法：

c.fld fd, uimm5<<3(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<3)

fd ←mem[address+7:address]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

说明：

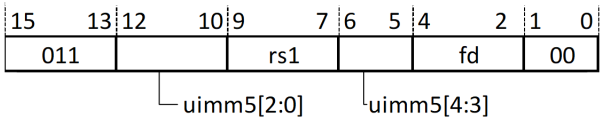
rs1 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

fd 编码代表寄存器如下:

- 000: f8
- 001: f9
- 010: f10
- 011: f11
- 100: f12
- 101: f13
- 110: f14
- 111: f15

指令格式：



14.6.13　C.FLDSP——浮点双字堆栈加载指令

语法：

c.fldsp fd, uimm6<<3(sp)

操作：

address ← sp+ zero_extend(uimm6<<3)

fd ←mem[address+7:address]

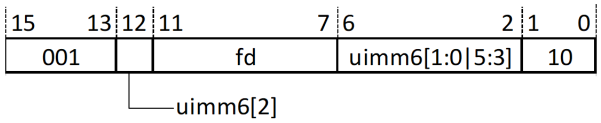
执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

指令格式：



14.6.14　C.FSD——浮点双字存储指令

语法：

c.fsd fs2，uimm5<<3(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<3)

mem[address+7:address] ←fs2

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

说明：

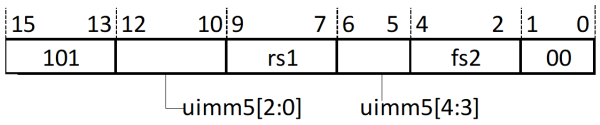
fs1 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

rs2 编码代表寄存器如下:

- 000: f8
- 001: f9
- 010: f10
- 011: f11
- 100: f12
- 101: f13
- 110: f14
- 111: f15

指令格式：



14.6.15　C.FSDSP——浮点双字堆栈存储指令

语法：

c.fsdsp fs2, uimm6<<3(sp)

操作：

address ← sp+ zero_extend(uimm6<<3)

mem[address+7:address] ←fs2

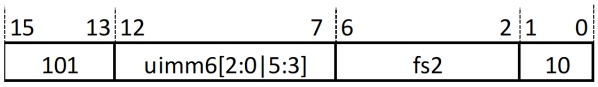
执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：



14.6.16　C.J——无条件跳转指令

语法：

c.j label

操作：

next pc ← current pc + sign_extend(imm<<1);

执行权限：

M mode/S mode/U mode

异常：

无

说明：

汇编器根据 label 算出 imm11

指令跳转范围为±2KB 地址空间

指令格式：

15　　13	12　　2	1　　0
101	imm11[10|3|8:7|9|5|6|2:0|4]	01

14.6.17　C.JALR——寄存器跳转子程序指令

语法：

c.jalr rs1

操作：

next pc ← rs1;

x1←current pc + 2;

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 != 0。

MMU 打开时，跳转范围是全部 512GB 地址空间。

MMU 关闭时，跳转范围是全部 1TB 地址空间。

指令格式：

15　　13	12	11　　7	6　　2	1　　0
100	1	rs1	00000	10

14.6.18　C.JR——寄存器跳转指令

语法：

c.jr rs1

操作：

next pc = rs1;

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 != 0。

MMU 打开时，跳转范围是全部 512GB 地址空间。

MMU 关闭时，跳转范围是全部 1TB 地址空间。

指令格式：

15　13	12	11　7	6　2	1　0
100	0	rs1	00000	10

14.6.19　C.LD——双字加载指令

语法：

c.ld rd, uimm5<<3(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<3)

rd ←mem[address+7:address]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

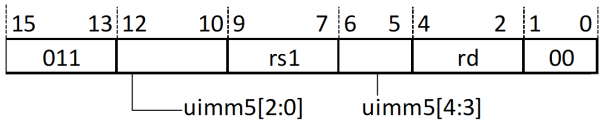
说明：

rs1/rd 编码代表寄存器如下:

- 000: x8

- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.20　C.LDSP——双字堆栈加载指令

语法：

c.ldsp rd, uimm6<<3(sp)

操作：

address ← sp+ zero_extend(uimm6<<3)

rd ←mem[address+7:address]

执行权限：

M mode/S mode/U mode

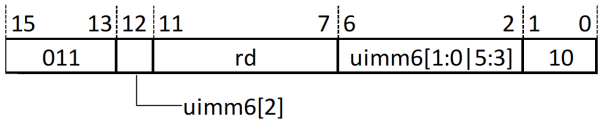
异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

说明：

rd != 0

指令格式：



14.6.21　C.LI——立即数传送指令

语法：

c.li rd, imm6

操作：

rd ←sign_extend(imm6)

执行权限：

M mode/S mode/U mode

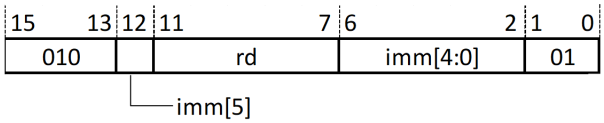
异常：

无

说明：

rd != 0。

指令格式：



14.6.22　C.LUI——高位立即数传送指令

语法：

c.lui rd, nzimm6

操作：

rd ←sign_extend(nzimm6<<12)

执行权限：

M mode/S mode/U mode

异常：

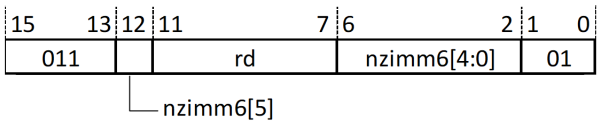
无

说明：

rd != 0。

Nzimm6 != 0。

指令格式：



14.6.23　C.LW——字加载指令

语法：

c.lw rd, uimm5<<2(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<2)

tmp[31:0] ←mem[address+3:address]

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

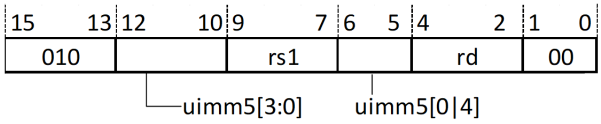
加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

说明：

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.24　C.LWSP——字堆栈加载指令

语法：

c.lwsp rd, uimm6<<2(sp)

操作：

address ← sp+ zero_extend(uimm6<<2)

tmp[31:0] ←mem[address+3:address]

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

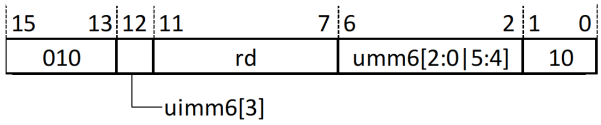
异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常

说明：

rd != 0

指令格式：



14.6.25　C.MV——数据传送指令

语法：

c.mv rd, rs2

操作：

rd ← rs2;

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs2 != 0, rd !=0。

指令格式：

15 13	12	11 7	6 2	1 0
100	0	rd	rs2	10

14.6.26　C.NOP——空指令

语法：

c.nop

操作：

无操作

执行权限：

M mode/S mode/U mode

异常：

无

指令格式：

15 13	12	11 7	6 2	1 0
000	0	00000	00000	01

14.6.27　C.OR——按位或指令

语法：

c.or rd, rs2

操作：

rd ← rs1 | rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 = rd

rd/rs1 编码代表寄存器如下:

- 000: x8
- 001: x9

- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：

15　　13	12	11　10	9　　7	6　5	4　　2	1　0
100	0	11	rs1/rd	10	rs2	01

14.6.28　C.SD——双字存储指令

语法：

c.sd rs2, uimm5<<3(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<3)

mem[address+7:address] ←rs2

执行权限：

M mode/S mode/U mode

异常：

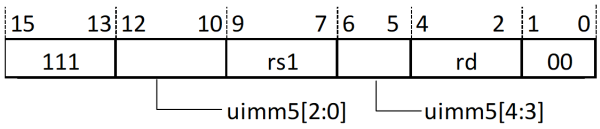
存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

说明：

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.29　C.SDSP——双字堆栈存储指令

语法：

c.fsdsp rs2, uimm6<<3(sp)

操作：

address ← sp+ zero_extend(uimm6<<3)

mem[address+7:address] ←rs2

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

15 13	12 7	6 2	1 0
111	uimm6[2:0|5:3]	rs2	10

14.6.30　C.SLLI——立即数逻辑左移指令

语法：

c.slli rd, nzuimm6

操作：

rd ←rs1 << nzuimm6

执行权限：

M mode/S mode/U mode

异常：

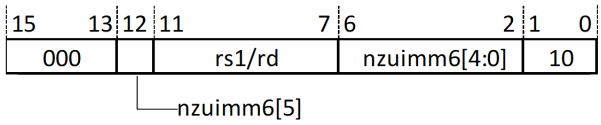
无

说明：

rs1==rd

rd/rs1 != 0，nzuimm6 != 0

指令格式：



14.6.31　C.SRAI——立即数算数右移指令

语法：

c.srli rd, nzuimm6

操作：

rd ←rs1 >>>nzuimm6

执行权限：

M mode/S mode/U mode

异常：

无

说明：

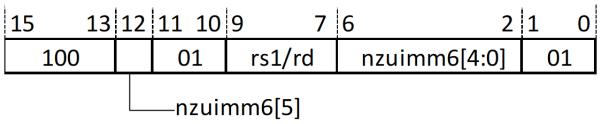
nzuimm6 != 0

rs1 == rd

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.32　C.SRLI——立即数逻辑右移指令

语法：

c.srli rd, nzuimm6

操作：

rd ←rs1 >> nzuimm6

执行权限：

M mode/S mode/U mode

异常：

无

说明：

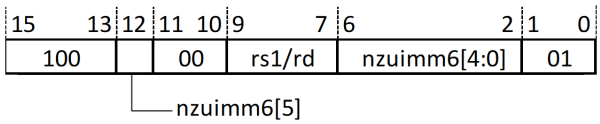
nzuimm6 != 0

rs1 == rd

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.33　C.SW——字存储指令

语法：

c.sw rs2, uimm5<<2(rs1)

操作：

address ← rs1+ zero_extend(uimm5<<2)

mem[address+3:address] ←rs2

执行权限：

M mode/S mode/U mode

异常：

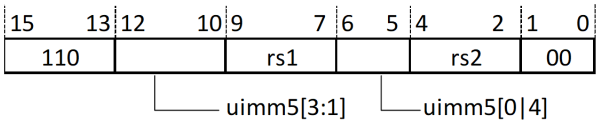
存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

说明：

rs1/rs2 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：



14.6.34　C.SWSP——字堆栈存储指令

语法：

c.swsp rs2, uimm6<<2(sp)

操作：

address ← sp+ zero_extend(uimm6<<2)

mem[address+3:address] ←rs2

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常

指令格式：

15　　13	12　　7	6　　2	1　　0
110	uimm6[3:0|5:4]	rs2	10

14.6.35　C.SUB——有符号减法指令

语法：

c.sub rd, rs2

操作：

rd ← rs1 - rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 == rd

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：

15　　13	12	11　10	9　　7	6　5	4　　2	1　0
100	0	11	rs1/rd	00	rs2	01

14.6.36　C.SUBW——低 32 位有符号减法指令

语法：

c.subw rd, rs2

操作：

tmp[31:0] ← rs1[31:0] - rs2[31:0]

rd ←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 == rd

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式

15　　13	12	11　10	9　　7	6　5	4　　2	1　0
100	1	11	rs1/rd	00	rs2	01

14.6.37　C.XOR——按位异或指令

语法：

c.xor rd, rs2

操作：

rd ← rs1 ^ rs2

执行权限：

M mode/S mode/U mode

异常：

无

说明：

rs1 == rd

rs1/rd 编码代表寄存器如下:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

指令格式：

15　13	12	11　10	9　7	6　5	4　2	1　0
100	0	11	rs1/rd	01	rs2	01

14.7　附录 A-8 伪指令列表

RISC-V 实现了一系列的伪指令，在此列出仅供参考，按英文字母顺序排列。

伪指令	基础指令	含义
beqz rs, offset	beq rs, x0, offset	寄存器为零分支跳转
bnez rs, offset	bne rs, x0, offset	寄存器不为零分支跳转

下页继续

表 14.1 – 续上页

伪指令	基础指令	含义
blez rs, offset	bge x0,rs,offset	寄存器小于等于零跳转
bgez rs, offset	bge rs, x0, offset	寄存器大于等于零跳转
bltz rs, offset	blt rs, x0, offset	寄存器小于零跳转
bgtz rs, offset	blt x0, xs, offset	寄存器大于零跳转
bgt rs, rt, offset	blt rt, rs, offset	比较大于分支跳转
ble rs, rt, offset	bge rt, rs, offset	比较小于等于分支跳转
bgtu rs, rt, offset	bltu rt, rs, offset	无符号比较大于分支跳转
bleu rs, rt, offset	bgeu rt, rs, offset	无符号比较小于等于分支跳转
call offset	auipc x6, offset[31:12] jalr x1, x6, offset[11:0]	跳转 4KB-4GB 空间的函数
csrc csr, rs	csrrc x0, csr, rs	清除控制寄存器中对应比特
csrci csr, imm	csrrci x0, csr, imm	清除控制寄存器低 6 位中对应比特
csrs csr, rs	csrrs x0, csr, rs	置位控制寄存器中对应比特
csrsi csr, imm	csrrsi x0, csr, imm	置位控制寄存器低 6 位中对应比特
csrw csr, rs	csrrw x0, csr, rs	写控制寄存器中对应比特
csrwi csr, imm	csrrwi x0, csr, imm	写控制寄存器低 6 位中对应比特
fabs.d rd , rs	fsgnjx.d rd, rs,rs	双精度浮点数取绝对值
fabs.s rd , rs	fsgnjx.s rd, rs,rs	单精度浮点数取绝对值
fence	fence iorw, iorw	存储和外设同步指令
fl{w|d} rd, symbol, rt	auipc rt, symbol[31:12] fl{w|d} rd, symbol[11:0](rt)	4GB 地址空间浮点加载指令
fmv.d rd, rs	fsgnj.d rd, rs,rs	双精度浮点复制指令
fmv.s rd, rs	fsgnj.s rd, rs,rs	单精度浮点复制指令
fneg.d rd, rs	fsgnjn.d rd, rs,rs	双精度浮点取负指令
fneg.s rd, rs	fsgnjn.s rd, rs,rs	单精度浮点取负指令
frcsr rd	csrrs x0, fcsr, x0	浮点控制寄存器读取指令
frflags rd	csrrs rd, fflags,x0	浮点异常位读取指令
frrm rd	csrrs rd, frm,x0	浮点舍入位读取指令
fscsr rs	csrrw x0, fcsr,rs	写浮点控制寄存器指令
fscsr rd, rs	csrrs rd, fcsr, rs	浮点控制寄存器读写指令
fsflags rs	csrrw x0, fcsr,rs	写浮点异常位指令
fsflags rd, rs	csrrs rd, fcsr, rs	浮点异常位读写指令
fsflagsi imm	csrrwi x0, fflags,imm	立即数写浮点异常位指令
fsflagsi rd, imm	csrrwi rd, fflags, imm	浮点异常位立即数读写指令
fsrm rs	csrrw x0, frm,rs	写浮点舍入位指令
fsrm rd, rs	csrrs rd, frm, rs	浮点舍入位读写指令

下页继续

表 14.1 – 续上页

伪指令	基础指令	含义
fsrmi imm	csrrwi x0, frm,imm	立即数写浮点舍入位指令
fsrmi rd, imm	csrrwi rd, frm, imm	浮点舍入位立即数读写指令
fs{w|d} rd, symbol, rt	auipc rt, symbol[31:12] fs{w|d} rd, symbol[11:0](rt)	4GB 地址空间浮点存储指令
j offset	jal x0, offset	直接跳转指令
jal offset	jal x1, offset	子程序跳转和链接指令
jalr rs	jalr x1, rs, 0	子程序跳转寄存器和链接寄存器指令
jr rs	jalr x0, rs, 0	跳转寄存器指令
la rd, symbol	auipc rd, symbol[31:12] addi rd, rd, symbol[11:0]	指令地址加载指令
li rd, immediate	根据立即数大小拆分为多条指令	立即数加载指令
l{b|h|w|d} rd, symbol, rt	auipc rt, symbol[31:12] l{b|h|w|d} rd, symbol[11:0](rt)	4GB 地址空间加载指令
mv rd, rs	addi rd, rs, 0	数据传送指令
neg rd, rs	sub rd, x0, rs	寄存器取负指令
negw rd, rs	subw rd, x0, rs	寄存器低 32 位取负指令
nop	addi x0,x0,0	空指令
not rd, rs	xori rd, rs, -1	寄存器取反指令
rdcycle[h] rd	csrrs rd, cycle[h], x0	周期数读取指令
rdinstret[h] rd	csrrs rd, instret[h], x0	指令数读取指令
rdtime[h] rd	csrrs rd, time[h], x0	真实时钟读取指令
ret	jalr x0, x1,0	子程序返回指令
s{b|h|w|d} rd, symbol, rt	auipc rt, symbol[31:12] s{b|h|w|d} rd, symbol[11:0](rt)	4GB 地址空间存储指令
seqz rd, rs	sltiu rd, rs, 1	寄存器为 0 置 1 指令
sextw rd, rs	addiw rd, rs, 0	符号位扩展指令
sgtz rd, rs	slt rd, rs, x0, rs	寄存器大于 0 置 1 指令
sltz rd, rs	slt rd, rs, rs, x0	寄存器小于 0 置 1 指令
snez rd, rs	sltu rd, rs, x0, rs	寄存器不为 0 置 1 指令
tail offset	auipc x6, offset[31:12] jalr x0, x6, offset[11:0]	寄存器不链接跳转子程序指令

第十五章　附录 B 平头哥扩展指令术语

除了标准中定义的 GC 指令集外，C910 实现了自定义的指令集，包括 Cache 指令子集，同步指令子集，算数运算指令子集，位操作指令子集，存储指令子集以及浮点半精度指令子集。

其中，Cache 指令子集、同步指令子集、算数运算指令子集、位操作指令子集以及存储指令子集需要在 mxstatus.theadisaee == 1 时方可正常执行，否则产生非法指令异常；浮点半精度指令子集需要在 mstatus.fs !=2’b00 时方可正常执行，否则产生非法指令异常，以下按照不同指令子集扩展对每条指令做具体描述。

15.1　附录 B-1 Cache 指令术语

Cache 指令子集实现了对 cache 的操作，每条指令位宽为 32 位。

以下指令按英文字母顺序排列。

15.1.1　DCACHE.CALL——DCACHE 清全部脏表项指令

语法：

dcache.call

操作：

clear 所有 L1 dcache 表项，将所有 dirty 表项写回到下一级存储，仅操作当前核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	00001	00000	000	00000	0001011

15.1.2　DCACHE.CIALL——DCACHE 清全部脏表项后无效指令

语法：

dcache.ciall

操作：

将所有 L1 dcache dirty 表项写回到下一级存储后，无效所有表项。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	00011	00000	000	00000	0001011

15.1.3　DCACHE.CIPA ——DCACHE 按物理地址清脏表项并无效

语法：

dcache.cipa rs1

操作：

将 rs1 中物理地址所属的 dcache/L2cache 表项写回下级存储并无效该表项，操作所有核和 L2CACHE。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	01011	rs1	000	00000	0001011

15.1.4 DCACHE.CISW——DCACHE 按 way/set 清脏表项并无效指令

语法：

dcache.cisw rs1

操作：

按照 rs1 中指定的 way/set 将 L1 dache dirty 表项写回到下一级存储并无效该表项，仅操作当前核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

C910 dcache 为两路组相联，rs1[31] 为 way 编码，rs1[w:6] 为 set 编码。当 dcache 为 32K 时,w 为 13，dcache 为 64K 时，w 为 14。

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00011	rs1	000	00000	0001011

15.1.5 DCACHE.CIVA——DCACHE 按虚拟地址清脏表项并无效

语法：

dcache.civa rs1

操作：

将 rs1 指定虚拟地址所属的 dcache/L2 cache 表项写回到下级存储，并无效该表项，操作当前核和 L2CACHE，并根据虚拟地址共享属性决定是否广播到其他核。

执行权限：

M mode/S mode/U mode

异常：

非法指令异常/加载指令页面错误异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，mxstatus.ucme =1，U mode 下可以执行该指令。
- mxstatus.theadisaee=1，mxstatus.ucme =0，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00111	rs1	000	00000	0001011

15.1.6　DCACHE.CPA——DCACHE 按物理地址清脏表项

语法：

dcache.cpa rs1

操作：

将 rs1 中物理地址所对应的 dcache/l2cache 表项写回到下一级存储，操作所有核和 L2CACHE。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	01001	rs1	000	00000	0001011

15.1.7　DCACHE.CPAL1 ——L1DCACHE 按物理地址清脏表项

语法：

dcache.cpal1 rs1

操作：　将 rs1 中物理地址所对应的 dcache 表项写回到下一级存储，操作所有核 L1CACHE。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	01000	rs1	000	00000	0001011

15.1.8　DCACHE.CVA——DCACHE 按虚拟地址清脏表项

语法：

dcache.cva rs1

操作：

将 rs1 中虚拟地址所对应的 dcache/l2cache 表项写回到下一级存储，操作当前核和 L2CACHE，并根据虚拟地址共享属性决定是否广播到其他核

执行权限：

M mode/S mode

异常：

非法指令异常/加载指令页面错误异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00101	rs1	000	00000	0001011

15.1.9　DCACHE.CVAL1——L1DCACHE 按虚拟地址清脏表项

语法：

dcache.cval1 rs1

操作：

将 rs1 中虚拟地址所对应的 dcache 表项写回到下一级存储，操作所有核 L1CACHE

执行权限：

M mode/S mode/U mode

异常：

非法指令异常/加载指令页面错误异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，mxstatus.ucme =0，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00100	rs1	000	00000	0001011

15.1.10　DCACHE.IPA ——DCACHE 按物理地址无效指令

语法：

dcache.ipa rs1

操作：

将 rs1 中物理地址所对应的 dcache/l2 cache 表项无效，操作所有核和 L2CACHE。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	01010	rs1	000	00000	0001011

15.1.11　DCACHE.ISW ——DCACHE 按 set/way 无效指令

语法：

dcache.isw rs1

操作：

无效指定 SET 和 WAY 的 dcache 表项，仅操作当前核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

C910 dcache 为两路组相联，rs1[31] 为 way 编码，rs1[w:6] 为 set 编码。当 dcache 为 32K 时,w 为 13，dcache 为 64K 时，w 为 14。

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00010	rs1	000	00000	0001011

15.1.12　DCACHE.IVA ——DCACHE 按虚拟地址无效指令

语法：

dcache.iva rs1

操作：

将 rs1 中虚拟地址所对应的 dcache/l2 cache 表项无效，操作当前核和 L2CACHE，并根据虚拟地址共享属性决定是否广播到其他核。

执行权限：

M mode/S mode

异常：

非法指令异常/加载指令页面错误异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	00110	rs1	000	00000	0001011

15.1.13　DCACHE.IALL——DCACHE 无效所有表项指令

语法：

dcache.iall

操作：

无效所有 L1 dcache 表项，仅操作当前核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	00010	00000	000	00000	0001011

15.1.14　ICACHE.IALL——ICACHE 无效所有表项指令

语法：

icache.iall

操作：

无效所有 icache 表项，仅操作当前核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	10000	00000	000	00000	0001011

15.1.15　ICACHE.IALLS——ICACHE 广播无效所有表项指令

语法：

icache.ialls

操作：

无效所有 icache 表项，并广播其他核去无效各自所有 icache 表项，操作所有核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	10001	00000	000	00000	0001011

15.1.16　ICACHE.IPA——ICACHE 按物理地址无效表项指令

语法：

icache.ipa rs1

操作：

将 rs1 中物理地址所对应的 icache 表项无效，操作所有核。

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	11000	rs1	000	00000	0001011

15.1.17　ICACHE.IVA——ICACHE 按虚拟地址无效表项指令

语法：

icache.iva rs1

操作：

将 rs1 中虚拟地址所对应的 icache 表项无效，操作当前核，并根据虚拟地址共享属性决定是否广播到其他核。

执行权限：

M mode/S mode/U mode

异常：

非法指令异常/加载指令页面错误异常

说明：

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，mxstatus.ucme=1，U mode 下可以执行该指令。

mxstatus.theadisaee=1，mxstatus.ucme=0，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000001	10000	rs1	000	00000	0001011

15.1.18　L2CACHE.CALL——L2CACHE 清所有脏表项指令

语法：

l2cache.call

操作：

将 l2cache 中所有 dirty 表项写回到下一级存储

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	10101	00000	000	00000	0001011

15.1.19　L2CACHE.CIALL——L2CACHE 清所有脏表项并无效指令

语法：

l2cache.ciall

操作：

将 l2cache 中所有 dirty 表项写回到下一级存储后无效所有 l2 表项

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

- mxstatus.theadisaee=0，执行该指令产生非法指令异常。
- mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	10111	00000	000	00000	0001011

15.1.20　L2CACHE.IALL——L2CACHE 无效指令

语法：

l2cache.iall

操作：

将 l2cache 中所有表项无效

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

- mxstatus.cskisayee=0，执行该指令产生非法指令异常。
- mxstatus.cskisayee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	10110	00000	000	00000	0001011

15.1.21　DCACHE.CSW ——DCACHE 按 set/way 清脏表项

语法：

dcache.csw rs1

操作：

按 SET 和 WAY 将 dcache 中的脏表项回写到下一级存储器

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

C910 dcache 为两路组相联，rs1[31] 为 way 编码，rs1[w:6] 为 set 编码。当 dcache 为 32K 时,w 为 13，dcache 为 64K 时，w 为 14。

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

15.2　附录 B-2 多核同步指令术语

同步指令子集实现了多核同步指令的扩展，每条指令位宽为 32 位，以下指令按英文字母顺序排列。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0	0	0	0	0	0	1	0	0	0	0	1	rs1			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

图 15.1: DCACHE.CSW

15.2.1　SFENCE.VMAS——虚拟内存同步广播指令

语法：

sfence.vmas rs1,rs2

操作：

虚拟内存的无效和同步操作，需要广播到 cluster 里的其他核

执行权限：

M mode/S mode

异常：

非法指令异常

说明：

rs1：虚拟地址，rs2：asid

- rs1=x0，rs2=x0 时，无效 TLB 中所有表项，并广播到 cluster 中的其他核
- rs1!=x0，rs2=x0 时，无效 TLB 中所有命中 rs1 虚拟地址的表项，并广播到 cluster 中的其他核。
- rs1=x0，rs2!=x0 时，无效 TLB 中所有命中 rs2 进程号的表项，并广播到 cluster 中的其他核。
- rs1!=x0，rs2!=x0 时，无效 TLB 中所有命中 rs1 虚拟地址和 rs2 进程号的表项，并广播到 cluster 中的其他核。

mxstatus.theadisaee=0，执行该指令产生非法指令异常。

mxstatus.theadisaee=1，U mode 下执行该指令产生非法指令异常。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000010	rs2	rs1	000	00000	0001011

15.2.2　SYNC——同步指令

语法：

sync

操作：

该指令保证前序所有指令比该指令早退休，后续所有指令比该指令晚退休

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	11000	00000	000	00000	0001011

15.2.3　SYNC.I——同步清空指令

语法：

sync.i

操作：

该指令保证前序所有指令比该指令早退休，后续所有指令比该指令晚退休，该指令退休时清空流水线

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	11010	00000	000	00000	0001011

15.2.4　SYNC.IS——同步清空广播指令

语法：

sync.is

操作：

该指令保证前序所有指令比该指令早退休，后续所有指令比该指令晚退休，该指令退休时清空流水线，并将该请求广播给其他核

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	11011	00000	000	00000	0001011

15.2.5　SYNC.S——同步广播指令

语法：

sync.s

操作：

该指令保证前序所有指令比该指令早退休，后续所有指令比该指令晚退休，并将该请求广播给其他核

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000000	11001	00000	000	00000	0001011

15.3　附录 B-3 算术运算指令术语

算术运算指令子集实现了对算术指令的扩展，每条指令位宽为 32 位。

以下指令按英文字母顺序排列。

15.3.1　ADDSL——寄存器移位相加指令

语法：

addsl rd rs1, rs2, imm2

操作：

rd ← rs1+ rs2<<imm2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00000	imm2	rs2	rs1	001	rd	0001011

15.3.2　MULA——乘累加指令

语法：

mula rd, rs1, rs2

操作：

rd ← rd+ (rs1 * rs2)[63:0]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	00	rs2	rs1	001	rd	0001011

15.3.3　MULAH——低 16 位乘累加指令

语法：

mulah rd, rs1, rs2

操作：

tmp[31:0] ← rd[31:0]+ (rs1[15:0] * rs[15:0])

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00101	00	rs2	rs1	001	rd	0001011

15.3.4　MULAW——低 32 位乘累加指令

语法：

mulaw rd, rs1, rs2

操作：

tmp[31:0] ← rd[31:0]+ (rs1[31:0] * rs[31:0])[31:0]

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	10	rs2	rs1	001	rd	0001011

15.3.5　MULS——乘累减指令

语法：

muls rd, rs1, rs2

操作：

rd ← rd- (rs1 * rs2)[63:0]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	01	rs2	rs1	001	rd	0001011

15.3.6　MULSH——低 16 位乘累减指令

语法：

mulsh rd, rs1, rs2

操作：

tmp[31:0] ← rd[31:0]- (rs1[15:0] * rs[15:0])

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00101	01	rs2	rs1	001	rd	0001011

15.3.7　MULSW——低 32 位乘累减指令

语法：

mulaw rd, rs1, rs2

操作：

tmp[31:0] ← rd[31:0]- (rs1[31:0] * rs[31:0])

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	11	rs2	rs1	001	rd	0001011

15.3.8　MVEQZ——寄存器为 0 传送指令

语法：

mveqz rd, rs1, rs2

操作： if (rs2 == 0)

rd ← rs1

else

rd ← rd

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	00	rs2	rs1	001	rd	0001011

15.3.9　MVNEZ——寄存器非 0 传送指令

语法：

mvnez rd, rs1, rs2

操作：

if (rs2 != 0)

rd ← rs1

else

rd ← rd

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	01	rs2	rs1	001	rd	0001011

15.3.10　SRRI——循环右移指令

语法：

srri rd, rs1, imm6

操作：

rd ← rs1 >>>> imm6

rs1 原值右移，左侧移入右侧移出位

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
000100	imm6	rs1	001	rd	0001011

15.3.11　SRRIW——低 32 位循环右移指令

语法：

srriw rd, rs1, imm5

操作：

rd ← sign_extend(rs1[31:0] >>>> imm5)

rs1[31:0] 原值右移，左侧移入右侧移出位

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001010	imm5	rs1	001	rd	0001011

15.4　附录 B-4 位操作指令术语

位操作指令子集实现了对位运算指令的扩展，每条指令位宽为 32 位。

以下指令按英文字母顺序排列。

15.4.1　EXT——寄存器连续位提取符号位扩展指令

语法：

ext rd, rs1, imm1,imm2

操作：

rd←sign_extend(rs1[imm1:imm2])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

若 imm1<imm2，该指令行为不可预测

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
imm1	imm2	rs1	010	rd	0001011

15.4.2　EXTU——寄存器连续位提取零扩展指令

语法：

extu rd, rs1, imm1,imm2

操作：

rd←zero_extend(rs1[imm1:imm2])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

说明：

若 imm1<imm2，该指令行为不可预测

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
imm1	imm2	rs1	011	rd	0001011

15.4.3　FF0——快速找 0 指令

语法：

ff0 rd, rs1

操作：

从 rs1 最高位开始查找第一个为 0 的位，结果写回 rd。如果 rs1 的最高位为 0，则结果为 0，如果 rs1 中没有 0，结果为 64

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10000	10	00000	rs1	001	rd	0001011

15.4.4　FF1——快速找 1 指令

语法：

ff1 rd, rs1

操作：

从 rs1 最高位开始查找第一个为 1 的位，将该位的索引写回 rd。如果 rs1 的最高位为 1，则结果为 0，如果 rs1 中没有 1，结果为 64

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10000	11	00000	rs1	001	rd	0001011

15.4.5　REV——字节倒序指令

语法：

rev rd, rs1

操作：

rd[63:56] ←rs1[7:0]

rd[55:48] ←rs1[15:8]

rd[47:40] ←rs1[23:16]

rd[39:32] ←rs1[31:24]

rd[31:24] ←rs1[39:32]

rd[23:16] ←rs1[47:40]

rd[15:8] ←rs1[55:48]

rd[7:0] ←rs1[63:56]

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10000	01	00000	rs1	001	rd	0001011

15.4.6　REVW——低 32 位字节倒序指令

语法：

revw rd, rs1

操作：

tmp[31:24] ←rs1[7:0]

tmp [23:16] ←rs1[15:8]

tmp [15:8] ←rs1[23:16]

tmp [7:0] ←rs1[31:24]

rd ←sign_extend(tmp[31:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10010	00	00000	rs1	001	rd	0001011

15.4.7　TST——比特为 0 测试指令

语法：

tst rd, rs1, imm6

操作：

if(rs1[imm6] == 1)

　　rd←1

else

　　rd←0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 26	25 20	19 15	14 12	11 7	6 0
100010	imm6	rs1	001	rd	0001011

15.4.8　TSTNBZ——字节为 0 测试指令

语法：

tstnbz rd, rs1

操作：

rd[63:56] ← (rs1[63:56] == 0) ? 8’ hff : 8’ h0

rd[55:48] ← (rs1[55:48] == 0) ? 8’ hff : 8’ h0

rd[47:40] ← (rs1[47:40] == 0) ? 8’ hff : 8’ h0

rd[39:32] ← (rs1[39:32] == 0) ? 8’ hff : 8’ h0

rd[31:24] ← (rs1[31:24] == 0) ? 8’ hff : 8’ h0

rd[23:16] ← (rs1[23:16] == 0) ? 8’ hff : 8’ h0

rd[15:8] ← (rs1[15:8] == 0) ? 8’ hff : 8’ h0

rd[7:0] ← (rs1[7:0] == 0) ? 8’ hff : 8’ h0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10000	00	00000	rs1	001	rd	0001011

15.5　附录 B-5 存储指令术语

存储指令子集实现了对存储指令的扩展，每条指令位宽为 32 位。

以下指令按英文字母顺序排列。

15.5.1　FLRD——浮点寄存器移位双字加载指令

语法：

flrd rd, rs1, rs2, imm2

操作：

rd ←mem[(rs1+rs2<<imm2)+7: (rs1+rs2<<imm2)]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常, 非法指令异常

说明：

mxstatus.theadisaee=1’ b0 或 mstatus.fs =2’ b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01100	imm2	rs2	rs1	110	rd	0001011

15.5.2　FLRW——浮点寄存器移位字加载指令

语法：

flrw rd, rs1, rs2, imm2

操作：

rd ←one_extend(mem[(rs1+rs2<<imm2)+3: (rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常，非法指令异常

说明：

mxstatus.theadisaee=1’b0 或 mstatus.fs =2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	imm2	rs2	rs1	110	rd	0001011

15.5.3 FLURD——浮点寄存器低 32 位移位双字加载指令

语法：

flurd rd, rs1, rs2, imm2

操作：

rd ←mem[(rs1+rs2[31:0]<<imm2)+7: (rs1+rs2[31:0]<<imm2)]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

mxstatus.theadisaee=1’b0 或 mstatus.fs = 2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01110	imm2	rs2	rs1	110	rd	0001011

15.5.4　FLURW——浮点寄存器低 32 位移位字加载指令

语法：

flurw rd, rs1, rs2, imm2

操作：

rd ←one_extend(mem[(rs1+rs2[31:0]<<imm2)+3: (rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

mxstatus.theadisaee=1’b0 或 mstatus.fs = 2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01010	imm2	rs2	rs1	110	rd	0001011

15.5.5　FSRD——浮点寄存器移位双字存储指令

语法：

fsrd rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)+7: (rs1+rs2<<imm2)] ←rd[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

mxstatus.theadisaee=1’b0 或 mstatus.fs =2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01100	imm2	rs2	rs1	111	rd	0001011

15.5.6　FSRW——浮点寄存器移位字存储指令

语法：

fsrw rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)+3: (rs1+rs2<<imm2)] ←rd[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

mxstatus.theadisaee=1’b0 或 mstatus.fs =2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	imm2	rs2	rs1	111	rd	0001011

15.5.7　FSURD——浮点寄存器低 32 位移位双字存储指令

语法：

fsurd rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)+7: (rs1+rs2[31:0]<<imm2)] ←rd[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

mxstatus.theadisaee=1’b0 或 mstatus.fs = 2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01110	imm2	rs2	rs1	111	rd	0001011

15.5.8　FSURW——浮点寄存器低 32 位移位字存储指令

语法：

fsurw rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)+3: (rs1+rs2[31:0]<<imm2)] ←rd[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

mxstatus.theadisaee=1’b0 或 mstatus.fs = 2’b00 时，该指令产生非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01010	imm2	rs2	rs1	111	rd	0001011

15.5.9　LBIA——符号位扩展字节加载基地址自增指令

语法：

lbia rd, (rs1), imm5,imm2

操作：

rd ←sign_extend(mem[rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00011	imm2	imm5	rs1	100	rd	0001011

15.5.10　LBIB——基地址自增符号位扩展字节加载指令

语法：

lbib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←sign_extend(mem[rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00001	imm2	imm5	rs1	100	rd	0001011

15.5.11　LBUIA——零扩展字节加载基地址自增指令

语法：

lbuia rd, (rs1), imm5,imm2

操作：

rd ←zero_extend(mem[rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10011	imm2	imm5	rs1	100	rd	0001011

15.5.12　LBUIB——基地址自增零扩展字节加载指令

语法：

lbuib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←zero_extend(mem[rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10001	imm2	imm5	rs1	100	rd	0001011

15.5.13　LDD——双寄存器加载指令

语法：

ldd rd1,rd2, (rs1),imm2

操作：

address←rs1 + zero_extend(imm2<<4)

rd1←mem[address+7:address]

rd2←mem[address+15:address+8]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rd1,rd2 ,rs1 互相不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11111	imm2	rd2	rs1	100	rd1	0001011

15.5.14　LDIA——符号位扩展双字加载基地址自增指令

语法：

ldia rd, (rs1), imm5,imm2

操作：

rd ←sign_extend(mem[rs1+7:rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01111	imm2	imm5	rs1	100	rd	0001011

15.5.15　LDIB——基地址自增符号位扩展双字加载指令

语法：

ldib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←sign_extend(mem[rs1+7:rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01101	imm2	imm5	rs1	100	rd	0001011

15.5.16　LHIA——符号位扩展半字加载基地址自增指令

语法：

lhia rd, (rs1), imm5,imm2

操作：

rd ←sign_extend(mem[rs1+1:rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00111	imm2	imm5	rs1	100	rd	0001011

15.5.17　LHIB——基地址自增符号位扩展半字加载指令

语法：

lhib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←sign_extend(mem[rs1+1:rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00101	imm2	imm5	rs1	100	rd	0001011

15.5.18　LHUIA——零扩展半字加载基地址自增指令

语法：

lhuia rd, (rs1), imm5,imm2

操作：

rd ←zero_extend(mem[rs1+1:rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10111	imm2	imm5	rs1	100	rd	0001011

15.5.19　LHUIB——基地址自增零扩展半字加载指令

语法：

lhuib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←zero_extend(mem[rs1+1:rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10101	imm2	imm5	rs1	100	rd	0001011

15.5.20　LRB——寄存器移位符号位扩展字节加载指令

语法：

lrb rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00000	imm2	rs2	rs1	100	rd	0001011

15.5.21　LRBU——寄存器移位零扩展扩展字节加载指令

语法：

lrbu rd, rs1, rs2, imm2

操作：

rd ←zero_extend(mem[(rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10000	imm2	rs2	rs1	100	rd	0001011

15.5.22　LRD——寄存器移位双字加载指令

语法：

lrd rd, rs1, rs2, imm2

操作：

rd ←mem[(rs1+rs2<<imm2)+7: (rs1+rs2<<imm2)]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01100	imm2	rs2	rs1	100	rd	0001011

15.5.23　LRH——寄存器移位符号位扩展半字加载指令

语法：

lrh rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2<<imm2)+1: (rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	imm2	rs2	rs1	100	rd	0001011

15.5.24　LRHU——寄存器移位零扩展扩展半字加载指令

语法：

lrhu rd, rs1, rs2, imm2

操作：

rd ←zero_extend(mem[(rs1+rs2<<imm2)+1: (rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10100	imm2	rs2	rs1	100	rd	0001011

15.5.25　LRW——寄存器移位符号位扩展字加载指令

语法：

lrw rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2<<imm2)+3: (rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	imm2	rs2	rs1	100	rd	0001011

15.5.26　LRWU——寄存器移位零扩展扩展字加载指令

语法：

lrwu rd, rs1, rs2, imm2

操作：

rd ←zero_extend(mem[(rs1+rs2<<imm2)+3: (rs1+rs2<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11000	imm2	rs2	rs1	100	rd	0001011

15.5.27　LURB——寄存器低 32 位移位符号位扩展字节加载指令

语法：

lurb rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00010	imm2	rs2	rs1	100	rd	0001011

15.5.28　LURBU——寄存器低 32 位移位零扩展字节加载指令

语法：

lurbu rd, rs1, rs2, imm2

操作：

rd ←zero_extend(mem[(rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10010	imm2	rs2	rs1	100	rd	0001011

15.5.29　LURD——寄存器低 32 位移位双字加载指令

语法：

lurd rd, rs1, rs2, imm2

操作：

rd ←mem[(rs1+rs2[31:0]<<imm2)+7: (rs1+rs2[31:0]<<imm2)]

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01110	imm2	rs2	rs1	100	rd	0001011

15.5.30　LURH——寄存器低 32 位移位符号位扩展半字加载指令

语法：

lurh rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2[31:0]<<imm2)+1:

(rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00110	imm2	rs2	rs1	100	rd	0001011

15.5.31　LURHU——寄存器低 32 位移位零扩展半字加载指令

语法：

lurhu rd, rs1, rs2, imm2

操作：

rd ←zero_extend(mem[(rs1+rs2[31:0]<<imm2)+1:

(rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
10110	imm2	rs2	rs1	100	rd	0001011

15.5.32　LURW——寄存器低 32 位移位符号位扩展字加载指令

语法：

lurw rd, rs1, rs2, imm2

操作：

rd ←sign_extend(mem[(rs1+rs2[31:0]<<imm2)+3:

(rs1+rs2[31:0]<<imm2)])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数，高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01010	imm2	rs2	rs1	100	rd	0001011

15.5.33　LURWU——寄存器低 32 位移位零扩展字加载指令

语法：

lwd rd1, rd2, (rs1),imm2

操作：

address←rs1+zero_extend(imm2<<3)

rd1 ←sign_extend(mem[address+3: address])

rd2 ←sign_extend(mem[address+7: address+4])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rd1,rd2 ,rs1 互相不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11010	imm2	rs2	rs1	100	rd	0001011

15.5.34　LWD——符号位扩展双寄存器字加载指令

语法：

lwd rd, imm7(rs1)

操作：

address←rs1+sign_extend(imm7)

rd ←sign_extend(mem[address+31: address])

rd+1 ←sign_extend(mem[address+63: address+32])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11100	imm2	rd2	rs1	100	rd1	0001011

15.5.35　LWIA——符号位扩展字加载基地址自增指令

语法：

lwia rd, (rs1), imm5,imm2

操作：

rd ←sign_extend(mem[rs1+3:rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01011	imm2	imm5	rs1	100	rd	0001011

15.5.36　LWIB——基地址自增符号位扩展字加载指令

语法：

lwib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←sign_extend(mem[rs1+3:rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01001	imm2	imm5	rs1	100	rd	0001011

15.5.37　LWUD——零扩展双寄存器字加载指令

语法：

lwud rd1,rd2, (rs1),imm2

操作：

address←rs1+zero_extend(imm2<<3)

rd1 ←zero_extend(mem[address+3: address])

rd2 ←zero_extend(mem[address+7: address+4])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

说明：

rd1,rd2 ,rs1 互相不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11110	imm2	rd2	rs1	100	rd1	0001011

15.5.38　LWUIA——零扩展字加载基地址自增指令

语法：

lwuia rd, (rs1), imm5,imm2

操作：

rd ←zero_extend(mem[rs1+3:rs1])

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11011	imm2	imm5	rs1	100	rd	0001011

15.5.39　LWUIB——基地址自增零扩展字加载指令

语法：

lwuib rd, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

rd ←zero_extend(mem[rs1+3:rs1])

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常。

说明：

rd 和 rs1 不可相等

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11001	imm2	imm5	rs1	100	rd	0001011

15.5.40　SBIA——字节存储基地址自增指令

语法：

sbia rs2, (rs1), imm5,imm2

操作：

mem[rs1]←rs2[7:0]

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00011	imm2	imm5	rs1	101	rs2	0001011

15.5.41　SBIB——基地址自增字节存储指令

语法：

sbib rs2, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

mem[rs1] ←rs2[7:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00001	imm2	imm5	rs1	101	rs2	0001011

15.5.42　SDD——双寄存器存储指令

语法：

sdd rd1,rd2, (rs1),imm2

操作：

address←rs1 + zero_extend(imm2<<4)

mem[address+7:address] ←rd1

mem[address+15:address+8]←rd2

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11111	imm2	rd2	rs1	101	rd1	0001011

15.5.43　SDIA——双字存储基地址自增指令

语法：

sdia rs2, (rs1), imm5,imm2

操作：

mem[rs1+7:rs1]←rs2[63:0]

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01111	imm2	imm5	rs1	101	rs2	0001011

15.5.44　SDIB——基地址自增双字存储指令

语法：

sdib rs2, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

mem[rs1+7:rs1] ←rs2[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01101	imm2	imm5	rs1	101	rs2	0001011

15.5.45　SHIA——半字存储基地址自增指令

语法：

shia rs2, (rs1), imm5,imm2

操作：

mem[rs1+1:rs1]←rs2[15:0]

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00111	imm2	imm5	rs1	101	rs2	0001011

15.5.46　SHIB——基地址自增半字存储指令

语法：

shib rs2, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

mem[rs1+1:rs1] ←rs2[15:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00101	imm2	imm5	rs1	101	rs2	0001011

15.5.47　SRB——寄存器移位字节存储指令

语法：

srb rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)] ←rd[7:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00000	imm2	imm5	rs1	101	rd	0001011

15.5.48　SRD——寄存器移位双字存储指令

语法：

srd rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)+7: (rs1+rs2<<imm2)] ←rd[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01100	imm2	rs2	rs1	101	rd	0001011

15.5.49　SRH——寄存器移位半字存储指令

语法：

srh rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)+1: (rs1+rs2<<imm2)] ←rd[15:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00100	imm2	rs2	rs1	101	rd	0001011

15.5.50　SRW——寄存器移位字存储指令

语法：

srw rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2<<imm2)+3: (rs1+rs2<<imm2)] ←rd[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01000	imm2	rs2	rs1	101	rd	0001011

15.5.51　SURB——寄存器低 32 位移位字节存储指令

语法：

surb rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)] ←rd[7:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数, 高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00010	imm2	rs2	rs1	101	rd	0001011

15.5.52　SURD——寄存器低 32 位移位双字存储指令

语法：

surd rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)+7: (rs1+rs2[31:0]<<imm2)] ←rd[63:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数, 高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01110	imm2	rs2	rs1	101	rd	0001011

15.5.53　SURH——寄存器低 32 位移位半字存储指令

语法：

surh rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)+1: (rs1+rs2[31:0]<<imm2)] ←rd[15:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数, 高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
00110	imm2	rs2	rs1	101	rd	0001011

15.5.54　SURW——寄存器低 32 位移位字存储指令

语法：

surw rd, rs1, rs2, imm2

操作：

mem[(rs1+rs2[31:0]<<imm2)+3: (rs1+rs2[31:0]<<imm2)] ←rd[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

说明：

rs2[31:0] 是无符号数, 高位 [63:32] 补 0 进行地址运算

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01010	imm2	rs2	rs1	101	rd	0001011

15.5.55　SWIA——字存储基地址自增指令

语法：

swia rs2, (rs1), imm5,imm2

操作：

mem[rs1+3:rs1]←rs2[31:0]

rs1←rs1 + sign_extend(imm5 << imm2)

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01011	imm2	imm5	rs1	101	rs2	0001011

15.5.56　SWIB——基地址自增字存储指令

语法：

swib rs2, (rs1), imm5,imm2

操作：

rs1←rs1 + sign_extend(imm5 << imm2)

mem[rs1+3:rs1] ←rs2[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
01001	imm2	imm5	rs1	101	rs2	0001011

15.5.57　SWD——双寄存器低 32 位存储指令

语法：

swd rd1,rd2,(rs1),imm2

操作：

address←rs1+ zero_extend(imm2<<3)

mem[address+3:address] ←rd1[31:0]

mem[address+7:address+4]←rd2[31:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
11100	imm2	rd2	rs1	101	rd1	0001011

15.6　附录 B-6 浮点半精度指令术语

浮点半精度指令子集实现了对浮点半精度的支持，每条指令位宽为 32 位，以下指令按英文字母顺序排列。

15.6.1　FADD.H——半精度浮点加法指令

语法：

fadd.h fd, fs1, fs2, rm

操作：

fd ← fs1 + fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fadd.h fd, fs1,fs2,rne。
- 3’b001: 向零舍入，对应的汇编指令 fadd.h fd, fs1,fs2,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fadd.h fd, fs1,fs2,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fadd.h fd, fs1,fs2,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fadd.h fd, fs1,fs2,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fadd.h fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000010	fs2	fs1	rm	fd	1010011

15.6.2　FCLASS.H——半精度浮点分类指令

语法：

fclass.h rd, fs1

操作：

if (fs1 = -inf)

　　rd ← 64’h1

if (fs1 = -norm)

　　rd ← 64’h2

if (fs1 = -subnorm)

　　rd ← 64’h4

if (fs1 = -zero)

　　rd ← 64’h8

if (fs1 = +zero)

　　rd ← 64’h10

if (fs1 = +subnorm)

rd ← 64’ h20

if (fs1 = +norm)

rd ← 64’ h40

if (fs1 = +inf)

rd ← 64’ h80

if (fs1 = sNaN)

rd ← 64’ h100

if (fs1 = qNaN)

rd ← 64’ h200

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110010	00000	fs1	001	rd	1010011

15.6.3　FCVT.D.H——半精度浮点转换成双精度浮点指令

语法：

fcvt.d.h fd, fs1

操作：

fd ← half_convert_to_double(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100001	00010	fs1	000	fd	1010011

15.6.4　FCVT.H.D——双精度浮点转换成半精度浮点指令

语法：

fcvt.h.d fd, fs1, rm

操作：

fd ← double_convert_to_half(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.d fd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.d fd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.d fd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.d fd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.d fd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.d fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100010	00001	fs1	rm	fd	1010011

15.6.5　FCVT.H.L——有符号长整型转换成半精度浮点数指令

语法：

fcvt.h.l fd, rs1, rm

操作：

fd ← signed_long_convert_to_half(rs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX/OF

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.l fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.l fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.l fd,rs1,fdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.l fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.l fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.l fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101010	00010	rs1	rm	fd	1010011

15.6.6　FCVT.H.LU——无符号长整型转换成半精度浮点数指令

语法：

fcvt.h.lu fd, rs1, rm

操作：

fd ← unsigned_long_convert_to_half_fp(rs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX/OF

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.lu fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.lu fd, rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.lu fd, rs1,fdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.lu fd, rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.lu fd, rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.lu fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101010	00011	rs1	rm	fd	1010011

15.6.7　FCVT.H.S——单精度浮点转换成半精度浮点指令

语法：

fcvt.h.s fd, fs1, rm

操作：

fd ← single_convert_to_half(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.s fd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.s fd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.s fd,fs1,fdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.s fd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.s fd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.s fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100010	00000	fs1	rm	fd	1010011

15.6.8　FCVT.H.W——有符号整型转换成半精度浮点数指令

语法：

fcvt.h.w fd, rs1, rm

操作：

fd ← signed_int_convert_to_half(rs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX/OF

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.w fd,rs1,rne。

- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.w fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.w fd,rs1,fdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.w fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.w fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.w fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101010	00000	rs1	rm	fd	1010011

15.6.9 FCVT.H.WU——无符号整型转换成半精度浮点数指令

语法：

fcvt.h.wu fd, rs1, rm

操作：

fd ← unsigned_int_convert_to_half_fp(rs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NX/OF

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.h.wu fd,rs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.h.wu fd,rs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.h.wu fd,rs1,fdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.h.wu fd,rs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.h.wu fd,rs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。

- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.h.wu fd, rs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1101010	00001	rs1	rm	fd	1010011

15.6.10　FCVT.L.H——半精度浮点转换成有符号长整型指令

语法：

fcvt.l.h rd, fs1, rm

操作：

rd ← half_convert_to_signed_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.l.h rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.l.h rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.l.h rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.l.h rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.l.h rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.l.h rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100010	00010	fs1	rm	rd	1010011

15.6.11　FCVT.LU.H——半精度浮点转换成无符号长整型指令

语法：

fcvt.lu.h rd, fs1, rm

操作：

rd ← half_convert_to_unsigned_long(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.lu.h rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.lu.h rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.lu.h rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.lu.h rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.lu.h rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.lu.h rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100010	00011	fs1	rm	rd	1010011

15.6.12　FCVT.S.H——半精度浮点转换成单精度浮点指令

语法：

fcvt.s.h fd, fs1

操作：

fd ← half_convert_to_single(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0100000	00010	fs1	000	fd	1010011

15.6.13　FCVT.W.H——半精度浮点转换成有符号整型指令

语法：

fcvt.w.h rd, fs1, rm

操作：

tmp ← half_convert_to_signed_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.w.h rd,fs1,rne。

- 3’b001: 向零舍入，对应的汇编指令 fcvt.w.h rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.w.h rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.w.h rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.w.h rd,fs1,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.w.h rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100010	00000	fs1	rm	rd	1010011

15.6.14　FCVT.WU.H——半精度浮点转换成无符号整型指令

语法：

fcvt.wu.h rd, fs1, rm

操作：

tmp ← half_convert_to_unsigned_int(fs1)

rd←sign_extend(tmp)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fcvt.wu.h rd,fs1,rne。
- 3’b001: 向零舍入，对应的汇编指令 fcvt.wu.h rd,fs1,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fcvt.wu.h rd,fs1,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fcvt.wu.h rd,fs1,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fcvt.wu.h rd,fs1,rmm。

- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fcvt.wu.h rd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1100010	00001	fs1	rm	rd	1010011

15.6.15　FDIV.H——半精度浮点除法指令

语法：

fdiv.h fd, fs1, fs2, rm

操作：

fd ← fs1 / fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/DZ/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fdiv.h fs1,fs2,rne。
- 3’b001: 向零舍入，对应的汇编指令 fdiv.h fd fs1,fs2,rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fdiv.h fd, fs1,fs2,rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fdiv.h fd, fs1,fs2,rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fdiv.h fd, fs1,fs2,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fdiv.h fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001110	fs2	fs1	rm	fd	1010011

15.6.16　FEQ.H——半精度浮点比较相等指令

语法：

feq.h rd, fs1, fs2

操作：

if(fs1 == fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010010	fs2	fs1	010	rd	1010011

15.6.17　FLE.H——半精度浮点比较小于等于指令

语法：

fle.h rd, fs1, fs2

操作：

if(fs1 <= fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010010	fs2	fs1	000	rd	1010011

15.6.18　FLH——半精度浮点加载指令

语法：

flh fd, imm12(rs1)

操作：

address←rs1+sign_extend(imm12)

fd[15:0] ← mem[(address+1):address]

fd[63:16] ← 48’hffffffffffff

执行权限：

M mode/S mode/U mode

异常：

加载指令非对齐访问异常、加载指令访问错误异常、加载指令页面错误异常、非法指令异常

影响标志位：

无

指令格式：

31 20	19 15	14 12	11 7	6 0
imm12[11:0]	rs1	001	rd	0000111

15.6.19　FLT.H——半精度浮点比较小于指令

语法：

flt.h rd, fs1, fs2

操作：

if(fs1 < fs2)

　　rd ← 1

else

　　rd ← 0

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1010010	fs2	fs1	001	rd	1010011

15.6.20　FMADD.H——半精度浮点乘累加指令

语法：

fmadd.h fd, fs1, fs2, fs3, rm

操作：

fd ← fs1*fs2 + fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3, rdn。

- 3’b011: 向正无穷舍入，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmadd.h fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	10	fs2	fs1	rm	fd	1000011

15.6.21　FMAX.H——半精度浮点取最大值指令

语法：

fmax.h fd, fs1, fs2

操作：

if(fs1 >= fs2)

　　fd ← fs1

else

　　fd ← fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010110	fs2	fs1	001	fd	1010011

15.6.22　FMIN.H——半精度浮点取最小值指令

语法：

fmin.h fd, fs1, fs2

操作：

if(fs1 >= fs2)

　　fd ← fs2

else

　　fd ← fs1

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010110	fs2	fs1	000	fd	1010011

15.6.23　FMSUB.H——半精度浮点乘累减指令

语法：

fmsub.h fd, fs1, fs2, fs3, rm

操作：

fd ← fs1*fs2 - fs3

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmsub.h fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmsub.h fd,fs1, fs2, fs3, rtz。

- 3’b010: 向负无穷舍入，对应的汇编指令 fmsub.h fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmsub.h fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmsub.h fd, fs1, fs2, fs3,rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmsub.h fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	10	fs2	fs1	rm	fd	1000111

15.6.24　FMUL.H——半精度浮点乘法指令

语法：

fmul.h fd, fs1, fs2, rm

操作：

fd ← fs1 * fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fmul.h fd, fs1, fs2, rne。
- 3’b001: 向零舍入，对应的汇编指令 fmul.h fd, fs1, fs2, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fmul.h fd, fs1, fs2, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fmul.h fd, fs1, fs2, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fmul.h fd, fs1,fs2 , rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。

- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fmul.h fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0001010	fs2	fs1	rm	fd	1010011

15.6.25　FMV.H.X——半精度浮点写传送指令

语法：

fmv.h.x fd, rs1

操作：

fd[15:0] ← rs1[15:0]

fd[63:16] ← 48’hffffffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1111010	00000	rs1	000	fd	1010011

15.6.26　FMV.X.H——半精度浮点寄存器读传送指令

语法：

fmv.x.h rd, fs1

操作：

tmp[15:0] ← fs1[15:0]

rd ← sign_extend(tmp[15:0])

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
1110010	00000	fs1	000	rd	1010011

15.6.27　FNMADD.H——半精度浮点乘累加取负指令

语法：

fnmadd.h fd, fs1, fs2, fs3, rm

操作：

fd ←-(fs1*fs2 + fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmadd.h fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	10	fs2	fs1	rm	fd	1001111

15.6.28　FNMSUB.H——半精度浮点乘累减取负指令

语法：

fnmsub.h fd, fs1, fs2, fs3, rm

操作：

fd ← -(fs1*fs2 - fs3)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/UF/IX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3, rne。
- 3’b001: 向零舍入，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3, rtz。
- 3’b010: 向负无穷舍入，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3, rdn。
- 3’b011: 向正无穷舍入，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3, rup。
- 3’b100: 就近向大值舍入，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3, rmm。
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fnmsub.h fd,fs1, fs2, fs3。

指令格式：

31 27	26 25	24 20	19 15	14 12	11 7	6 0
fs3	10	fs2	fs1	rm	fd	1001011

15.6.29　FSGNJ.H——半精度浮点符号注入指令

语法：

fsgnj.h fd, fs1, fs2

操作：

fd[14:0] ← fs1[14:0]

fd[15] ← fs2[15]

fd[63:16] ← 48’hffffffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010010	fs2	fs1	000	fd	1010011

15.6.30　FSGNJN.H——半精度浮点符号取反注入指令

语法：

fsgnjn.h fd, fs1, fs2

操作：

fd[14:0] ← fs1[14:0]

fd[15] ← ! fs2[15]

fd[63:16] ← 48’hffffffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010010	fs2	fs1	001	fd	1010011

15.6.31　FSGNJX.H——半精度浮点符号异或注入指令

语法：

fsgnjx.h fd, fs1, fs2

操作：

fd[14:0] ← fs1[14:0]

fd[15] ← fs1[15] ^ fs2[15]

fd[63:16] ← 48’hffffffffffff

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

无

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0010010	fs2	fs1	010	fd	1010011

15.6.32　FSH——半精度浮点存储指令

语法：

fsh fs2, imm12(fs1)

操作：

address←fs1+sign_extend(imm12)

mem[(address+1):address] ← fs2[15:0]

执行权限：

M mode/S mode/U mode

异常：

存储指令非对齐访问异常、存储指令访问错误异常、存储指令页面错误异常、非法指令异常

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
imm12[11:5]	fs2	fs1	001	imm12[4:0]	0100111

15.6.33　FSQRT.H——半精度浮点开方指令

语法：

fsqrt.h fd, fs1, rm

操作：

fd ← sqrt(fs1)

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsqrt.h fd, fs1,rne
- 3’b001: 向零舍入，对应的汇编指令 fsqrt.h fd, fs1,rtz
- 3’b010: 向负无穷舍入，对应的汇编指令 fsqrt.h fd, fs1,rdn
- 3’b011: 向正无穷舍入，对应的汇编指令 fsqrt.h fd, fs1,rup
- 3’b100: 就近向大值舍入，对应的汇编指令 fsqrt.h fd, fs1,rmm
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsqrt.h fd, fs1。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0101110	00000	fs1	rm	fd	1010011

15.6.34　FSUB.H——半精度浮点减法指令

语法：

fsub.h fd, fs1, fs2, rm

操作：

fd ← fs1 - fs2

执行权限：

M mode/S mode/U mode

异常：

非法指令异常

影响标志位：

浮点状态位 NV/OF/NX

说明：

rm 决定舍入模式:

- 3’b000: 就近向偶数舍入，对应的汇编指令 fsub.h fd, fs1,fs2,rne
- 3’b001: 向零舍入，对应的汇编指令 fsub.h fd, fs1,fs2,rtz
- 3’b010: 向负无穷舍入，对应的汇编指令 fsub.h fd, fs1,fs2,rdn
- 3’b011: 向正无穷舍入，对应的汇编指令 fsub.h fd, fs1,fs2,rup
- 3’b100: 就近向大值舍入，对应的汇编指令 fsub.h fd, fs1,fs2,rmm
- 3’b101: 暂未使用，不会出现该编码。
- 3’b110: 暂未使用，不会出现该编码。
- 3’b111: 动态舍入，根据浮点控制寄存器 fcsr 中的 rm 位来决定舍入模式，对应的汇编指令 fsub.h fd, fs1,fs2。

指令格式：

31 25	24 20	19 15	14 12	11 7	6 0
0000110	fs2	fs1	rm	fd	1010011

第十六章　附录 C 控制寄存器

本附录对机器模式控制寄存器、超级用户模式控制寄存器和用户模式控制寄存器进行详细说明。

16.1　附录 C-1 机器模式控制寄存器

机器模式控制寄存器按照功能分为：机器模式信息寄存器组、机器模式异常配置寄存器组、机器模式异常处理寄存器组、机器模式内存保护寄存器组、机器模式计数器和机器模式计数器配置寄存器组。

16.1.1　机器模式信息寄存器组

16.1.1.1　机器模式供应商编号寄存器（MVENDORID）

机器模式供应商编号寄存器（MVENDORID）存储了平头哥半导体有限公司的厂商编号信息，目前尚未定义，值为全零。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式只读，即非机器模式访问和机器模式写操作访问都会导致非法指令异常。

16.1.1.2　机器模式架构编号寄存器（MARCHID）

机器模式架构编号寄存器（MARCHID）存储了处理器核的架构编号，用于存放平头哥半导体有限公司产品的内部编号，其复位值由产品本身决定。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式只读，即非机器模式访问和机器模式写操作访问都会导致非法指令异常。

16.1.1.3　机器模式硬件实现编号寄存器（MIMPID）

机器模式硬件实现编号寄存器（MIMPID）存储了处理器核的硬件实现编号。C910 目前未实现该寄存器，读访问为 0 值。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式只读，即非机器模式访问和机器模式写操作访问都会导致非法指令异常。

16.1.1.4　机器模式逻辑内核编号寄存器（MHARTID）

机器模式逻辑内核编号寄存器（MHARTID）存储了处理器核的硬件逻辑内核编号。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式只读，即非机器模式访问和机器模式写操作访问都会导致非法指令异常。

16.1.2　机器模式异常配置寄存器组

16.1.2.1　机器模式处理器状态寄存器（MSTATUS）

机器模式处理器状态寄存器（MSTATUS）存储了处理器在机器模式下的状态和控制信息，包括全局中断有效位、异常保留中断有效位、异常保留特权模式位等。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

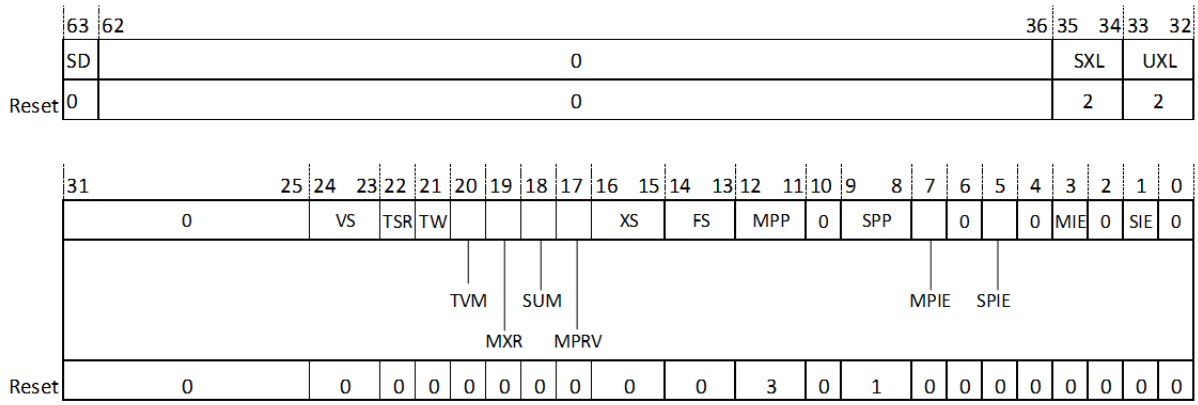


图 16.1: 机器模式处理器状态寄存器（MSTATUS）

SIE-超级用户模式中断使能位：

- 当 SIE 为 0 时，超级用户中断无效；
- 当 SIE 为 1 时，超级用户中断有效；

该位会被 reset 清零，处理器被降级到超级用户模式响应中断时被清零；在处理器退出中断服务程序时被置为 SPIE 的值。

MIE-机器模式中断使能位：

- 当 MIE 为 0 时，机器模式中断无效；
- 当 MIE 为 1 时，机器模式中断有效；

该位会被 reset 清零，处理器在机器模式响应中断时被清零；在处理器退出中断服务程序时被置为 MPIE 的值。

SPIE-超级用户模式保留中断使能位：

该位用于保存处理器在超级用户模式响应中断前 SIE 位的值。

该位会被 reset 清零，在处理器退出中断服务程序时被置 1。

MPIE-机器模式保留中断使能位：

该位用于保存处理器在机器模式响应中断前 MIE 位的值。

该位会被 reset 清零，在处理器退出中断异常服务程序时被置 1。

SPP-超级用户模式保留特权状态位：

该位用于保存处理器在超级用户模式进入异常服务程序前的特权状态。

- 当 SPP 为 2’b00 时，表示处理器进入异常服务程序前处于用户模式；
- 当 SPP 为 2’b01 时，表示处理器进入异常服务程序前处于超级用户模式；

该位会被 reset 置 2’b01。

MPP-机器模式保留特权状态位：

该位用于保存处理器在机器模式进入异常服务程序前的特权状态。

- 当 MPP 为 2’b00 时，表示处理器进入异常服务程序前处于用户模式；
- 当 MPP 为 2’b01 时，表示处理器进入异常服务程序前处于超级用户模式；
- 当 MPP 为 2’b11 时，表示处理器进入异常服务程序前处于机器模式；

该位会被 reset 置 2’b11。

FS-浮点单元状态位：

根据浮点状态位，可以判断上下文切换的时候，是否需要保存浮点相关寄存器。

- 当 FS 为 2’b00 时，浮点单元处于关闭状态，此时访问浮点相关寄存器会产生异常。
- 当 FS 为 2’b01 时，浮点单元处于初始化状态。
- 当 FS 为 2’b10 时，浮点单元处于 clean 态。
- 当 FS 为 2’b11 时，浮点单元处于 dirty 态，表明浮点寄存器和控制寄存器被修改过。

XS-扩展单元状态位：

C910 没有扩展单元，固定为 0。

MPRV-修改特权模式：

- 当 MPRV=1 时，加载和存储请求执行时根据 MPP 中的特权态进行执行。
- 当 MPRV=0 时，加载和存储请求执行时根据当前处理器所处特权模式进行执行。

SUM-允许超级用户模式下访问 U 态虚拟内存空间

- 当 SUM=1 时，超级用户模式下，加载、存储和取值请求可以访问标记为用户态的虚拟内存空间。

- 当 SUM=0 时，超级用户模式下，加载、存储和取值请求不可以访问标记为用户态的虚拟内存空间。

MXR-允许加载请求访问标记为可执行的内存空间

- 当 MXR=1 时，允许加载请求访问标记为可执行或者可读的虚拟内存空间。
- 当 MXR=0 时，允许加载请求只能访问标记为可读的虚拟内存空间。

TVM-陷阱虚拟内存

- 当 TVM=1 时，超级用户模式读写 satp 控制寄存器以及执行 sfence 指令，产生非法指令异常。
- 当 TVM=0 时，超级用户模式可以读写 satp 控制寄存器以及执行 sfence 指令。

TW-超时等待

- 当 TW=1 时，超级用户模式执行低功耗指令 wfi，产生非法指令异常。
- 当 TW=0 时，超级用户模式执行低功耗指令 wfi。

TSR-陷阱 sret

- 当 TSR=1 时，超级用户模式执行 sret 指令，产生非法指令异常。
- 当 TSR=0 时，允许超级用户模式执行 sret 指令。

VS-矢量单元状态位

根据矢量状态位，可以判断上下文切换的时候，是否需要保存矢量相关寄存器。

- 当 VS 为 2’b00 时，矢量单元处于关闭状态，此时访问矢量相关寄存器会产生异常。
- 当 VS 为 2’b01 时，矢量单元处于初始化状态。
- 当 VS 为 2’b10 时，矢量单元处于 clean 态。
- 当 VS 为 2’b11 时，矢量单元处于 dirty 态，表明矢量寄存器和矢量控制寄存器被修改过。

VS 位仅当配置矢量执行单元时有效，不配置时恒为 0。

UXL-寄存器位宽

只读，固定值是 2，表示在 U 态下，寄存器的位宽是 64bit。

SXL-寄存器位宽

只读，固定值是 2，表示在 S 态下，寄存器的位宽是 64bit。

SD-浮点、矢量和扩展单元 dirty 状态总和位

- 当 SD=1 时，表明浮点或矢量或扩展单元处在 dirty 状态。
- 当 SD=0 时，表明浮点和矢量和扩展单元处都不处在 dirty 状态。

16.1.2.2　机器模式处理器指令集特性寄存器（MISA）

机器模式处理器指令集特性寄存器（MISA）存储了处理器所支持的指令集架构特性。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

C910 支持的指令集架构为 RV64GC，对应的 MISA 寄存器复位值为 0x800000000094112d。具体的赋值规则请参考 RISCV 官方文档《riscv-privileged》。

C910 不支持动态配置 MISA 寄存器，对该寄存器进行写操作不产生任何效果。

16.1.2.3　机器模式异常降级控制寄存器（MEDELEG）

机器模式异常降级控制寄存器（MEDELEG）可以将超级用户和用户模式发生的异常降级到超级用户模式响应。MEDELEG 低 16 比特和异常向量表一一对应，可以选择将哪些异常降级到超级用户模式响应。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

16.1.2.4　机器模式中断降级控制寄存器（MIDELEG）

机器模式中断降级控制寄存器（MIDELEG）可以将超级用户模式中断降级到超级用户模式响应。

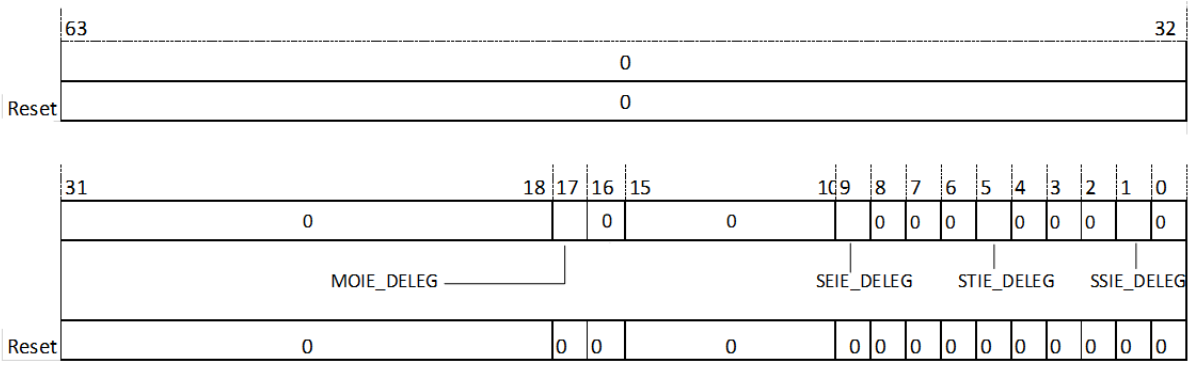


图 16.2: 机器模式中断降级控制寄存器（MIDELEG）

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

16.1.2.5　机器模式中断使能控制寄存器（MIE）

机器模式中断使能控制寄存器（MIE）用于控制不同中断类型的使能和屏蔽。该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

SSIE-超级用户模式软件中断使能位：

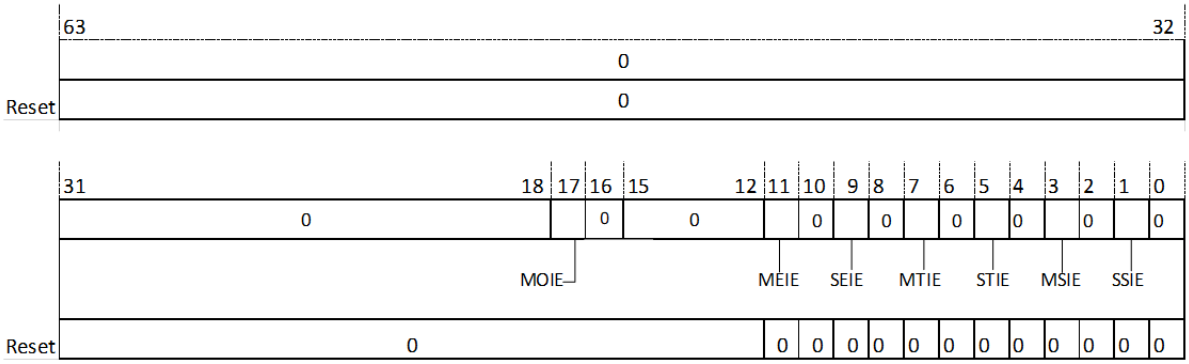


图 16.3: 机器模式中断使能控制寄存器（MIE）

- 当 SEIE 为 0 时，超级用户模式软件外部中断无效。
- 当 SEIE 为 1 时，超级用户模式软件外部中断有效。

MSIE-机器模式软件中断使能位：

- 当 MSIE 为 0 时，机器模式软件中断无效。
- 当 MSIE 为 1 时，机器模式软件中断有效。

STIE-超级用户模式计时器中断使能位：

- 当 STIE 为 0 时，超级用户模式计时器中断无效。
- 当 STIE 为 1 时，超级用户模式计时器外部中断有效。

MTIE-机器模式计时器中断使能位：

- 当 MTIE 为 0 时，机器模式计时器中断无效。
- 当 MTIE 为 1 时，机器模式计时器中断有效。

SEIE-超级用户模式外部中断使能位：

- 当 SEIE 为 0 时，超级用户模式外部中断无效。
- 当 SEIE 为 1 时，超级用户模式外部中断有效。

MEIE-机器模式外部中断使能位：

- 当 MEIE 为 0 时，机器模式外部中断无效。
- 当 MEIE 为 1 时，机器模式外部中断有效。

MOIE-机器模式事件计数器溢出中断使能位：

- 当 MOIE 为 0 时，机器模式计数器溢出中断无效。
- 当 MOIE 为 1 时，机器模式计数器溢出中断有效。

16.1.2.6　机器模式向量基址寄存器（MTVEC）

机器模式向量基址寄存器（MTVEC）用于配置异常服务程序的入口地址。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

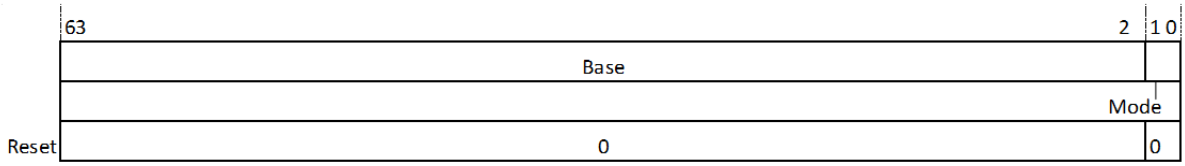


图 16.4: 机器模式向量基址寄存器（MTVEC）

BASE-向量基址位：

向量基址位指示了异常服务程序入口地址的高 62 位，将此基址拼接 2’b00 即可得到异常服务程序入口地址。

该位会被 reset 清零。

MODE-向量入口模式位：

- 当 MODE[1:0] 为 2’b00 时，异常和中断都统一使用 BASE 地址作为异常入口地址。
- 当 MODE[1:0] 为 2’b01 时，异常使用 BASE 地址作为入口地址，中断使用 BASE + 4*cause。

16.1.2.7　机器模式计数器访问授权寄存器（MCOUNTEREN）

机器模式计数器访问授权寄存器（mcounteren），用于授权超级用户模式是否可以访问用户模式计数器。

具体请参考 performance_test 。

16.1.3　机器模式异常处理寄存器组

16.1.3.1　机器模式异常临时数据备份寄存器（MSCRATCH）

机器模式异常临时数据备份寄存器（MSCRATCH）用于处理器在异常服务程序中备份临时数据。一般用来存储机器模式本地上下文空间的入口指针值。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

16.1.3.2　机器模式异常保留程序计数器寄存器（MEPC）

机器模式异常保留程序计数器（MEPC）用于存储程序从异常服务程序退出时的程序计数器值（即 PC 值）。C910 支持 16 位宽指令，MEPC 的值以 16 位宽对齐，最低位为零。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

16.1.3.3　机器模式异常事件向量寄存器（MCAUSE）

机器模式异常事件向量寄存器（MCAUSE）用于保存触发异常的异常事件向量号，用于在异常服务程序中处理对应事件。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

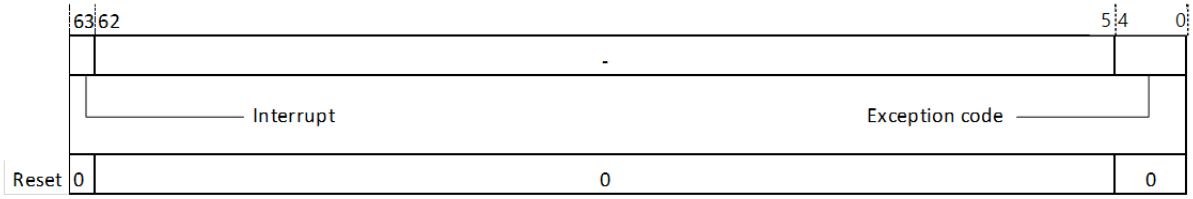


图 16.5: 机器模式异常事件向量寄存器（MCAUSE）

Interrupt-中断标记位：

- 当 Interrupt 位为 0 时，表示触发异常的来源不是中断，Exception Code 按照异常解析。
- 当 Interrupt 位为 1 时，表示触发异常的来源是中断，Exception Code 按照中断解析。

Exception Code-异常向量号位：

在处理器进入异常时，异常向量位会被更新为异常来源的对应值。

16.1.3.4　机器模式中断等待状态寄存器（MIP）

机器模式中断等待状态寄存器（MIP）用于保存处理器的中断等待状态。当处理器出现中断无法立即响应的情况时，MIP 寄存器中的对应位会被置位。

写 CLINT 中的 MSIP 和 SSIP 寄存器可以出发对应的中断，中断有效后可以通过 MIP 中对应的 bit 为 MSIP bit 以及 SSIP bit 进行查询。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

	63 … 32
	0
Reset	0

	31 … 18	17	16	15 … 12	11	10	9	8	7	6	5	4	3	2	1	0
	0			0		0		0		0		0		0		0
		MOIP	MCIP		MEIP		SEIP		MTIP		STIP		MSIP		SSIP	
Reset	0				0	0	0	0	0	0	0	0	0	0	0	0

图 16.6: 机器模式中断等待状态寄存器（MIP）

SSIP-超级用户模式软件中断等待位：

- 当 SSIP 为 0 时，处理器当前没有处于等待状态的超级用户模式软件中断。
- 当 SSIP 为 1 时，处理器当前有处于等待状态的超级用户模式软件中断。

M 态可读写，SSIP delegate 到 S 态之后，S 态可读写，否则 S 态只读。

MSIP-机器模式软件中断等待位：

- 当 MSIP 为 0 时，处理器当前没有处于等待状态的机器模式软件中断。
- 当 MSIP 为 1 时，处理器当前有处于等待状态的机器模式软件中断。

此 bit 为只读。

STIP-超级用户模式计时器中断等待位：

- 当 STIP 为 0 时，处理器当前没有处于等待状态的超级用户模式计时器中断。
- 当 STIP 为 1 时，处理器当前有处于等待状态的超级用户模式计时器中断。

MTIP-机器模式计时器中断等待位：

- 当 MTIP 为 0 时，处理器当前没有处于等待状态的机器模式计时器中断。
- 当 MTIP 为 1 时，处理器当前有处于等待状态的机器模式计时器中断。

SEIP-超级用户模式外部中断等待位：

- 当 SEIP 为 0 时，处理器当前没有处于等待状态的超级用户模式外部中断。
- 当 SEIP 为 1 时，处理器当前有处于等待状态的超级用户模式外部中断。

MEIP-外部中断等待位：

- 当 MEIP 为 0 时，处理器当前没有处于等待状态的机器模式外部中断。
- 当 MEIP 为 1 时，处理器当前有处于等待状态的机器模式外部中断。

MOIP-机器模式事件计数器溢出中断等待位：

- 当 MOIP 为 0 时，处理器当前没有处于等待状态的机器模式计数器溢出中断。

- 当 MOIP 为 1 时，处理器当前有处于等待状态的机器模式计数器溢出中断。

16.1.4　机器模式内存保护寄存器组

机器模式内存保护寄存器组是和内存保护单元设置相关的控制寄存器。

16.1.4.1　机器模式物理内存保护配置寄存器（PMPCFG）

机器模式物理内存保护配置寄存器（PMPCFG）用于配置物理内存的访问权限、地址匹配模式。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

具体信息请参考*物理内存保护设置寄存器（PMPCFG）*。

16.1.4.2　机器模式物理内存地址寄存器（PMPADDR）

机器模式物理内存地址寄存器（PMPADDR）用于配置物理内存的每个表项的地址区间。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

具体信息请参考*物理内存保护地址寄存器（PMPADDR）*。

16.1.5　机器模式计数器寄存器组

机器模式计数器寄存器组属于性能监测单元，用于统计程序运行中的软件信息和部分硬件信息，供软件开发人员进行程序优化。

16.1.5.1　机器模式周期计数器（MCYCLE）

机器模式周期计数器（MCYCLE）用于存储处理器已经执行的周期数，当处理器处于执行状态（即非低功耗状态）下，MCYCLE 寄存器就会在每个执行周期自增计数。

周期计数器为 64 位，周期计数器会被 reset 清零。

具体信息请参考*事件计数器*。

16.1.5.2　机器模式退休指令计数器（MINSTRET）

机器模式退休指令计数器（MINSTRET）用于存储处理器已经退休的指令数，MINSTRET 寄存器会在每条指令退休时自增计数。

退休指令计数器为 64 位，退休计数器会被 reset 清零。

具体信息请参考*事件计数器*。

16.1.5.3　机器模式事件计数器（MHPMCOUNTERn）

机器模式事件计数器（MHPMCOUNTERn）用于对事件进行计数。

事件计数器为 64 位，事件计数器会被 reset 清零。

具体信息请参考事件计数器 。

16.1.6　机器模式计数器配置寄存器组

机器模式计数器配置寄存器用于给机器模式事件计数器选择计数的事件。

16.1.6.1　机器模式事件选择器（MHPMEVENTn）

机器模式计数器配置寄存器 (MHPMEVENTn) 用于给机器模式事件计数器选择的事件，mhpmevent3-31 和 mhpmcounter3-31 一一对应。在 C910 中，各事件计数器只能对固定事件进行计数，因此，mhpmevent3-31 只能写入指定数值。

事件选择器 64 位，周期计数器会被 reset 清零。

具体信息请参考性能监测事件选择寄存器 。

16.1.7　机器模式处理器控制和状态扩展寄存器组

C910 对处理器和状态扩展了部分寄存器，包含：机器模式扩展状态寄存器（MXSTATUS）、机器模式硬件控制寄存器（MHCR）、机器模式硬件操作寄存器（MCOR）、机器模式 L2Cache 控制寄存器（MCCR2）、机器模式隐式操作寄存器（MHINT）、机器模式复位向量基址寄存器（MRVBR）、超级用户态计数器写使能寄存器（MCOUNTERWEN）、机器模式事件中断使能寄存器（MCOUNTERINTEN）、机器模式事件上溢出标注寄存器（MCOUNTEROF）。

16.1.7.1　机器模式扩展状态寄存器（MXSTATUS）

机器模式扩展状态寄存器（MXSTATUS）存储了处理器当前所处特权模式和 C910 扩展功能开关位。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

PMDU-用户模式性能监测计数使能位：

当 PMDU 为 0 时，用户模式下允许性能计数器计数。

当 PMDU 为 1 时，用户模式下禁止性能计数器计数。

PMDS-超级用户模式性能监测计数使能位：

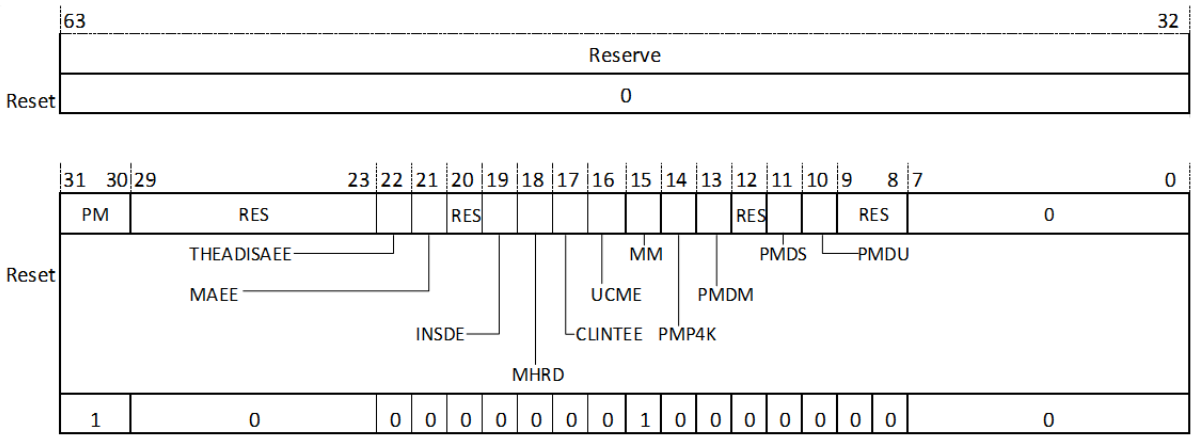


图 16.7: 机器模式扩展状态寄存器（MXSTATUS）

当 PMDS 为 0 时，超级用户模式下允许性能计数器计数。

当 PMDS 为 1 时，超级用户模式下禁止性能计数器计数。

PMDM-机器模式性能监测计数使能位：

当 PMDM 为 0 时，机器模式下允许性能计数器计数。

当 PMDM 为 1 时，机器模式下禁止性能计数器计数。

PMP4K-PMP 最小粒度控制位：

C910 当前只支持 PMP 最小粒度为 4K，不受该位影响。

MM-非对齐访问使能位：

当 MM 为 0 时，不支持非对齐访问，非对齐访问将产生非对齐异常。

当 MM 为 1 时，支持非对齐访问，硬件处理非对齐访问。(C910 默认值为 1)

UCME-U 态执行扩展 cache 指令：

当 UCME 为 0 时，用户模式不能执行扩展的 cache 操作指令，产生非法指令异常。

当 UCME 为 1 时，用户模式可以执行扩展的 cache 操作指令。

CLINTEE-Clint 计时器/软件中断超级用户扩展使能位：

当 CLINTEE 为 0 时，CLINT 发起的超级用户软件中断和计时器中断不会被响应。

当 CLINTEE 为 1 时，CLINT 发起的超级用户软件中断和计时器中断可以被响应。

MHRD-关闭硬件回填：

当 MHRD 为 0 时，TLB 缺失后，硬件会进行硬件回填。

当 MHRD 为 1 时，TLB 缺失后，硬件不会进行硬件回填。

INSDE-关闭 Icache snoop Dcache 功能：

当 INSDE 为 0 时，Icache 缺失后，会 snoop Dcache。

当 INSDE 为 1 时，Icache 缺失后，不会 snoop Dcache。

MAEE-扩展 MMU 地址属性：

当 MAEE 为 0 时，不扩展 MMU 地址属性。

当 MAEE 为 1 时，MMU 的 pte 中扩展地址属性位，用户可以配置页面的地址属性。

THEADISAEE-使能扩展指令集：

当 THEADISAEE 为 0 时，使用 C910 扩展指令集时产生非法指令异常。

当 THEADISAEE 为 1 时，可以使用 C910 扩展指令集。

PM-处理器所处特权模式：

当 PM=2’b00 时，表征当前处理器运行在用户模式。

当 PM=2’b01 时，表征当前处理器运行在超级用户模式。

当 PM=2’b11 时，表征当前处理器运行在机器模式。(Reset 后为机器模式)

16.1.7.2　机器模式硬件配置寄存器（MHCR）

机器模式硬件配置寄存器（MHCR）用于对处理器进行配置，包括性能和功能。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

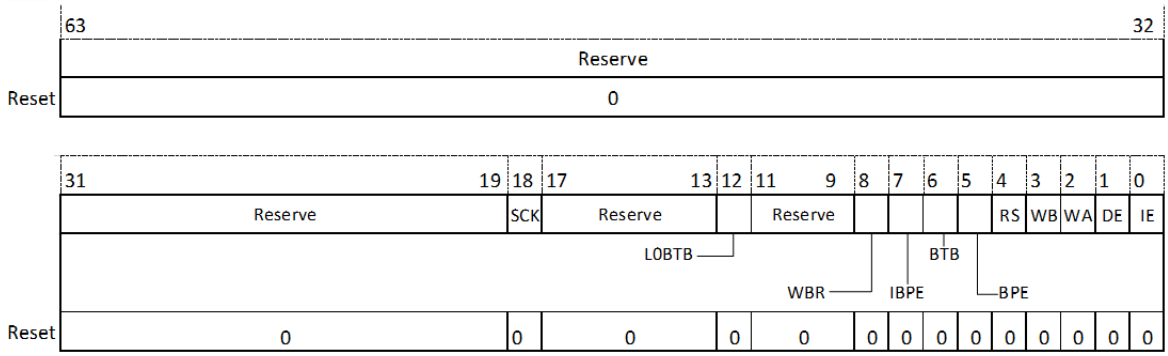


图 16.8: 机器模式硬件配置寄存器（MHCR）

IE-Icache 使能位：

当 IE=0 时，Icache 关闭。

当 IE=1 时，Icache 打开。

DE-Dcache 使能位：

当 DE=0 时，Dcache 关闭。

当 DE=1 时，Dcache 打开。

WA-高速缓存写分配设置位：

当 WA=0 时，数据高速缓存为 write non-allocate 模式。

当 WA=1 时，数据高速缓存为 write allocate 模式。

WB-高速缓存写回设置位：

当 WB=0 时，数据高速缓存为写直模式。

当 WB=1 时，数据高速缓存为写回模式。

C910 只支持写回模式，WB 固定为 1。

RS-地址返回栈设置位：

当 RS=0 时，返回栈关闭。

当 RS=1 时，返回栈开启。

BPE-允许预测跳转设置位：

当 BPE=0 时，预测跳转关闭。

当 BPE=1 时，预测跳转开启。

BTB-分支目标预测使能位：

当 BTB=0 时，分支目标预测关闭。

当 BTB=1 时，分支目标预测开启。

IBPE-间接分支跳转预测跳转使能位：

当 IBPE=0 时，间接分支跳转预测关闭。

当 IBPE=1 时，间接分支跳转预测开启。

WBR-写突发传输使能位：

当 WBR=0 时，不支持写突发传输。

当 WBR=1 时，支持写突发传输。

C910 默认为 1，不可设置。

L0BTB-第一级分支目标预测使能位：

当 L0BTB=0 时，第一级分支目标预测关闭。

当 L0BTB=1 时，第一级分支目标预测开启。

SCK-**系统和处理器的时钟比：**

该位用来表示系统和处理器的时钟比，其计算公式为：时钟比 =SCK+1，CPU 上有对应引脚引出。SCK 在 reset 时被配置且不能在之后改变。

16.1.7.3　机器模式硬件操作寄存器（MCOR）

机器模式硬件操作寄存器（MCOR）用于对高速缓存和分支预测部件进行相关操作。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

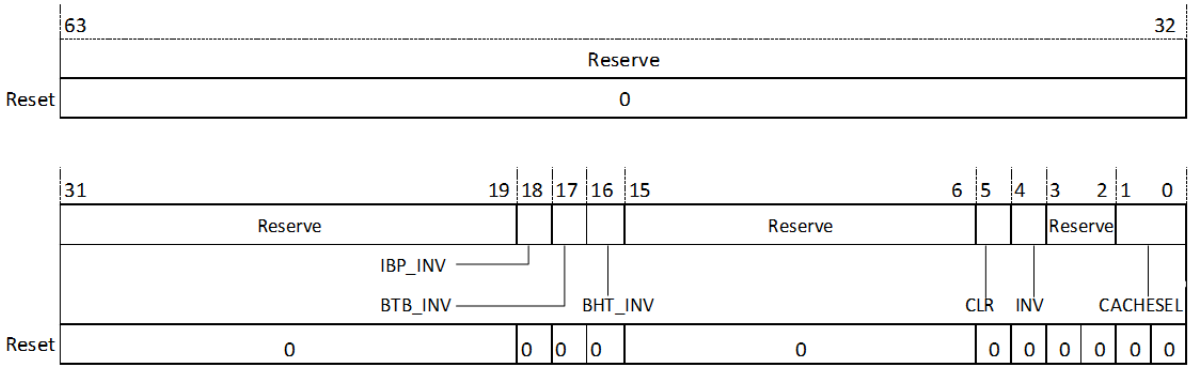


图 16.9: 机器模式硬件操作寄存器（MCOR）

CACHESEL-高速缓存选择位：

当 CACHE_SEL=2’b01 时，选中指令高速缓存。

当 CACHE_SEL=2’b10 时，选中数据高速缓存。

当 CACHE_SEL=2’b11 时，选中指令和数据高速缓存。

INV-高速缓存无效化设置位：

当 INV=0 时，高速缓存不进行无效化。

当 INV=1 时，高速缓存进行无效化。

CLR-高速缓存脏表现清除设置位：

当 CLR=0 时，高速缓存被标记为脏的表项不会被写到片外。

当 CLR=1 时，高速缓存被标记为脏的表项会被写到片外。

BHT_INV-BHT 无效设置位：

当 BHT_INV=0 时，分支历史表内的数据不进行无效化。

当 BHT_INV=1 时，分支历史表内的数据进行无效化。

BTB_INV-BTB 无效设置位：

当 BTB_INV=0 时，分支目标缓冲器内的数据不进行无效化。

当 BTB_INV=1 时，分支目标缓冲器内的数据进行无效化。

IBP_INV-IBP 无效设置位：

当 IBP_INV=0 时，间接跳转分支预测的数据不进行无效化。

当 IBP_INV=1 时，间接跳转分支预测的数据进行无效化。

以上所有无效化操作和清脏表现操作，在写的时候置高，在操作完成时，清 0。

16.1.7.4　机器模式 L2Cache 控制寄存器（MCCR2）

机器模式 L2Cache 控制寄存器（MCCR2）用来配置共享的二级高速缓存中各个存储器的访问延时，二级高速缓存有效 / 无效，指令预取能力和 TLB 预取使能。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

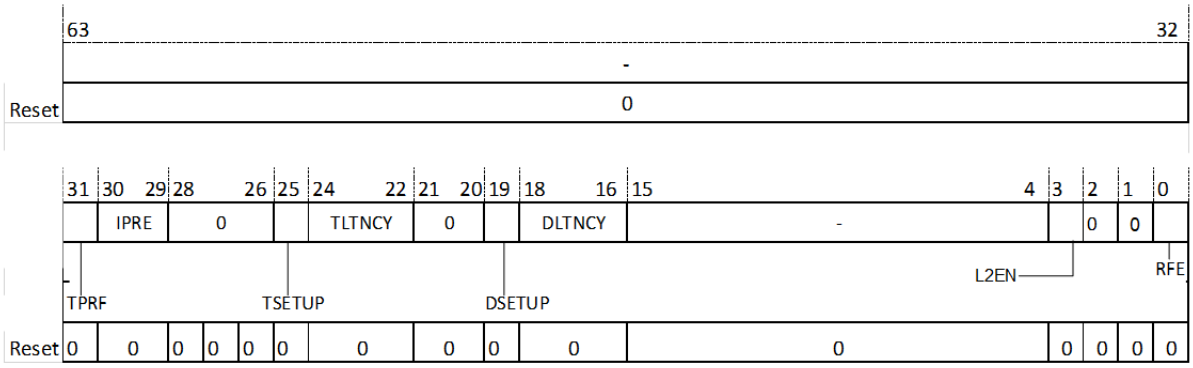


图 16.10: 机器模式 L2Cache 控制寄存器（MCCR2）

RFE-数据访问读分配使能位：

当 RFE=0 时，数据访问 L2Cache 缺失时，不回填 L2Cache。

当 RFE=1 时，数据访问 L2Cache 缺失时，回填 L2Cache。

L2EN-L2Cache 使能位：

当 L2EN=0 时，L2Cache 关闭。

当 L2EN=1 时，L2Cache 开启。（C910 固定为 1）

DLTNCY-L2Cache DATA RAM 访问周期配置位：

当 DLTNCY=0 时，DATA RAM 访问周期为 1。

当 DLTNCY=1 时，DATA RAM 访问周期为 2。

当 DLTNCY 为 2 时，DATA RAM 访问周期为 3。

当 DLTNCY 为 3 时，DATA RAM 访问周期为 4。

当 DLTNCY 为 4 时，DATA RAM 访问周期为 5。

当 DLTNCY 为 5 时，DATA RAM 访问周期为 6。

当 DLTNCY 为 6 时，DATA RAM 访问周期为 7。

当 DLTNCY 为 7 时，DATA RAM 访问周期为 8。

DSETUP-L2Cache DATA RAM 的 setup 配置位

当 DSETUP 为 0 时，DATA RAM 不需要额外的 setup 周期；

当 DSETUP 为 1 时，DATA RAM 需要额外的一个 setup 周期。

TLTNCY-L2Cache TAG RAM 的访问周期配置位：

当 TLTNCY 为 0 时，TAG RAM 访问周期为 1；

当 TLTNCY 为 1 时，TAG RAM 访问周期为 2；

当 TLTNCY 为 2 时，TAG RAM 访问周期为 3；

当 TLTNCY 为 3 时，TAG RAM 访问周期为 4；

当 TLTNCY 为 4 时，TAG RAM 访问周期为 5。

TSETUP-L2 CACHE TAG RAM 的 setup 配置位：

当 TSETUP 为 0 时，TAG RAM 不需要额外的 setup 周期；

当 TSETUP 为 1 时，TAG RAM 需要额外的 1 个 setup 周期。

IPRF-L2Cache 指令预取能力：

指示取指请求访问 L2Cache 缺失时预取的缓存行数量：

当 IPRF 为 0 时，L2Cache 指令预取功能关闭；

当 IPRF 为 1 时，预取 1 条缓存行；

当 IPRF 为 2 时，预取 2 条缓存行；

当 IPRF 为 3 时，预取 3 条缓存行。

TPRF-L2Cache TLB 预取使能：

当 **TPRF** 为 0 时，L2Cache TLB 预取功能关闭；

当 TPRF 为 1 时，L2Cache TLB 预取功能开启。

16.1.7.5　机器模式隐式操作寄存器（MHINT）

机器模式隐式操作寄存器（MHINT）用于高速缓存多种功能开关控制。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

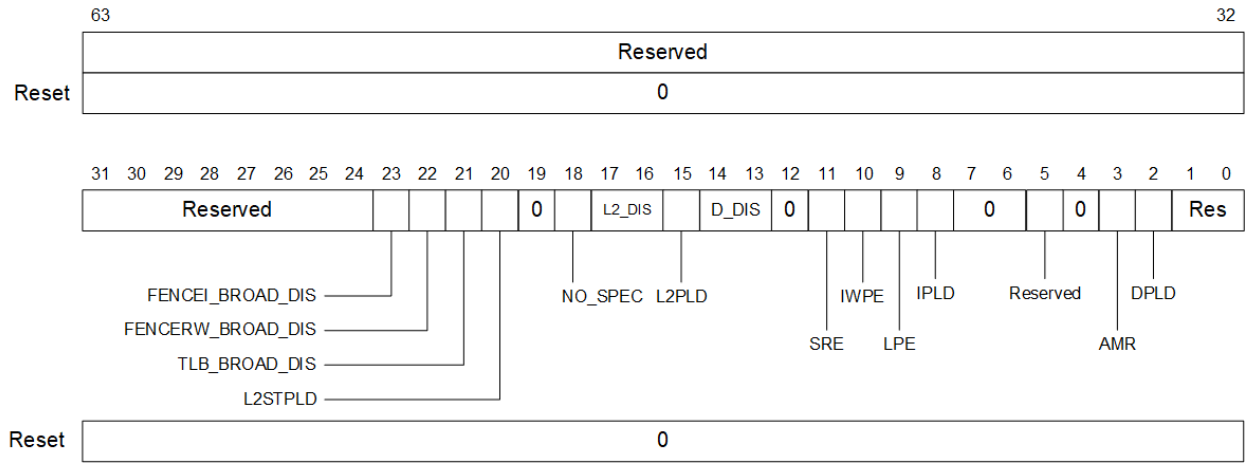


图 16.11: 机器模式隐式操作寄存器（MHINT）

DPLD-DCACHE 预取使能位：

当 DPLD 为 0 时，DCACHE 预取关闭；

当 DPLD 为 1 时，DCACHE 预取开启。

AMR-L1 Cache 写分配策略自动调整使能位：

当 AMR 为 0 时，写分配策略由访问地址的页面属性 WA 决定。

当 AMR 为 1 时，在出现连续多条缓存行的存储操作时后续连续地址的存储操作不再写入 L1 Cache。

IPLD-ICACHE 预取使能位：

当 IPLD 为 0 时，ICACHE 预取关闭。

当 IPLD 为 1 时，ICACHE 预取开启。

LPE-循环加速使能位：

当 LPE 为 0 时，循环加速关闭。

当 LPE 为 1 时，循环加速开启。

IWPE-ICACHE 路预测使能位：

当 IWPE 为 0 时，ICACHE 路预测关闭；

当 IWPE 为 1 时，ICACHE 路预测开启。

SRE-单退休模式位：

当 SRE 为 0 时，单退休模式关闭；

当 SRE 为 1 时，单退休模式开启。

D_DIS-DCACHE 预取缓存行数量：

当 DPLD 为 0 时，预取 2 条缓存行。

当 DPLD 为 1 时，预取 4 条缓存行。

当 DPLD 为 2 时，预取 8 条缓存行。

当 DPLD 为 3 时，预取 16 条缓存行。

默认为 0。

L2PLD-L2CACHE 预取使能位：

当 L2PLD 为 0 时，L2CACHE 预取关闭；

当 L2PLD 为 1 时，L2CACHE 预取开启。

L2_DIS-L2CACHE 预取缓存行数量：

当 L2_DIS 为 0 时，预取 8 条缓存行；

当 L2_DIS 为 1 时，预取 16 条缓存行；

当 L2_DIS 为 2 时，预取 32 条缓存行；

当 L2_DIS 为 3 时，预取 64 条缓存行；

L2Cache 的预取是在 L1Cache 预取的基础上再次进行预取。

NO_SPEC-SPEC FAIL 预测功能使能位：

当 NO_SPEC 为 0 时，spec fail 预测功能关闭；

当 NO_SPEC 为 1 时，spec fail 预测功能开启。

L2STPLD-L2 Cache Store 预取使能位：

当 L2STPLD 为 0 时，L2 CACHE store 预取关闭；

当 L2STPLD 为 1 时，L2 CACHE store 预取开启。

TLB_BROAD_DIS-TLB fence 操作广播取消位：

当 TLB_BROAD_DIS 为 0 时，sfence.vma 指令操作广播到其他核；

当 TLB_BROAD_DIS 为 1 时，sfence.vma 指令操作不广播。

单核情况下，该位不存在。

FENCERW_BROAD_DIS-fence 操作广播取消位：

当 FENCERW_BROAD_DIS 为 0 时，fence 指令操作广播到其他核；

当 FENCERW_BROAD_DIS 为 1 时，fence 指令操作不广播。

单核情况下，该位不存在。

FENCEI_BROAD_DIS-fence.i 操作广播取消位：

当 FENCEI_BROAD_DIS 为 0 时，fence.i 指令操作广播到其他核；

当 FENCEI_BROAD_DIS 为 1 时，fence.i 指令操作不广播。

单核情况下，该位不存在。

16.1.7.6　机器模式复位向量基址寄存器（MRVBR）

机器模式复位寄存器（MRVBR）用来保存复位异常向量的基址。每个 C910 核心拥有独立的 MRVBR 寄存器。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式只读，即非机器模式访问都会导致非法指令异常。

	63 … 40	39 … 0
	Reserved	Reset vector base
Reset	0	(Input pin)

图 16.12: 机器模式复位向量基址寄存器（MRVBR）

Reset vector base-复位基址：

控制核心的复位基址。

16.1.7.7　超级户态计数器写使能寄存器（MCOUNTERWEN）

超级用户态计数器写使能寄存器（MCOUNTERWEN），用于授权超级用户模式是否可以写超级用户模式事件计数器。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

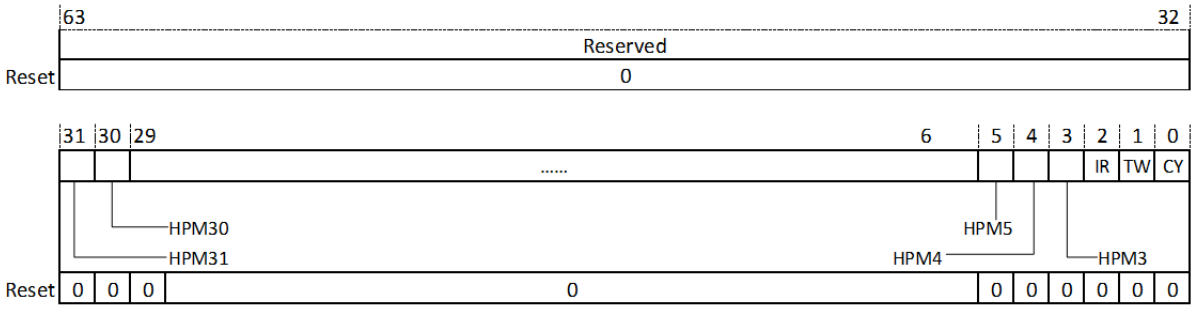


图 16.13: 超级用户态计数器写使能寄存器（MCOUNTERWEN）

当 mcounterwen.bit[n] 为 1 时，允许超级用户模式下写对应 shpmcounter。

当 mcounterwen.bit[n] 为 0 时，不允许超级用户模式下写对应的 shpmcounter，否则产生非法指令异常。

16.1.7.8　机器模式事件中断使能寄存器（MCOUNTERINTEN）

机器模式事件中断使能寄存器（MCOUNTERINTEN），用于使能各事件计数上溢出时产生中断。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

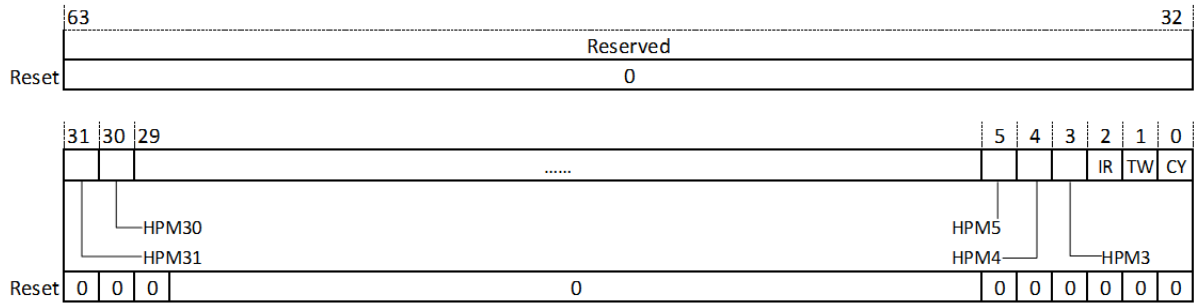


图 16.14: 机器模式事件中断使能寄存器（MCOUNTERINTEN）

当 mcounterinten.bit[n] 为 1 时，对应 mhpmcounter 上溢出时触发中断。

当 mcounterinten.bit[n] 为 0 时，对应 mhpmcounter 上溢出时不触发中断。

16.1.7.9　机器模式事件上溢出标注寄存器（MCOUNTEROF）

机器模式事件上溢出标注寄存器（MCOUNTEROF），用于表示各事件计数是否发生了上溢出。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

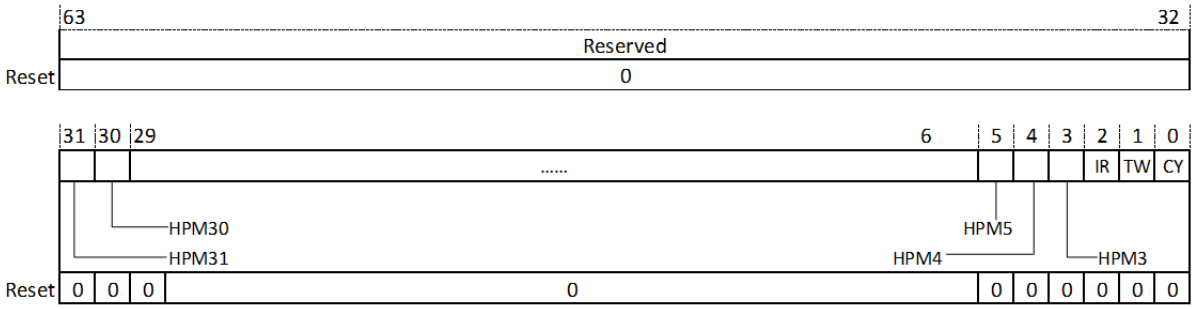


图 16.15: 机器模式事件上溢出标注寄存器（MCOUNTEROF）

当 mcounterof.bit[n] 为 1 时，对应 mhpmcounter 发生了上溢出。

当 mcounterof.bit[n] 为 0 时，对应 mhpmcounter 未发生上溢出。

16.1.8　机器模式 Cache 访问扩展寄存器组

机器模式 Cache 访问扩展寄存器用于直接读取 L1 和 L2 高速缓存中内容，便于对高速缓存进行调试。

16.1.8.1　机器模式 Cache 指令寄存器（MCINS）

机器模式 Cache 指令寄存器（MCINS）用于向 L1 或 L2 高速缓存发起读请求。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

	63 … 1	0
	-	R
Reset	0	0

图 16.16: 机器模式 Cache 指令寄存器（MCINS）

R-Cache 读访问：

- 当 R 为 0 时，不发起 Cache 读请求。
- 当 R 为 1 时，发起 Cache 读请求。

16.1.8.2　机器模式 Cache 访问索引寄存器（MCINDEX）

机器模式 Cache 访问索引寄存器（MCINDEX）用于配置读请求访问的 Cache 位置信息。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

	63 … 32
	-
Reset	0

	31 … 28	27 … 25	24 … 21	20 … 0
	RID	-	WAY	INDEX
Reset	0	0	0	0

图 16.17: 机器模式 Cache 访问索引寄存器（MCINDEX）

RID-RAM 标志位：

指示访问的 RAM 信息。

- 当 RID 为 0 时，表示访问的是 ICACHE TAG RAM。
- 当 RID 为 1 时，表示访问的是 ICACHE DATA RAM。
- 当 RID 为 2 时，表示访问的是 DCACHE TAG RAM。
- 当 RID 为 3 时，表示访问的是 DCACHE DATA RAM。
- 当 RID 为 4 时，表示访问的是 L2CACHE TAG RAM。
- 当 RID 为 5 时，表示访问的是 L2CACHE DATA RAM。
- 当 RID 为 12 时，表示访问的是 DCACHE LD TAG RAM。

WAY-Cache 路信息：

指示 RAM 访问的路位置信息。

INDEX-Cache 索引：

指示 RAM 访问的索引位置信息。

16.1.8.3　机器模式 Cache 数据寄存器（MCDATA0/1）

机器模式 Cache 数据寄存器（MCDATA0/1）用于记录读取 L1 或 L2 高速缓存的数据。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

	63 … 0
	DATA
Reset	0

图 16.18: 机器模式 Cache 访问数据寄存器（MCDATA）

表 16.1: 机器模式 Cache 访问数据寄存器与 RAM 类型对应关系

RAM 类型	CDATA 内容
ICACHE TAG	CDATA0[39:12]: TAG CDATA0[0]: VALID
ICACHE DATA	CDATA0~CDATA1: 128bit DATA
DCACHE TAG	CDATA0[39:12]: TAG CDATA0[2]: DIRTY CDATA0[1]: SHARED CDATA0[0]: VALID
DCACHE DATA	CDATA0~CDATA1: 128bit DATA
L2CACHE TAG	CDATA0[39:12]: TAG CDATA[1]: DIRTY CDATA0[0]: VALID
L2CACHE DATA	CDATA0~CDATA1: 128bit DATA

16.1.9 机器模式处理器型号寄存器组

16.1.9.1 机器模式处理器型号寄存器（MCPUID）

机器模式处理器型号寄存器（MCPUID）存储了处理器型号信息，具体定义请参考《C-SKY 产品 ID 定义规范（4.0 版)》。其复位值由产品本身决定。

16.1.9.2 片上总线基地址寄存器（MAPBADDR）

该寄存器反映处理器片上寄存器（CLINT，PLIC）的基地址。该寄存器的值由硬件集成决定。

16.1.10 多核扩展寄存器组

16.1.10.1 Snoop 监听使能寄存器（MSMPR）

侦听使能寄存器，控制核心能否处理侦听请求。每个核心独立配置本核是否能够处理侦听请求，顶层一致性总线根据各个核的侦听状态控制侦听请求的发送。读写权限为机器模式可读写。

该寄存器的宽度为 64 位，只有 bit 0 有定义，其余是 Reserved。

bit 0：SMPEN-核心侦听使能位

- 当 SMPEN 为 1’b0 时，核心不能够处理侦听请求，顶层屏蔽发送侦听请求给核心。(复位值)
- 当 SMPEN 为 1’b1 时，核心能够处理侦听请求，顶层发送侦听请求给核心。

处理器核心在下电前，必须设置核心对应的 SMPEN=0，以关闭核心的侦听功能。核心上电后，软件在打开 D-Cache 和 MMU 之前，必须设置核心的 SMPEN=1。核心在正常工作时（包括 WFI 模式），必须保持 SMPEN=1，否则结果不可预期。

16.2　附录 C-2 超级用户模式控制寄存器

超级用户模式控制寄存器按照功能分为：超级用户模式异常配置寄存器组、超级用户模式异常处理寄存器组、超级用户模式地址转换寄存器组。

16.2.1　超级用户模式异常配置寄存器组

当异常和中断被降级到超级用户模式响应时，跟机器模式一样，需要通过超级用户模式异常配置寄存器组进行异常的配置。

16.2.1.1　超级用户模式处理器状态寄存器（SSTATUS）

超级用户模式处理器状态寄存器（SSTATUS）存储了处理器在超级用户模式下的状态和控制信息，包括全局中断有效位、异常保留中断有效位、异常保留特权模式位等，是 MSTATUS 的部分映射。

该寄存器的位长是 64 位，寄存器的读写权限是机器模式可读写，即非机器模式访问都会导致非法指令异常。

	63	62 … 36	35 34	33 32
	SD	0	0	UXL
Reset	0	0	2	2

	31 … 25	24 23	22	21	20	19	18	17	16 15	14 13	12 11	10 9	8	7	6	5	4	3	2	1	0
	0	VS	0	0	0			0	XS	FS	0	0	SPP	0	0		0	0	0	SIE	0
					MXR		SUM									SPIE					
Reset	0	0	0	0	0	0	0	0	0	0	3	0	1	0	0	0	0	0	0	0	0

图 16.19: 超级用户模式处理器状态寄存器（SSTATUS）

具体信息请参考机器模式处理器状态寄存器（*MSTATUS*）。

16.2.1.2　超级用户模式中断使能控制寄存器（SIE）

超级用户模式中断使能控制寄存器（SIE）用于控制不同中断类型的使能和屏蔽，是 MIE 的部分映射。该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，超级用户模式下的写权限由对应位的 mideleg 决定，用户模式访问会导致非法指令异常。

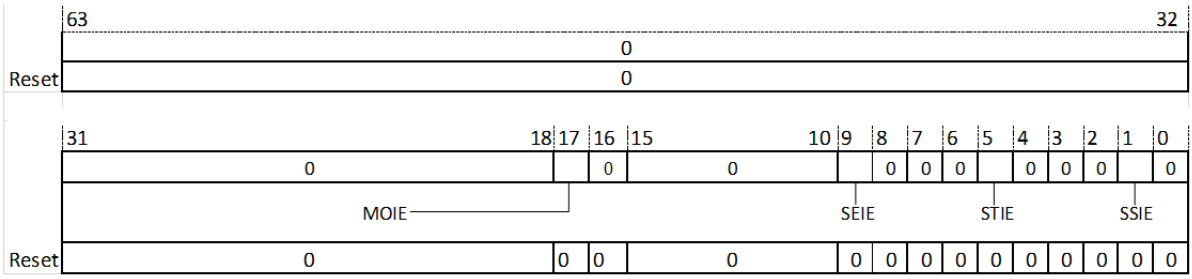


图 16.20: 超级用户模式中断使能控制寄存器（SIE）

具体信息请参考*机器模式中断使能控制寄存器（MIE）*。

16.2.1.3　超级用户模式向量基址寄存器（STVEC）

超级用户模式向量基址寄存器（STVEC）用于配置异常服务程序的入口地址。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读写，用户模式访问都会导致非法指令异常。

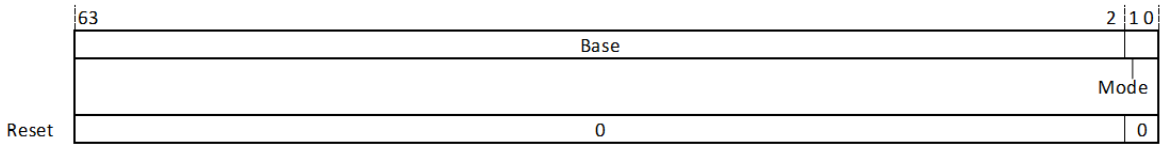


图 16.21: 超级用户模式向量基址寄存器（STVEC）

具体信息请参考*机器模式向量基址寄存器（MTVEC）*。

16.2.1.4　超级用户模式计数器访问授权寄存器（SCOUNTEREN）

超级模式计数器访问授权寄存器（scounteren），用于授权用户模式是否可以访问用户模式计数器。

具体信息，请参考*超级用户模式计数器访问授权寄存器（scounteren）*。

16.2.2　超级用户模式异常处理寄存器组

16.2.2.1　超级用户模式异常临时数据备份寄存器（SSCRATCH）

超级用户模式异常临时数据备份寄存器（SSCRATCH）用于处理器在异常服务程序中备份临时数据。一般用来存储超级用户模式本地上下文空间的入口指针值。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读写，用户模式访问会导致非法指令异常。

16.2.2.2　超级用户模式异常保留程序计数器寄存器（SEPC）

超级用户模式异常保留程序计数器（SEPC）用于存储程序从异常服务程序退出时的程序计数器值（即 PC 值）。C910 支持 16 位宽指令，SEPC 的值以 16 位宽对齐，最低位为零。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读写，用户模式访问会导致非法指令异常。

16.2.2.3　超级用户模式异常事件向量寄存器（SCAUSE）

超级用户模式异常事件向量寄存器（SCAUSE）用于保存触发异常的异常事件向量号，用于在异常服务程序中处理对应事件。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读写，用户模式访问都会导致非法指令异常。

16.2.2.4　超级用户模式中断等待状态寄存器（SIP）

超级用户模式中断等待状态寄存器（SIP）用于保存处理器的中断等待状态。当处理器出现中断无法立即响应的情况时，SIP 寄存器中的对应位会被置位。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，写权限由对应位的 mideleg 决定，用户模式访问都会导致非法指令异常。

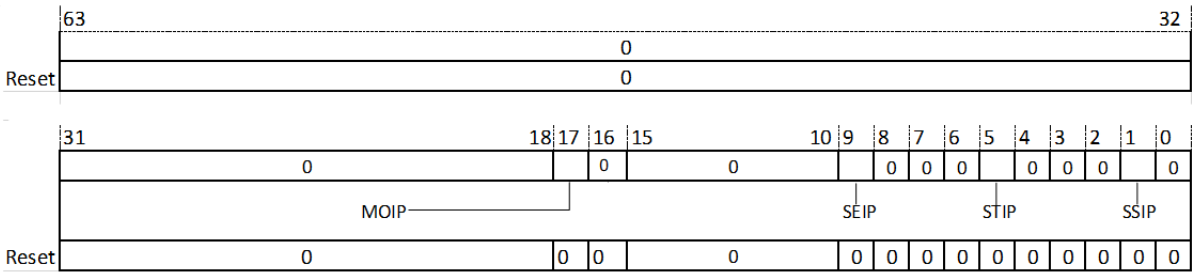


图 16.22: 超级用户模式中断等待状态寄存器（SIP）

16.2.3　超级用户模式地址转换寄存器组

超级用户模式下，需要访问虚拟内存空间。超级用户模式地址转换寄存器（SATP）用于控制 MMU 单元的模式切换、硬件回填基地址和进程号。

16.2.3.1　超级用户模式地址转换寄存器（SATP）

超级用户模式地址转换寄存器（SATP）用于控制 MMU 单元的模式切换、硬件回填基地址和进程号。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读写，用户模式访问会导致非法指令异常。

具体信息请参考 virtual_mem_manage_satp 。

16.2.4　超级用户模式处理器控制和状态扩展寄存器组

16.2.4.1　超级用户模式扩展状态寄存器（SXSTATUS）

超级用户模式扩展状态寄存器（SXSTATUS）是机器模式扩展状态寄存器（MXSTATUS）的映射，具体信息请参考*机器模式扩展状态寄存器（MXSTATUS）*。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，只有 MM 位可写，用户模式访问会导致非法指令异常。

16.2.4.2　超级用户模式硬件控制寄存器（SHCR）

超级用户模式硬件控制寄存器（SHCR）是机器模式硬件控制寄存器（MHCR）的映射，具体信息请参考*机器模式硬件配置寄存器（MHCR）*。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

16.2.4.3　超级用户模式事件溢出中断使能寄存器（SCOUNTERINTEN）

超级用户模式溢出中断使能寄存器（SCOUNTERINTEN）是机器模式中事件溢出中断使能寄存器（MCOUNTERINTEN）的映射，具体信息请参考*机器模式事件中断使能寄存器（MCOUNTERINTEN）*。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

当 mcounterwen.bit[n] 为 1 时，scounterinten.bit[n] 决定对应 shpmcounter 溢出时是否产生中断。

16.2.4.4　超级用户模式事件上溢出标注寄存器（SCOUNTEROF）

超级用户模式事件上溢出标注寄存器（SCOUNTEROF）是机器模式事件上溢出标注寄存器（MCOUNTEROF）的映射，具体信息请参考*机器模式事件上溢出标注寄存器（MCOUNTEROF）*。

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

当 mcounterwen.bit[n] 为 1 时，scounterof.bit[n] 表示对应 shpmcounter 是否产生溢出。

16.2.4.5　超级用户模式周期计数器（SCYCLE）

超级用户模式周期计数器（SCYCLE）用于存储处理器已经执行的周期数，当处理器处于执行状态（即非低功耗状态）下，SCYCLE 寄存器就会在每个执行周期自增计数。

周期计数器为 64 位，周期计数器会被 reset 清零。

具体信息请参考事件计数器 。

16.2.4.6　超级用户模式退休指令计数器（SINSTRET）

超级用户模式退休指令计数器（SINSTRET）用于存储处理器已经退休的指令数，SINSTRET 寄存器会在每条指令退休时自增计数。

退休指令计数器为 64 位，退休指令计数器会被 reset 清零。

具体信息请参考事件计数器 。

16.2.4.7　超级用户模式事件计数器（SHPMCOUNTERn）

超级用户模式事件计数器（SHPMCOUNTERn）是机器模式事件计数器（MHPMCOUNTERn）的映射。

具体信息请参考事件计数器 。

16.2.5　超级用户模式 MMU 扩展寄存器

C910 中 MMU 单元扩展了 MMU 相关寄存器，实现软件回填功能，软件可以直接对 TLB 进行读写等操作。

16.2.5.1　超级用户模式 MMU 控制寄存器（SMCIR）

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

具体信息请参考 virtual_mem_manage_smcir 。

16.2.5.2　超级用户模式 MMU 控制寄存器（SMIR）

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

具体信息请参考*MMU Index 寄存器（SMIR）*。

16.2.5.3　超级用户模式 MMU 控制寄存器（SMEH）

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

具体信息请参考*MMU EntryHi 寄存器（SMEH）*。

16.2.5.4　超级用户模式 MMU 控制寄存器（SMEL）

该寄存器的位长是 64 位，寄存器的读写权限是超级用户模式可读，用户模式访问会导致非法指令异常。

具体信息请参考*MMU EntryLo 寄存器（SMEL）*。

16.3　附录 C-3 用户模式控制寄存器

用户模式控制寄存器按照功能主要分为浮点寄存器、计数器和矢量控制寄存器。

16.3.1　用户模式浮点控制寄存器组

16.3.1.1　浮点异常累积状态寄存器（FFLAGS）

浮点异常累积状态寄存器（FFLAGS）是浮点控制状态寄存器（FCSR）的异常累积域映射，具体信息请参考浮点控制状态寄存器（*FCSR*）。

16.3.1.2　浮点动态舍入模式寄存器（FRM）

浮点动态舍入寄存器（FRM）是浮点控制状态寄存器（FCSR）的舍入模式域映射，具体信息请参考浮点控制状态寄存器（*FCSR*）。

16.3.1.3　浮点控制状态寄存器（FCSR）

浮点控制状态寄存器（FCSR）用于记录浮点的异常累积和舍入模式控制。

该寄存器的位长是 64 位，该寄存器任何模式都可以读写。

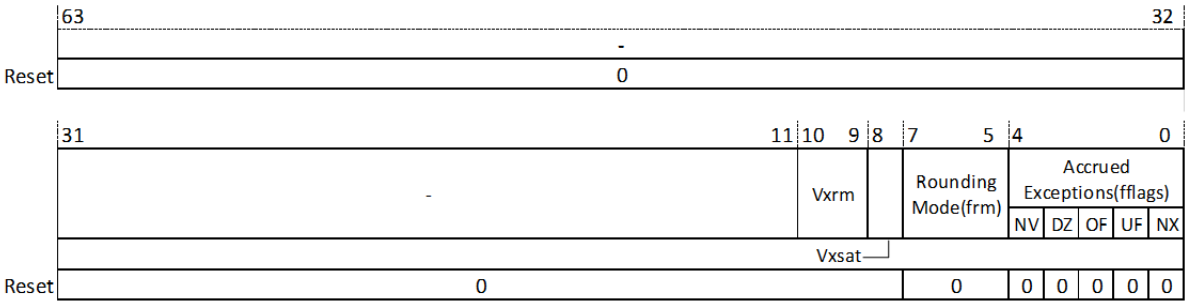


图 16.23: 浮点控制状态寄存器（FCSR）

NX-非精确异常：

- 当 NX=0 时，没有产生非精确异常。
- 当 NX=1 时，产生非精确异常。

UF-下溢异常：

- 当 UF=0 时，没有产生下溢异常。
- 当 UF=1 时，产生下溢异常。

OF-上溢异常：

- 当 OF=0 时，没有产生上溢异常。
- 当 OF=1 时，产生上溢异常。

DZ-除 0 异常：

- 当 DZ=0 时，没有产生除 0 异常。
- 当 DZ=1 时，产生除 0 异常。

NV-无效操作数异常：

- 当 NV=0 时，没有产生无效操作数异常。
- 当 NV=1 时，产生无效操作数异常。

RM-舍入模式：

- 当 RM=0 时，RNE 舍入模式，向最近偶数舍入。
- 当 RM=1 时，RTZ 舍入模式，向 0 舍入。
- 当 RM=2 时，RDN 舍入模式，向负无穷舍入。
- 当 RM=3 时，RUP 舍入模式，向正无穷舍入。
- 当 RM=4 时，RMM 舍入模式，向最近舍入。

VXSAT-矢量溢出标志位：

VXSAT 对应位的映射。

VXRM-矢量舍入模式位：

VXRM 对应位的映射。

16.3.2　用户模式计数/计时寄存器组

16.3.2.1　用户模式周期计数器（CYCLE）

用户模式周期计数器（CYCLE）用于存储处理器已经执行的周期数，当处理器处于执行状态（即非低功耗状态）下，CYCLE 寄存器就会在每个执行周期自增计数。

周期计数器为 64 位，周期计数器会被 reset 清零。

具体信息请参考事件计数器 。

16.3.2.2　用户模式时间计数器（TIME）

用户模式时间计数器（TIME）是 MTIME 的只读映射。

具体信息请参考事件计数器 。

16.3.2.3　用户模式退休指令计数器（INSTRET）

用户模式退休指令计数器（MINSTRET）用于存储处理器已经退休的指令数，INSTRET 寄存器会在每条指令退休时自增计数。

退休指令计数器为 64 位，退休指令计数器会被 reset 清零。

具体信息请参考事件计数器 。

16.3.2.4　用户模式事件计数器（HPMCOUNTERn）

用户模式事件计数器（HPMCOUNTERn）是机器模式事件计数器（MHPMCOUNTERn）的映射。

具体信息请参考事件计数器 。

16.3.3　用户模式扩展浮点控制寄存器组

16.3.3.1　用户模式浮点扩展控制寄存器（FXCR）

用户模式浮点扩展控制寄存器（FXCR）用于浮点扩展功能开关和浮点异常累积位。

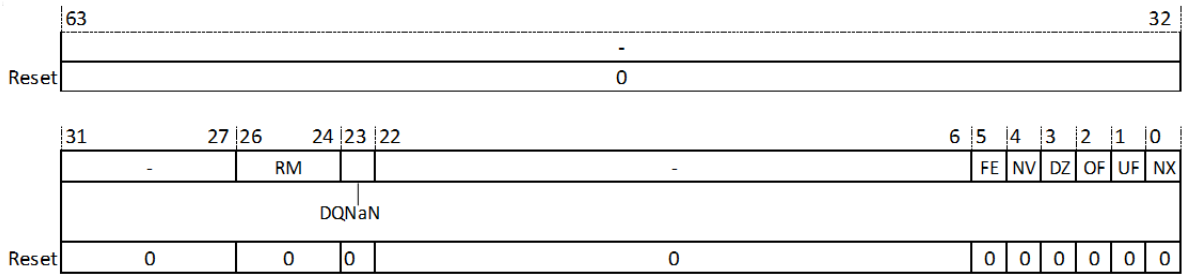


图 16.24: 浮点扩展控制寄存器（FXCR）

NX-非精确异常：

FCSR 对应位的映射。

UF-下溢异常：

FCSR 对应位的映射。

OF-上溢异常：

FCSR 对应位的映射。

DZ-除 0 异常：

FCSR 对应位的映射。

NV-无效操作数异常：

FCSR 对应位的映射。

FE-浮点异常累积位：

当有任何一个浮点异常发生时，该位将被置为 1。

DQNaN-输出 QNaN 模式位：

当 DQNaN 为 0 时，计算输出的 QNaN 值为默认的固定值。

当 DQNaN 为 1 时，计算输出的 QNaN 值根 IEEE754 标准一致。

RM-舍入模式：

FCSR 对应位的映射。